PYTHON PROGRAMMING PRACTICE BOOK

১ থেকে 274টি সমস্যার বিস্তারিত সমাধান, অ্যালগরিদম, কোড এবং ব্যাখ্যা + ৬টি Practical Project
(Beginner to Professional Step-by-Step Guide)

📘 শিক্ষার্থীদের জন্য গুরুত্বপূর্ণ নির্দেশনাবলী (Instructions for Students)

প্রিয় শিক্ষার্থী, Python Programming Practice Book-এ তোমাকে স্বাগতম! প্রোগ্রামিং কোনো মুখস্থ করার বিষয় নয়, এটি হলো সাইকেল চালানো বা সাঁতার কাটার মতো একটি ব্যবহারিক দক্ষতা। তুমি যত বেশি নিজে কীবোর্ডে কোড টাইপ করবে এবং ভুল থেকে শিখবে, তোমার প্রোগ্রামিং দক্ষতা তত দ্রুত বৃদ্ধি পাবে।

এই বইটি থেকে সর্বোচ্চ সুবিধা পেতে নিচের নির্দেশনাবলী অক্ষরে অক্ষরে মেনে চলো:

১. কীভাবে প্রতিটি সমস্যা অনুশীলন করবে? (Step-by-Step Practice Guide)

প্রতিটি সমস্যার সমাধান করার জন্য নিচের ৪টি ধাপ অনুসরণ করবে:

1. প্রথমে নিজে চিন্তা করো: যেকোনো সমস্যা খোলার পর সরাসরি কোড সেকশন দেখবে না। প্রথমে 'সমস্যা' এবং 'কীভাবে চিন্তা করব?' অংশটি মনোযোগ দিয়ে পড়ো।

2. অ্যালগরিদম বোঝো: লজিক বোঝার পর খাতা-কলমে বা নিজের মাথায় সমাধানের ধাপগুলো (Algorithm) সাজিয়ে নাও।

3. নিজে টাইপ করে কোড রান করো: কখনো কোড Copy-Paste করবে না। নিজের কম্পিউটার বা মোবাইলের Python IDE (VS Code, PyCharm, IDLE বা Online Compiler)-এ প্রতিটি লাইন নিজে টাইপ করো।

4. ভিন্ন ইনপুট দিয়ে যাচাই করো: বইয়ে দেওয়া Sample Input ছাড়াও নিজের ইচ্ছামতো বিভিন্ন মান (যেমন বড় সংখ্যা, শূন্য, ঋণাত্মক সংখ্যা ইত্যাদি) দিয়ে কোড রান করে দেখো আউটপুট ঠিক আসে কিনা।

২. অনুশীলনী ও বাড়ির কাজ (Mandatory Practice Problems)

- এই বইয়ের 274টি প্রধান সমস্যার প্রতিটির শেষে 'অনুশীলনের জন্য' ৩টি করে অতিরিক্ত সমস্যা দেওয়া আছে।
- অর্থাৎ, এখানে মোট 274 × ৩ = 822টি অতিরিক্ত প্র্যাকটিস প্রবলেম রয়েছে।
- এর বাইরে শেষে ৬টি পূর্ণাঙ্গ Practical Project (Experiment) দেওয়া আছে — সেগুলো অবশ্যই নিজে রান করে খাতায় রিপোর্ট তৈরি করবে।
- প্রধান সমস্যার কোড বোঝার পর সেই লজিক ব্যবহার করে নিচের ৩টি সমস্যা অবশ্যই নিজে নিজে সমাধান করবে। এটিই তোমার আসল পরীক্ষা ও বাড়ির কাজ।

৩. কোড লেখার সেরা নিয়মাবলী (Coding Best Practices)

- **Indentation (স্পেস বা ট্যাব):** পাইথনে if, else, for, while বা def লেখার পর পরবর্তী লাইনে অবশ্যই ৪টি স্পেস (বা ১টি Tab) দিতে হয়। Indentation ভুল হলে IndentationError হবে।
- অর্থপূর্ণ ভেরিয়েবল নাম: ভেরিয়েবলের নাম সবসময় কাজের সাথে মিল রেখে দেবে। যেমন বয়সের জন্য a না লিখে age লিখবে, যোগফলের জন্য t না লিখে total লিখবে।
- **Case Sensitivity:** পাইথনে ছোট হাতের ও বড় হাতের অক্ষর আলাদা। print() এর জায়গায় Print() লিখলে কাজ করবে না।
- কমেন্ট করা **(Comments):** কোডের গুরুত্বপূর্ণ লাইনের পাশে # দিয়ে ছোট করে লিখে রাখো লাইনটি কী কাজ করছে।

৪. ভুল হলে বা Error আসলে কী করবে? (Debugging Mindset)

> **[Important]** কোড রান করতে গিয়ে Error বা লাল লেখা আসলে ঘাবড়ে যাবে না বা ভয় পাবে না। পৃথিবীর সেরা প্রোগ্রামারদের কোডেও প্রতিদিন হাজারবার Error আসে!

- **Error Message** পড়ো**:** স্ক্রিনে যে Error দেয়, তার একদম নিচের লাইনটি পড়ো। সেখানে লেখা থাকে কী ধরনের ভুল হয়েছে (যেমন SyntaxError, NameError, TypeError) এবং কত নম্বর লাইনে ভুল হয়েছে।
- বইয়ের **Common Mistakes** দেখো**:** প্রতিটি সমস্যার নিচে 'Common Mistakes' সেকশন দেওয়া আছে। শিক্ষার্থীরা সাধারণত কী ভুল করে তা সেখানে ব্যাখ্যা করা আছে। তোমার ভুলের সাথে সেটি মিলিয়ে নাও।

৫. প্রতিদিনের অনুশীলনের রুটিন (Daily Practice Routine)

একদিনে সব শেষ করার চেষ্টা না করে নিয়মিত অল্প অল্প করে অনুশীলন করো। নিচের রুটিনটি অনুসরণ করতে পারো:

সময়কাল	লক্ষ্যমাত্রা	করণীয়
১ম ৩০ মিনিট	নতুন সমস্যা শেখা	বই থেকে ৩-৫টি সমস্যার লজিক বোঝা ও কোড টাইপ করে রান করা
পরবর্তী ৩০ মিনিট	নিজে নিজে সমাধান	ওই সমস্যাগুলোর নিচের 'অনুশীলনের জন্য' দেওয়া অতিরিক্ত সমস্যাগুলো সমাধান করা
সপ্তাহের শেষে	রিভিশন ও প্রজেক্ট	পুরো সপ্তাহের শেখা কোডগুলো না দেখে লেখার চেষ্টা করা

> **[মনে রাখবে]** "Practice makes a programmer perfect!" প্রতিদিন অন্তত ১ ঘণ্টা কোডিং অনুশীলন চালিয়ে যাও। শুভকামনা!

Topic: Basic Output

সমস্যা 1: Hello World বার্তা স্ক্রিনে প্রিন্ট করুন।

সমস্যা

কম্পিউটার স্ক্রিনে একটি সাধারণ বার্তা "Hello World" প্রদর্শন করতে হবে। এটি যেকোনো প্রোগ্রামিং ভাষা শেখার প্রথম পদক্ষেপ।

কী কী জানা থাকতে হবে?

- print() ফাংশন
- String Data Type (ডাবল কোটেশন বা সিঙ্গেল কোটেশনের ব্যবহার)

কীভাবে চিন্তা করব?

আমাদের উদ্দেশ্য হলো পাইথনের বিল্ট-ইন আউটপুট ফাংশন print() ব্যবহার করে স্ক্রিনে কিছু দেখানো। print() ফাংশনের ভেতরে আমরা যা লিখব তা যদি String (টেক্সট) হয়, তবে তাকে অবশ্যই ডাবল কোটেশন (" ") বা সিঙ্গেল কোটেশন (' ') এর ভেতরে রাখতে হবে।

> **[Best Practice]** সর্বদা পাইথনের কোড লেখার সময় ছোট হাতের অক্ষর (lowercase) দিয়ে print ফাংশন লিখবেন।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: print() ফাংশন কল করা এবং এর ভেতরে "Hello World" স্ট্রিংটি পাস করা।

Step 3: প্রোগ্রাম শেষ করা।

Python Code

```
# Displaying Hello World on the screen
print("Hello World")
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Hello World")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Hello World

ব্যাখ্যা

এখানে print() হলো পাইথনের একটি Built-in Function যা কনসোলে আউটপুট দেখায়। "Hello World" হলো একটি String বা শব্দগুচ্ছ। কোটেশন চিহ্ন পাইথনকে বুঝিয়ে দেয় যে এটি সাধারণ টেক্সট।

Common Mistakes

- print বানান ভুল করে Print বা PRINT লেখা (পাইথন Case-sensitive)।
- কোটেশন চিহ্ন (" " বা ' ') দিতে ভুলে যাওয়া।
- ব্র্যাকেট () সঠিকভাবে বন্ধ না করা।

অনুশীলনের জন্য

1. স্ক্রিনে আপনার দেশের নাম (যেমন: Bangladesh) প্রিন্ট করুন।

2. স্ক্রিনে "Python Programming is Fun!" বার্তাটি প্রদর্শন করুন।

3. একই print() ফাংশন ব্যবহার করে আপনার স্কুল বা বিশ্ববিদ্যালয়ের নাম প্রদর্শন করুন।

সমস্যা 2: আপনার নিজের নাম স্ক্রিনে সুন্দরভাবে প্রিন্ট করুন।

সমস্যা

স্ক্রিনে আপনার সম্পূর্ণ নাম সুন্দরভাবে ফরম্যাট করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- print() ফাংশন
- String Formatting

কীভাবে চিন্তা করব?

আগের সমস্যার মতোই আমরা print() ফাংশন ব্যবহার করব। তবে আউটপুটটি যেন দেখতে সুন্দর ও প্রফেশনাল হয়, সেজন্য নামের আগে একটি ছোট বার্তা যেমন 'My name is' যুক্ত করে দিতে পারি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: print() ফাংশনের ভেতরে বার্তা এবং নাম একসাথে স্ট্রিং হিসেবে লেখা।

Step 3: স্ক্রিনে ফলাফল প্রদর্শন করা।

Python Code

```
# Printing personal name cleanly
print("My name is: Abdur Rahman")
```

Built-in Function / Shortcut Method

```
# Variable based approach
# name = "Abdur Rahman"
# print(f"My name is: {name}")
```

Sample Input / Output Table

Sample Input	Sample Output
None	My name is: Abdur Rahman

ব্যাখ্যা

print() ফাংশনের ভেতরে আমরা কোলন (:) এবং স্পেস ব্যবহার করে আউটপুটটিকে পড়ার উপযোগী (Readable) করেছি।

Common Mistakes

- স্ট্রিংয়ের ভেতরে স্পেস না দেওয়া, যার ফলে শব্দগুলো একসাথে লেগে যায়।
- বাংলা বা ইংরেজি যাই লিখুন না কেন, কোটেশন চিহ্নের ভেতরে না রাখা।

অনুশীলনের জন্য

1. আপনার বাবার নাম স্ক্রিনে প্রদর্শন করুন।

2. আপনার প্রিয় শিক্ষকের নাম স্ক্রিনে প্রিন্ট করুন।

3. আপনার প্রিয় বন্ধুর নাম ফরম্যাট করে প্রিন্ট করুন।

সমস্যা 3: আপনার সম্পূর্ণ ঠিকানা একাধিক লাইনে প্রিন্ট করুন।

সমস্যা

আপনার বর্তমান ঠিকানা (যেমন: বাড়ি নম্বর, এলাকা, জেলা ও দেশ) আলাদা আলাদা লাইনে (Multiple lines) প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- print() ফাংশন
- Triple Quotes (''' ''') অথবা Newline Character (\n)

কীভাবে চিন্তা করব?

একাধিক লাইনে আউটপুট দেখানোর জন্য আমরা দুটি পদ্ধতি অবলম্বন করতে পারি: ১) একাধিক print() ফাংশন ব্যবহার করা, অথবা ২) একটিমাত্র print() ফাংশনের ভেতর Escape Character (\n) ব্যবহার করা। নতুনদের জন্য \n ব্যবহার করে বা একাধিক print() লিখে লাইন ব্রেক করা সবচেয়ে সহজ।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: প্রথম লাইনে বাড়ির নম্বর ও রাস্তার নাম প্রিন্ট করা।

Step 3: দ্বিতীয় লাইনে শহরের নাম প্রিন্ট করা।

Step 4: তৃতীয় লাইনে দেশের নাম প্রিন্ট করা।

Step 5: প্রোগ্রাম শেষ করা।

Python Code

```
# Printing address in multiple lines using multiple print statements
print("House No: 12, Road No: 5")
print("Dhanmondi, Dhaka")
print("Bangladesh")
```

Built-in Function / Shortcut Method

```
# Using Escape Character (\n) in a single print statement
# print("House No: 12, Road No: 5\nDhanmondi, Dhaka\nBangladesh")
```

Sample Input / Output Table

Sample Input	Sample Output
None	House No: 12, Road No: 5 Dhanmondi, Dhaka Bangladesh

ব্যাখ্যা

পাইথনে প্রতিটি print() ফাংশন স্বয়ংক্রিয়ভাবে লাইনের শেষে একটি Newline তৈরি করে। তাই পর পর তিনটি print() লিখলে আউটপুট তিনটি আলাদা লাইনে দেখায়।

Common Mistakes

- একই লাইনে সব ঠিকানা লিখে ফেলা।
- \n ব্যবহার করার সময় স্লাশ (/) এবং ব্যাকস্লাশ (\) গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. আপনার স্কুল বা কলেজের সম্পূর্ণ ঠিকানা ৩ লাইনে প্রিন্ট করুন।

2. আপনার কর্মস্থলের ঠিকানা একাধিক লাইনে প্রদর্শন করুন।

3. একটি চিঠি বা দরখাস্তের প্রাপকের ঠিকানা ৪ লাইনে প্রিন্ট করুন।

সমস্যা 4: নিজের ব্যক্তিগত তথ্য (নাম, বয়স, জেলা, দেশ) একাধিক লাইনে প্রদর্শন করুন।

সমস্যা

একজন শিক্ষার্থীর মৌলিক ব্যক্তিগত তথ্য যেমন নাম, বয়স, জেলার নাম এবং দেশের নাম পরপর আলাদা লাইনে একটি বায়োডাটার মতো প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- print() ফাংশন
- String ও Numeric Data একসাথে প্রিন্ট করার ধারণা

কীভাবে চিন্তা করব?

আমরা প্রতিটি তথ্যের জন্য আলাদা print() ব্যবহার করব। যাতে প্রতিটি তথ্য (Name, Age, District, Country) স্ক্রিনে একটি সুন্দর ছক বা তালিকার মতো দেখায়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: প্রথম print() দিয়ে নাম প্রদর্শন করা।

Step 3: দ্বিতীয় print() দিয়ে বয়স প্রদর্শন করা।

Step 4: তৃতীয় ও চতুর্থ print() দিয়ে যথাক্রমে জেলা ও দেশ প্রদর্শন করা।

Step 5: প্রোগ্রাম শেষ করা।

Python Code

```
# Displaying personal information cleanly
print("Name    : Abdur Rahman")
print("Age     : 22 Years")
print("District: Dhaka")
print("Country : Bangladesh")
```

Built-in Function / Shortcut Method

```
# Using multi-line triple quoted string
# info = '''Name    : Abdur Rahman
# Age     : 22 Years
# District: Dhaka
# Country : Bangladesh'''
# print(info)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Name : Abdur Rahman

```
Age     : 22 Years
District: Dhaka
Country : Bangladesh
```

ব্যাখ্যা

এখানে কোলন (:) গুলোকে একই বরাবর রাখার জন্য আমরা স্পেস ব্যবহার করেছি, যা আউটপুটকে একটি প্রফেশনাল টেবিল বা ফর্মের মতো রূপ দিয়েছে।

Common Mistakes

- তথ্যগুলো এলোমেলোভাবে লেখা যার ফলে দেখতে অগোছালো লাগে।
- বয়স লেখার সময় সংখ্যার সাথে কোটেশন না দিলে বা String হিসেবে সঠিকভাবে যুক্ত না করা।

অনুশীলনের জন্য

1. আপনার পছন্দের ৫টি বইয়ের নাম এবং লেখকের নাম লিস্ট আকারে প্রিন্ট করুন।

2. আপনার পরিবারের সদস্যদের নাম ও সম্পর্ক একাধিক লাইনে প্রিন্ট করুন।

3. একটি পণ্যের মেমো (পণ্যের নাম, দাম, পরিমাণ) স্ক্রিনে প্রদর্শন করুন।

সমস্যা 5: Escape Character (\n, \t, \", \\) ব্যবহার করে একটি সুন্দর আউটপুট তৈরি করুন।

সমস্যা

পাইথনের বিশেষ Escape Sequence যেমন নতুন লাইন (\n), ট্যাব স্পেস (\t), ডাবল কোটেশন (\") এবং ব্যাকস্লাশ (\\) ব্যবহার করে একটি ফরম্যাটেড টেক্সট প্রিন্ট করতে হবে।

কী কী জানা থাকতে হবে?

- Escape Sequence (\n, \t, \", \\)
- print() ফাংশন

কীভাবে চিন্তা করব?

স্ট্রিংয়ের ভেতরে কিছু বিশেষ ক্যারেক্টার সরাসরি প্রিন্ট করা যায় না (যেমন কোটেশনের ভেতরে কোটেশন)। ব্যাকস্লাশ (\) দিয়ে আমরা পাইথনকে বোঝাই যে পরবর্তী ক্যারেক্টারটি একটি বিশেষ নির্দেশ। \n নতুন লাইন তৈরি করে, \t চারটি স্পেসের সমান জায়গা (Tab) তৈরি করে।

> **[Important]** উইন্ডোজের ফাইল পাথ প্রিন্ট করতে হলে সবসময় ডাবল ব্যাকস্লাশ (\\) অথবা Raw String (r'...') ব্যবহার করতে হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: print() ফাংশনের ভেতরে \t ব্যবহার করে শিরোনামে ট্যাব স্পেস দেওয়া।

Step 3: \" ব্যবহার করে কোনো শব্দকে কোটেশনের ভেতর বন্দি করা।

Step 4: \\ ব্যবহার করে ব্যাকস্লাশ চিহ্ন প্রিন্ট করা।

Step 5: ফলাফল স্ক্রিনে প্রদর্শন করা।

Python Code

```
# Demonstrating escape characters
print("Course:\tPython Programming")
print("Topic:\tEscape Characters")
print("Status:\t\"Completed\"")
print("Path:\tC:\\Users\\Python\\Notes")
```

Built-in Function / Shortcut Method

```
# Raw string approach (for file paths)
# print(r"Path:\tC:\Users\Python\Notes")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Course: Python Programming Topic: Escape Characters Status: "Completed" Path: C:\Users\Python\Notes

ব্যাখ্যা

\t এর কারণে 'Course:' এর পর একটি Tab স্পেস তৈরি হয়েছে। \" ব্যবহারের ফলে 'Completed' শব্দটি কোটেশনসহ প্রিন্ট হয়েছে। আর \\ লেখার ফলে স্ক্রিনে একটি ব্যাকস্লাশ (\) প্রিন্ট হয়েছে।

Common Mistakes

- ফরোয়ার্ড স্লাশ (/) দিয়ে Escape Character লেখার চেষ্টা করা (যেমন /n বা /t)।
- ফাইল পাথ লেখার সময় সিঙ্গেল ব্যাকস্লাশ দেওয়া, যা পাইথন Error বা ভুল আউটপুট দেয়।

অনুশীলনের জন্য

1. \t ব্যবহার করে একটি ৩ কলামের রুটিন বা সময়সূচী প্রিন্ট করুন।

2. স্ক্রিনে "Honesty is the best policy" বাক্যটি ডাবল কোটেশনসহ প্রিন্ট করুন।

3. \n এবং \t একসাথে ব্যবহার করে একটি সিঁড়ির (Staircase) মতো প্যাটার্ন তৈরি করুন।

সমস্যা 6: print() ফাংশনের sep ব্যবহার করে একাধিক মান নির্দিষ্ট Separator দিয়ে প্রিন্ট করুন।

সমস্যা

print() ফাংশনে কমা (,) দিয়ে একাধিক শব্দ বা মান লিখলে সাধারণত তাদের মাঝে স্পেস দেখা যায়। sep প্যারামিটার ব্যবহার করে স্পেসের বদলে হাইফেন (-), কমা বা অন্য যেকোনো চিহ্ন দিয়ে মানগুলো আলাদা করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- print() ফাংশনের sep (Separator) প্যারামিটার
- Multiple Arguments in print()

কীভাবে চিন্তা করব?

print() ফাংশনে যখন আমরা কমা দিয়ে একাধিক মান দিই যেমন `print('2026', '07', '01')`, তখন পাইথন ডিফল্টভাবে তাদের মাঝে একটি স্পেস বসায়। আমরা যদি `sep='-'` লিখে দিই, তবে স্পেসের বদলে হাইফেন বসে যাবে এবং তারিখটি সুন্দর ফরম্যাটে দেখাবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: print() ফাংশনে তারিখ, মাস এবং বছর আলাদা আর্গুমেন্ট হিসেবে দেওয়া।

Step 3: sep='-' প্যারামিটার যুক্ত করে আর্গুমেন্টগুলোর মাঝখানে হাইফেন বসানো।

Step 4: আউটপুট প্রদর্শন করা।

Python Code

```
# Printing a formatted date using sep parameter
print("2026", "07", "01", sep="-")

# Printing items separated by arrow
print("Dhaka", "Chittagong", "Sylhet", sep=" -> ")
```

Built-in Function / Shortcut Method

```
# Using join method (Alternative shortcut)
# print("-".join(["2026", "07", "01"]))
```

Sample Input / Output Table

Sample Input	Sample Output

```
None                                    2026-07-01
                                        Dhaka -> Chittagong -> Sylhet
```

ব্যাখ্যা

sep='-' নির্দেশ করায় print() ফাংশন প্রতিটি কমা (,) এর জায়গায় একটি করে হাইফেন (-) বসিয়ে দিয়েছে। দ্বিতীয় উদাহরণে sep=' -> ' দেওয়ার ফলে শহরের নামগুলোর মাঝে তীর চিহ্ন বসেছে।

Common Mistakes

- sep প্যারামিটারটি print() এর একদম শেষে না লিখে মাঝে লিখে ফেলা।
- sep= এর মান হিসেবে স্ট্রিং (কোটেশন) না দিয়ে সরাসরি চিহ্ন লিখে দেওয়া।

অনুশীলনের জন্য

1. sep=':' ব্যবহার করে সময় (যেমন 10:30:45) প্রিন্ট করুন।

2. sep=' * ' ব্যবহার করে ৩টি সংখ্যার গুণনের রূপ (যেমন 5 * 4 * 3) প্রিন্ট করুন।

3. sep=',' ব্যবহার করে ৫টি ফলের নাম কমা দিয়ে আলাদা করে প্রিন্ট করুন।

সমস্যা 7: print() ফাংশনের end ব্যবহার করে একাধিক print() স্টেটমেন্ট একই লাইনে প্রদর্শন করুন।

সমস্যা

সাধারণত পাইথনে পরপর দুটি print() লিখলে আউটপুট দুটি আলাদা লাইনে আসে। end প্যারামিটার ব্যবহার করে একাধিক print() স্টেটমেন্টের আউটপুট একই লাইনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- print() ফাংশনের end প্যারামিটার

কীভাবে চিন্তা করব?

পাইথনের print() ফাংশনের ডিফল্ট end প্যারামিটার হলো নতুন লাইন (`end='\n'`)। আমরা যদি প্রথম print() ফাংশনের শেষে `end=' '` (স্পেস) বা `end=''` (খালি স্ট্রিং) দিয়ে দিই, তবে পরবর্তী print() এর আউটপুট নতুন লাইনে না গিয়ে একই লাইনের একদম পাশে বসে যাবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: প্রথম print() ফাংশন লেখা এবং এর শেষে end=' ' যুক্ত করা।

Step 3: দ্বিতীয় print() ফাংশন লেখা।

Step 4: ফলাফল একই লাইনে প্রদর্শন করা।

Python Code

```
# Printing multiple statements on the same line using end parameter
print("Hello", end=" ")
print("World!", end=" ")
print("Welcome to Python.")
```

Built-in Function / Shortcut Method

```
# Combining in single print
# print("Hello", "World!", "Welcome to Python.")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Hello World! Welcome to Python.

ব্যাখ্যা

প্রথম দুটি print() ফাংশনের শেষে `end=' '` দেওয়ার কারণে পাইথন নতুন লাইনে যায়নি, বরং একটি স্পেস দিয়ে পরবর্তী print() এর আউটপুট একই লাইনে যুক্ত করেছে।

Common Mistakes

- end=' ' এর ভেতর কোনো স্পেস না দিলে শব্দগুলো একসাথে লেগে যাওয়া (যেমন HelloWorld!)।
- end প্যারামিটার লেখার সময় কোটেশন দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. তিনটি আলাদা print() ব্যবহার করে 'I', 'Love', 'Python' কথাটি একই লাইনে প্রিন্ট করুন।

2. লুপ বা একাধিক print() এর মাধ্যমে ১ থেকে ৫ পর্যন্ত সংখ্যাগুলো পাশাপাশি কমা দিয়ে প্রিন্ট করুন।

3. end=' --- ' ব্যবহার করে দুটি আলাদা বাক্যাংশ একই লাইনে যুক্ত করুন।

Topic: Variable, Input & Data Type

সমস্যা 8: ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে তাদের যোগফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর (User) কাছ থেকে কীবোর্ডের মাধ্যমে দুটি সংখ্যা ইনপুট নিতে হবে এবং তাদের যোগফল নির্ণয় করে স্ক্রিনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- input() ফাংশন
- int() বা float() Type Casting
- যোগ (+) অপারেটর

কীভাবে চিন্তা করব?

input() ফাংশন দিয়ে আমরা যখন ব্যবহারকারীর কাছ থেকে কিছু ইনপুট নিই, পাইথন সেটিকে String বা টেক্সট হিসেবে গ্রহণ করে। যদি আমরা দুটি সংখ্যা সরাসরি যোগ করি (যেমন '5' + '10'), তবে আউটপুট হবে '510'। তাই গাণিতিক যোগ করার জন্য আমাদের অবশ্যই ইনপুটটিকে int() বা float() ফাংশন দিয়ে সংখ্যায় রূপান্তর (Type Cast) করতে হবে।

> **[Important]** ইনপুট নেওয়ার সময় ব্যবহারকারী দশমিক সংখ্যা দিতে পারে মনে হলে int() এর পরিবর্তে float() ব্যবহার করা সবচেয়ে নিরাপদ।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ব্যবহারকারীর কাছ থেকে প্রথম সংখ্যাটি ইনপুট নেওয়া এবং int() বা float() দিয়ে রূপান্তর করে num1 ভেরিয়েবলে রাখা।

Step 3: ব্যবহারকারীর কাছ থেকে দ্বিতীয় সংখ্যাটি ইনপুট নেওয়া এবং num2 ভেরিয়েবলে রাখা।

Step 4: total = num1 + num2 সূত্র দিয়ে যোগফল নির্ণয় করা।

Step 5: যোগফল স্ক্রিনে প্রিন্ট করা।

Python Code

```
# Taking two numbers from user and finding their sum
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

total = num1 + num2
print("The sum is:", total)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("The sum is:", int(input("Enter first number: ")) + int(input("Enter
second number: ")))
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter first number: 15              The sum is: 40
Enter second number: 25
```

ব্যাখ্যা

`int(input(...))` লেখার মাধ্যমে ইনপুট নেওয়া স্ট্রিং মানটি সাথে সাথেই Integer বা পূর্ণসংখ্যায় রূপান্তরিত হয়েছে। এরপর + অপারেটর দিয়ে তাদের যোগ করে total ভেরিয়েবলে রাখা হয়েছে।

Common Mistakes

- int() বা float() ব্যবহার না করা, যার ফলে দুটি সংখ্যা যোগ না হয়ে পাশাপাশি বসে যায় (String Concatenation)।
- ভেরিয়েবলের নাম লেখার সময় স্পেস ব্যবহার করা (যেমন `first num` এর বদলে `first_num` লিখতে হবে)।

অনুশীলনের জন্য

1. ব্যবহারকারীর কাছ থেকে দুটি দশমিক সংখ্যা (float) ইনপুট নিয়ে তাদের যোগফল নির্ণয় করুন।

2. ব্যবহারকারীর কাছ থেকে তিনটি সংখ্যা ইনপুট নিয়ে তাদের মোট যোগফল নির্ণয় করুন।

3. আপনার বর্তমান বয়স ইনপুট নিয়ে তার সাথে ১০ যোগ করে ১০ বছর পর আপনার বয়স কত হবে তা প্রদর্শন করুন।

সমস্যা 9: ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে তাদের বিয়োগফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে প্রথম সংখ্যাটি থেকে দ্বিতীয় সংখ্যাটি বিয়োগ করতে হবে এবং ফলাফল প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- input() ফাংশন
- int() বা float() Type Casting
- বিয়োগ (-) অপারেটর

কীভাবে চিন্তা করব?

আগের সমস্যার মতোই আমরা দুটি সংখ্যা ইনপুট নেব। এবার আমরা বিয়োগফল বের করার জন্য মাইনাস (-) চিহ্ন বা Subtraction Operator ব্যবহার করব। প্রথম সংখ্যাটি ছোট এবং দ্বিতীয়টি বড় হলে ফলাফল ঋণাত্মক (Negative) আসবে, যা পাইথন সহজেই হিসাব করতে পারে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: প্রথম সংখ্যা ইনপুট নিয়ে num1 এ সংরক্ষণ করা।

Step 3: দ্বিতীয় সংখ্যা ইনপুট নিয়ে num2 তে সংরক্ষণ করা।

Step 4: result = num1 - num2 সূত্র প্রয়োগ করা।

Step 5: বিয়োগফল প্রিন্ট করা।

Python Code

```
# Finding subtraction of two user inputs
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

difference = num1 - num2
print("The difference is:", difference)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Difference:", int(input("First: ")) - int(input("Second: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 50 Enter second number: 20	The difference is: 30

ব্যাখ্যা

`difference = num1 - num2` লাইনে num1 থেকে num2 বিয়োগ করে difference ভেরিয়েবলে রাখা হয়েছে এবং print() দিয়ে প্রদর্শন করা হয়েছে।

Common Mistakes

- বড় সংখ্যা থেকে ছোট সংখ্যা বিয়োগ করার সময় ভেরিয়েবলের ক্রম (Order) ভুল করা।
- ইনপুট নেওয়ার সময় int() করতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. আপনার কাছে মোট কত টাকা ছিল এবং কত টাকা খরচ করেছেন তা ইনপুট নিয়ে অবশিষ্ট টাকা নির্ণয় করুন।

2. বর্তমান সাল এবং আপনার জন্মসাল ইনপুট নিয়ে আপনার বয়স হিসাব করুন।

3. দুটি দশমিক সংখ্যার বিয়োগফল নির্ণয় করুন।

সমস্যা 10: ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে তাদের গুণফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে তাদের গুণফল (Product) নির্ণয় করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- input() ফাংশন
- গুণ (*) অপারেটর
- Type Casting

কীভাবে চিন্তা করব?

প্রোগ্রামিংয়ে গুণের কাজ করার জন্য ক্রস চিহ্ন (x) এর পরিবর্তে অ্যাস্টেরিস্ক বা তারকা চিহ্ন (*) ব্যবহার করা হয়। আমরা দুটি সংখ্যা ইনপুট নিয়ে * অপারেটর দিয়ে গুণফল বের করব।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে num1 এবং num2 ভেরিয়েবলে সংরক্ষণ করা।

Step 3: product = num1 * num2 সূত্র দিয়ে গুণফল বের করা।

Step 4: গুণফল স্ক্রিনে প্রদর্শন করা।

Python Code

```
# Multiplying two user inputted numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

product = num1 * num2
print("The product is:", product)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Product:", float(input("First: ")) * float(input("Second: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 12.5	The product is: 50.0

```
Enter second number: 4
```

ব্যাখ্যা

এখানে আমরা `float()` ব্যবহার করেছি যাতে পূর্ণসংখ্যা এবং দশমিক উভয় ধরনের ইনপুট নিয়েই গুণফল সঠিকভাবে নির্ণয় করা যায়।

Common Mistakes

- গুণের জন্য x বা X ব্যবহার করা, যা পাইথনে Syntax Error তৈরি করে।
- স্ট্রিংয়ের সাথে সংখ্যা গুণ করে ফেলা (যেমন '5' * 3 দিলে '555' আউটপুট আসবে)।

অনুশীলনের জন্য

1. একটি খাতার দাম এবং আপনি কয়টি খাতা কিনবেন তা ইনপুট নিয়ে মোট খরচ নির্ণয় করুন।

2. একজন শ্রমিকের দৈনিক মজুরি এবং কাজের দিন সংখ্যা ইনপুট নিয়ে মোট আয় নির্ণয় করুন।

3. তিনটি সংখ্যার গুণফল নির্ণয় করুন।

সমস্যা 11: ব্যবহারকারীর কাছ থেকে দুটি সংখ্যা ইনপুট নিয়ে তাদের ভাগফল নির্ণয় করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে প্রথম সংখ্যাটিকে দ্বিতীয় সংখ্যাটি দ্বারা ভাগ করতে হবে এবং ভাগফল (Quotient) প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- ভাগ (/) অপারেটর
- float() Type Casting

কীভাবে চিন্তা করব?

পাইথনে সাধারণ ভাগের জন্য ফরোয়ার্ড স্লাশ (/) অপারেটর ব্যবহার করা হয়। পাইথনে সাধারণ ভাগ করলে ফলাফল সবসময় Float (দশমিক সংখ্যা) হিসেবে আসে, এমনকি দুটি সংখ্যা নিঃশেষে বিভাজ্য হলেও (যেমন 10 / 2 = 5.0)। তবে মনে রাখতে হবে, শূন্য (0) দিয়ে কোনো সংখ্যাকে ভাগ করা যায় না।

> **[Note]** পাইথনে / অপারেটর সর্বদা দশমিক মান দেয়। পূর্ণসংখ্যা ভাগফল চাইলে // (Floor Division) ব্যবহার করতে হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ভাজ্য (Dividend) ইনপুট নিয়ে num1 এ রাখা।

Step 3: ভাজক (Divisor) ইনপুট নিয়ে num2 তে রাখা।

Step 4: quotient = num1 / num2 সূত্র দিয়ে ভাগফল নির্ণয় করা।

Step 5: ভাগফল প্রদর্শন করা।

Python Code

```
# Dividing two numbers
num1 = float(input("Enter dividend: "))
num2 = float(input("Enter divisor: "))

quotient = num1 / num2
print("The quotient is:", quotient)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Quotient:", float(input("Dividend: ")) / float(input("Divisor: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter dividend: 25 Enter divisor: 4	The quotient is: 6.25

ব্যাখ্যা

`/` অপারেটরটি num1 কে num2 দ্বারা ভাগ করে সঠিক দশমিক ভাগফল 6.25 প্রদান করেছে।

Common Mistakes

- ভাগ করার জন্য ব্যাকস্লাশ (\) ব্যবহার করা।
- ভাজক হিসেবে শূন্য (0) ইনপুট দেওয়া, যা `ZeroDivisionError` তৈরি করে।

অনুশীলনের জন্য

1. মোট দূরত্ব এবং মোট সময় ইনপুট নিয়ে গড় গতিবেগ (Speed = Distance / Time) নির্ণয় করুন।

2. মোট বিলের পরিমাণ এবং বন্ধুদের সংখ্যা ইনপুট নিয়ে জনপ্রতি কত টাকা দিতে হবে তা নির্ণয় করুন।

3. একটি বাৎসরিক আয় ইনপুট নিয়ে মাসিক গড় আয় বের করুন।

সমস্যা 12: দুটি সংখ্যার ভাগশেষ (Remainder) নির্ণয় করুন।

সমস্যা

একটি সংখ্যাকে অপর একটি সংখ্যা দ্বারা ভাগ করলে কত ভাগশেষ থাকে তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Modulus (%) অপারেটর

কীভাবে চিন্তা করব?

প্রোগ্রামিংয়ে ভাগশেষ বের করার জন্য Modulus বা শতকরা চিহ্ন (%) ব্যবহার করা হয়। যেমন আমরা যদি 14 কে 5 দিয়ে ভাগ করি, তবে ৫ দুগুণে ১০ আর ভাগশেষ থাকে ৪। `14 % 5` লিখলে পাইথন আমাদের ৪ ফলাফল দেবে। জোড়-বিজোড় নির্ণয় বা বিভিন্ন লজিকে এটি প্রচুর ব্যবহৃত হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি পূর্ণসংখ্যা ইনপুট নেওয়া।

Step 3: remainder = num1 % num2 সূত্র প্রয়োগ করা।

Step 4: ভাগশেষ প্রিন্ট করা।

Python Code

```
# Finding remainder using modulus operator
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

remainder = num1 % num2
print("The remainder is:", remainder)
```

Built-in Function / Shortcut Method

```
# Using divmod() built-in function which returns (quotient, remainder)
# q, r = divmod(num1, num2)
# print("The remainder is:", r)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 17 Enter second number: 5	The remainder is: 2

ব্যাখ্যা

১৭ কে ৫ দিয়ে ভাগ করলে ৩ বার যায় (৩ x ৫ = ১৫) এবং অবশিষ্ট থাকে ২। `%` অপারেটরটি সরাসরি সেই অবশিষ্ট বা ভাগশেষ ২ বের করে দিয়েছে।

Common Mistakes

- % চিহ্ন দেখে এটিকে শতকরা বা পারসেন্টেজ হিসাব মনে করা। প্রোগ্রামিংয়ে এটি শুধুমাত্র ভাগশেষ দেয়।
- দশমিক সংখ্যার ভাগশেষ বের করার চেষ্টা করা (সাধারণত পূর্ণসংখ্যার ভাগশেষ বের করা হয়)।

অনুশীলনের জন্য

1. একটি সংখ্যা ইনপুট নিয়ে তাকে ১০ দিয়ে ভাগ করলে ভাগশেষ কত হয় তা বের করুন (এটি সংখ্যার শেষ ডিজিট)।

2. ১০০ কে একটি ইনপুট করা সংখ্যা দিয়ে ভাগ করে ভাগশেষ বের করুন।

3. divmod() ফাংশন ব্যবহার করে ভাগফল ও ভাগশেষ একসাথে প্রদর্শন করুন।

সমস্যা 13: দুটি সংখ্যার গড় (Average) নির্ণয় করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে তাদের গাণিতিক গড় (Average) নির্ণয় করে স্ক্রিনে দেখাতে হবে।

কী কী জানা থাকতে হবে?

- গড়ের সূত্র: (যোগফল) / রাশিগুলোর সংখ্যা
- Operator Precedence (ব্র্যাকেটের ব্যবহার)

কীভাবে চিন্তা করব?

দুটি সংখ্যার গড় বের করতে হলে প্রথমে সংখ্যা দুটিকে যোগ করতে হবে, তারপর সেই যোগফলকে ২ দিয়ে ভাগ করতে হবে। এখানে ব্র্যাকেট দেওয়া অত্যন্ত গুরুত্বপূর্ণ। আমরা যদি `num1 + num2 / 2` লিখি, তবে পাইথনের নিয়মে ভাগের কাজ আগে হবে। তাই আমাদের অবশ্যই `(num1 + num2) / 2` লিখতে হবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে num1 ও num2 তে রাখা।

Step 3: average = (num1 + num2) / 2 সূত্র প্রয়োগ করা।

Step 4: গড় মান প্রিন্ট করা।

Python Code

```
# Finding average of two numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
```

```
average = (num1 + num2) / 2
print("The average is:", average)
```

Built-in Function / Shortcut Method

```
# Using statistics module
# import statistics
# print("Average:", statistics.mean([num1, num2]))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 20 Enter second number: 30	The average is: 25.0

ব্যাখ্যা

`(num1 + num2)` ব্র্যাকেটের ভেতর থাকায় প্রথমে যোগের কাজ হয়েছে (২০+৩০=৫০)। তারপর তাকে ২ দিয়ে ভাগ করে ২৫.০ গড় পাওয়া গেছে।

Common Mistakes

- যোগ করার সময় ব্র্যাকেট দিতে ভুলে যাওয়া (`num1 + num2 / 2`), যা ভুল উত্তর দেয়।
- ২ এর বদলে অন্য কোনো সংখ্যা দিয়ে ভাগ করা।

অনুশীলনের জন্য

1. বাংলা ও ইংরেজি পরীক্ষার নম্বর ইনপুট নিয়ে গড় নম্বর বের করুন।

2. দুটি পণ্যের মূল্যের গড় বের করুন।

3. বছরের প্রথম দুই মাসের বৃষ্টির পরিমাণের গড় বের করুন।

সমস্যা 14: তিনটি সংখ্যার গড় (Average) নির্ণয় করুন।

সমস্যা

তিনটি সংখ্যা ইনপুট নিয়ে তাদের যোগফলকে ৩ দিয়ে ভাগ করে গড় নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- গড় নির্ণয়ের সূত্র
- ইনপুট ও টাইপ কাস্টিং

কীভাবে চিন্তা করব?

তিনটি সংখ্যার গড়ের ক্ষেত্রে আমরা তিনটি ভেরিয়েবলে মান নেব, তাদের যোগফলকে একটি ব্র্যাকেটে রাখব এবং ৩ দিয়ে ভাগ করব। সূত্র: `(num1 + num2 + num3) / 3`।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: তিনটি সংখ্যা ইনপুট নিয়ে num1, num2 এবং num3 ভেরিয়েবলে রাখা।

Step 3: average = (num1 + num2 + num3) / 3 সূত্র প্রয়োগ করা।

Step 4: ফলাফল প্রদর্শন করা।

Python Code

```
# Finding average of three numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))

average = (num1 + num2 + num3) / 3
print("The average is:", average)
```

Built-in Function / Shortcut Method

```
# Shortcut using list and sum()
# nums = [float(input("Num 1: ")), float(input("Num 2: ")), float(input("Num 3: "))]
# print("Average:", sum(nums) / len(nums))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 10 Enter second number: 20 Enter third number: 30	The average is: 20.0

ব্যাখ্যা

তিনটি সংখ্যার যোগফল ৬০ কে ৩ দ্বারা ভাগ করে সঠিক গড় ২০.০ নির্ণয় করা হয়েছে।

Common Mistakes

- ৩টি সংখ্যা যোগ করার পর ৩ এর বদলে ২ দিয়ে ভাগ করে ফেলা।
- ব্র্যাকেট না দেওয়া।

অনুশীলনের জন্য

1. ৪টি বিষয়ের পরীক্ষার নম্বর ইনপুট নিয়ে গড় জিপিএ বা নম্বর বের করুন।

2. সপ্তাহের প্রথম ৩ দিনের তাপমাত্রার গড় নির্ণয় করুন।

3. ৫টি সংখ্যার গড় নির্ণয় করার প্রোগ্রাম লিখুন।

সমস্যা 15: একটি Temporary Variable ব্যবহার করে দুটি সংখ্যার মান অদলবদল (Swap) করুন।

সমস্যা

দুটি ভেরিয়েবলের মান একে অপরের সাথে অদলবদল (Swap) করতে হবে একটি তৃতীয় বা Temporary ভেরিয়েবলের সাহায্য নিয়ে। যেমন a=5 ও b=10 থাকলে অদলবদলের পর a=10 ও b=5 হবে।

কী কী জানা থাকতে হবে?

- Variable Assignment
- Swapping Logic

কীভাবে চিন্তা করব?

মনে করুন আপনার ডান হাতে চায়ের কাপ এবং বাম হাতে দুধের গ্লাস আছে। আপনি যদি কাপ দুটির পানীয় অদলবদল করতে চান, তবে সরাসরি করা যাবে না, একটি তৃতীয় ফাঁকা গ্লাস লাগবে। প্রথমে চায়ের কাপ থেকে চা ফাঁকা গ্লাসে ঢালবেন। তারপর দুধের গ্লাস থেকে দুধ চায়ের কাপে ঢালবেন। শেষে ফাঁকা গ্লাসে রাখা চা দুধের গ্লাসে ঢালবেন। প্রোগ্রামিংয়েও এই তৃতীয় গ্লাসটি হলো `temp` ভেরিয়েবল।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে num1 ও num2 তে রাখা।

Step 3: temp নামের একটি ভেরিয়েবলে num1 এর মান রাখা (`temp = num1`)।

Step 4: num1 ভেরিয়েবলে num2 এর মান রাখা (`num1 = num2`)।

Step 5: num2 ভেরিয়েবলে temp এর মান রাখা (`num2 = temp`)।

Step 6: অদলবদল করা মান দুটি প্রিন্ট করা।

Python Code

```
# Swapping two variables using a temporary variable
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print("Before swapping: num1 =", num1, "num2 =", num2)

# Using temp variable
```

```
temp = num1
num1 = num2
num2 = temp

print("After swapping: num1 =", num1, "num2 =", num2)
```

Built-in Function / Shortcut Method

```
# Pythonic direct swap shortcut (No temp needed)
# num1, num2 = num2, num1
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 15 Enter second number: 45	Before swapping: num1 = 15 num2 = 45 After swapping: num1 = 45 num2 = 15

ব্যাখ্যা

`temp = num1` লেখার ফলে num1 এর মানটি temp এ নিরাপদে সংরক্ষিত হয়েছে। এরপর num1 ও num2 এর মান পরিবর্তন করে সফলভাবে অদলবদল করা হয়েছে।

Common Mistakes

- temp ভেরিয়েবলে মান না রেখেই সরাসরি `num1 = num2` লিখে ফেলা, যার ফলে num1 এর আসল মান হারিয়ে যায়।
- অ্যাসাইনমেন্টের ক্রম ভুল করা।

অনুশীলনের জন্য

1. দুটি স্ট্রিং বা শব্দের মান একটি temp ভেরিয়েবল দিয়ে Swap করুন।

2. তিনটি ভেরিয়েবল a, b, c এর মান চক্রাকারে (a যাবে b তে, b যাবে c তে, c যাবে a তে) পরিবর্তন করুন।

3. দুটি বন্ধুর বয়সের মান ইনপুট নিয়ে অদলবদল করে প্রদর্শন করুন।

সমস্যা 16: কোনো Temporary Variable ব্যবহার না করে দুটি সংখ্যার মান অদলবদল (Swap) করুন।

সমস্যা

কোনো তৃতীয় বা Temporary ভেরিয়েবল ব্যবহার না করেই শুধুমাত্র গাণিতিক হিসাব বা পাইথনের বিশেষ নিয়মের মাধ্যমে দুটি ভেরিয়েবলের মান অদলবদল করতে হবে।

কী কী জানা থাকতে হবে?

- Python Tuple Unpacking
- Arithmetic Swapping Logic

কীভাবে চিন্তা করব?

অন্যান্য ভাষায় গাণিতিক যোগ-বিয়োগ দিয়ে (a = a+b; b = a-b; a = a-b) এটি করা হয়। কিন্তু পাইথনে একটি অত্যন্ত চমৎকার ও সহজ পদ্ধতি রয়েছে যাকে 'Tuple Unpacking' বলা হয়। আমরা এক লাইনেই `a, b = b, a` লিখে দুটি ভেরিয়েবলের মান কোনো তৃতীয় ভেরিয়েবল ছাড়াই অদলবদল করতে পারি।

> **[Best Practice]** পাইথনে সবসময় `a, b = b, a` পদ্ধতি ব্যবহার করে মান অদলবদল করা সবচেয়ে দ্রুত ও প্রফেশনাল।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে num1 ও num2 তে রাখা।

Step 3: `num1, num2 = num2, num1` নির্দেশ ব্যবহার করে মান অদলবদল করা।

Step 4: নতুন মানগুলো প্রিন্ট করা।

Python Code

```
# Swapping without temporary variable (Pythonic way)
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print("Before swap: num1 =", num1, "num2 =", num2)

# Swapping without temp
num1, num2 = num2, num1

print("After swap: num1 =", num1, "num2 =", num2)
```

Built-in Function / Shortcut Method

```
# Mathematical logic without temp:
# num1 = num1 + num2
# num2 = num1 - num2
# num1 = num1 - num2
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 100 Enter second number: 200	Before swap: num1 = 100 num2 = 200 After swap: num1 = 200 num2 = 100

ব্যাখ্যা

পাইথন ডানপাশের `num2, num1` এর মানগুলোকে একটি জোড় (Tuple) হিসেবে তৈরি করে এবং সাথে সাথে বামপাশের ভেরিয়েবলগুলোতে বসে যায়। এটি পাইথনের একটি অনন্য বৈশিষ্ট্য।

Common Mistakes

- কমা দেওয়ার সময় ভেরিয়েবলের নাম ভুল করা।
- গাণিতিক পদ্ধতিতে করার সময় যোগ-বিয়োগের ক্রমে ভুল করা।

অনুশীলনের জন্য

1. যোগ ও বিয়োগ অপারেটর ব্যবহার করে কোনো temp ভেরিয়েবল ছাড়াই দুটি সংখ্যা Swap করুন।

2. গুণ ও ভাগ অপারেটর ব্যবহার করে দুটি সংখ্যা Swap করুন।

3. একসাথে ৩টি ভেরিয়েবলের মান `x, y, z = z, x, y` পদ্ধতিতে অদলবদল করুন।

সমস্যা 17: একটি Integer মানকে Float-এ রূপান্তর (Type Casting) করে প্রদর্শন করুন।

সমস্যা

একটি পূর্ণসংখ্যা (Integer) ভেরিয়েবলকে দশমিক সংখ্যায় (Float) রূপান্তর করতে হবে এবং তার মান ও নতুন Data Type প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- float() ফাংশন
- type() ফাংশন

কীভাবে চিন্তা করব?

পূর্ণসংখ্যা (যেমন: 10) এর শেষে কোনো দশমিক বিন্দু থাকে না। আমরা যখন `float(10)` লিখব, তখন পাইথন সেটির শেষে একটি `.0` যুক্ত করে দশমিক সংখ্যা বানিয়ে দেবে (যেমন: 10.0)। ভেরিয়েবলের ডাটা টাইপ দেখার জন্য আমরা `type()` ফাংশন ব্যবহার করি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি পূর্ণসংখ্যা ইনপুট নেওয়া বা ভেরিয়েবলে সংরক্ষণ করা।

Step 3: float() ফাংশন ব্যবহার করে সেটিকে Float-এ রূপান্তর করা।

Step 4: রূপান্তরিত মান এবং তার type প্রিন্ট করা।

Python Code

```
# Converting Integer to Float
integer_num = 25
print("Original value:", integer_num, "| Type:", type(integer_num))

# Type casting to float
float_num = float(integer_num)
print("Converted value:", float_num, "| Type:", type(float_num))
```

Built-in Function / Shortcut Method

```
# Direct casting inside print
# print(float(25), type(float(25)))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original value: 25 | Type: <class 'int'> Converted value: 25.0 | Type: <class 'float'>

ব্যাখ্যা

`float(integer_num)` ফাংশনটি ২৫ কে ২৫.০ তে রূপান্তর করেছে। `type()` ফাংশনটি নিশ্চিত করেছে যে এটি এখন `<class 'float'>`।

Common Mistakes

- float শব্দের বানান ভুল করা।

- টাইপ কাস্টিং করার পর সেই মানটি নতুন ভেরিয়েবলে সংরক্ষণ না করা।

অনুশীলনের জন্য

1. ব্যবহারকারীর কাছ থেকে একটি পূর্ণসংখ্যা ইনপুট নিয়ে সেটিকে Float-এ রূপান্তর করুন।

2. ১০০ পূর্ণসংখ্যাটিকে float-এ রূপান্তর করে ৫ দিয়ে ভাগ করুন।

3. -৫০ (ঋণাত্মক পূর্ণসংখ্যা) কে float-এ রূপান্তর করুন।

সমস্যা 18: একটি Float মানকে Integer-এ রূপান্তর (Type Casting) করে প্রদর্শন করুন।

সমস্যা

একটি দশমিক সংখ্যা (Float) কে পূর্ণসংখ্যায় (Integer) রূপান্তর করতে হবে এবং তার মান প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- int() ফাংশন
- Type Casting

কীভাবে চিন্তা করব?

দশমিক সংখ্যাকে যখন `int()` ফাংশন দিয়ে পূর্ণসংখ্যায় রূপান্তর করা হয়, তখন পাইথন দশমিকের পরের অংশটুকু পুরোপুরি বাদ দিয়ে দেয় (Truncate করে)। যেমন 15.89 কে int করলে তা 15 হয়ে যায় (এটি রাউন্ড বা আসন্ন মান করে না, সরাসরি দশমিকের পর বাদ দেয়)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি দশমিক সংখ্যা ভেরিয়েবলে সংরক্ষণ করা।

Step 3: int() ফাংশন দিয়ে সেটিকে Integer-এ রূপান্তর করা।

Step 4: ফলাফল ও টাইপ প্রদর্শন করা।

Python Code

```
# Converting Float to Integer
float_num = 45.78
print("Original float:", float_num)

# Type casting to int
int_num = int(float_num)
print("Converted integer:", int_num)
print("New Type:", type(int_num))
```

Built-in Function / Shortcut Method

```
# Direct int casting
# print("Integer value:", int(45.78))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original float: 45.78 Converted integer: 45 New Type: <class 'int'>

ব্যাখ্যা

`int(45.78)` লেখার ফলে দশমিক অংশ `.78` বাদ গিয়ে শুধুমাত্র পূর্ণসংখ্যা `45` পাওয়া গেছে।

Common Mistakes

- মনে করা যে 45.78 কে int করলে 46 হবে (int রাউন্ড করে না, শুধু দশমিক অংশ ছেঁটে ফেলে)।
- রাউন্ড মান চাইলে int() এর বদলে round() ব্যবহার করতে হয়।

অনুশীলনের জন্য

1. 99.99 কে Integer-এ রূপান্তর করে ফলাফল দেখুন।

2. ব্যবহারকারীর কাছ থেকে একটি পণ্যের দশমিক দাম ইনপুট নিয়ে তার পূর্ণসংখ্যা দাম প্রদর্শন করুন।

3. একটি float ভেরিয়েবল ও একটি int ভেরিয়েবল যোগ করে ফলাফলকে int-এ রূপান্তর করুন।

সমস্যা 19: একটি সংখ্যাসূচক String-কে Integer-এ রূপান্তর (Type Casting) করুন।

সমস্যা

কোটেশনের ভেতর থাকা একটি সংখ্যাসূচক স্ট্রিং (যেমন "123") কে পূর্ণসংখ্যায় (Integer) রূপান্তর করে তার সাথে গাণিতিক যোগ বা গুণ করতে হবে।

কী কী জানা থাকতে হবে?

- int() ফাংশন
- String vs Integer

কীভাবে চিন্তা করব?

কোটেশনের ভেতর যেকোনো সংখ্যা থাকলে পাইথন তাকে টেক্সট বা String মনে করে (যেমন `str_num = '100'`)। এর সাথে ১০ যোগ করলে এরর হবে। তাই আমাদের `int(str_num)` ব্যবহার করে সেটিকে আসল সংখ্যায় রূপান্তর করে গাণিতিক কাজ করতে হবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি সংখ্যাসূচক স্ট্রিং ভেরিয়েবলে রাখা।

Step 3: int() ফাংশন দিয়ে স্ট্রিংকে পূর্ণসংখ্যায় রূপান্তর করা।

Step 4: রূপান্তরিত সংখ্যার সাথে একটি সংখ্যা যোগ করে প্রমাণ করা যে এটি এখন গাণিতিক সংখ্যা।

Step 5: ফলাফল প্রিন্ট করা।

Python Code

```
# Converting numeric string to integer
num_str = "250"
print("Original string:", num_str, type(num_str))

# Convert to integer
num_int = int(num_str)
result = num_int + 50

print("After conversion and adding 50:", result)
print("New Type:", type(num_int))
```

Built-in Function / Shortcut Method

```
# Direct conversion inside operation
# print(int("250") + 50)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original string: 250 <class 'str'> After conversion and adding 50: 300 New Type: <class 'int'>

ব্যাখ্যা

স্ট্রিং "250" কে `int()` দ্বারা রূপান্তরিত করার পরই তার সাথে 50 যোগ করে 300 পাওয়া সম্ভব হয়েছে।

Common Mistakes

- অক্ষরযুক্ত স্ট্রিং (যেমন '123a' বা 'hello') কে int() করার চেষ্টা করা, যা `ValueError` তৈরি করে।
- দশমিকযুক্ত স্ট্রিং (যেমন '12.5') কে সরাসরি `int()` করা (আগে `float()` করতে হয়)।

অনুশীলনের জন্য

1. দুটি সংখ্যাসূচক স্ট্রিং "50" এবং "40" কে int-এ রূপান্তর করে তাদের গুণফল বের করুন।

2. ইনপুট হিসেবে নেওয়া জন্মসাল স্ট্রিংকে int-এ রূপান্তর করে বয়স বের করুন।

3. "1000" স্ট্রিংটিকে float-এ রূপান্তর করুন।

সমস্যা 20: বিভিন্ন ধরনের Variable-এর Data Type প্রদর্শন করুন।

সমস্যা

পাইথনের বিভিন্ন মৌলিক ডাটা টাইপ যেমন Integer, Float, String এবং Boolean ভেরিয়েবল তৈরি করে তাদের Data Type স্ক্রিনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- type() ফাংশন
- Basic Data Types (int, float, str, bool)

কীভাবে চিন্তা করব?

পাইথনে ভেরিয়েবলের ডাটা টাইপ আগে থেকে ঘোষণা করতে হয় না (Dynamically Typed)। আমরা ভেরিয়েবলে যে ধরনের মান রাখি, পাইথন স্বয়ংক্রিয়ভাবে তার টাইপ নির্ধারণ করে। আমরা `type()` ফাংশন ব্যবহার করে প্রতিটি ভেরিয়েবলের ডাটা টাইপ জেনে নিতে পারি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি পূর্ণসংখ্যা ভেরিয়েবলে রাখা।

Step 3: একটি দশমিক সংখ্যা ভেরিয়েবলে রাখা।

Step 4: একটি টেক্সট বা স্ট্রিং ভেরিয়েবলে রাখা।

Step 5: একটি True বা False (Boolean) মান ভেরিয়েবলে রাখা।

Step 6: type() ফাংশন দিয়ে প্রতিটি ভেরিয়েবলের ডাটা টাইপ প্রদর্শন করা।

Python Code

```
# Checking data types of different variables
age = 22                # Integer
height = 5.8            # Float
name = "Abdur Rahman"   # String
is_student = True       # Boolean

print("Value:", age, "| Type:", type(age))
print("Value:", height, "| Type:", type(height))
print("Value:", name, "| Type:", type(name))
print("Value:", is_student, "| Type:", type(is_student))
```

Built-in Function / Shortcut Method

```
# Printing types directly
# print(type(10), type(10.5), type("hello"), type(True))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Value: 22 | Type: <class 'int'> Value: 5.8 | Type: <class 'float'> Value: Abdur Rahman | Type: <class 'str'> Value: True | Type: <class 'bool'>

ব্যাখ্যা

`type()` ফাংশনটি ভেরিয়েবলের ভেতরের মান যাচাই করে তার ক্লাস বা টাইপ যেমন int, float, str বা bool প্রদর্শন করেছে।

Common Mistakes

- Boolean মান লেখার সময় `True` বা `False` এর প্রথম অক্ষর ছোট হাতের লেখা (যেমন true বা false), যা পাইথনে ভুল।
- type() ফাংশনকে print() এর ভেতর না লিখে শুধু `type(age)` লিখলে স্ক্রিনে কিছু দেখা যাবে না।

অনুশীলনের জন্য

1. একটি List এবং একটি Tuple তৈরি করে তাদের Data Type প্রদর্শন করুন।

2. একটি ভেরিয়েবলে None মান রেখে তার Data Type পরীক্ষা করুন।

3. ব্যবহারকারীর কাছ থেকে input() নিয়ে সরাসরি তার Data Type পরীক্ষা করুন (দেখবেন সবসময় str আসবে)।

Topic: Operators

সমস্যা 21: Arithmetic Operator ব্যবহার করে দুটি সংখ্যার বিভিন্ন গাণিতিক ফলাফল নির্ণয় করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে পাইথনের সকল মৌলিক গাণিতিক অপারেটর (+, -, *, /, %, //, **) ব্যবহার করে তাদের ফলাফল একযোগে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Arithmetic Operators (+, -, *, /, %, //, **)

কীভাবে চিন্তা করব?

পাইথনে মোট ৭টি প্রধান গাণিতিক অপারেটর রয়েছে। আমরা দুটি সংখ্যা ইনপুট নিয়ে এই ৭টি অপারেটরের ফলাফল পরপর প্রদর্শন করব। যাতে একজন শিক্ষার্থী সবগুলোর কাজ একসাথে বুঝতে পারে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে a ও b তে রাখা।

Step 3: +, -, *, /, %, // এবং ** অপারেটর ব্যবহার করে আলাদা আলাদা হিসাব করা।

Step 4: সবগুলো ফলাফল স্ক্রিনে প্রিন্ট করা।

Python Code

```
# Demonstrating all Arithmetic Operators
a = int(input("Enter first number (a): "))
b = int(input("Enter second number (b): "))

print("Addition (a + b):", a + b)
print("Subtraction (a - b):", a - b)
print("Multiplication (a * b):", a * b)
print("Normal Division (a / b):", a / b)
print("Modulus / Remainder (a % b):", a % b)
print("Floor Division (a // b):", a // b)
print("Exponentiation / Power (a ** b):", a ** b)
```

Built-in Function / Shortcut Method

```
# None needed as this is a comprehensive demonstration
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter first number (a): 10
Enter second number (b): 3
```

```
Addition (a + b): 13
Subtraction (a - b): 7
Multiplication (a * b): 30
Normal Division (a / b):
3.3333333333333335
Modulus / Remainder (a % b): 1
Floor Division (a // b): 3
Exponentiation / Power (a ** b):
1000
```

ব্যাখ্যা

এখানে `a / b` দিয়েছে ৩.৩৩৩... কিন্তু `a // b` দশমিক অংশ বাদ দিয়ে পূর্ণসংখ্যা ৩ দিয়েছে। আর `a ** b` মানে হলো ১০ এর পাওয়ার ৩ অর্থাৎ ১০০০।

Common Mistakes

- দ্বিতীয় সংখ্যা হিসেবে শূন্য (0) ইনপুট দেওয়া (ভাগ এবং মডুলাসে এরর হবে)।
- `//` এবং `/` এর পার্থক্য বুঝতে না পারা।

অনুশীলনের জন্য

1. ১৫ এবং ৪ ইনপুট দিয়ে এই ৭টি অপারেটরের ফলাফল যাচাই করুন।

2. দুটি দশমিক সংখ্যার ওপর গাণিতিক অপারেটরগুলো প্রয়োগ করুন।

3. ৩টি সংখ্যার ওপর যোগ, গুণ এবং পাওয়ার একসাথে প্রয়োগ করুন।

সমস্যা 22: Assignment Operator ব্যবহার করে একটি Variable-এর মান পরিবর্তন করুন।

সমস্যা

অ্যাসাইনমেন্ট অপারেটর (=, +=, -=, *=, /=) ব্যবহার করে একটি ভেরিয়েবলের প্রাথমিক মান ধাপে ধাপে পরিবর্তন (Update) করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Assignment Operators (=, +=, -=, *=, /=)

কীভাবে চিন্তা করব?

সাধারণত আমরা `=` দিয়ে ভেরিয়েবলে মান রাখি। যদি আমরা কোনো ভেরিয়েবলের বর্তমান মানের সাথে ৫ যোগ করতে চাই, তবে `x = x + 5` লিখি। শর্টকাট অ্যাসাইনমেন্ট অপারেটর দিয়ে এটিকে `x += 5` লেখা হয়। একইভাবে `-=`, `*=` কাজ করে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি ভেরিয়েবলে প্রাথমিক মান ১০ রাখা।

Step 3: `+=` দিয়ে ৫ যোগ করে মান আপডেট ও প্রদর্শন করা।

Step 4: `-=` দিয়ে ৩ বিয়োগ করে মান আপডেট ও প্রদর্শন করা।

Step 5: `*=` দিয়ে ২ গুণ করে প্রদর্শন করা।

Python Code

```
# Demonstrating Assignment Operators
num = 10
print("Initial value:", num)

num += 5   # Same as: num = num + 5
print("After num += 5:", num)

num -= 3   # Same as: num = num - 3
print("After num -= 3:", num)

num *= 2   # Same as: num = num * 2
print("After num *= 2:", num)

num /= 4   # Same as: num = num / 4
print("After num /= 4:", num)
```

Built-in Function / Shortcut Method

```
# Short assignment is itself the shortcut method
```

Sample Input / Output Table

Sample Input	Sample Output
None	Initial value: 10 After num += 5: 15 After num -= 3: 12 After num *= 2: 24

```
After num /= 4: 6.0
```

ব্যাখ্যা

`num += 5` লেখার ফলে num এর আগের মান ১০ এর সাথে ৫ যোগ হয়ে নতুন মান ১৫ হয়েছে এবং সেটিই num এ সংরক্ষিত হয়েছে।

Common Mistakes

- `+=` লেখার সময় `=+` লিখে ফেলা (যেমন `num =+ 5`), যা যোগ না করে ভেরিয়েবলের মান সরাসরি ধনাত্মক ৫ করে দেয়।
- ভেরিয়েবল আগে থেকে ঘোষণা না করেই `+=` ব্যবহার করা।

অনুশীলনের জন্য

1. একটি ভেরিয়েবলের মান ১০০ নিয়ে সেটিকে `//=` এবং `%=` দিয়ে পরিবর্তন করুন।

2. একটি স্ট্রিং ভেরিয়েবলের সাথে `+=` ব্যবহার করে নতুন শব্দ যুক্ত করুন।

3. লুপে বা কাউন্টারে `count += 1` এর ব্যবহার অনুশীলন করুন।

সমস্যা 23: Comparison Operator ব্যবহার করে দুটি সংখ্যার তুলনা করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে Comparison বা Relational Operator (==, !=, >, <, >=, <=) ব্যবহার করে তাদের তুলনা করতে হবে এবং সত্য (True) বা মিথ্যা (False) ফলাফল দেখতে হবে।

কী কী জানা থাকতে হবে?

- Comparison Operators
- Boolean Output (True/False)

কীভাবে চিন্তা করব?

তুলনামূলক অপারেটর দুটি মানের মধ্যে তুলনা করে এবং ফলাফল হিসেবে সবসময় Boolean মান (True অথবা False) প্রদান করে। যেমন আমরা যদি চেক করি `10 > 5`, তবে পাইথন বলবে `True`। আর `10 == 5` দিলে বলবে `False`।

> **[Warning]** মনে রাখবেন `=` হলো অ্যাসাইনমেন্ট অপারেটর (মান রাখার জন্য), আর `==` হলো Comparison অপারেটর (তুলনা করার জন্য)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নিয়ে x ও y তে রাখা।

Step 3: `==`, `!=`, `>`, `<`, `>=`, `<=` অপারেটরগুলো দিয়ে তুলনা করা।

Step 4: প্রতিটি তুলনার Boolean ফলাফল স্ক্রিনে প্রিন্ট করা।

Python Code

```
# Demonstrating Comparison Operators
x = int(input("Enter first number (x): "))
y = int(input("Enter second number (y): "))

print("x is equal to y (x == y):", x == y)
print("x is not equal to y (x != y):", x != y)
print("x is greater than y (x > y):", x > y)
print("x is less than y (x < y):", x < y)
print("x is greater than or equal to y (x >= y):", x >= y)
print("x is less than or equal to y (x <= y):", x <= y)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number (x): 20 Enter second number (y): 15	x is equal to y (x == y): False x is not equal to y (x != y): True x is greater than y (x > y): True x is less than y (x < y): False x is greater than or equal to y (x >= y): True x is less than or equal to y (x <= y): False

ব্যাখ্যা

যেহেতু ২০ সংখ্যাটি ১৫ এর চেয়ে বড়, তাই `x > y` এবং `x != y` এর মান `True` এসেছে।

Common Mistakes

- সমান কিনা যাচাই করার জন্য ডাবল ইকুয়াল (`==`) এর জায়গায় সিঙ্গেল ইকুয়াল (`=`) ব্যবহার করা।
- `>=` লেখার বদলে `=>` লেখা, যা Syntax Error।

অনুশীলনের জন্য

1. দুটি স্ট্রিংয়ের মধ্যে `==` এবং `!=` ব্যবহার করে তুলনা করুন।

2. আপনার বয়স ১৮ এর সমান বা বেশি কিনা (`age >= 18`) তা যাচাই করুন।

3. দুটি সমান সংখ্যা (যেমন ৫০ ও ৫০) ইনপুট দিয়ে সব অপারেটরের আউটপুট দেখুন।

সমস্যা 24: Logical Operator ব্যবহার করে একাধিক শর্ত যাচাই করুন।

সমস্যা

Logical Operator (and, or, not) ব্যবহার করে একাধিক শর্ত বা Condition একসাথে যাচাই করতে হবে।

কী কী জানা থাকতে হবে?

- Logical Operators (and, or, not)

কীভাবে চিন্তা করব?

যখন আমাদের একাধিক শর্ত একসাথে পরীক্ষা করতে হয়, তখন লজিক্যাল অপারেটর লাগে।
১) `and`: সবগুলো শর্ত True হলে তবেই True হবে।
২) `or`: যেকোনো একটি শর্ত True হলেই True হবে।
৩) `not`: সত্যকে মিথ্যা এবং মিথ্যাকে সত্য বানায় (Opposite)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: তিনটি সংখ্যা ইনপুট নেওয়া (a, b, c)।

Step 3: `and` দিয়ে যাচাই করা a কি b এবং c উভয়ের চেয়ে বড় কিনা।

Step 4: `or` দিয়ে যাচাই করা a কি b অথবা c যেকোনো একটির চেয়ে বড় কিনা।

Step 5: `not` দিয়ে বিপরীত ফলাফল দেখা।

Python Code

```
# Demonstrating Logical Operators
age = int(input("Enter your age: "))
has_id = input("Do you have ID card? (yes/no): ") == "yes"

# AND operator: Both conditions must be True
can_enter = (age >= 18) and has_id
print("Can enter restricted area (age >= 18 AND has ID):", can_enter)

# OR operator: At least one condition must be True
discount = (age < 12) or (age > 60)
print("Eligible for special discount (age < 12 OR age > 60):", discount)
```

```
# NOT operator: Reverses the boolean result
print("Is NOT eligible for discount:", not discount)
```

Built-in Function / Shortcut Method

```
# Chained comparison shortcut in Python
# print("Age between 18 and 30:", 18 <= age <= 30)
```

Sample Input / Output Table

Sample Input	Sample Output

`Enter your age: 25`
`Do you have ID card? (yes/no):`
`yes`
	`Can enter restricted area (age >= 18 AND has ID): True`
	`Eligible for special discount (age < 12 OR age > 60): False`
	`Is NOT eligible for discount: True`

ব্যাখ্যা

ব্যবহারকারীর বয়স ২৫ যা ১৮ এর বেশি এবং তার ID আছে। উভয় শর্ত সত্য হওয়ায় `and` অপারেটরটি `True` ফলাফল দিয়েছে।

Common Mistakes

- অন্যান্য ভাষার মতো `&&` বা `||` লেখার চেষ্টা করা (পাইথনে সরাসরি `and`, `or` লিখতে হয়)।
- ছোট-বড় হাতের অক্ষরের ভুল করা (যেমন And, OR লেখা)।

অনুশীলনের জন্য

1. একটি সংখ্যা ১০ এর চেয়ে বড় এবং ৫০ এর চেয়ে ছোট কিনা তা `and` দিয়ে যাচাই করুন।

2. আজ ছুটির দিন অথবা আপনার শরীর খারাপ কিনা তা `or` দিয়ে যাচাই করুন।

3. একটি সংখ্যা শূন্য নয় কিনা তা `not` অপারেটর দিয়ে যাচাই করুন।

সমস্যা 25: Identity Operator (is, is not) ব্যবহার করে দুটি Variable একই Object কিনা পরীক্ষা করুন।

সমস্যা

Identity Operator (`is` এবং `is not`) ব্যবহার করে দুটি ভেরিয়েবল মেমোরিতে একই Object নির্দেশ করছে কিনা তা পরীক্ষা করতে হবে।

কী কী জানা থাকতে হবে?

- Identity Operators (is, is not)
- id() ফাংশন

কীভাবে চিন্তা করব?

`==` যাচাই করে দুটি ভেরিয়েবলের মান (Value) সমান কিনা। আর `is` যাচাই করে দুটি ভেরিয়েবল মেমোরিতে একই জায়গায় (Same Memory Address/Object) আছে কিনা। পাইথনে ছোট পূর্ণসংখ্যা এবং একই স্ট্রিং মেমোরি সাশ্রয়ের জন্য একই অবজেক্ট হিসেবে থাকে। তবে দুটি আলাদা List এর মান সমান হলেও তারা একই অবজেক্ট হয় না।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি List তৈরি করা যাদের ভেতরের মান একই।

Step 3: `==` দিয়ে তাদের মান তুলনা করা।

Step 4: `is` দিয়ে তারা একই অবজেক্ট কিনা তুলনা করা।

Step 5: `id()` ফাংশন দিয়ে তাদের মেমোরি অ্যাড্রেস প্রদর্শন করা।

Python Code

```
# Demonstrating Identity Operators (is, is not)
list1 = [10, 20, 30]
list2 = [10, 20, 30]
list3 = list1  # Pointing to the same object in memory

print("list1 == list2 (Same values?):", list1 == list2)
print("list1 is list2 (Same memory object?):", list1 is list2)
print("list1 is list3 (Same memory object?):", list1 is list3)

print("Memory ID of list1:", id(list1))
print("Memory ID of list2:", id(list2))
print("Memory ID of list3:", id(list3))
```

Built-in Function / Shortcut Method

```
# Checking with is not
# print("list1 is not list2:", list1 is not list2)
```

Sample Input / Output Table

Sample Input	Sample Output
None	list1 == list2 (Same values?): True

```
list1 is list2 (Same memory
object?): False
list1 is list3 (Same memory
object?): True
Memory ID of list1:
139804560123456
Memory ID of list2:
139804560124096
Memory ID of list3:
139804560123456
```

ব্যাখ্যা

list1 এবং list2 এর মান সমান হওয়ায় `==` True হয়েছে। কিন্তু তারা মেমোরিতে দুটি আলাদা লিস্ট হওয়ায় `list1 is list2` False হয়েছে। অন্যদিকে `list3 = list1` করার কারণে তারা একই অবজেক্ট, তাই `is` True হয়েছে।

Common Mistakes

- মান তুলনা করার জন্য `==` এর জায়গায় `is` ব্যবহার করা।
- মনে করা যে দুটি ভেরিয়েবলের মান এক হলেই তাদের মেমোরি আইডি এক হবে।

অনুশীলনের জন্য

1. দুটি পূর্ণসংখ্যা a = 50 এবং b = 50 তৈরি করে `is` অপারেটর দিয়ে পরীক্ষা করুন (দেখবেন True আসবে)।

2. দুটি আলাদা ডিকশনারি তৈরি করে `is` এবং `is not` যাচাই করুন।

3. None যাচাই করার সময় সর্বদা `x is None` ব্যবহার করুন।

সমস্যা 26: Membership Operator (in, not in) ব্যবহার করে কোনো Element Collection-এর মধ্যে আছে কিনা যাচাই করুন।

সমস্যা

Membership Operator (`in` এবং `not in`) ব্যবহার করে কোনো নির্দিষ্ট অক্ষর বা শব্দ একটি String, List বা Collection-এর মধ্যে উপস্থিত আছে কিনা তা যাচাই করতে হবে।

কী কী জানা থাকতে হবে?

- Membership Operators (in, not in)

কীভাবে চিন্তা করব?

কোনো তালিকার ভেতর কোনো উপাদান খুঁজার জন্য লুপ চালিয়ে খোঁজার দরকার নেই। পাইথনে `in` অপারেটর দিয়ে খুব সহজেই জানা যায় উপাদানটি আছে কিনা। থাকলে এটি `True` দেয়, না থাকলে `False` দেয়। আর `not in` ঠিক তার উল্টো কাজ করে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ফলের একটি List এবং একটি String তৈরি করা।

Step 3: `in` ব্যবহার করে লিস্টে একটি নির্দিষ্ট ফল আছে কিনা যাচাই করা।

Step 4: `not in` ব্যবহার করে স্ট্রিংয়ে কোনো অক্ষর নেই কিনা যাচাই করা।

Step 5: ফলাফল প্রদর্শন করা।

Python Code

```
# Demonstrating Membership Operators (in, not in)
fruits = ["apple", "banana", "mango", "orange"]
text = "Python Programming Practice"

print("Is 'mango' in fruits list?:", "mango" in fruits)
print("Is 'grape' in fruits list?:", "grape" in fruits)

print("Is 'Program' inside text?:", "Program" in text)
print("Is 'Java' NOT in text?:", "Java" not in text)
```

Built-in Function / Shortcut Method

```
# Direct conditional check
# if "apple" in fruits: print("Found Apple!")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Is 'mango' in fruits list?: True Is 'grape' in fruits list?: False Is 'Program' inside text?: True Is 'Java' NOT in text?: True

ব্যাখ্যা

যেহেতু 'mango' আমাদের fruits লিস্টের ভেতর আছে, তাই `"mango" in fruits` আমাদের `True` ফলাফল দিয়েছে।

Common Mistakes

- বানানে বা বড়-ছোট হাতের অক্ষরে ভুল করা (যেমন 'Mango' in fruits দিলে False আসবে কারণ লিস্টে 'mango' ছোট হাতের)।
- ডিকশনারির ক্ষেত্রে Value খোঁজার জন্য সরাসরি `in` ব্যবহার করা (ডিকশনারিতে সরাসরি `in` লিখলে তা শুধু Key খোঁজে)।

অনুশীলনের জন্য

1. ব্যবহারকারীর কাছ থেকে একটি অক্ষর ইনপুট নিয়ে সেটি Vowel লিস্টে (`['a','e','i','o','u']`) আছে কিনা `in` দিয়ে যাচাই করুন।

2. একটি বাক্যে 'Python' শব্দটি আছে কিনা তা পরীক্ষা করুন।

3. ১ থেকে ১০ এর একটি লিস্ট তৈরি করে ৭ সংখ্যাটি আছে কিনা যাচাই করুন।

সমস্যা 27: Bitwise Operator ব্যবহার করে দুটি সংখ্যার Bitwise Operation প্রদর্শন করুন।

সমস্যা

দুটি সংখ্যার ওপর পাইথনের Bitwise Operators (&, |, ^, ~, <<, >>) প্রয়োগ করে বাইনারি বিট লেভেলে গাণিতিক কাজ প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Binary Number System
- Bitwise Operators (&, |, ^, ~, <<, >>)

কীভাবে চিন্তা করব?

বিটওয়াইজ অপারেটরগুলো সংখ্যাকে সরাসরি বাইনারি (0 এবং 1) বিটে রূপান্তর করে কাজ করে।
১) `&` (Bitwise AND): উভয় বিট 1 হলে 1 হবে।
২) `|` (Bitwise OR): যেকোনো একটি বিট 1 হলেই 1 হবে।
৩) `^` (Bitwise XOR): দুটি বিট ভিন্ন হলে 1 হবে।
৪) `<<` (Left Shift): বিটগুলোকে বামে সরিয়ে দেয় (সংখ্যা দ্বিগুণ হয়)।
৫) `>>` (Right Shift): বিটগুলোকে ডানে সরিয়ে দেয় (সংখ্যা অর্ধেক হয়)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি ছোট পূর্ণসংখ্যা যেমন a = 10 (বাইনারি 1010) এবং b = 4 (বাইনারি 0100) নেওয়া।

Step 3: &, |, ^, << এবং >> অপারেটর প্রয়োগ করা।

Step 4: ফলাফল স্ক্রিনে প্রদর্শন করা।

Python Code

```
# Demonstrating Bitwise Operators
a = 10  # Binary: 1010
b = 4   # Binary: 0100

print("Bitwise AND (a & b):", a & b)   # 1010 & 0100 = 0000 (0)
print("Bitwise OR (a | b):", a | b)    # 1010 | 0100 = 1110 (14)
print("Bitwise XOR (a ^ b):", a ^ b)   # 1010 ^ 0100 = 1110 (14)
print("Bitwise NOT (~a):", ~a)         # -(10+1) = -11
print("Left Shift (a << 1):", a << 1)  # Shifts bits left by 1 (Doubles: 20)
print("Right Shift (a >> 1):", a >> 1) # Shifts bits right by 1 (Halves: 5)
```

Built-in Function / Shortcut Method

```
# Viewing binary representation using bin()
# print("Binary of 10:", bin(10))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Bitwise AND (a & b): 0 Bitwise OR (a | b): 14 Bitwise XOR (a ^ b): 14 Bitwise NOT (~a): -11 Left Shift (a << 1): 20 Right Shift (a >> 1): 5

ব্যাখ্যা

১০ এবং ৪ এর বাইনারি যথাক্রমে ১০১০ এবং ০১০০। বিটওয়াইজ AND করলে প্রতিটি বিট গুণ হয়, যার ফলে ০০০০ বা ০ পাওয়া গেছে। Left Shift একবার করলে মান দ্বিগুণ হয়ে ২০ হয়েছে।

Common Mistakes

- লজিক্যাল `and`, `or` এর জায়গায় বিটওয়াইজ `&`, `|` ব্যবহার করা।
- দশমিক সংখ্যা বা ফ্লোটের ওপর বিটওয়াইজ অপারেটর চালানোর চেষ্টা করা (এটি শুধুমাত্র পূর্ণসংখ্যায় কাজ করে)।

অনুশীলনের জন্য

1. ১২ এবং ৫ এর ওপর Bitwise AND এবং OR প্রয়োগ করুন।

2. একটি সংখ্যাকে Left Shift 2 (`x << 2`) করে দেখুন মান ৪ গুণ হয় কিনা।

3. `bin()` ফাংশন ব্যবহার করে কোনো সংখ্যার বাইনারি রূপ প্রিন্ট করুন।

সমস্যা 28: একটি সংখ্যার Power নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি ভিত্তি (Base) এবং একটি ঘাত (Exponent/Power) ইনপুট নিয়ে সূচকের মান বা Power নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Exponentiation Operator (**)
- pow() ফাংশন

কীভাবে চিন্তা করব?

কোনো সংখ্যার পাওয়ার বের করার অর্থ হলো সংখ্যাটিকে ততবার নিজের সাথে গুণ করা। যেমন ৫ এর পাওয়ার ৩ মানে হলো ৫ x ৫ x ৫ = ১২৫। পাইথনে এটি করার সবচেয়ে সহজ উপায় হলো ডাবল অ্যাস্টেরিস্ক (`**`) অপারেটর ব্যবহার করা।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ভিত্তি (base) ইনপুট নেওয়া।

Step 3: পাওয়ার (power) ইনপুট নেওয়া।

Step 4: result = base ** power সূত্র প্রয়োগ করা।

Step 5: ফলাফল প্রদর্শন করা।

Python Code

```
# Finding power of a number using ** operator
base = float(input("Enter base number: "))
exponent = float(input("Enter exponent / power: "))

result = base ** exponent
print("Result is:", result)
```

Built-in Function / Shortcut Method

```
# Using built-in pow() function or math module
# import math
# print("Result:", pow(base, exponent))
# print("Math pow:", math.pow(base, exponent))
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter base number: 5` `Enter exponent / power: 3`	`Result is: 125.0`

ব্যাখ্যা

`base ** exponent` অর্থাৎ `5 ** 3` হিসাব করে সঠিক ১২৫.০ ফলাফল বের করা হয়েছে।

Common Mistakes

- পাওয়ার দেওয়ার জন্য `^` চিহ্ন ব্যবহার করা (পাইথনে `^` হলো বিটওয়াইজ XOR, পাওয়ার নয়)।
- বড় সংখ্যার পাওয়ার বের করতে গিয়ে মেমোরি ওভারফ্লো চিন্তা করা (পাইথন অনেক বড় পূর্ণসংখ্যার হিসাব অনায়াসেই করতে পারে)।

অনুশীলনের জন্য

1. ২ এর পাওয়ার ১০ (2^{10}) নির্ণয় করুন।

2. কোনো সংখ্যার পাওয়ার 0.5 দিয়ে তার বর্গমূল (Square Root) বের করুন।

3. pow() ফাংশন ব্যবহার করে ৩টি আর্গুমেন্ট (`pow(base, exp, mod)`) এর কাজ যাচাই করুন।

সমস্যা 29: Modulus Operator ব্যবহার করে ভাগশেষ নির্ণয় করুন।

সমস্যা

দুটি পূর্ণসংখ্যা ইনপুট নিয়ে Modulus Operator (%) ব্যবহার করে তাদের ভাগশেষ প্রদর্শন করুন এবং একটি বাস্তব উদাহরণ দেখুন।

কী কী জানা থাকতে হবে?

- Modulus Operator (%)

কীভাবে চিন্তা করব?

ভাগশেষ বের করার জন্য `%` ব্যবহার করা হয়। এটি জোড়-বিজোড় নির্ণয়, সময় রূপান্তর (যেমন সেকেন্ড থেকে মিনিট) ইত্যাদিতে খুবই কার্যকর।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: মোট সেকেন্ড ইনপুট নেওয়া।

Step 3: `seconds % 60` দিয়ে অবশিষ্ট সেকেন্ড বের করা।

Step 4: `seconds // 60` দিয়ে মিনিট বের করা।

Step 5: মিনিট ও সেকেন্ড প্রদর্শন করা।

Python Code

```
# Practical use of modulus operator: Converting seconds to minutes and seconds
total_seconds = int(input("Enter total seconds: "))

minutes = total_seconds // 60
remaining_seconds = total_seconds % 60

print(total_seconds, "seconds is equal to:", minutes, "minutes and",
remaining_seconds, "seconds")
```

Built-in Function / Shortcut Method

```
# Using divmod
# m, s = divmod(total_seconds, 60)
# print(f"{m} minutes and {s} seconds")
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter total seconds: 130`	`130 seconds is equal to: 2 minutes and 10 seconds`

ব্যাখ্যা

১৩০ কে ৬০ দিয়ে ভাগ করলে ভাগশেষ থাকে ১০ (`130 % 60 = 10`), যা আমাদের অবশিষ্ট সেকেন্ড বের করে দিয়েছে।

Common Mistakes

- ভাগশেষ এবং ভাগফল গুলিয়ে ফেলা।
- ঋণাত্মক সংখ্যার মডুলাস করার সময় পাইথনের নিয়ম না জানা।

অনুশীলনের জন্য

1. মোট দিন সংখ্যা ইনপুট নিয়ে কত সপ্তাহ ও কত দিন হয় তা `%` এবং `//` দিয়ে বের করুন।

2. একটি তিন অঙ্কের সংখ্যা ইনপুট নিয়ে `% 10` ব্যবহার করে তার শেষ ডিজিট বের করুন।

3. একটি সংখ্যা ৫ দ্বারা নিঃশেষে বিভাজ্য কিনা তা `% 5 == 0` দিয়ে যাচাই করুন।

সমস্যা 30: Floor Division Operator ব্যবহার করে পূর্ণসংখ্যা ভাগফল নির্ণয় করুন।

সমস্যা

Floor Division Operator (`//`) ব্যবহার করে দুটি সংখ্যা ভাগ করার পর দশমিক অংশ বাদ দিয়ে নিচের পূর্ণসংখ্যাটি নির্ণয় করুন।

কী কী জানা থাকতে হবে?

- Floor Division Operator (//)
- Normal Division vs Floor Division

কীভাবে চিন্তা করব?

সাধারণ ভাগ (`/`) সবসময় দশমিক মান দেয়। কিন্তু যখন আমাদের শুধুমাত্র পূর্ণসংখ্যার ভাগফল দরকার হয় (যেমন ১৫ টি চকলেট ৪ জন শিশুর মাঝে সমানভাবে কয়টি করে দেওয়া যাবে), তখন আমরা ডাবল ফরোয়ার্ড স্লাশ (`//`) ব্যবহার করি। এটি দশমিক অংশ বাদ দিয়ে পূর্ণসংখ্যা প্রদান করে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: মোট চকলেট সংখ্যা এবং শিশুর সংখ্যা ইনপুট নেওয়া।

Step 3: `chocolates // children` ব্যবহার করে জনপ্রতি চকলেটের সংখ্যা বের করা।

Step 4: ফলাফল প্রদর্শন করা।

Python Code

```
# Finding integer quotient using Floor Division (//)
chocolates = int(input("Enter total chocolates: "))
children = int(input("Enter number of children: "))

per_child = chocolates // children
remaining = chocolates % children

print("Each child gets:", per_child, "chocolates")
print("Chocolates left:", remaining)
```

Built-in Function / Shortcut Method

```
# Comparing normal and floor division inside print
# print("Normal:", chocolates / children, "| Floor:", chocolates // children)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter total chocolates: 25	Each child gets: 6 chocolates

```
Enter number of children: 4        Chocolates left: 1
```

ব্যাখ্যা

২৫ কে ৪ দিয়ে ভাগ করলে ৬.২৫ হয়। কিন্তু `//` ব্যবহার করায় দশমিক অংশ বাদ গিয়ে প্রত্যেকে ৬টি করে চকলেট পেয়েছে।

Common Mistakes

- ঋণাত্মক সংখ্যার Floor Division বুঝতে ভুল করা (যেমন `-10 // 3` এর মান `-4` হয়, `-3` নয়, কারণ এটি নিচের দিকের পূর্ণসংখ্যায় যায়)।
- `//` লেখার সময় মাঝে স্পেস দেওয়া (`/ /`)।

অনুশীলনের জন্য

1. ১০০ টাকায় ৩৫ টাকা দামের কয়টি কলম কেনা যাবে তা `//` দিয়ে বের করুন।

2. মোট মাস সংখ্যা ইনপুট নিয়ে কত বছর ও কত মাস হয় তা বের করুন।

3. ঋণাত্মক সংখ্যা যেমন -১৫ কে ৪ দিয়ে Floor Division করে ফলাফল লক্ষ্য করুন।

Topic: Geometry & Formula Based

সমস্যা 31: একটি আয়তক্ষেত্রের দৈর্ঘ্য ও প্রস্থ ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি আয়তক্ষেত্রের (Rectangle) দৈর্ঘ্য ও প্রস্থ ইনপুট নিয়ে তার ক্ষেত্রফল (Area) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- আয়তক্ষেত্রের ক্ষেত্রফলের সূত্র: দৈর্ঘ্য x প্রস্থ
- float() Type Casting

কীভাবে চিন্তা করব?

যেকোনো জ্যামিতিক সমস্যার ক্ষেত্রে প্রথমেই আমাদের সঠিক সূত্রটি জানা দরকার। আয়তক্ষেত্রের ক্ষেত্রফল বের করার সূত্র হলো: `Area = Length x Width`। আমরা দৈর্ঘ্য ও প্রস্থ ইনপুট নিয়ে তাদের গুণ করব।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দৈর্ঘ্য (length) ইনপুট নেওয়া।

Step 3: প্রস্থ (width) ইনপুট নেওয়া।

Step 4: area = length * width সূত্র প্রয়োগ করা।

Step 5: ক্ষেত্রফল প্রদর্শন করা।

Python Code

```
# Finding area of a rectangle
length = float(input("Enter length of rectangle: "))
width = float(input("Enter width of rectangle: "))

area = length * width
print("The area of rectangle is:", area)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Area:", float(input("Length: ")) * float(input("Width: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter length of rectangle: 12.5 Enter width of rectangle: 8	The area of rectangle is: 100.0

ব্যাখ্যা

দৈর্ঘ্য ১২.৫ এবং প্রস্থ ৮ কে গুণ করে ক্ষেত্রফল ১০০.০ বর্গ একক নির্ণয় করা হয়েছে।

Common Mistakes

- দৈর্ঘ্য ও প্রস্থ যোগ করে ফেলা।
- ইনপুট নেওয়ার সময় int() ব্যবহার করা যা দশমিক দৈর্ঘ্য গ্রহণ করতে পারে না।

অনুশীলনের জন্য

1. একটি ঘরের দৈর্ঘ্য ও প্রস্থ মিটারে ইনপুট নিয়ে মেঝের ক্ষেত্রফল বের করুন।

2. একটি জমির দৈর্ঘ্য ও প্রস্থ ফুটে ইনপুট নিয়ে বর্গফুটে ক্ষেত্রফল বের করুন।

3. ক্ষেত্রফল এবং দৈর্ঘ্য ইনপুট নিয়ে প্রস্থ বের করার প্রোগ্রাম লিখুন।

সমস্যা 32: একটি আয়তক্ষেত্রের দৈর্ঘ্য ও প্রস্থ ইনপুট নিয়ে পরিসীমা নির্ণয় করুন।

সমস্যা

একটি আয়তক্ষেত্রের দৈর্ঘ্য ও প্রস্থ ইনপুট নিয়ে তার পরিসীমা (Perimeter) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- পরিসীমার সূত্র: 2 x (দৈর্ঘ্য + প্রস্থ)
- Operator Precedence

কীভাবে চিন্তা করব?

পরিসীমা হলো কোনো ক্ষেত্রের চারিদিকের মোট সীমানার দৈর্ঘ্য। আয়তক্ষেত্রের বিপরীত বাহুগুলো সমান হওয়ায় এর পরিসীমার সূত্র হলো `Perimeter = 2 * (length + width)`। এখানে ব্র্যাকেট দেওয়া বাধ্যতামূলক।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দৈর্ঘ্য ও প্রস্থ ইনপুট নেওয়া।

Step 3: perimeter = 2 * (length + width) সূত্র প্রয়োগ করা।

Step 4: পরিসীমা প্রিন্ট করা।

Python Code

```
# Finding perimeter of a rectangle
length = float(input("Enter length: "))
width = float(input("Enter width: "))

perimeter = 2 * (length + width)
print("The perimeter of rectangle is:", perimeter)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Perimeter:", 2 * (float(input("L: ")) + float(input("W: "))))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter length: 15 Enter width: 10	The perimeter of rectangle is: 50.0

ব্যাখ্যা

প্রথমে ব্র্যাকেটের ভেতর দৈর্ঘ্য ও প্রস্থ যোগ করা হয়েছে (১৫+১০=২৫)। তারপর ২ দিয়ে গুণ করে ৫০.০ পরিসীমা পাওয়া গেছে।

Common Mistakes

- ব্র্যাকেট না দেওয়া (`2 * length + width`), যার ফলে শুধু দৈর্ঘ্য ২ দিয়ে গুণ হয়।

- ২ গুণের জায়গায় গুণ চিহ্ন (*) দিতে ভুলে যাওয়া (`2(length + width)`), যা পাইথনে Syntax Error।

অনুশীলনের জন্য

1. একটি মাঠের চারিদিকে বেড়া দিতে কত মিটার তার লাগবে তা পরিসীমা দিয়ে বের করুন।

2. একটি বর্গক্ষেত্রের এক বাহুর দৈর্ঘ্য ইনপুট নিয়ে তার পরিসীমা (`4 * side`) বের করুন।

3. একটি ত্রিভুজের ৩ বাহুর দৈর্ঘ্য ইনপুট নিয়ে তার পরিসীমা (`a + b + c`) বের করুন।

সমস্যা 33: একটি বর্গক্ষেত্রের বাহুর দৈর্ঘ্য ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করুন।

সমস্যা

একটি বর্গক্ষেত্রের (Square) এক বাহুর দৈর্ঘ্য ইনপুট নিয়ে তার ক্ষেত্রফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- বর্গক্ষেত্রের ক্ষেত্রফল = বাহু x বাহু বা $(\text{বাহু})^2$

কীভাবে চিন্তা করব?

বর্গক্ষেত্রের চারটি বাহুই সমান। তাই এর ক্ষেত্রফল বের করার সূত্র হলো এক বাহুর দৈর্ঘ্যের বর্গ বা স্কয়ার। আমরা `side ** 2` অথবা `side * side` লিখে এটি বের করতে পারি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: বর্গক্ষেত্রের এক বাহুর দৈর্ঘ্য (side) ইনপুট নেওয়া।

Step 3: area = side ** 2 সূত্র প্রয়োগ করা।

Step 4: ক্ষেত্রফল প্রিন্ট করা।

Python Code

```
# Finding area of a square
side = float(input("Enter side length of square: "))

area = side ** 2
print("The area of square is:", area)
```

Built-in Function / Shortcut Method

```
# Using multiplication
# print("Area:", side * side)
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter side length of square: 9`	`The area of square is: 81.0`

ব্যাখ্যা

৯ এর বর্গ বা ৯ x ৯ হিসাব করে ক্ষেত্রফল ৮১.০ বের করা হয়েছে।

Common Mistakes

- বাহুর দৈর্ঘ্যকে ৪ দিয়ে গুণ করে ফেলা (সেটি পরিসীমা, ক্ষেত্রফল নয়)।
- পাওয়ার দেওয়ার জন্য `^` চিহ্ন দেওয়া।

অনুশীলনের জন্য

1. বর্গক্ষেত্রের ক্ষেত্রফল ইনপুট নিয়ে তার এক বাহুর দৈর্ঘ্য (`area ** 0.5`) বের করুন।

2. একটি বর্গাকার বাগানের ক্ষেত্রফল বের করুন যার এক বাহুর দৈর্ঘ্য ২৫ মিটার।

3. দুটি বর্গক্ষেত্রের বাহু ইনপুট নিয়ে তাদের ক্ষেত্রফলের পার্থক্য বের করুন।

সমস্যা 34: একটি বৃত্তের ব্যাসার্ধ ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে বৃত্তের ব্যাসার্ধ (Radius) ইনপুট নিয়ে তার ক্ষেত্রফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- বৃত্তের ক্ষেত্রফল = πr^2
- math.pi বা ৩.১৪১৬ এর মান

কীভাবে চিন্তা করব?

বৃত্তের ক্ষেত্রফলের সূত্র হলো `Area = π * r * r` বা `π * r²`। এখানে π (পাই) একটি ধ্রুবক যার মান আনুমানিক 3.1416। আমরা পাইথনের `math` মডিউল থেকে `math.pi` ব্যবহার করতে পারি অথবা সরাসরি 3.1416 বসাতে পারি।

অ্যালগরিদম

Step 1: math মডিউল ইমপোর্ট করা।

Step 2: বৃত্তের ব্যাসার্ধ (radius) ইনপুট নেওয়া।

Step 3: area = math.pi * (radius ** 2) সূত্র প্রয়োগ করা।

Step 4: round() ফাংশন দিয়ে ২ ঘর দশমিক পর্যন্ত ক্ষেত্রফল প্রিন্ট করা।

Python Code

```
# Finding area of a circle
import math

radius = float(input("Enter radius of circle: "))

area = math.pi * (radius ** 2)
print("The area of circle is:", round(area, 2))
```

Built-in Function / Shortcut Method

```
# Using fixed pi value 3.1416
# print("Area:", 3.1416 * float(input("Radius: ")) ** 2)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter radius of circle: 7	The area of circle is: 153.94

ব্যাখ্যা

`math.pi` এর মান ৩.১৪১৫৯... এর সাথে ব্যাসার্ধ ৭ এর বর্গ (৪৯) গুণ করে ১৫৩.৯৪ বর্গ একক ক্ষেত্রফল পাওয়া গেছে। `round(area, 2)` দিয়ে দশমিকের পর ২ ঘর রাখা হয়েছে।

Common Mistakes

- ব্যাসার্ধের বদলে ব্যাস (Diameter) ইনপুট নিলে তাকে ২ দিয়ে ভাগ না করে সরাসরি সূত্রে বসানো।
- `math` মডিউল ইমপোর্ট না করেই `math.pi` লেখা।

অনুশীলনের জন্য

1. বৃত্তের ব্যাস (Diameter) ইনপুট নিয়ে ক্ষেত্রফল বের করুন (`r = d / 2`)।

2. অর্ধবৃত্তের (Semicircle) ক্ষেত্রফল নির্ণয় করুন।

3. বৃত্তের ক্ষেত্রফল ইনপুট নিয়ে ব্যাসার্ধ বের করুন।

সমস্যা 35: একটি বৃত্তের ব্যাসার্ধ ইনপুট নিয়ে পরিধি নির্ণয় করুন।

সমস্যা

একটি বৃত্তের ব্যাসার্ধ ইনপুট নিয়ে তার পরিধি (Circumference) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- বৃত্তের পরিধি = 2 * π * r

কীভাবে চিন্তা করব?

বৃত্তের পরিধি হলো বৃত্তের বাইরের সীমানার সম্পূর্ণ দৈর্ঘ্য। এর সূত্র হলো `Circumference = 2 * π * r`।

অ্যালগরিদম

Step 1: math মডিউল ইমপোর্ট করা।

Step 2: ব্যাসার্ধ ইনপুট নেওয়া।

Step 3: circumference = 2 * math.pi * radius সূত্র প্রয়োগ করা।

Step 4: পরিধি প্রিন্ট করা।

Python Code

```
# Finding circumference of a circle
import math

radius = float(input("Enter radius of circle: "))

circumference = 2 * math.pi * radius
print("The circumference of circle is:", round(circumference, 2))
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Circumference:", 2 * 3.1416 * float(input("Radius: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter radius of circle: 10	The circumference of circle is: 62.83

ব্যাখ্যা

২ x ৩.১৪১৬ x ১০ হিসাব করে পরিধি ৬২.৮৩ নির্ণয় করা হয়েছে।

Common Mistakes

- ক্ষেত্রফল এবং পরিধির সূত্র গুলিয়ে ফেলা (`πr²` এর বদলে `2πr`)।
- গুণের চিহ্ন না দেওয়া।

অনুশীলনের জন্য

1. একটি চাকা একবার ঘুরলে কতটুকু দূরত্ব অতিক্রম করে তা পরিধি দিয়ে বের করুন।

2. বৃত্তের পরিধি ইনপুট নিয়ে ব্যাসার্ধ বের করুন।

3. একটি বৃত্তাকার মাঠের চারিদিকে ৫ বার দৌড়ালে মোট কত দূরত্ব হয় তা বের করুন।

সমস্যা 36: একটি ত্রিভুজের ভূমি ও উচ্চতা ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করুন।

সমস্যা

ত্রিভুজের ভূমি (Base) এবং উচ্চতা (Height) ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- ত্রিভুজের ক্ষেত্রফল = 0.5 x ভূমি x উচ্চতা

কীভাবে চিন্তা করব?

যেকোনো ত্রিভুজের ভূমি ও লম্ব উচ্চতা জানা থাকলে ক্ষেত্রফল বের করার সাধারণ সূত্র হলো: `Area = 0.5 * base * height` অথবা `(base * height) / 2`।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ভূমি (base) ইনপুট নেওয়া।

Step 3: উচ্চতা (height) ইনপুট নেওয়া।

Step 4: area = 0.5 * base * height সূত্র প্রয়োগ করা।

Step 5: ক্ষেত্রফল প্রদর্শন করা।

Python Code

```
# Finding area of a triangle given base and height
base = float(input("Enter base of triangle: "))
height = float(input("Enter height of triangle: "))

area = 0.5 * base * height
print("The area of triangle is:", area)
```

Built-in Function / Shortcut Method

```
# Using division by 2
# print("Area:", (base * height) / 2)
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter base of triangle: 14` `Enter height of triangle: 8`	`The area of triangle is: 56.0`

ব্যাখ্যা

ভূমি ১৪ এবং উচ্চতা ৮ গুণ করলে ১১২ হয়। তাকে ০.৫ দিয়ে গুণ করে বা ২ দিয়ে ভাগ করে ৫৬.০ ক্ষেত্রফল পাওয়া গেছে।

Common Mistakes

- ০.৫ দিয়ে গুণ করতে বা ২ দিয়ে ভাগ করতে ভুলে যাওয়া।
- উচ্চতা বলতে হেলানো বাহু মনে করা (এটি সর্বদা লম্ব উচ্চতা হতে হবে)।

অনুশীলনের জন্য

1. সমবাহু ত্রিভুজের এক বাহু ইনপুট নিয়ে ক্ষেত্রফল (`0.433 * side * side`) বের করুন।

2. ত্রিভুজের ক্ষেত্রফল ও ভূমি ইনপুট নিয়ে উচ্চতা বের করুন।

3. হেরনের সূত্র (`√(s(s-a)(s-b)(s-c))`) ব্যবহার করে ৩ বাহু দেওয়া থাকলে ক্ষেত্রফল বের করুন।

সমস্যা 37: একটি সমকোণী ত্রিভুজের ভূমি ও উচ্চতা ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করুন।

সমস্যা

সমকোণী ত্রিভুজের (Right-angled Triangle) ভূমি ও লম্বের দৈর্ঘ্য ইনপুট নিয়ে ক্ষেত্রফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- সমকোণী ত্রিভুজের ক্ষেত্রফল = 0.5 x ভূমি x লম্ব

কীভাবে চিন্তা করব?

সমকোণী ত্রিভুজের ক্ষেত্রে সমকোণ সংলগ্ন দুটি বাহুর একটি ভূমি এবং অপরটি লম্ব (উচ্চতা) হয়। তাই সাধারণ ত্রিভুজের ক্ষেত্রফলের সূত্রটিই এখানে সরাসরি প্রযোজ্য।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ভূমি ইনপুট নেওয়া।

Step 3: লম্ব বা উচ্চতা ইনপুট নেওয়া।

Step 4: area = (base * perpendicular) / 2 সূত্র প্রয়োগ করা।

Step 5: ফলাফল প্রদর্শন করা।

Python Code

```
# Finding area of a right-angled triangle
base = float(input("Enter base: "))
perpendicular = float(input("Enter perpendicular height: "))

area = (base * perpendicular) / 2
print("The area of right-angled triangle is:", area)
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Area:", 0.5 * float(input("Base: ")) * float(input("Height: ")))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter base: 10 Enter perpendicular height: 15	The area of right-angled triangle is: 75.0

ব্যাখ্যা

১০ এবং ১৫ এর গুণফল ১৫০ কে ২ দ্বারা ভাগ করে ৭৫.০ ক্ষেত্রফল নির্ণয় করা হয়েছে।

Common Mistakes

- অতিভুজ (Hypotenuse) কে উচ্চতা হিসেবে ইনপুট দেওয়া।
- ২ দিয়ে ভাগ না করা।

অনুশীলনের জন্য

1. পীথাগোরাসের সূত্র (`c = √(a² + b²)`) ব্যবহার করে ভূমি ও লম্ব দেওয়া থাকলে অতিভুজ বের করুন।

2. সমদ্বিবাহু সমকোণী ত্রিভুজের এক বাহু দেওয়া থাকলে ক্ষেত্রফল বের করুন।

3. ভূমি ও অতিভুজ দেওয়া থাকলে লম্ব বের করে তারপর ক্ষেত্রফল বের করুন।

সমস্যা 38: একটি Cylinder-এর ব্যাসার্ধ ও উচ্চতা ইনপুট নিয়ে আয়তন নির্ণয় করুন।

সমস্যা

একটি সিলিন্ডার বা বেলনের ভূমির ব্যাসার্ধ এবং উচ্চতা ইনপুট নিয়ে তার আয়তন (Volume) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- সিলিন্ডারের আয়তন = π x r^2 x h

কীভাবে চিন্তা করব?

সিলিন্ডারের ভূমি একটি বৃত্ত। তাই বৃত্তের ক্ষেত্রফল (`π $r^2$`) কে সিলিন্ডারের উচ্চতা (`h`) দিয়ে গুণ করলেই মোট আয়তন বা ভলিউম পাওয়া যায়। সূত্র: `Volume = π * $r^2$ * h`।

অ্যালগরিদম

Step 1: math মডিউল ইমপোর্ট করা।

Step 2: ব্যাসার্ধ (r) এবং উচ্চতা (h) ইনপুট নেওয়া।

Step 3: volume = math.pi * (r ** 2) * h সূত্র প্রয়োগ করা।

Step 4: আয়তন প্রিন্ট করা।

Python Code

```
# Finding volume of a cylinder
import math

radius = float(input("Enter radius of cylinder: "))
height = float(input("Enter height of cylinder: "))

volume = math.pi * (radius ** 2) * height
print("The volume of cylinder is:", round(volume, 2))
```

Built-in Function / Shortcut Method

```
# Using fixed pi
# print("Volume:", 3.1416 * (radius ** 2) * height)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter radius of cylinder: 5 Enter height of cylinder: 12	The volume of cylinder is: 942.48

ব্যাখ্যা

৩.১৪১৬ x ৫^2 x ১২ হিসাব করে সিলিন্ডারের আয়তন ৯৪২.৪৮ ঘন একক পাওয়া গেছে।

Common Mistakes

- উচ্চতার বর্গ করে ফেলা (শুধুমাত্র ব্যাসার্ধের বর্গ হবে)।

- ক্ষেত্রফল আর আয়তনের পার্থক্য বুঝতে না পারা।

অনুশীলনের জন্য

1. একটি শঙ্কু বা কোণের (Cone) আয়তন (`(1/3) * π * r² * h`) বের করুন।

2. সিলিন্ডারের বক্রতলের ক্ষেত্রফল (`2 * π * r * h`) বের করুন।

3. একটি গোলকের (Sphere) আয়তন (`(4/3) * π * r³`) বের করুন।

সমস্যা 39: একটি Cube-এর বাহুর দৈর্ঘ্য ইনপুট নিয়ে আয়তন নির্ণয় করুন।

সমস্যা

একটি ঘনক বা Cube-এর এক বাহুর দৈর্ঘ্য ইনপুট নিয়ে তার মোট আয়তন নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- ঘনকের আয়তন = বাহু³ বা side ** 3

কীভাবে চিন্তা করব?

ঘনক বা কিউবের (যেমন রুবিকস কিউব বা লুডুর ছক্কা) দৈর্ঘ্য, প্রস্থ এবং উচ্চতা সবই সমান। তাই এর আয়তন বের করার সূত্র হলো এক বাহুর দৈর্ঘ্যের কিউব বা ত্রিঘাত: `Volume = side ** 3`।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ঘনকের এক বাহুর দৈর্ঘ্য ইনপুট নেওয়া।

Step 3: volume = side ** 3 সূত্র প্রয়োগ করা।

Step 4: আয়তন প্রিন্ট করা।

Python Code

```
# Finding volume of a cube
side = float(input("Enter side length of cube: "))

volume = side ** 3
print("The volume of cube is:", volume)
```

Built-in Function / Shortcut Method

```
# Using multiplication
# print("Volume:", side * side * side)
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter side length of cube: 4`	`The volume of cube is: 64.0`

ব্যাখ্যা

৪ x ৪ x ৪ অর্থাৎ ৪ এর কিউব হিসাব করে ৬৪.০ ঘন একক পাওয়া গেছে।

Common Mistakes

- বাহুকে ৩ দিয়ে গুণ করে ফেলা (`side * 3`), যা সম্পূর্ণ ভুল।
- বর্গক্ষেত্র এবং ঘনকের সূত্র গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. একটি আয়তাকার ঘনবস্তুর (Cuboid) দৈর্ঘ্য, প্রস্থ ও উচ্চতা ইনপুট নিয়ে আয়তন (`l * w * h`) বের করুন।

2. ঘনকের আয়তন ইনপুট নিয়ে তার এক বাহুর দৈর্ঘ্য (`volume ** (1/3)`) বের করুন।

3. ঘনকের সমগ্র তলের ক্ষেত্রফল (`6 * side$^2$`) বের করুন।

সমস্যা 40: Fahrenheit তাপমাত্রাকে Celsius-এ রূপান্তর করুন।

সমস্যা

ফারেনহাইট স্কেলের তাপমাত্রা ইনপুট নিয়ে তাকে সেলসিয়াস স্কেলে রূপান্তর করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- তাপমাত্রা রূপান্তরের সূত্র: C = (F - 32) x 5 / 9

কীভাবে চিন্তা করব?

ফারেনহাইট থেকে সেলসিয়াসে নেওয়ার সূত্র হলো: `C = ((F - 32) * 5) / 9`। এখানে প্রথমে ফারেনহাইট মান থেকে ৩২ বিয়োগ করতে হবে, তারপর ৫ দিয়ে গুণ করে ৯ দিয়ে ভাগ করতে হবে। ব্র্যাকেট সঠিকভাবে দেওয়া জরুরি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ফারেনহাইট তাপমাত্রা (F) ইনপুট নেওয়া।

Step 3: celsius = ((F - 32) * 5) / 9 সূত্র প্রয়োগ করা।

Step 4: রূপান্তরিত সেলসিয়াস মান প্রিন্ট করা।

Python Code

```
# Converting Fahrenheit to Celsius
fahrenheit = float(input("Enter temperature in Fahrenheit: "))

celsius = ((fahrenheit - 32) * 5) / 9
print("Temperature in Celsius is:", round(celsius, 2))
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Celsius:", round(((float(input("Fahrenheit: ")) - 32) * 5) / 9, 2))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter temperature in Fahrenheit: 98.6	Temperature in Celsius is: 37.0

ব্যাখ্যা

মানুষের শরীরের স্বাভাবিক তাপমাত্রা ৯৮.৬ ডিগ্রি ফারেনহাইটকে সূত্রে বসিয়ে সঠিক ৩৭.০ ডিগ্রি সেলসিয়াস পাওয়া গেছে।

Common Mistakes

- ব্র্যাকেট না দিয়ে `F - 32 * 5 / 9` লেখা, যার ফলে গুণের কাজ আগে হয়ে ভুল উত্তর দেয়।
- ৫/৯ এর জায়গায় ৯/৫ দিয়ে ফেলা।

অনুশীলনের জন্য

1. ১০০ ডিগ্রি ফারেনহাইট (জ্বরের তাপমাত্রা) সেলসিয়াসে কত হয় বের করুন।

2. ফারেনহাইট থেকে কেলভিন স্কেলে তাপমাত্রা রূপান্তর করুন।

3. কোন তাপমাত্রায় সেলসিয়াস ও ফারেনহাইট স্কেল সমান মান দেয় (-৪০) তা কোড দিয়ে যাচাই করুন।

সমস্যা 41: Celsius তাপমাত্রাকে Fahrenheit-এ রূপান্তর করুন।

সমস্যা

সেলসিয়াস স্কেলের তাপমাত্রা ইনপুট নিয়ে তাকে ফারেনহাইট স্কেলে রূপান্তর করতে হবে।

কী কী জানা থাকতে হবে?

- তাপমাত্রা রূপান্তরের সূত্র: F = (C x 9 / 5) + 32

কীভাবে চিন্তা করব?

সেলসিয়াস থেকে ফারেনহাইটে নেওয়ার সূত্র হলো: `F = (C * 9 / 5) + 32` বা `(C * 1.8) + 32`। প্রথমে সেলসিয়াস মানকে ১.৮ দিয়ে গুণ করতে হবে, তারপর তার সাথে ৩২ যোগ করতে হবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: সেলসিয়াস তাপমাত্রা ইনপুট নেওয়া।

Step 3: fahrenheit = (celsius * 9 / 5) + 32 সূত্র প্রয়োগ করা।

Step 4: ফারেনহাইট মান প্রদর্শন করা।

Python Code

```
# Converting Celsius to Fahrenheit
celsius = float(input("Enter temperature in Celsius: "))

fahrenheit = (celsius * 9 / 5) + 32
print("Temperature in Fahrenheit is:", round(fahrenheit, 2))
```

Built-in Function / Shortcut Method

```
# Using decimal multiplier 1.8
# print("Fahrenheit:", (celsius * 1.8) + 32)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter temperature in Celsius: 25	Temperature in Fahrenheit is: 77.0

ব্যাখ্যা

ঘরের সাধারণ তাপমাত্রা ২৫ ডিগ্রি সেলসিয়াসকে ১.৮ দিয়ে গুণ করে (৪৫) তার সাথে ৩২ যোগ করে ৭৭.০ ফারেনহাইট বের করা হয়েছে।

Common Mistakes

- ৩২ যোগ করার বদলে বিয়োগ করা।
- ৯/৫ এর জায়গায় ৫/৯ লেখা।

অনুশীলনের জন্য

1. পানির স্ফুটনাঙ্ক ১০০ ডিগ্রি সেলসিয়াসকে ফারেনহাইটে রূপান্তর করুন (দেখবেন ২১২ আসবে)।

2. সেলসিয়াস থেকে কেলভিন (`C + 273.15`) স্কেলে রূপান্তর করুন।

3. আজকের দিনের তাপমাত্রা সেলসিয়াসে ইনপুট নিয়ে ফারেনহাইটে প্রদর্শন করুন।

সমস্যা 42: Kilometer-কে Meter-এ রূপান্তর করুন।

সমস্যা

দূরত্ব কিলোমিটারে ইনপুট নিয়ে তাকে মিটারে রূপান্তর করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- ১ কিলোমিটার = ১০০০ মিটার

কীভাবে চিন্তা করব?

আমরা জানি, ১ কিলোমিটার সমান ১০০০ মিটার। তাই যেকোনো কিলোমিটার মানকে মিটারে রূপান্তর করতে হলে তাকে ১০০০ দিয়ে গুণ করতে হবে। সূত্র: `meters = kilometers * 1000`।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: কিলোমিটার মান ইনপুট নেওয়া।

Step 3: meters = kilometers * 1000 সূত্র প্রয়োগ করা।

Step 4: মিটার মান প্রদর্শন করা।

Python Code

```
# Converting Kilometer to Meter
kilometers = float(input("Enter distance in kilometers: "))

meters = kilometers * 1000
print(kilometers, "km is equal to", meters, "meters")
```

Built-in Function / Shortcut Method

```
# Direct shortcut
# print(f"{km} km = {km * 1000} meters")
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter distance in kilometers: 4.5  4.5 km is equal to 4500.0 meters
```

ব্যাখ্যা

৪.৫ কিলোমিটারকে ১০০০ দিয়ে গুণ করে ৪৫০০.০ মিটার নির্ণয় করা হয়েছে।

Common Mistakes

- ১০০০ দিয়ে গুণ না করে ভাগ করে ফেলা।
- ১০০ দিয়ে গুণ করা (সেটি মিটারের বদলে হেক্টোমিটার বা সেন্টিমিটারের ভুল হিসাব)।

অনুশীলনের জন্য

1. মিটারকে সেন্টিমিটারে (`m * 100`) রূপান্তর করুন।

2. কিলোমিটারকে মাইলে (`km * 0.621371`) রূপান্তর করুন।

3. কেজিকে গ্রামে (`kg * 1000`) রূপান্তর করুন।

সমস্যা 43: Meter-কে Kilometer-এ রূপান্তর করুন।

সমস্যা

দূরত্ব মিটারে ইনপুট নিয়ে তাকে কিলোমিটারে রূপান্তর করতে হবে।

কী কী জানা থাকতে হবে?

- ১ কিলোমিটার = ১০০০ মিটার

কীভাবে চিন্তা করব?

ছোট একক (মিটার) থেকে বড় এককে (কিলোমিটার) যাওয়ার জন্য আমাদের ভাগ করতে হয়। যেহেতু ১০০০ মিটারে ১ কিলোমিটার, তাই মিটার মানকে ১০০০ দ্বারা ভাগ করতে হবে। সূত্র: `kilometers = meters / 1000`।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: মিটার মান ইনপুট নেওয়া।

Step 3: kilometers = meters / 1000 সূত্র প্রয়োগ করা।

Step 4: কিলোমিটার মান প্রিন্ট করা।

Python Code

```
# Converting Meter to Kilometer
meters = float(input("Enter distance in meters: "))
```

```
kilometers = meters / 1000
print(meters, "meters is equal to", kilometers, "km")
```

Built-in Function / Shortcut Method

```
# Shortcut Method
# print("Kilometers:", float(input("Meters: ")) / 1000)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter distance in meters: 3500	3500.0 meters is equal to 3.5 km

ব্যাখ্যা

৩৫০০ মিটারকে ১০০০ দ্বারা ভাগ করে ৩.৫ কিলোমিটার পাওয়া গেছে।

Common Mistakes

- ভাগ করার বদলে গুণ করে ফেলা।
- পূর্ণসংখ্যা ভাগফল (`//`) ব্যবহার করা, যা দশমিক দূরত্ব বাদ দিয়ে দেয়।

অনুশীলনের জন্য

1. গ্রামকে কিলোগ্রামে (`g / 1000`) রূপান্তর করুন।

2. সেন্টিমিটারকে মিটারে (`cm / 100`) রূপান্তর করুন।

3. মাইলকে কিলোমিটারে (`miles * 1.60934`) রূপান্তর করুন।

Topic: Conditional Logic (Control Flow)

সমস্যা 44: একটি সংখ্যা জোড় নাকি বিজোড় তা নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি পূর্ণসংখ্যা ইনপুট নিয়ে সেটি জোড় (Even) নাকি বিজোড় (Odd) তা conditional logic ব্যবহার করে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-else স্টেটমেন্ট
- Modulus Operator (%)

কীভাবে চিন্তা করব?

যেসব সংখ্যাকে ২ দ্বারা ভাগ করলে কোনো ভাগশেষ থাকে না অর্থাৎ ভাগশেষ শূন্য (0) হয়, তারা হলো জোড় সংখ্যা। আর ভাগশেষ ১ হলে তারা বিজোড় সংখ্যা। তাই আমরা `number % 2 == 0` শর্তটি `if` দিয়ে যাচাই করব। সত্য হলে Even, মিথ্যা হলে Odd।

> **[Important]** পাইথনে if এবং else এর ভিতরের লাইনগুলোতে অবশ্যই Indentation (৪টি স্পেস বা ১টি Tab) দিতে হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি পূর্ণসংখ্যা ইনপুট নেওয়া।

Step 3: if number % 2 == 0 শর্তটি পরীক্ষা করা।

Step 4: শর্ত সত্য হলে 'Even Number' প্রিন্ট করা।

Step 5: অন্যথায় else ব্লকে গিয়ে 'Odd Number' প্রিন্ট করা ◦

Python Code

```
# Checking if a number is Even or Odd
num = int(input("Enter a number: "))

if num % 2 == 0:
    print(num, "is an Even number")
else:
    print(num, "is an Odd number")
```

Built-in Function / Shortcut Method

```
# One-line conditional expression (Ternary operator)
# print("Even" if num % 2 == 0 else "Odd")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 18	18 is an Even number

ব্যাখ্যা

১৮ কে ২ দিয়ে ভাগ করলে ভাগশেষ শূন্য হয়। তাই `18 % 2 == 0` শর্তটি True হওয়ায় `if` ব্লকের কোডটি চলে Even আউটপুট দিয়েছে।

Common Mistakes

- `if` বা `else` এর শেষে কোলন (:) দিতে ভুলে যাওয়া।
- `== 0` এর জায়গায় সিঙ্গেল ইকুয়াল `= 0` দেওয়া।
- Indentation (৪টি স্পেস) ঠিকমতো না দেওয়া।

অনুশীলনের জন্য

1. একটি সংখ্যা ৩ দ্বারা নিঃশেষে বিভাজ্য কিনা যাচাই করুন।

2. একটি সংখ্যা জোড় এবং ১০ এর চেয়ে বড় কিনা একসাথে যাচাই করুন।

3. ব্যবহারকারীর ইনপুট করা দুটি সংখ্যার যোগফল জোড় নাকি বিজোড় তা বের করুন।

সমস্যা 45: একটি সংখ্যা ধনাত্মক, ঋণাত্মক নাকি শূন্য তা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি ধনাত্মক (Positive), ঋণাত্মক (Negative) নাকি শূন্য (Zero) তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-elif-else স্টেটমেন্ট
- Comparison Operators (<, >)

কীভাবে চিন্তা করব?

এখানে আমাদের তিনটি সম্ভাবনা আছে: ১) সংখ্যাটি শূন্যের চেয়ে বড় কিনা (`num > 0` হলে Positive), ২) সংখ্যাটি শূন্যের চেয়ে ছোট কিনা (`num < 0` হলে Negative), অথবা ৩) সংখ্যাটি শূন্যের সমান কিনা (`num == 0` হলে Zero)। একাধিক শর্ত যাচাইয়ের জন্য পাইথনে `if-elif-else` ব্যবহার করা হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি সংখ্যা ইনপুট নেওয়া।

Step 3: if num > 0 হলে 'Positive' প্রিন্ট করা।

Step 4: elif num < 0 হলে 'Negative' প্রিন্ট করা।

Step 5: else ব্লকে 'Zero' প্রিন্ট করা।

Python Code

```
# Checking if a number is Positive, Negative or Zero
num = float(input("Enter a number: "))

if num > 0:
    print("The number is Positive")
```

```
elif num < 0:
    print("The number is Negative")
else:
    print("The number is Zero")
```

Built-in Function / Shortcut Method

```
# None needed for basic control flow
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: -15.5	The number is Negative

ব্যাখ্যা

-১৫.৫ সংখ্যাটি শূন্যের চেয়ে ছোট। তাই `elif num < 0` শর্তটি সত্য হয়ে Negative আউটপুট দিয়েছে।

Common Mistakes

- `elif` এর বদলে অন্য ভাষার মতো `else if` লেখা (পাইথনে `elif` লিখতে হয়)।
- শর্ত লেখার পর কোলন (:) না দেওয়া।

অনুশীলনের জন্য

1. একটি তাপমাত্রা ইনপুট নিয়ে সেটি গরম (>৩০), ঠান্ডা (<১০) নাকি স্বাভাবিক তা বের করুন।

2. ব্যাংক ব্যালেন্স ইনপুট নিয়ে তা ধনাত্মক বা ঋণাত্মক (লোন) কিনা যাচাই করুন।

3. একটি সংখ্যা ধনাত্মক ও জোড় কিনা তা if-elif দিয়ে যাচাই করুন।

সমস্যা 46: দুটি সংখ্যার মধ্যে বড় সংখ্যাটি নির্ণয় করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে তাদের মধ্যে কোনটি বড় (Maximum) তা if-else শর্ত ব্যবহার করে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-else স্টেটমেন্ট
- Comparison Operator (>)

কীভাবে চিন্তা করব?

দুটি সংখ্যা num1 এবং num2 এর মধ্যে আমরা তুলনা করব। যদি `num1 > num2` হয়, তবে num1 বড়। অন্যথায় num2 বড়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নেওয়া।

Step 3: if num1 > num2 হলে num1 কে বড় ঘোষণা করা।

Step 4: else হলে num2 কে বড় ঘোষণা করা।

Python Code

```
# Finding maximum between two numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if num1 > num2:
    print("Maximum number is:", num1)
elif num2 > num1:
    print("Maximum number is:", num2)
else:
    print("Both numbers are equal")
```

Built-in Function / Shortcut Method

```
# Using built-in max() function
# print("Maximum:", max(num1, num2))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 45 Enter second number: 78	Maximum number is: 78.0

ব্যাখ্যা

যেহেতু ৭৮ সংখ্যাটি ৪৫ এর চেয়ে বড়, তাই `elif num2 > num1` ব্লকটি এক্সিকিউট হয়েছে।

Common Mistakes

- দুটি সংখ্যা সমান হওয়ার সম্ভাবনা (Equal case) বিবেচনা না করে শুধু if-else দেওয়া।
- ভেরিয়েবলের নাম ভুল করা।

অনুশীলনের জন্য

1. দুটি বন্ধুর বয়স ইনপুট নিয়ে কে বড় তা বের করুন।

2. দুটি পণ্যের দাম ইনপুট নিয়ে কোনটি বেশি দামি তা বের করুন।

3. max() ফাংশন ব্যবহার করে দুটি সংখ্যার বড়টি বের করুন।

সমস্যা 47: দুটি সংখ্যার মধ্যে ছোট সংখ্যাটি নির্ণয় করুন।

সমস্যা

দুটি সংখ্যা ইনপুট নিয়ে তাদের মধ্যে কোনটি ছোট (Minimum) তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-else স্টেটমেন্ট
- Comparison Operator (<)

কীভাবে চিন্তা করব?

বড় সংখ্যা বের করার লজিকের মতোই, আমরা এবার `<` (Less than) অপারেটর দিয়ে পরীক্ষা করব `num1 < num2` কিনা। সত্য হলে num1 ছোট, না হলে num2 ছোট।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: দুটি সংখ্যা ইনপুট নেওয়া।

Step 3: if num1 < num2 হলে num1 ছোট।

Step 4: elif num2 < num1 হলে num2 ছোট।

Step 5: else হলে উভয়ই সমান।

Python Code

```
# Finding minimum between two numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if num1 < num2:
    print("Minimum number is:", num1)
elif num2 < num1:
    print("Minimum number is:", num2)
else:
    print("Both numbers are equal")
```

Built-in Function / Shortcut Method

```
# Using built-in min() function
# print("Minimum:", min(num1, num2))
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter first number: 12` `Enter second number: 5`	`Minimum number is: 5.0`

ব্যাখ্যা

১২ এবং ৫ এর মধ্যে ৫ ছোট হওয়ায় `elif num2 < num1` শর্তটি সত্য হয়েছে।

Common Mistakes

- `<` এবং `>` চিহ্নের দিক গুলিয়ে ফেলা।
- Indentation ভুল করা।

অনুশীলনের জন্য

1. দুটি পরীক্ষার নম্বরের মধ্যে কম নম্বরটি বের করুন।

2. min() ফাংশন ব্যবহার করে দুটি দশমিক সংখ্যার ছোটটি বের করুন।

3. দুটি শহরের তাপমাত্রার মধ্যে কোনটিতে কম গরম তা বের করুন।

সমস্যা 48: তিনটি সংখ্যার মধ্যে সবচেয়ে বড় সংখ্যাটি নির্ণয় করুন।

সমস্যা

তিনটি সংখ্যা ইনপুট নিয়ে তাদের মধ্যে সবচেয়ে বড় (Largest) সংখ্যাটি লজিক্যাল অপারেটর ব্যবহার করে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Logical AND (and)
- if-elif-else

কীভাবে চিন্তা করব?

তিনটি সংখ্যা a, b, c থাকলে a তখনই সবচেয়ে বড় হবে যখন a একইসাথে b এর চেয়ে বড় এবং c এর চেয়ে বড় হবে (`a >= b and a >= c`)। একইভাবে b বড় হবে যদি `b >= a and b >= c` হয়। অন্যথায় c সবচেয়ে বড় হবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: তিনটি সংখ্যা a, b এবং c ইনপুট নেওয়া।

Step 3: if a >= b and a >= c হলে a বড়।

Step 4: elif b >= a and b >= c হলে b বড়।

Step 5: else হলে c বড়।

Python Code

```
# Finding largest among three numbers
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b and a >= c:
    print("Largest number is:", a)
elif b >= a and b >= c:
    print("Largest number is:", b)
else:
    print("Largest number is:", c)
```

Built-in Function / Shortcut Method

```
# Using built-in max() function
# print("Largest:", max(a, b, c))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 25 Enter second number: 89 Enter third number: 54	Largest number is: 89.0

ব্যাখ্যা

এখানে b (৮৯) অন্য দুটি সংখ্যা a (২৫) এবং c (৫৪) উভয়ের চেয়ে বড় হওয়ায় `elif` ব্লকটি সত্য হয়েছে।

Common Mistakes

- `and` অপারেটর ব্যবহার না করে শুধু `if a > b` দিয়ে সিদ্ধান্ত নেওয়া।
- `>=` এর বদলে শুধু `>` দিলে দুটি সংখ্যা সমান ও বড় হলে সঠিক কাজ না করা।

অনুশীলনের জন্য

1. ৪টি সংখ্যার মধ্যে সবচেয়ে বড় সংখ্যাটি বের করুন।

2. তিনজন বন্ধুর উচ্চতার মধ্যে সবচেয়ে ড্যাঙ্গা বা লম্বা কে তা বের করুন।

3. max() ফাংশনে একটি লিস্ট পাস করে সবচেয়ে বড় মান বের করুন।

সমস্যা 49: তিনটি সংখ্যার মধ্যে সবচেয়ে ছোট সংখ্যাটি নির্ণয় করুন।

সমস্যা

তিনটি সংখ্যা ইনপুট নিয়ে তাদের মধ্যে সবচেয়ে ছোট (Smallest) সংখ্যাটি নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Logical AND (and)
- Comparison Operator (<=)

কীভাবে চিন্তা করব?

সবচেয়ে বড় সংখ্যা বের করার মতোই, এবার আমরা পরীক্ষা করব কোনো একটি সংখ্যা অন্য দুটি সংখ্যার চেয়ে ছোট বা সমান কিনা। যেমন `a <= b and a <= c` হলে a সবচেয়ে ছোট।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: তিনটি সংখ্যা ইনপুট নেওয়া।

Step 3: if a <= b and a <= c হলে a সবচেয়ে ছোট।

Step 4: elif b <= a and b <= c হলে b সবচেয়ে ছোট।

Step 5: else হলে c সবচেয়ে ছোট।

Python Code

```
# Finding smallest among three numbers
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a <= b and a <= c:
    print("Smallest number is:", a)
elif b <= a and b <= c:
    print("Smallest number is:", b)
else:
    print("Smallest number is:", c)
```

Built-in Function / Shortcut Method

```
# Using built-in min() function
# print("Smallest:", min(a, b, c))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 15 Enter second number: 9 Enter third number: 22	Smallest number is: 9.0

ব্যাখ্যা

এখানে b (৯) অন্য দুটি সংখ্যার চেয়ে ছোট। তাই `elif` ব্লকটি কাজ করে ৯.০ আউটপুট দিয়েছে।

Common Mistakes

- শর্তে `and` এর জায়গায় ভুল করে `or` ব্যবহার করা।
- কোলন না দেওয়া।

অনুশীলনের জন্য

1. তিনটি পণ্যের দামের মধ্যে সবচেয়ে সস্তা পণ্যটির দাম বের করুন।

2. min() ফাংশন ব্যবহার করে ৪টি সংখ্যার ছোটটি বের করুন।

3. তিনটি পরীক্ষার নম্বরের মধ্যে সবচেয়ে কম নম্বরটি বের করুন।

সমস্যা 50: একটি বছর Leap Year কিনা নির্ণয় করুন।

সমস্যা

একটি সাল বা বছর ইনপুট নিয়ে সেটি অধিবর্ষ বা Leap Year কিনা তা নির্দিষ্ট গাণিতিক শর্তের মাধ্যমে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Leap Year-এর লজিক
- Modulus Operator (%)
- Logical Operators (and, or)

কীভাবে চিন্তা করব?

একটি বছর Leap Year হবে কিনা তা দুটি নিয়মে জানা যায়:
নিয়ম ১: বছরটি যদি ৪০০ দ্বারা নিঃশেষে বিভাজ্য হয় (`year % 400 == 0`), তবে সেটি নিশ্চিত Leap Year।

নিয়ম ২: বছরটি যদি ৪ দ্বারা বিভাজ্য হয় (`year % 4 == 0`) এবং একই সাথে ১০০ দ্বারা বিভাজ্য না হয় (`year % 100 != 0`), তবে সেটি Leap Year।
এই দুটি নিয়মের যেকোনো একটি সত্য হলেই লিপ ইয়ার।

> **[Note]** শতাব্দীর বছরগুলো (যেমন ১৭০০, ১৮০০, ১৯০০) ৪ দ্বারা বিভাজ্য হলেও লিপ ইয়ার নয়, যদি না তারা ৪০০ দ্বারা বিভাজ্য হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: বছর (year) ইনপুট নেওয়া।

Step 3: if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0) শর্ত যাচাই করা।

Step 4: সত্য হলে 'Leap Year' প্রিন্ট করা।

Step 5: মিথ্যা হলে 'Not a Leap Year' প্রিন্ট করা।

Python Code

```
# Checking if a year is Leap Year
year = int(input("Enter a year: "))

if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print(year, "is a Leap Year")
else:
    print(year, "is NOT a Leap Year")
```

Built-in Function / Shortcut Method

```
# Using calendar module
# import calendar
# print("Is Leap Year:", calendar.isleap(year))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a year: 2024	2024 is a Leap Year

ব্যাখ্যা

২০২৪ সালটি ১০০ দ্বারা বিভাজ্য নয় এবং ৪ দ্বারা নিঃশেষে বিভাজ্য (`2024 % 4 == 0`)। তাই এটি একটি Leap Year। (যদি ১৯০০ সাল দেওয়া হতো, তবে তা ১০০ দ্বারা বিভাজ্য কিন্তু ৪০০ দ্বারা নয়, তাই লিপ ইয়ার হতো না)।

Common Mistakes

- শুধুমাত্র `year % 4 == 0` দিয়ে লিপ ইয়ার যাচাই করা (যা ১৯০০ বা ২১০০ সালের মতো শতাব্দী বছরের জন্য ভুল উত্তর দেয়)।
- `and` এবং `or` এর ব্যবহারে ব্র্যাকেট না দেওয়া।

অনুশীলনের জন্য

1. ২০০০ সাল এবং ২১০০ সাল ইনপুট দিয়ে কোডটি যাচাই করুন।

2. calendar মডিউলের `calendar.isleap()` ব্যবহার করে লিপ ইয়ার বের করুন।

3. বর্তমান সাল ইনপুট নিয়ে সেটি লিপ ইয়ার কিনা যাচাই করুন।

সমস্যা 51: প্রাপ্ত নম্বর অনুযায়ী Grade নির্ণয় করুন।

সমস্যা

পরীক্ষায় প্রাপ্ত ১০০ এর মধ্যের নম্বর ইনপুট নিয়ে প্রচলিত গ্রেডিং সিস্টেম অনুযায়ী গ্রেড (A+, A, A-, B, C, F ইত্যাদি) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-elif-else স্টেটমেন্ট
- Range Checking

কীভাবে চিন্তা করব?

আমরা ওপর থেকে নিচের দিকে শর্ত যাচাই করব।
যদি নম্বর ৮০ বা তার বেশি হয়: A+
৭০ থেকে ৭৯ এর মধ্যে হলে: A
৬০ থেকে ৬৯ হলে: A-
৫০ থেকে ৫৯ হলে: B
৪০ থেকে ৪৯ হলে: C
৩৩ থেকে ৩৯ হলে: D
৩৩ এর কম হলে: F (Fail)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: নম্বর (marks) ইনপুট নেওয়া।

Step 3: if marks >= 80 হলে 'Grade: A+'।

Step 4: elif marks >= 70 হলে 'Grade: A'।

Step 5: এভাবে ধাপে ধাপে elif দিয়ে ৩৩ পর্যন্ত যাচাই করা।

Step 6: else ব্লকে (৩৩ এর কম হলে) 'Grade: F' প্রদর্শন করা।

Python Code

```
# Finding grade based on marks
marks = float(input("Enter your marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks entered!")
elif marks >= 80:
    print("Grade: A+")
elif marks >= 70:
    print("Grade: A")
elif marks >= 60:
    print("Grade: A-")
elif marks >= 50:
    print("Grade: B")
elif marks >= 40:
    print("Grade: C")
elif marks >= 33:
    print("Grade: D")
else:
    print("Grade: F (Fail)")
```

Built-in Function / Shortcut Method

```
# None needed for conditional grading
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter your marks (0-100): 75.5`	`Grade: A`

ব্যাখ্যা

৭৫.৫ নম্বরটি ৮০ এর চেয়ে ছোট কিন্তু ৭০ এর চেয়ে বড়। তাই `elif marks >= 70` শর্তটি সত্য হয়ে Grade: A প্রিন্ট হয়েছে।

Common Mistakes

- শর্তগুলো ছোট থেকে বড়র দিকে সাজানো (যেমন আগে `marks >= 33` দিলে ৮০ পেলেও D দেখিয়ে দেবে। সবসময় বড় শর্ত আগে দিতে হয়)।
- ১০০ এর বেশি বা ০ এর কম নম্বর ইনপুট দিলে Invalid চেক না করা।

অনুশীলনের জন্য

1. বিশ্ববিদ্যালয়ের ৪.০০ স্কেল অনুযায়ী জিপিএ নির্ণয় করার প্রোগ্রাম লিখুন।

2. ৫টি বিষয়ের নম্বর ইনপুট নিয়ে গড় বের করে তারপর গ্রেড নির্ণয় করুন।

3. গ্রেডের পাশাপাশি মন্তব্য (যেমন Excellent, Good, Poor) প্রিন্ট করুন।

সমস্যা 52: পরীক্ষার নম্বর অনুযায়ী Pass অথবা Fail নির্ণয় করুন।

সমস্যা

পরীক্ষায় প্রাপ্ত নম্বর ইনপুট নিয়ে পাস মার্কস (যেমন ৩৩ বা ৪০) এর ভিত্তিতে শিক্ষার্থী Pass করেছে নাকি Fail করেছে তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- if-else স্টেটমেন্ট

কীভাবে চিন্তা করব?

ধরে নিই পাস নম্বর হলো ৩৩। যদি ইনপুট করা নম্বর ৩৩ বা তার বেশি হয় (`marks >= 33`), তবে Pass। অন্যথায় Fail।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: নম্বর ইনপুট নেওয়া।

Step 3: if marks >= 33 যাচাই করা।

Step 4: সত্য হলে 'Congratulations! You Passed' প্রিন্ট করা।

Step 5: মিথ্যা হলে 'Sorry! You Failed' প্রিন্ট করা।

Python Code

```
# Checking Pass or Fail
marks = float(input("Enter obtained marks: "))

passing_mark = 33

if marks >= passing_mark:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

Built-in Function / Shortcut Method

```
# Short ternary version
# print("PASS" if float(input("Marks: ")) >= 33 else "FAIL")
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter obtained marks: 42`	`Result: PASS`

ব্যাখ্যা

৪২ নম্বরটি ৩৩ এর চেয়ে বড় হওয়ায় শিক্ষার্থী পাস করেছে।

Common Mistakes

- `marks > 33` লেখা (ঠিক ৩৩ পেলে ফেল দেখাবে, তাই অবশ্যই `marks >= 33` লিখতে হবে)।
- ইনপুটকে float বা int না করা।

অনুশীলনের জন্য

1. পাস মার্কস ৪০ ধরে একটি পরীক্ষার Pass/Fail নির্ণয় করুন।

2. তত্ত্বীয় (Theory) এবং ব্যবহারিক (Practical) উভয় বিষয়ে আলাদাভাবে পাস করতে হবে—এমন শর্তে Pass/Fail নির্ণয় করুন।

3. ৩টি বিষয়ের কোনো একটিতে ৩৩ এর কম পেলে Fail দেখানোর প্রোগ্রাম লিখুন।

সমস্যা 53: একজন ব্যক্তির বয়স অনুযায়ী ভোট দেওয়ার যোগ্যতা নির্ণয় করুন।

সমস্যা

ব্যক্তির বয়স ইনপুট নিয়ে তিনি ভোট দেওয়ার যোগ্য (Eligible to Vote) কিনা তা নির্ণয় করতে হবে। সাধারণত ১৮ বছর বা তার বেশি বয়সে ভোট দেওয়া যায়।

কী কী জানা থাকতে হবে?

- if-else স্টেটমেন্ট
- Comparison Operator (>=)

কীভাবে চিন্তা করব?

ভোট দেওয়ার ন্যূনতম বয়স হলো ১৮। আমরা বয়স ইনপুট নিয়ে যাচাই করব `age >= 18` কিনা। সত্য হলে Eligible, মিথ্যা হলে Not Eligible।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: বয়স (age) ইনপুট নেওয়া।

Step 3: if age >= 18 পরীক্ষা করা।

Step 4: সত্য হলে যোগ্য বার্তা প্রদর্শন করা।

Step 5: মিথ্যা হলে আর কত বছর অপেক্ষা করতে হবে তা হিসাব করে প্রদর্শন করা।

Python Code

```
# Checking voting eligibility
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    years_left = 18 - age
    print("You are NOT eligible to vote.")
    print("Please wait for", years_left, "more years.")
```

Built-in Function / Shortcut Method

```
# Short conditional
# print("Eligible" if int(input("Age: ")) >= 18 else "Not Eligible")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter your age: 15	You are NOT eligible to vote. Please wait for 3 more years.

ব্যাখ্যা

১৫ বছর বয়স ১৮ এর কম। তাই else ব্লকে গিয়ে ১৮ থেকে ১৫ বিয়োগ করে জানিয়ে দেওয়া হয়েছে যে আরও ৩ বছর অপেক্ষা করতে হবে।

Common Mistakes

- ঋণাত্মক বয়স বা অবাস্তব বয়স (যেমন ১৫০) ইনপুট দিলে যাচাই না করা।
- `>=` এর জায়গায় `>` দেওয়া।

অনুশীলনের জন্য

1. ড্রাইভিং লাইসেন্স পাওয়ার যোগ্যতা (বয়স >= ১৮) যাচাই করুন।

2. সরকারি চাকরির আবেদন করার বয়সসীমা (১৮ থেকে ৩০ বছর) যাচাই করুন।

3. পেনশন পাওয়ার বয়স (৬০ বছর বা বেশি) হয়েছে কিনা যাচাই করুন।

সমস্যা 54: একটি Character Alphabet কিনা নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি Character (অক্ষর বা চিহ্ন) ইনপুট নিয়ে সেটি ইংরেজি বর্ণমালার অক্ষর (Alphabet: a-z বা A-Z) কিনা তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- String method isalpha()
- Comparison with character ranges ('a' to 'z')

কীভাবে চিন্তা করব?

কোনো ক্যারেক্টার ইংরেজি বর্ণমালা কিনা তা যাচাই করার সবচেয়ে সহজ উপায় হলো পাইথনের বিল্ট-ইন স্ট্রিং মেথড `.isalpha()` ব্যবহার করা। অথবা আমরা ম্যানুয়ালি চেক করতে পারি ক্যারেক্টারটি 'a' থেকে 'z' অথবা 'A' থেকে 'Z' এর মধ্যে আছে কিনা।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি ক্যারেক্টার ইনপুট নেওয়া।

Step 3: `if char >= 'a' and char <= 'z' or char >= 'A' and char <= 'Z'` শর্ত পরীক্ষা করা।

Step 4: সত্য হলে 'It is an Alphabet' প্রিন্ট করা।

Step 5: অন্যথায় 'It is NOT an Alphabet' প্রিন্ট করা।

Python Code

```
# Checking if a character is an Alphabet (Manual Method)
char = input("Enter a single character: ")

if ('a' <= char <= 'z') or ('A' <= char <= 'Z'):
    print(char, "is an Alphabet")
else:
    print(char, "is NOT an Alphabet")
```

Built-in Function / Shortcut Method

```
# Using built-in isalpha() method
# print(char, "is Alphabet" if char.isalpha() else "is NOT Alphabet")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a single character: K	K is an Alphabet

ব্যাখ্যা

'K' অক্ষরটি 'A' থেকে 'Z' এর সীমানার মধ্যে অবস্থিত। তাই শর্তটি সত্য হয়ে এটি একটি Alphabet বা বর্ণমালা হিসেবে চিহ্নিত হয়েছে।

Common Mistakes

- একাধিক অক্ষর একসাথে ইনপুট দেওয়া (যেমন 'abc')।
- ছোট হাতের ও বড় হাতের উভয় শর্ত যাচাই না করা।

অনুশীলনের জন্য

1. isalpha() মেথড ব্যবহার করে ইনপুট করা ক্যারেক্টারটি বর্ণমালা কিনা যাচাই করুন।

2. একটি চিহ্ন যেমন '#' বা '$' ইনপুট দিয়ে আউটপুট লক্ষ্য করুন।

3. ইনপুট করা অক্ষরটি বাংলা বর্ণমালা কিনা তা যাচাই করার চেষ্টা করুন।

সমস্যা 55: একটি Character Digit কিনা নির্ণয় করুন।

সমস্যা

একটি Character ইনপুট নিয়ে সেটি কোনো অংক বা Digit (0 থেকে 9) কিনা তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- String method isdigit()
- Comparison ('0' to '9')

কীভাবে চিন্তা করব?

ক্যারেক্টারটি যদি '0' থেকে '9' এর মধ্যে থাকে, তবে সেটি একটি অংক বা Digit। পাইথনে `.isdigit()` মেথড দিয়েও এটি এক লাইনে যাচাই করা যায়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি ক্যারেক্টার ইনপুট নেওয়া।

Step 3: if '0' <= char <= '9' শর্ত যাচাই করা।

Step 4: সত্য হলে 'Digit' প্রিন্ট করা, অন্যথায় 'Not a Digit' প্রিন্ট করা।

Python Code

```
# Checking if a character is a Digit
char = input("Enter a single character: ")

if '0' <= char <= '9':
```

```
    print(char, "is a Digit")
else:
    print(char, "is NOT a Digit")
```

Built-in Function / Shortcut Method

```
# Using built-in isdigit() method
# print(char, "is a Digit" if char.isdigit() else "is NOT a Digit")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a single character: 7	7 is a Digit

ব্যাখ্যা

'7' ক্যারেক্টারটি '0' এবং '9' এর মধ্যে রয়েছে। তাই এটি একটি Digit।

Common Mistakes

- ইনপুট নেওয়ার পর `int(char)` করে ফেলা (এখানে আমরা ক্যারেক্টার হিসেবেই যাচাই করছি, তাই int করার দরকার নেই)।
- দশমিক বিন্দু (.) কে ডিজিট মনে করা।

অনুশীলনের জন্য

1. isdigit() মেথড ব্যবহার করে ইনপুট যাচাই করুন।

2. ইনপুট করা ক্যারেক্টারটি Alphabet, Digit নাকি Special Character তা if-elif-else দিয়ে একসাথে বের করুন।

3. একটি সম্পূর্ণ স্ট্রিং (যেমন '12345') এর সব অক্ষর ডিজিট কিনা তা `str.isdigit()` দিয়ে যাচাই করুন।

সমস্যা 56: একটি Alphabet Vowel নাকি Consonant তা নির্ণয় করুন।

সমস্যা

একটি ইংরেজি বর্ণমালা ইনপুট নিয়ে সেটি স্বরবর্ণ (Vowel: a, e, i, o, u) নাকি ব্যঞ্জনবর্ণ (Consonant) তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- in অপারেটর
- String lower() বা upper() মেথড

কীভাবে চিন্তা করব?

ইংরেজি বর্ণমালায় ৫টি স্বরবর্ণ আছে: a, e, i, o, u। বড় হাতের ও ছোট হাতের মিলিয়ে আমরা একটি লিস্ট বা স্ট্রিং তৈরি করতে পারি `vowels = 'aeiouAEIOU'`। ইনপুট করা ক্যারেক্টারটি যদি এই vowels এর ভেতর থাকে (`char in vowels`), তবে সেটি Vowel। অন্যথায় সেটি Consonant।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি অক্ষর ইনপুট নেওয়া।

Step 3: অক্ষরটিকে ছোট হাতের অক্ষরে রূপান্তর করা (`char.lower()`) যাতে বড়-ছোট হাতের সমস্যা না হয়।

Step 4: if char in ['a', 'e', 'i', 'o', 'u'] পরীক্ষা করা।

Step 5: সত্য হলে Vowel, মিথ্যা হলে Consonant প্রিন্ট করা।

Python Code

```
# Checking Vowel or Consonant
char = input("Enter an alphabet character: ")

# Convert to lower case to handle both uppercase and lowercase
lower_char = char.lower()

if lower_char in ['a', 'e', 'i', 'o', 'u']:
    print(char, "is a Vowel")
elif lower_char.isalpha():
    print(char, "is a Consonant")
else:
    print("Invalid input! Please enter an alphabet.")
```

Built-in Function / Shortcut Method

```
# Direct string check
# print("Vowel" if char.lower() in "aeiou" else "Consonant")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter an alphabet character: E	E is a Vowel

ব্যাখ্যা

'E' কে `.lower()` করায় তা 'e' হয়েছে, যা আমাদের ভাওয়েল লিস্টের ভেতর রয়েছে। তাই এটি Vowel হিসেবে চিহ্নিত হয়েছে।

Common Mistakes

- বড় হাতের অক্ষর (যেমন 'A') ইনপুট দিলে এবং কোডে শুধু ছোট হাতের চেক করলে Consonant দেখিয়ে দেওয়া।
- সংখ্যা বা চিহ্ন ইনপুট দিলেও Consonant দেখিয়ে দেওয়া (তাই আগে `isalpha()` যাচাই করা ভালো)।

অনুশীলনের জন্য

1. একটি সম্পূর্ণ শব্দের প্রথম অক্ষরটি Vowel নাকি Consonant তা বের করুন।

2. 'y' অক্ষরটিকে Semi-vowel হিসেবে চিহ্নিত করার শর্ত যুক্ত করুন।

3. ইনপুট করা অক্ষরটি স্বরবর্ণ না হলে তা প্রিন্ট করুন।

সমস্যা 57: একটি Alphabet Uppercase নাকি Lowercase তা নির্ণয় করুন।

সমস্যা

একটি ইংরেজি বর্ণমালা ইনপুট নিয়ে সেটি বড় হাতের (Uppercase: A-Z) নাকি ছোট হাতের (Lowercase: a-z) তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- isupper() এবং islower() মেথড
- Comparison with 'A'-'Z' and 'a'-'z'

কীভাবে চিন্তা করব?

অক্ষরটি যদি 'A' থেকে 'Z' এর মধ্যে থাকে, তবে সেটি Uppercase। আর 'a' থেকে 'z' এর মধ্যে থাকলে সেটি Lowercase। পাইথনের বিল্ট-ইন `.isupper()` এবং `.islower()` দিয়েও এটি সহজেই করা যায়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি ক্যারেক্টার ইনপুট নেওয়া।

Step 3: if 'A' <= char <= 'Z' হলে 'Uppercase Character' প্রিন্ট করা।

Step 4: elif 'a' <= char <= 'z' হলে 'Lowercase Character' প্রিন্ট করা।

Step 5: else ব্লকে 'Not an alphabet' প্রিন্ট করা।

Python Code

```
# Checking Uppercase or Lowercase
char = input("Enter an alphabet character: ")

if 'A' <= char <= 'Z':
    print(char, "is an UPPERCASE character")
elif 'a' <= char <= 'z':
    print(char, "is a lowercase character")
```

```
else:
    print(char, "is not an alphabet character")
```

Built-in Function / Shortcut Method

```
# Using built-in methods
# if char.isupper(): print("UPPERCASE")
# elif char.islower(): print("lowercase")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter an alphabet character: m	m is a lowercase character

ব্যাখ্যা

'm' অক্ষরটি 'a' এবং 'z' এর সীমানার মধ্যে থাকায় এটি একটি lowercase বা ছোট হাতের অক্ষর হিসেবে চিহ্নিত হয়েছে।

Common Mistakes

- সংখ্যা ইনপুট দিলে এরর বা ভুল আউটপুট দেওয়া।
- isupper() মেথড লেখার পর ব্র্যাকেট () না দেওয়া।

অনুশীলনের জন্য

1. ইনপুট করা ছোট হাতের অক্ষরকে বড় হাতের এবং বড় হাতের অক্ষরকে ছোট হাতের অক্ষরে রূপান্তর করুন।

2. isupper() মেথড দিয়ে একটি বাক্যের সব অক্ষর বড় হাতের কিনা যাচাই করুন।

3. ব্যবহারকারীর নামের প্রথম অক্ষর বড় হাতের কিনা যাচাই করুন।

সমস্যা 58: তিনটি বাহুর দৈর্ঘ্য দিয়ে Triangle গঠন সম্ভব কিনা নির্ণয় করুন।

সমস্যা

তিনটি বাহুর দৈর্ঘ্য ইনপুট নিয়ে জ্যামিতিক শর্ত অনুযায়ী তাদের দ্বারা একটি ত্রিভুজ (Triangle) গঠন করা সম্ভব কিনা তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- ত্রিভুজ গঠনের শর্ত: যেকোনো দুই বাহুর যোগফল তৃতীয় বাহু অপেক্ষা বড় হতে হবে

কীভাবে চিন্তা করব?

জ্যামিতির উপপাদ্য অনুযায়ী, একটি ত্রিভুজ তখনই আঁকা সম্ভব যখন তার যেকোনো দুটি বাহুর যোগফল তৃতীয় বাহুটির চেয়ে বড় হয়। অর্থাৎ তিনটি বাহু a, b, c হলে অবশ্যই ৩টি শর্ত একসাথে মানতে হবে:

১) `a + b > c`

২) `b + c > a`

৩) `a + c > b`

এই তিনটির কোনো একটিও মিথ্যা হলে ত্রিভুজ গঠন সম্ভব নয়।

> **[Best Practice]** ত্রিভুজ গঠনের এই উপপাদ্যটি প্রোগ্রামিং ও জ্যামিতিতে সবচেয়ে জনপ্রিয় ইন্টারভিউ প্রশ্নগুলোর একটি।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: তিনটি বাহুর দৈর্ঘ্য a, b এবং c ইনপুট নেওয়া।

Step 3: if (a + b > c) and (b + c > a) and (a + c > b) শর্ত পরীক্ষা করা।

Step 4: সত্য হলে 'Triangle is Valid (ত্রিভুজ গঠন সম্ভব)' প্রিন্ট করা।

Step 5: মিথ্যা হলে 'Triangle is NOT Valid (সম্ভব নয়)' প্রিন্ট করা।

Python Code

```
# Checking validity of a triangle given three sides
a = float(input("Enter side 1: "))
b = float(input("Enter side 2: "))
c = float(input("Enter side 3: "))

if (a + b > c) and (b + c > a) and (a + c > b):
    print("Valid Triangle! A triangle CAN be formed.")
else:
    print("Invalid Triangle! A triangle CANNOT be formed.")
```

Built-in Function / Shortcut Method

```
# Sorting shortcut logic: sum of two smallest > largest
# sides = sorted([a, b, c])
# print("Valid" if sides[0] + sides[1] > sides[2] else "Invalid")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter side 1: 5	Valid Triangle! A triangle CAN be

```
Enter side 2: 7                         formed.
Enter side 3: 10
```

ব্যাখ্যা

এখানে যেকোনো দুই বাহুর যোগফল তৃতীয় বাহু অপেক্ষা বড় (যেমন ৫+৭=১২ > ১০)। তাই এই মাপ দিয়ে ত্রিভুজ আঁকা সম্ভব। (কিন্তু যদি ১, ২ এবং ৫ দেওয়া হতো, তবে ১+২=৩ যা ৫ এর চেয়ে ছোট, তাই ত্রিভুজ আঁকা যেত না)।

Common Mistakes

- `>` চিহ্নের বদলে `>=` দেওয়া (দুটি বাহুর যোগফল তৃতীয়টির সমান হলেও ত্রিভুজ হয় না, একটি সরলরেখা হয়ে যায়)।
- তিনটি বাহুর মধ্যে শুধু যেকোনো একটি যোগফল পরীক্ষা করা।

অনুশীলনের জন্য

1. ত্রিভুজটি সমবাহু (Equilateral), সমদ্বিবাহু (Isosceles) নাকি বিষমবাহু (Scalene) তা নির্ণয় করুন।

2. ত্রিভুজের তিনটি কোণ (Angles) ইনপুট নিয়ে তাদের যোগফল ১৮০ ডিগ্রি কিনা তা দিয়ে ত্রিভুজের বৈধতা যাচাই করুন।

3. পীথাগোরাসের উপপাদ্য মেনে সমকোণী ত্রিভুজ গঠন সম্ভব কিনা যাচাই করুন।

Topic: Loop & Iteration

সমস্যা 59: ১ থেকে ১০ পর্যন্ত সংখ্যা প্রিন্ট করুন।

সমস্যা

for loop বা while loop ব্যবহার করে স্ক্রিনে ১ থেকে ১০ পর্যন্ত সংখ্যাগুলো পরপর প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- for loop
- range() ফাংশন

কীভাবে চিন্তা করব?

প্রোগ্রামিংয়ে কোনো কাজ বারবার করার জন্য লুপ (Loop) ব্যবহার করা হয়। পাইথনে আমরা `range()` ফাংশন দিয়ে একটি সংখ্যার ধারা তৈরি করতে পারি। মনে রাখতে হবে, `range(start, stop)` এর শেষ সংখ্যাটি (stop) লুপে অন্তর্ভুক্ত হয় না। তাই ১ থেকে ১০ পর্যন্ত প্রিন্ট করতে হলে আমাদের `range(1, 11)` লিখতে হবে।

> **[Note]** range(start, stop) সবসময় stop এর আগের সংখ্যা পর্যন্ত চলে। তাই ১০ পেতে হলে ১১

> লিখতে হয়।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: for লুপ চালিয়ে i ভেরিয়েবলের মান ১ থেকে ১১ এর পূর্ব পর্যন্ত (অর্থাৎ ১০ পর্যন্ত) ঘোরানো।

Step 3: প্রতিবার লুপ ঘোরার সময় i এর মান প্রিন্ট করা।

Python Code

```
# Printing numbers from 1 to 10 using for loop
for i in range(1, 11):
    print(i, end=" ")
```

Built-in Function / Shortcut Method

```
# Using while loop
# i = 1
# while i <= 10:
#     print(i, end=" ")
#     i += 1
```

Sample Input / Output Table

Sample Input	Sample Output
None	1 2 3 4 5 6 7 8 9 10

ব্যাখ্যা

`range(1, 11)` ১ থেকে শুরু হয়ে ১০ পর্যন্ত মোট ১০ বার লুপ ঘুরিয়েছে। `end=" "` দেওয়ার কারণে সংখ্যাগুলো নিচে নিচে না গিয়ে একই লাইনে স্পেস দিয়ে প্রিন্ট হয়েছে।

Common Mistakes

- `range(1, 10)` লেখা, যার ফলে ১ থেকে ৯ পর্যন্ত প্রিন্ট হয় (১০ বাদ পড়ে যায়)।
- while লুপ ব্যবহার করলে `i += 1` দিতে ভুলে যাওয়া, যা লুপকে অনন্তকাল (Infinite loop) চালিয়ে দেয়।

অনুশীলনের জন্য

1. ০ থেকে ৯ পর্যন্ত সংখ্যা প্রিন্ট করুন (`range(10)`)।

2. ১১ থেকে ২০ পর্যন্ত সংখ্যা প্রিন্ট করুন।

3. ১ থেকে ১০ পর্যন্ত সংখ্যাগুলো প্রতি লাইনে একটি করে প্রিন্ট করুন।

সমস্যা 60: ১ থেকে ১০০ পর্যন্ত সংখ্যা প্রিন্ট করুন।

সমস্যা

১ থেকে ১০০ পর্যন্ত সকল পূর্ণসংখ্যা লুপ ব্যবহার করে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- for loop
- range(1, 101)

কীভাবে চিন্তা করব?

১ থেকে ১০ প্রিন্ট করার মতোই, এবার আমরা `range(1, 101)` ব্যবহার করব যাতে লুপটি ১ থেকে শুরু হয়ে ১০০ পর্যন্ত চলে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: for লুপে range(1, 101) সেট করা।

Step 3: প্রতিটি সংখ্যা প্রিন্ট করা।

Python Code

```
# Printing numbers from 1 to 100
for num in range(1, 101):
    print(num, end=" ")
```

Built-in Function / Shortcut Method

```
# Printing using join and list
# print(" ".join(map(str, range(1, 101))))
```

Sample Input / Output Table

Sample Input	Sample Output
None	1 2 3 4 5 ... 98 99 100 (Prints all numbers from 1 to 100)

ব্যাখ্যা

এখানে `range(1, 101)` এর কারণে লুপটি ১০০ বার চলেছে এবং প্রতিটি সংখ্যা প্রদর্শন করেছে।

Common Mistakes

- `range(1, 100)` দেওয়া, যার ফলে ৯৯ এ গিয়ে লুপ থেমে যায়।
- বড় লুপ চালানোর সময় আউটপুট দেখতে অসুবিধা হলে নতুন লাইনে ব্রেক না দেওয়া।

অনুশীলনের জন্য

1. ১০০ থেকে ২০০ পর্যন্ত সংখ্যা প্রিন্ট করুন।

2. ১ থেকে ১০০ পর্যন্ত সংখ্যাগুলো প্রতি ১০টি পর পর নতুন লাইনে প্রিন্ট করুন।

3. ৫০ থেকে ১০০ পর্যন্ত সংখ্যা প্রিন্ট করুন।

সমস্যা 61: ১ থেকে N পর্যন্ত সংখ্যা প্রিন্ট করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি সংখ্যা N ইনপুট নিয়ে ১ থেকে সেই N পর্যন্ত সকল সংখ্যা প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- input() ও int()
- Dynamic range(1, N + 1)

কীভাবে চিন্তা করব?

ব্যবহারকারী N এর মান যত দেবে (যেমন ১৫ বা ৫০), আমাদের লুপটি ১ থেকে শুরু হয়ে ঠিক তত পর্যন্ত চলবে। যেহেতু range এর শেষ মানটি বাদ পড়ে যায়, তাই আমাদের `range(1, N + 1)` লিখতে হবে।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: ব্যবহারকারীর কাছ থেকে N এর মান ইনপুট নেওয়া।

Step 3: for i in range(1, N + 1) লুপ চালানো।

Step 4: i এর মান প্রিন্ট করা।

Python Code

```
# Printing numbers from 1 to N
N = int(input("Enter upper limit (N): "))

print("Numbers from 1 to", N, "are:")
for i in range(1, N + 1):
    print(i, end=" ")
```

Built-in Function / Shortcut Method

```
# While loop approach
```

```
# i = 1
# while i <= N: print(i, end=" "); i += 1
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter upper limit (N): 15`	`Numbers from 1 to 15 are:` `1 2 3 4 5 6 7 8 9 10 11 12 13 14 15`

ব্যাখ্যা

ব্যবহারকারী N হিসেবে ১৫ ইনপুট দিয়েছে। তাই `range(1, 15 + 1)` অর্থাৎ `range(1, 16)` হয়ে ১ থেকে ১৫ পর্যন্ত সংখ্যাগুলো প্রদর্শিত হয়েছে।

Common Mistakes

- `range(1, N)` দেওয়া, যার ফলে শেষ সংখ্যাটি N বাদ পড়ে যায়।
- N কে int-এ রূপান্তর না করা।

অনুশীলনের জন্য

1. ব্যবহারকারীর কাছ থেকে start এবং stop দুটি সংখ্যা ইনপুট নিয়ে start থেকে stop পর্যন্ত সংখ্যা প্রিন্ট করুন।

2. ১ থেকে N পর্যন্ত শুধু জোড় সংখ্যাগুলো প্রিন্ট করুন।

3. ১ থেকে N পর্যন্ত ৫ এর গুণিতকগুলো প্রিন্ট করুন।

সমস্যা 62: N থেকে ১ পর্যন্ত উল্টো ক্রমে সংখ্যা প্রিন্ট করুন।

সমস্যা

একটি সংখ্যা N ইনপুট নিয়ে সেই সংখ্যা থেকে শুরু করে নিচের দিকে ১ পর্যন্ত উল্টো ক্রমে (Descending Order) সংখ্যাগুলো প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- range() এর step parameter
- Negative step (-1)

কীভাবে চিন্তা করব?

`range(start, stop, step)` ফাংশনে ৩টি আর্গুমেন্ট দেওয়া যায়। যদি আমরা ওপর থেকে নিচের দিকে বা উল্টো ক্রমে আসতে চাই, তবে আমাদের step এর মান -1 দিতে হবে। যেমন N থেকে ১ পর্যন্ত আসার জন্য আমাদের লিখতে হবে `range(N, 0, -1)` (যেখানে 0 এর আগের সংখ্যা ১ এ এসে লুপ থামবে)।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: N ইনপুট নেওয়া।

Step 3: for i in range(N, 0, -1) লুপ চালানো।

Step 4: i এর মান প্রিন্ট করা।

Python Code

```
# Printing numbers from N down to 1 in reverse order
N = int(input("Enter starting number (N): "))

print("Countdown from", N, "to 1:")
for i in range(N, 0, -1):
    print(i, end=" ")
```

Built-in Function / Shortcut Method

```
# Using reversed() function
# for i in reversed(range(1, N + 1)): print(i, end=" ")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter starting number (N): 10	Countdown from 10 to 1: 10 9 8 7 6 5 4 3 2 1

ব্যাখ্যা

`range(10, 0, -1)` লেখার কারণে লুপটি ১০ থেকে শুরু হয়েছে এবং প্রতিবার ১ করে কমে শেষপর্যন্ত ১ এ এসে থেমেছে।

Common Mistakes

- step হিসেবে `-1` না দেওয়া (`range(N, 0)` দিলে লুপ একবারও চলবে না)।
- stop মান হিসেবে ১ দেওয়া (`range(N, 1, -1)` দিলে ২ এ এসে থেমে যাবে, ১ প্রিন্ট হবে না)।

অনুশীলনের জন্য

1. ২০ থেকে ২ পর্যন্ত উল্টো ক্রমে শুধু জোড় সংখ্যাগুলো প্রিন্ট করুন (`range(20, 0, -2)`)।

2. ১০০ থেকে শূন্য পর্যন্ত ১০ করে কমিয়ে প্রিন্ট করুন (100, 90, 80...)।

3. while লুপ ব্যবহার করে একটি রকেট লঞ্চের কাউন্টডাউন (10 থেকে 0 এবং 'Blast Off!') তৈরি করুন।

সমস্যা 63: ১ থেকে ১০০ পর্যন্ত সংখ্যার যোগফল নির্ণয় করুন।

সমস্যা

১ থেকে ১০০ পর্যন্ত সকল পূর্ণসংখ্যার সমষ্টি বা যোগফল লুপ চালিয়ে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Accumulator Variable (total = 0)
- Loop summation Logic

কীভাবে চিন্তা করব?

ধারাবাহিক সংখ্যার যোগফল বের করার জন্য লুপ শুরু হওয়ার আগে আমাদের একটি ভেরিয়েবল `total = 0` রাখতে হবে (যাকে Accumulator বলে)। এরপর লুপে প্রতিটি সংখ্যা আসার পর আমরা সেই সংখ্যাটিকে `total` এর সাথে যোগ করে দেব: `total = total + i` বা `total += i`। লুপ শেষে total এর ভেতরেই মোট যোগফল থাকবে।

অ্যালগরিদম

Step 1: total নামের একটি ভেরিয়েবলের মান 0 রাখা।

Step 2: for লুপ চালিয়ে ১ থেকে ১০০ পর্যন্ত সংখ্যাগুলো নেওয়া।

Step 3: প্রতিবার total এর সাথে লুপের সংখ্যাটি যোগ করা (`total += i`)।

Step 4: লুপ শেষ হওয়ার পর total এর মান প্রদর্শন করা।

Python Code

```
# Finding sum of numbers from 1 to 100
total = 0

for i in range(1, 101):
    total += i

print("The sum of numbers from 1 to 100 is:", total)
```

Built-in Function / Shortcut Method

```
# Mathematical formula: n * (n + 1) // 2
# print("Sum:", 100 * 101 // 2)
```

```
# Or built-in sum()
# print("Sum:", sum(range(1, 101)))
```

Sample Input / Output Table

Sample Input	Sample Output
None	The sum of numbers from 1 to 100 is: 5050

ব্যাখ্যা

লুপটি ১ থেকে ১০০ পর্যন্ত ১০০ বার চলেছে। প্রতিবার নতুন সংখ্যাটি total এর সাথে যুক্ত হয়ে চূড়ান্ত যোগফল ৫০৫০ তৈরি করেছে।

Common Mistakes

- `total = 0` ভেরিয়েবলটি লুপের ভেতরে রাখা (যার ফলে প্রতিবার total শূন্য হয়ে যায় এবং শুধু শেষ সংখ্যা ১০০ উত্তর আসে)।
- `print(total)` লাইনটি লুপের ভেতরে (Indented) রাখা, যা প্রতিবার যোগফল প্রিন্ট করে। লুপের বাইরে রাখতে হবে।

অনুশীলনের জন্য

1. ১ থেকে ৫০ পর্যন্ত সংখ্যার যোগফল বের করুন।

2. sum() ফাংশন ব্যবহার করে ১ থেকে ৫০০ পর্যন্ত যোগফল বের করুন।

3. while লুপ ব্যবহার করে ১ থেকে ১০০ এর যোগফল বের করুন।

সমস্যা 64: ১ থেকে N পর্যন্ত সংখ্যার যোগফল নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি সংখ্যা N ইনপুট নিয়ে ১ থেকে N পর্যন্ত সকল সংখ্যার যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Accumulator Variable
- Dynamic range summation

কীভাবে চিন্তা করব?

আগের সমস্যার মতোই, শুধু ১০০ এর জায়গায় আমরা ব্যবহারকারীর ইনপুট করা `N` ব্যবহার করব এবং লুপটি `range(1, N + 1)` পর্যন্ত চালাব।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 ঘোষণা করা।

Step 3: for i in range(1, N + 1) চালিয়ে total += i করা।

Step 4: total প্রদর্শন করা।

Python Code

```
# Finding sum of numbers from 1 to N
N = int(input("Enter N: "))
total = 0

for i in range(1, N + 1):
    total += i

print("The sum from 1 to", N, "is:", total)
```

Built-in Function / Shortcut Method

```
# Mathematical shortcut formula
# print("Sum:", N * (N + 1) // 2)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 10	The sum from 1 to 10 is: 55

ব্যাখ্যা

১ থেকে ১০ পর্যন্ত সংখ্যাগুলো (১+২+৩+৪+৫+৬+৭+৮+৯+১০) যোগ হয়ে মোট ৫৫ হয়েছে।

Common Mistakes

- N এর মান যুক্ত না করা (`range(1, N)` দেওয়া)।
- total আপডেট না করে `total = i` লিখে ফেলা।

অনুশীলনের জন্য

1. ১ থেকে ১০০ এর বদলে ১০ থেকে N পর্যন্ত যোগফল বের করুন।

2. বীজগাণিতিক সূত্র `N * (N + 1) // 2` দিয়ে লুপ ছাড়াই যোগফল বের করুন।

3. ব্যবহারকারীর ইনপুট করা start এবং end এর মধ্যবর্তী সব সংখ্যার যোগফল বের করুন।

সমস্যা 65: ১ থেকে N পর্যন্ত জোড় সংখ্যার যোগফল নির্ণয় করুন।

সমস্যা

১ থেকে N পর্যন্ত যতগুলো জোড় (Even) সংখ্যা আছে, শুধুমাত্র তাদের যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- for loop
- Conditional if inside loop বা step parameter range(2, N+1, 2)

কীভাবে চিন্তা করব?

জোড় সংখ্যার যোগফল বের করার দুটি উপায় আছে: ১) ১ থেকে N পর্যন্ত লুপ চালিয়ে ভেতরে `if i % 2 == 0` হলে যোগ করা। অথবা ২) সবচেয়ে বুদ্ধিমানের কাজ হলো ২ থেকে শুরু করে ২ ঘর করে লাফ দিয়ে লুপ চালানো: `range(2, N + 1, 2)`। এতে লুপ শুধুমাত্র জোড় সংখ্যাগুলোতেই যায় এবং কোড দ্রুত কাজ করে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 সেট করা।

Step 3: for i in range(2, N + 1, 2) চালানো।

Step 4: total এর সাথে i যোগ করা।

Step 5: total প্রিন্ট করা।

Python Code

```
# Sum of Even numbers from 1 to N
N = int(input("Enter N: "))
total = 0

for i in range(2, N + 1, 2):
    total += i

print("Sum of even numbers from 1 to", N, "is:", total)
```

Built-in Function / Shortcut Method

```
# Using if condition inside normal range
# total = sum(i for i in range(1, N + 1) if i % 2 == 0)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 10	Sum of even numbers from 1 to 10

```
                                        is: 30
```

ব্যাখ্যা

১ থেকে ১০ এর মধ্যে জোড় সংখ্যাগুলো হলো ২, ৪, ৬, ৮ এবং ১০। এদের যোগফল (২+৪+৬+৮+১০) = ৩০।

Common Mistakes

- `range(2, N)` দেওয়া, যার ফলে N নিজে জোড় হলেও যোগ হয় না।
- step হিসেবে 2 না দিয়ে সাধারণ range দেওয়া যার ফলে বিজোড় সংখ্যাও যোগ হয়ে যায়।

অনুশীলনের জন্য

1. ১ থেকে ১০০ পর্যন্ত জোড় সংখ্যার যোগফল বের করুন।

2. if শর্ত ব্যবহার করে জোড় সংখ্যা যোগ করার কোড লিখুন।

3. জোড় সংখ্যাগুলোর যোগফলের পাশাপাশি মোট কয়টি জোড় সংখ্যা ছিল তা গণনা (Count) করুন।

সমস্যা 66: ১ থেকে N পর্যন্ত বিজোড় সংখ্যার যোগফল নির্ণয় করুন।

সমস্যা

১ থেকে N পর্যন্ত যতগুলো বিজোড় (Odd) সংখ্যা আছে, শুধুমাত্র তাদের যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(1, N+1, 2)
- Summation loop

কীভাবে চিন্তা করব?

বিজোড় সংখ্যা ১ থেকে শুরু হয় এবং প্রতিবার ২ করে বাড়ে (১, ৩, ৫, ৭...) তাই আমরা লুপটি ১ থেকে শুরু করে ২ ঘর করে লাফ দেব: `range(1, N + 1, 2)`।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(1, N + 1, 2) লুপ চালানো।

Step 4: total += i করা।

Step 5: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of Odd numbers from 1 to N
N = int(input("Enter N: "))
```

```
total = 0

for i in range(1, N + 1, 2):
    total += i

print("Sum of odd numbers from 1 to", N, "is:", total)
```

Built-in Function / Shortcut Method

```
# Using mathematical property: Sum of first k odd numbers is k^2
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 10	Sum of odd numbers from 1 to 10 is: 25

ব্যাখ্যা

১ থেকে ১০ এর মধ্যে বিজোড় সংখ্যা হলো ১, ৩, ৫, ৭, ৯। এদের যোগফল ২৫।

Common Mistakes

- লুপ শূন্য (0) বা ২ থেকে শুরু করা।
- step দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. ১ থেকে ৫০ পর্যন্ত বিজোড় সংখ্যার যোগফল বের করুন।

2. ১ থেকে N পর্যন্ত বিজোড় সংখ্যাগুলোর গড় বের করুন।

3. while লুপ ব্যবহার করে বিজোড় সংখ্যার যোগফল বের করুন।

সমস্যা 67: ১ থেকে N পর্যন্ত ৫ দ্বারা বিভাজ্য সংখ্যাগুলোর যোগফল নির্ণয় করুন।

সমস্যা

১ থেকে N পর্যন্ত যে সকল সংখ্যা ৫ দ্বারা নিঃশেষে বিভাজ্য (যেমন ৫, ১০, ১৫, ২০...), শুধুমাত্র তাদের যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(5, N+1, 5) বা if i % 5 == 0

কীভাবে চিন্তা করব?

আমরা চাইলে ১ থেকে N পর্যন্ত লুপ চালিয়ে ভেতরে `if i % 5 == 0` চেক করতে পারি। অথবা সরাসরি ৫ থেকে শুরু করে ৫ ঘর করে লাফ দিয়ে লুপ চালাতে পারি `range(5, N + 1, 5)`।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(5, N + 1, 5) লুপ চালানো।

Step 4: total এর সাথে i যোগ করা।

Step 5: ফলাফল প্রদর্শন করা।

Python Code

```
# Sum of numbers divisible by 5 from 1 to N
N = int(input("Enter N: "))
total = 0

for i in range(5, N + 1, 5):
    total += i

print("Sum of numbers divisible by 5 up to", N, "is:", total)
```

Built-in Function / Shortcut Method

```
# Using if check approach
# total = sum(i for i in range(1, N + 1) if i % 5 == 0)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 25	Sum of numbers divisible by 5 up to 25 is: 75

ব্যাখ্যা

১ থেকে ২৫ এর মধ্যে ৫ দ্বারা বিভাজ্য সংখ্যাগুলো হলো ৫, ১০, ১৫, ২০ এবং ২৫। এদের মোট যোগফল ৭৫।

Common Mistakes

- `range(1, N + 1, 5)` দেওয়া (যা ১, ৬, ১১... তৈরি করে যা ৫ দ্বারা বিভাজ্য নয়। অবশ্যই ৫ থেকে শুরু করতে হবে `range(5, ...)`)।

- N এর মান ৫ এর চেয়ে ছোট হলে কী হবে তা বিবেচনা না করা (কোড অবশ্য ক্র্যাশ করবে না, যোগফল ০ দেবে)।

অনুশীলনের জন্য

1. ১ থেকে ১০০ এর মধ্যে ৪ দ্বারা বিভাজ্য সংখ্যাগুলোর যোগফল বের করুন।

2. ১ থেকে N পর্যন্ত ১০ দ্বারা বিভাজ্য সংখ্যাগুলোর যোগফল বের করুন।

3. ১ থেকে N পর্যন্ত ৫ দ্বারা বিভাজ্য সংখ্যাগুলো যোগ না করে শুধু স্ক্রিনে প্রিন্ট করুন।

সমস্যা 68: ১ থেকে N পর্যন্ত ৭ দ্বারা বিভাজ্য সংখ্যাগুলোর যোগফল নির্ণয় করুন।

সমস্যা

১ থেকে N পর্যন্ত যে সকল সংখ্যা ৭ দ্বারা নিঃশেষে বিভাজ্য, তাদের সমষ্টি নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(7, N+1, 7)

কীভাবে চিন্তা করব?

৫ দ্বারা বিভাজ্য সমস্যার মতোই, আমরা এবার ৭ থেকে শুরু করে ৭ ঘর করে লাফ দেব: `range(7, N + 1, 7)`।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(7, N + 1, 7) লুপ চালানো।

Step 4: total += i করা।

Step 5: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of numbers divisible by 7 from 1 to N
N = int(input("Enter N: "))
total = 0

for i in range(7, N + 1, 7):
    total += i

print("Sum of numbers divisible by 7 up to", N, "is:", total)
```

Built-in Function / Shortcut Method

```
# Using comprehension
```

```
# print("Sum:", sum(x for x in range(1, N + 1) if x % 7 == 0))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 30	Sum of numbers divisible by 7 up to 30 is: 70

ব্যাখ্যা

১ থেকে ৩০ এর মধ্যে ৭ দ্বারা বিভাজ্য সংখ্যা হলো ৭, ১৪, ২১ এবং ২৮। এদের যোগফল ৭০।

Common Mistakes

- ৭ এর গুণিতকগুলো ঠিকমতো লুপে আনতে না পারা।
- লুপের শেষে প্রিন্ট না করা।

অনুশীলনের জন্য

1. ১ থেকে ২০০ এর মধ্যে ৯ দ্বারা বিভাজ্য সংখ্যাগুলোর যোগফল বের করুন।

2. ১ থেকে N পর্যন্ত ১৩ দ্বারা বিভাজ্য সংখ্যাগুলো প্রিন্ট করুন।

3. ৭ দ্বারা বিভাজ্য সংখ্যাগুলোর মোট সংখ্যা (Count) নির্ণয় করুন।

সমস্যা 69: ১ থেকে N পর্যন্ত ৩ এবং ৫ উভয় দ্বারা বিভাজ্য সংখ্যাগুলো প্রিন্ট করুন।

সমস্যা

১ থেকে N পর্যন্ত এমন সংখ্যাগুলো খুঁজে বের করতে হবে যা একই সাথে ৩ এবং ৫ উভয় সংখ্যা দিয়েই নিঃশেষে বিভাজ্য (যেমন ১৫, ৩০, ৪৫... ইত্যাদি)।

কী কী জানা থাকতে হবে?

- Logical AND (and)
- Modulus (%)
- if condition inside loop

কীভাবে চিন্তা করব?

কোনো সংখ্যা একই সাথে ৩ এবং ৫ উভয় দ্বারা বিভাজ্য হওয়ার অর্থ হলো সংখ্যাটি ৩ দ্বারা ভাগ করলেও ভাগশেষ ০ হবে এবং ৫ দ্বারা ভাগ করলেও ভাগশেষ ০ হবে (`i % 3 == 0 and i % 5 == 0`)। গাণিতিকভাবে, ৩ ও ৫ এর ল.সা.গু ১৫ হওয়ায় সংখ্যাগুলো মূলত ১৫ এর গুণিতক হয়। তবে নতুনদের জন্য `and` অপারেটর দিয়ে চেক করাই সবচেয়ে ভালো।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for লুপে ১ থেকে N পর্যন্ত প্রতিটি সংখ্যা i নেওয়া।

Step 3: if (i % 3 == 0) and (i % 5 == 0) শর্ত যাচাই করা।

Step 4: শর্ত সত্য হলে i এর মান প্রিন্ট করা।

Python Code

```
# Numbers divisible by both 3 and 5
N = int(input("Enter N: "))

print("Numbers divisible by both 3 and 5 up to", N, "are:")
for i in range(1, N + 1):
    if i % 3 == 0 and i % 5 == 0:
        print(i, end=" ")
```

Built-in Function / Shortcut Method

```
# Since LCM of 3 and 5 is 15, we can directly loop step 15
# for i in range(15, N + 1, 15): print(i, end=" ")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 50	Numbers divisible by both 3 and 5 up to 50 are: 15 30 45

ব্যাখ্যা

১ থেকে ৫০ এর মধ্যে শুধুমাত্র ১৫, ৩০ এবং ৪৫ সংখ্যাগুলোই ৩ এবং ৫ উভয় দ্বারা ভাগ করা যায়।

Common Mistakes

- `and` এর জায়গায় ভুল করে `or` দেওয়া (যার ফলে ৩ দিয়ে ভাগ যাওয়া সব সংখ্যা এবং ৫ দিয়ে ভাগ যাওয়া সব সংখ্যা চলে আসে)।
- ব্র্যাকেট না দিয়ে লজিক গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. ১ থেকে ১০০ এর মধ্যে ২ এবং ৩ উভয় দ্বারা বিভাজ্য সংখ্যাগুলো প্রিন্ট করুন।

2. ১ থেকে N পর্যন্ত ৩ অথবা ৫ যেকোনো একটি দ্বারা বিভাজ্য সংখ্যাগুলো প্রিন্ট করুন (`or` ব্যবহার করে)।

3. জনপ্রিয় 'FizzBuzz' সমস্যাটি সমাধান করুন (৩ দ্বারা বিভাজ্য হলে Fizz, ৫ দ্বারা হলে Buzz, উভয়ের দ্বারা হলে FizzBuzz)।

সমস্যা 70: একটি সংখ্যায় মোট কতটি Digit আছে তা নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি সংখ্যা ইনপুট নিয়ে সেই সংখ্যায় মোট কতটি অংক বা Digit আছে তা লুপ বা শর্টকাট পদ্ধতিতে গণনা করতে হবে (যেমন 4582 সংখ্যায় ৪টি ডিজিট আছে)।

কী কী জানা থাকতে হবে?

- while loop
- Floor Division (//)
- len() ও str() shortcut

কীভাবে চিন্তা করব?

গাণিতিক উপায়ে ডিজিট গোনার জন্য আমরা একটি while লুপ চালাতে পারি। প্রতিবার সংখ্যাটিকে ১০ দিয়ে Floor Division (`num = num // 10`) করলে সংখ্যার শেষ ডিজিটটি বাদ পড়ে যায়। এভাবে কতবার ভাগ করলে সংখ্যাটি শূন্য (0) হয়, সেই সংখ্যাটিই হলো মোট ডিজিট সংখ্যা। আর সবচেয়ে সহজ শর্টকাট হলো সংখ্যাটিকে String বানিয়ে তার দৈর্ঘ্য `len(str(num))` মেপে নেওয়া।

অ্যালগরিদম

Step 1: প্রোগ্রাম শুরু করা।

Step 2: একটি সংখ্যা ইনপুট নেওয়া এবং count = 0 রাখা।

Step 3: সংখ্যাটি 0 হলে count = 1 করা।

Step 4: while num > 0 লুপ চালিয়ে প্রতিবার num = num // 10 করা এবং count += 1 করা।

Step 5: count প্রদর্শন করা।

Python Code

```
# Counting digits in a number using while loop
num = int(input("Enter a positive integer: "))
count = 0

temp = abs(num) # Handling negative numbers
if temp == 0:
    count = 1
else:
    while temp > 0:
        temp = temp // 10
        count += 1
```

```
print("Total number of digits:", count)
```

Built-in Function / Shortcut Method

```
# String conversion shortcut (Most popular in Python)
# print("Total digits:", len(str(abs(int(input("Number: "))))))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a positive integer: 98765	Total number of digits: 5

ব্যাখ্যা

৯৮৭৬৫ কে ১০ দিয়ে ভাগ করতে করতে ৫ বার ভাগ করার পর সংখ্যাটি শূন্য হয়েছে। তাই লুপটি ৫ বার ঘুরে count এর মান ৫ বানিয়েছে।

Common Mistakes

- ইনপুট হিসেবে শূন্য (0) দিলে লুপ না ঘুরে আউটপুট 0 ডিজিট দেখানো (শূন্য নিজেও ১টি ডিজিট)।
- ঋণাত্মক সংখ্যা দিলে লুপ অসীম হওয়া বা ভুল দেওয়া (তাই `abs()` করা জরুরি)।

অনুশীলনের জন্য

1. `len(str(num))` ব্যবহার করে এক লাইনে ডিজিট গণনা করুন।

2. একটি সংখ্যায় জোড় ডিজিট কয়টি এবং বিজোড় ডিজিট কয়টি আছে তা আলাদাভাবে গণনা করুন।

3. কোনো সংখ্যার প্রথম ডিজিট এবং শেষ ডিজিট বের করুন।

সমস্যা 71: একটি সংখ্যার সব Digit-এর যোগফল নির্ণয় করুন।

সমস্যা

একটি পূর্ণসংখ্যা ইনপুট নিয়ে তার ভিতরের সকল অংক বা ডিজিটের যোগফল নির্ণয় করতে হবে (যেমন 1234 এর ডিজিটগুলোর যোগফল 1+2+3+4 = 10)।

কী কী জানা থাকতে হবে?

- Modulus (% 10)
- Floor Division (// 10)
- while loop

কীভাবে চিন্তা করব?

যেকোনো সংখ্যাকে ১০ দিয়ে ভাগশেষ (`num % 10`) করলে তার একদম শেষ ডিজিটটি পাওয়া যায় (যেমন ১২৩৪ % ১০ = ৪)। আমরা এই শেষ ডিজিটটি যোগফলের সাথে যুক্ত করব। তারপর সংখ্যাটিকে ১০ দিয়ে Floor Division (`num // 10`) করে শেষ ডিজিটটি বাদ দিয়ে দেব (১২৩৪ হয়ে যাবে ১২৩)। এভাবে সংখ্যাটি শূন্য হওয়া পর্যন্ত লুপ চালাব।

অ্যালগরিদম

Step 1: সংখ্যাটি ইনপুট নেওয়া।

Step 2: sum_digits = 0 রাখা।

Step 3: while num > 0 লুপ চালানো।

Step 4: last_digit = num % 10 বের করে sum_digits এর সাথে যোগ করা।

Step 5: num = num // 10 করে সংখ্যাটি ছোট করা।

Step 6: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of all digits of a number
num = int(input("Enter a number: "))
temp = abs(num)
sum_digits = 0

while temp > 0:
    digit = temp % 10
    sum_digits += digit
    temp = temp // 10

print("Sum of digits is:", sum_digits)
```

Built-in Function / Shortcut Method

```
# Pythonic shortcut using list comprehension / generator
# print("Sum:", sum(int(d) for d in str(abs(num))))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 456	Sum of digits is: 15

ব্যাখ্যা

৪৫৬ এর ডিজিটগুলো হলো ৪, ৫ এবং ৬। এদের যোগফল (৪+৫+৬) = ১৫।

Common Mistakes

- `temp % 10` এর জায়গায় `temp / 10` করা।
- লুপের ভেতর `temp = temp // 10` করতে ভুলে যাওয়া (যার ফলে Infinite Loop তৈরি হয়)।

অনুশীলনের জন্য

1. একটি সংখ্যার সব ডিজিটের গুণফল (Product of digits) বের করুন।

2. শুধুমাত্র জোড় ডিজিটগুলোর যোগফল বের করুন।

3. String conversion ও for লুপ ব্যবহার করে ডিজিট যোগ করুন।

সমস্যা 72: একটি সংখ্যাকে উল্টো (Reverse) করে প্রদর্শন করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে তার ডিজিটগুলোর ক্রম উল্টো করে নতুন সংখ্যা তৈরি ও প্রদর্শন করতে হবে (যেমন 1234 দিলে আউটপুট হবে 4321)।

কী কী জানা থাকতে হবে?

- Number reversal algorithm
- while loop
- reversed_num = reversed_num * 10 + digit

কীভাবে চিন্তা করব?

আগের সমস্যার মতোই আমরা প্রতিবার `num % 10` দিয়ে শেষ ডিজিটটি বের করব। নতুন উল্টো সংখ্যা তৈরির জন্য একটি ভেরিয়েবল `rev = 0` রাখব। প্রতিবার লুপে `rev = rev * 10 + digit` করব। যেমন প্রথমে ৪, তারপর ৪x১০+৩=৪৩, তারপর ৪৩x১০+২=৪৩২, শেষে ৪৩২১। আর স্ট্রিং স্লাইসিং `str(num)[::-1]` দিয়ে এক লাইনেই করা যায়।

অ্যালগরিদম

Step 1: সংখ্যা ইনপুট নেওয়া।

Step 2: rev_num = 0 রাখা।

Step 3: while num > 0 লুপ চালানো।

Step 4: digit = num % 10 বের করা।

Step 5: rev_num = (rev_num * 10) + digit করা।

Step 6: num = num // 10 করা।

Step 7: rev_num প্রিন্ট করা।

Python Code

```
# Reversing a number using while loop
```

```
num = int(input("Enter a number: "))
temp = abs(num)
rev_num = 0

while temp > 0:
    digit = temp % 10
    rev_num = (rev_num * 10) + digit
    temp = temp // 10

if num < 0:
    rev_num = -rev_num

print("Reversed number is:", rev_num)
```

Built-in Function / Shortcut Method

```
# String slicing shortcut [::-1]
# print("Reversed:", int(str(num)[::-1]))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 3579	Reversed number is: 9753

ব্যাখ্যা

৩৫৭৯ সংখ্যার শেষ ডিজিট থেকে শুরু করে প্রতিবার ১০ দিয়ে গুণ করে নতুন ডিজিট যোগ করার মাধ্যমে ৯৭৫৩ উল্টো সংখ্যাটি গঠিত হয়েছে।

Common Mistakes

- `rev_num * 10` না করে শুধু ডিজিট যোগ করে ফেলা (যার ফলে উল্টো না হয়ে ডিজিটগুলোর যোগফল বের হয়ে যায়)।
- শেষে শূন্য থাকা সংখ্যা (যেমন 1200) রিভার্স করলে গাণিতিক নিয়মে 21 আসে (যা সঠিক), কিন্তু স্ট্রিংয়ে করলে 0021 আসতে পারে।

অনুশীলনের জন্য

1. স্ট্রিং স্লাইসিং `[::-1]` ব্যবহার করে একটি সংখ্যা উল্টো করুন।

2. উল্টো করার পর আসল সংখ্যা ও উল্টো সংখ্যার পার্থক্য (Difference) বের করুন।

3. একটি তিন অঙ্কের সংখ্যা ইনপুট নিয়ে রিভার্স করুন।

সমস্যা 73: break স্টেটমেন্ট ব্যবহার করে Loop থেকে বের হয়ে আসার উদাহরণ তৈরি করুন।

সমস্যা

break স্টেটমেন্টের কাজ বোঝানোর জন্য একটি লুপ তৈরি করুন, যা নির্দিষ্ট কোনো শর্ত (যেমন ৫ পাওয়া গেলে বা নেগেটিভ সংখ্যা ইনপুট দিলে) পূরণ হলেই সাথে সাথে লুপ বন্ধ করে সম্পূর্ণ বের হয়ে আসবে।

কী কী জানা থাকতে হবে?

- break স্টেটমেন্ট
- Loop Control Statements

কীভাবে চিন্তা করব?

`break` কি-ওয়ার্ডটির কাজ হলো লুপের স্বাভাবিক চলন মাঝপথেই জোরপূর্বক থামিয়ে দেওয়া। যেমন আমরা ১ থেকে ১০ পর্যন্ত প্রিন্ট করতে চাই, কিন্তু যদি লুপের মান ৫ হয়, তবে আমরা `break` দিয়ে লুপ বন্ধ করে দেব। ফলে ৫ এর পরের সংখ্যাগুলো আর প্রিন্ট হবে না।

অ্যালগরিদম

Step 1: ১ থেকে ১০ পর্যন্ত for লুপ চালানো।

Step 2: লুপের ভেতর if i == 5 শর্ত দেওয়া।

Step 3: শর্ত সত্য হলে break কি-ওয়ার্ড এক্সিকিউট করা।

Step 4: অন্যথায় i এর মান প্রিন্ট করা।

Python Code

```
# Demonstrating break statement
print("Looping from 1 to 10, but stopping when 5 is reached:")

for i in range(1, 11):
    if i == 5:
        print("\nCondition (i == 5) met! Breaking the loop.")
        break
    print(i, end=" ")

print("Program finished.")
```

Built-in Function / Shortcut Method

```
# Break inside infinite while loop
# while True:
#     if input("Type 'exit' to stop: ") == 'exit': break
```

Sample Input / Output Table

Sample Input	Sample Output

None	`Looping from 1 to 10, but stopping when 5 is reached:`
	`1 2 3 4`
	`Condition (i == 5) met! Breaking the loop.`
	`Program finished.`

ব্যাখ্যা

লুপটি ১ থেকে ৪ পর্যন্ত স্বাভাবিকভাবে চলেছে। কিন্তু যখনই `i == 5` হয়েছে, `break` স্টেটমেন্টটি কার্যকর হয়ে লুপটি সম্পূর্ণরূপে বন্ধ করে দিয়েছে।

Common Mistakes

- `break` লেখার পর কোনো কোড লেখা (break এর নিচের লাইনে লুপের ভেতর কিছু লিখলে তা আর কখনো এক্সিকিউট হয় না—Unreachable code)।
- break এবং continue এর পার্থক্য বুঝতে না পারা।

অনুশীলনের জন্য

1. একটি অসীম লুপ (`while True`) তৈরি করুন যা ব্যবহারকারী 'q' না লেখা পর্যন্ত চলতে থাকবে।

2. ১ থেকে ১০০ এর মধ্যে প্রথম কোন সংখ্যাটি ১৭ দ্বারা বিভাজ্য তা খুঁজে বের করে break দিয়ে লুপ থামান।

3. লিস্টে কোনো নির্দিষ্ট উপাদান খোঁজার পর break করার কোড লিখুন।

সমস্যা 74: continue স্টেটমেন্ট ব্যবহার করে নির্দিষ্ট Iteration বাদ দিয়ে Loop চালিয়ে যান।

সমস্যা

continue স্টেটমেন্ট ব্যবহার করে লুপের চলমান Iteration বা ধাপটি স্কিপ (Skip) করে পরবর্তী ধাপে চলে যাওয়ার একটি স্পষ্ট উদাহরণ তৈরি করুন।

কী কী জানা থাকতে হবে?

- continue স্টেটমেন্ট

কীভাবে চিন্তা করব?

`break` যেমন পুরো লুপ বন্ধ করে দেয়, `continue` কিন্তু লুপ বন্ধ করে না। এটি শুধুমাত্র বর্তমান ধাপের নিচের কোডটুকু স্কিপ করে লুপের পরবর্তী ধাপে চলে যায়। যেমন আমরা ১ থেকে ১০ প্রিন্ট করতে চাই, কিন্তু ৫ সংখ্যাটিকে বাদ দিতে চাই। তখন `if i == 5: continue` লিখলে ৫ প্রিন্ট না হয়ে লুপ ৬ এ চলে যাবে।

অ্যালগরিদম

Step 1: ১ থেকে ১০ পর্যন্ত for লুপ চালানো।

Step 2: if i == 5 হলে continue এক্সিকিউট করা।

Step 3: লুপের ভেতর print(i) রাখা (যা continue হলে চলবে না)।

Python Code

```
# Demonstrating continue statement
print("Printing 1 to 10 except the number 5:")

for i in range(1, 11):
    if i == 5:
        continue  # Skips the rest of the loop block for i = 5
    print(i, end=" ")
```

Built-in Function / Shortcut Method

```
# Filtering using comprehension instead of continue
# print([x for x in range(1, 11) if x != 5])
```

Sample Input / Output Table

Sample Input	Sample Output
None	Printing 1 to 10 except the number 5: 1 2 3 4 6 7 8 9 10

ব্যাখ্যা

আউটপুটে লক্ষ্য করুন ৪ এর পর সরাসরি ৬ এসেছে। ৫ সংখ্যাটি স্কিপ বা বাদ পড়ে গেছে কারণ `continue` স্টেটমেন্টটি `print(i)` লাইনটি চালাতে দেয়নি।

Common Mistakes

- while লুপে `continue` ব্যবহার করার সময় কাউন্টার (`i += 1`) continue এর নিচে রাখা (যার ফলে লুপ একই স্থানে আটকে Infinite loop হয়ে যায়)।
- continue দিয়ে পুরো লুপ বন্ধ হবে মনে করা।

অনুশীলনের জন্য

1. ১ থেকে ২০ এর মধ্যে ৩ দ্বারা বিভাজ্য সংখ্যাগুলো `continue` দিয়ে স্কিপ করে বাকিগুলো প্রিন্ট করুন।

2. continue ব্যবহার করে ১ থেকে ১০ এর মধ্যে শুধু বিজোড় সংখ্যাগুলো প্রিন্ট করুন (জোড় হলে continue)।

3. একটি স্ট্রিং প্রিন্ট করার সময় স্পেসগুলো continue দিয়ে বাদ দিন।

Topic: Number Theory Problems

সমস্যা 75: একটি সংখ্যার Factorial নির্ণয় করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একটি সংখ্যা N ইনপুট নিয়ে তার ফ্যাক্টোরিয়াল (Factorial: N!) নির্ণয় করতে হবে। যেমন 5! = 5 x 4 x 3 x 2 x 1 = 120।

কী কী জানা থাকতে হবে?

- Accumulator Variable (fact = 1)
- for loop / math.factorial()

কীভাবে চিন্তা করব?

ফ্যাক্টোরিয়াল হলো ১ থেকে শুরু করে ওই সংখ্যা পর্যন্ত সকল পূর্ণসংখ্যার গুণফল। যেহেতু আমরা গুণ করব, তাই আমাদের শুরুর ভেরিয়েবল `fact` এর মান অবশ্যই 1 রাখতে হবে (0 রাখলে সব গুণফল 0 হয়ে যাবে)। এরপর ১ থেকে N পর্যন্ত লুপ চালিয়ে প্রতিবার `fact = fact * i` করব। মনে রাখতে হবে, 0! এর মান সবসময় 1।

অ্যালগরিদম

Step 1: সংখ্যা N ইনপুট নেওয়া।

Step 2: if N < 0 হলে ফ্যাক্টোরিয়াল সম্ভব নয় জানানো।

Step 3: elif N == 0 হলে 1 প্রদর্শন করা।

Step 4: else fact = 1 রেখে ১ থেকে N পর্যন্ত লুপ চালিয়ে fact *= i করা।

Step 5: ফলাফল প্রিন্ট করা।

Python Code

```
# Finding Factorial of a number
num = int(input("Enter a positive integer: "))

if num < 0:
    print("Factorial does not exist for negative numbers")
elif num == 0:
    print("The factorial of 0 is 1")
else:
    fact = 1
    for i in range(1, num + 1):
        fact *= i
    print("The factorial of", num, "is:", fact)
```

Built-in Function / Shortcut Method

```
# Using built-in math.factorial()
# import math
# print("Factorial:", math.factorial(num))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a positive integer: 5	The factorial of 5 is: 120

ব্যাখ্যা

১ x ২ x ৩ x ৪ x ৫ হিসাব করে মোট ১২০ গুণফল নির্ণয় করা হয়েছে।

Common Mistakes

- `fact = 0` রাখা, যার ফলে যেকোনো সংখ্যার ফ্যাক্টোরিয়াল শূন্য হয়ে যায়।
- ঋণাত্মক সংখ্যার ফ্যাক্টোরিয়াল বের করার চেষ্টা করা।

অনুশীলনের জন্য

1. while লুপ ব্যবহার করে ফ্যাক্টোরিয়াল বের করুন।

2. math.factorial() ফাংশন ব্যবহার করে ১০ এর ফ্যাক্টোরিয়াল বের করুন।

3. ১ থেকে ১০ পর্যন্ত প্রতিটি সংখ্যার ফ্যাক্টোরিয়াল একটি টেবিলে প্রিন্ট করুন।

সমস্যা 76: N-তম Fibonacci সংখ্যা নির্ণয় করুন।

সমস্যা

ফিবোনাচ্চি ধারার (Fibonacci Series: 0, 1, 1, 2, 3, 5, 8...) N-তম পদ বা সংখ্যাটি নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Fibonacci logic: next_num = a + b
- Variable Swapping

কীভাবে চিন্তা করব?

ফিবোনাচ্চি ধারার প্রথম দুটি সংখ্যা হলো 0 এবং 1। পরবর্তী প্রতিটি সংখ্যা তার আগের দুটি সংখ্যার যোগফলের সমান (যেমন 0+1=1, 1+1=2, 1+2=3, 2+3=5)। আমরা দুটি ভেরিয়েবল a = 0 এবং b = 1 দিয়ে শুরু করব এবং লুপ চালিয়ে a ও b এর মান আপডেট করব।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: if N == 1 হলে 0 এবং N == 2 হলে 1 প্রিন্ট করা।

Step 3: else a = 0 এবং b = 1 সেট করে ৩ থেকে N পর্যন্ত লুপ চালানো।

Step 4: প্রতিবার c = a + b করা এবং a = b, b = c করে এগিয়ে যাওয়া।

Step 5: b এর মান প্রদর্শন করা।

Python Code

```
# Finding the N-th Fibonacci number
N = int(input("Enter which Fibonacci term you want (N): "))

if N <= 0:
    print("Please enter a positive integer greater than 0")
elif N == 1:
    print("The 1st Fibonacci number is: 0")
elif N == 2:
    print("The 2nd Fibonacci number is: 1")
else:
    a, b = 0, 1
    for i in range(3, N + 1):
        c = a + b
        a, b = b, c
    print("The", N, "th Fibonacci number is:", b)
```

Built-in Function / Shortcut Method

```
# Using recursion or golden ratio formula (Binet's Formula)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter which Fibonacci term you want (N): 7	The 7 th Fibonacci number is: 8

ব্যাখ্যা

ফিবোনাচ্চি ধারার প্রথম ৭টি পদ হলো: 0, 1, 1, 2, 3, 5, 8। তাই ৭ম পদ হিসেবে 8 আউটপুট এসেছে।

Common Mistakes

- প্রথম পদ 0 এর বদলে 1 ধরে হিসাব শুরু করা।
- ভেরিয়েবল আপডেট করার সময় a ও b এর মান এলোমেলো করা।

অনুশীলনের জন্য

1. ফিবোনাচ্চি ধারার প্রথম Nটি সংখ্যা প্রিন্ট করুন।

2. ১০০ এর চেয়ে ছোট সকল ফিবোনাচ্চি সংখ্যা প্রিন্ট করুন।

3. একটি ইনপুট করা সংখ্যা ফিবোনাচ্চি ধারায় আছে কিনা যাচাই করুন।

সমস্যা 77: একটি সংখ্যা Prime কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি মৌলিক সংখ্যা (Prime Number) কিনা তা নির্ণয় করতে হবে। যে সংখ্যা ১ এবং ওই সংখ্যা ছাড়া অন্য কোনো সংখ্যা দ্বারা নিঃশেষে বিভাজ্য নয়, তাকে মৌলিক সংখ্যা বলে।

কী কী জানা থাকতে হবে?

- Prime Number logic
- break স্টেটমেন্ট
- Boolean flag বা for-else

কীভাবে চিন্তা করব?

১ মৌলিক সংখ্যা নয়, তাই সংখ্যাটি অবশ্যই ১ এর চেয়ে বড় হতে হবে। আমরা ২ থেকে শুরু করে ওই সংখ্যার বর্গমূল বা পূর্ববর্তী সংখ্যা (`num // 2`) পর্যন্ত লুপ চালিয়ে দেখব কোনো সংখ্যা দিয়ে নিঃশেষে ভাগ করা যায় কিনা। যদি কোনো সংখ্যা দিয়ে ভাগ চলে যায় (`num % i == 0`), তবে সেটি Prime নয়। আর একবারও ভাগ না গেলে সেটি Prime।

> **[Best Practice]** মৌলিক সংখ্যা যাচাই করার জন্য ২ থেকে num পর্যন্ত না চালিয়ে `num ** 0.5` (বর্গমূল) পর্যন্ত চালালে কোড হাজার গুণ দ্রুত কাজ করে।

অ্যালগরিদম

Step 1: সংখ্যা num ইনপুট নেওয়া।

Step 2: if num <= 1 হলে 'Not Prime' প্রিন্ট করা।

Step 3: else is_prime = True ধরে ২ থেকে num এর পূর্ব পর্যন্ত লুপ চালানো।

Step 4: যদি num % i == 0 হয়, তবে is_prime = False করে break করা।

Step 5: লুপ শেষে is_prime True হলে Prime, না হলে Not Prime প্রদর্শন করা।

Python Code

```
# Checking if a number is Prime
num = int(input("Enter a number: "))
```

```
if num <= 1:
    print(num, "is NOT a Prime number")
else:
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        print(num, "is a Prime number")
    else:
        print(num, "is NOT a Prime number")
```

Built-in Function / Shortcut Method

```
# Using sympy module
# import sympy
# print("Is Prime:", sympy.isprime(num))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 29	29 is a Prime number

ব্যাখ্যা

২৯ কে ২ থেকে ৫ (২৯ এর বর্গমূলের কাছাকাছি) পর্যন্ত কোনো সংখ্যা দ্বারাই ভাগ করা যায় না। তাই এটি একটি মৌলিক বা Prime সংখ্যা।

Common Mistakes

- ১ কে মৌলিক সংখ্যা হিসেবে আউটপুট দেওয়া (১ মৌলিক বা যৌগিক কোনোটিই নয়)।
- ২ কে যৌগিক বলা (২ হলো একমাত্র জোড় মৌলিক সংখ্যা)।

অনুশীলনের জন্য

1. sympy.isprime() ব্যবহার করে মৌলিক সংখ্যা যাচাই করুন।

2. ২ থেকে ১০০ এর মধ্যে কয়টি মৌলিক সংখ্যা আছে তা বের করুন।

3. একটি সংখ্যার নিকটবর্তী মৌলিক সংখ্যাটি খুঁজে বের করুন।

সমস্যা 78: ১ থেকে N পর্যন্ত সব Prime Number প্রদর্শন করুন।

সমস্যা

১ থেকে N পর্যন্ত যতগুলো মৌলিক সংখ্যা আছে, তাদের সবাইকে স্ক্রিনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Nested Loop (লুপের ভেতর লুপ)
- Prime validation logic

কীভাবে চিন্তা করব?

আমরা বাইরে একটি লুপ চালাব যা ২ থেকে N পর্যন্ত প্রতিটি সংখ্যা `num` নেবে। এরপর ভিতরের লুপটি দিয়ে আমরা যাচাই করব সেই `num` টি মৌলিক কিনা। মৌলিক হলে প্রিন্ট করব।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for num in range(2, N + 1) লুপ চালানো।

Step 3: প্রতিটি num এর জন্য is_prime পরীক্ষা করা।

Step 4: মৌলিক হলে প্রিন্ট করা।

Python Code

```
# Printing all Prime numbers from 1 to N
N = int(input("Enter upper limit (N): "))

print("Prime numbers between 1 and", N, "are:")
for num in range(2, N + 1):
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
```

Built-in Function / Shortcut Method

```
# Using list comprehension and all()
# print([x for x in range(2, N+1) if all(x % y != 0 for y in range(2,
int(x**0.5)+1))])
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter upper limit (N): 30          Prime numbers between 1 and 30
                                   are:
                                   2 3 5 7 11 13 17 19 23 29
```

ব্যাখ্যা

২ থেকে ৩০ এর মধ্যে মোট ১০টি মৌলিক সংখ্যা খুঁজে বের করে প্রদর্শন করা হয়েছে।

Common Mistakes

- Nested loop এর Indentation ভুল করা।
- ভিতরের লুপে `is_prime` ভেরিয়েবলটি প্রতিবার True করতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. ১ থেকে ১০০ এর মধ্যে সকল মৌলিক সংখ্যার যোগফল বের করুন।

2. ১ থেকে N পর্যন্ত মৌলিক সংখ্যাগুলোর মোট সংখ্যা গণনা করুন।

3. Sieve of Eratosthenes অ্যালগরিদম সম্পর্কে জেনে তা দিয়ে প্রাইম বের করুন।

সমস্যা 79: দুটি সংখ্যার GCD নির্ণয় করুন।

সমস্যা

দুটি সংখ্যার গরিষ্ঠ সাধারণ গুণনীয়ক বা গ.সা.গু (Greatest Common Divisor - GCD / HCF) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Euclidean Algorithm
- math.gcd()

কীভাবে চিন্তা করব?

গ.সা.গু হলো সেই সবচেয়ে বড় সংখ্যা যা দিয়ে উভয় সংখ্যাকেই নিঃশেষে ভাগ করা যায়। ইউক্লিডের পদ্ধতি অনুযায়ী, ছোট সংখ্যা দিয়ে বড় সংখ্যাকে ভাগ করার পর যে ভাগশেষ থাকে, সেই ভাগশেষ দিয়ে আবার ভাজককে ভাগ করতে হয়। ভাগশেষ শূন্য না হওয়া পর্যন্ত এই প্রক্রিয়া চালিয়ে গেলে শেষ ভাজকটিই হলো গ.সা.গু।

অ্যালগরিদম

Step 1: দুটি সংখ্যা a ও b ইনপুট নেওয়া।

Step 2: while b != 0 লুপ চালানো।

Step 3: a, b = b, a % b করে মান আপডেট করা।

Step 4: লুপ শেষে a এর মান প্রদর্শন করা।

Python Code

```
# Finding GCD using Euclidean Algorithm
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

a, b = num1, num2
while b != 0:
    a, b = b, a % b

print("The GCD of", num1, "and", num2, "is:", a)
```

Built-in Function / Shortcut Method

```
# Using built-in math.gcd()
# import math
# print("GCD:", math.gcd(num1, num2))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter first number: 48 Enter second number: 18	The GCD of 48 and 18 is: 6

ব্যাখ্যা

৪৮ এবং ১৮ উভয়কেই ৬ দ্বারা নিঃশেষে ভাগ করা যায় এবং ৬-ই সবচেয়ে বড় সংখ্যা। তাই গ.সা.গু ৬।

Common Mistakes

- লুপের শর্ত ভুল দেওয়া।
- মান অদলবদল করার সময় temp ভেরিয়েবল বা tuple unpacking ঠিকমতো না করা।

অনুশীলনের জন্য

1. math.gcd() ব্যবহার করে ৩টি সংখ্যার গ.সা.গু বের করুন।

2. ১ থেকে শুরু করে দুটি সংখ্যার ছোটটি পর্যন্ত লুপ চালিয়ে সাধারণ পদ্ধতিতে গ.সা.গু বের করুন।

3. দুটি সংখ্যার গ.সা.গু ১ হলে তাদের সহমৌলিক (Co-prime) প্রমাণ করুন।

সমস্যা 80: দুটি সংখ্যার LCM নির্ণয় করুন।

সমস্যা

দুটি সংখ্যার লঘিষ্ঠ সাধারণ গুণিতক বা ল.সা.গু (Least Common Multiple - LCM) নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- LCM formula: (a * b) // GCD(a, b)
- math.lcm()

কীভাবে চিন্তা করব?

আমরা জানি, দুটি সংখ্যার গুণফল তাদের গ.সা.গু এবং ল.সা.গু এর গুণফলের সমান (`a * b = GCD * LCM`)। তাই আমরা খুব সহজেই গ.সা.গু বের করে দুটি সংখ্যার গুণফলকে গ.সা.গু দিয়ে ভাগ করলেই ল.সা.গু পেয়ে যাব। সূত্র: `LCM = (num1 * num2) // GCD`।

অ্যালগরিদম

Step 1: দুটি সংখ্যা ইনপুট নেওয়া।

Step 2: ইউক্লিডের পদ্ধতিতে GCD বের করা।

Step 3: lcm = (num1 * num2) // gcd সূত্র প্রয়োগ করা।

Step 4: ল.সা.গু প্রিন্ট করা।

Python Code

```
# Finding LCM using GCD
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

# Finding GCD first
a, b = num1, num2
while b != 0:
    a, b = b, a % b
gcd = a

lcm = (num1 * num2) // gcd
print("The LCM of", num1, "and", num2, "is:", lcm)
```

Built-in Function / Shortcut Method

```
# Using built-in math.lcm() (Python 3.9+)
# import math
# print("LCM:", math.lcm(num1, num2))
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter first number: 12` `Enter second number: 15`	`The LCM of 12 and 15 is: 60`

ব্যাখ্যা

১২ এবং ১৫ উভয়ের গুণিতকগুলোর মধ্যে সবচেয়ে ছোট সাধারণ গুণিতক হলো ৬০ (১২x৫=৬০, ১৫x৪=৬০)।

Common Mistakes

- সাধারণ ভাগ (`/`) ব্যবহার করা যার ফলে আউটপুটে দশমিক `.0` চলে আসে।
- GCD শূন্য হলে ZeroDivisionError হওয়া (যদিও ধনাত্মক সংখ্যার ক্ষেত্রে হয় না)।

অনুশীলনের জন্য

1. math.lcm() ব্যবহার করে ৩টি সংখ্যার ল.সা.গু বের করুন।

2. বড় সংখ্যাটি থেকে শুরু করে ১ করে বাড়িয়ে লুপ চালিয়ে গাণিতিক উপায়ে ল.সা.গু বের করুন।

3. দুটি মৌলিক সংখ্যার ল.সা.গু যে তাদের গুণফলের সমান তা কোড দিয়ে প্রমাণ করুন।

সমস্যা 81: একটি সংখ্যা Perfect Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি Perfect Number কিনা তা যাচাই করতে হবে। কোনো সংখ্যার প্রকৃত গুণনীয়কগুলোর (নিজের সংখ্যাটি ছাড়া বাকি ভাজকগুলো) যোগফল যদি মূল সংখ্যাটির সমান হয়, তবে তাকে Perfect Number বলে (যেমন 6 এর ভাজক 1, 2, 3 এবং এদের যোগফল 1+2+3 = 6)।

কী কী জানা থাকতে হবে?

- Divisor / Factor finding
- Loop summation

কীভাবে চিন্তা করব?

আমরা ১ থেকে শুরু করে `num // 2` পর্যন্ত লুপ চালাব। যেসব সংখ্যা দিয়ে আমাদের `num` নিঃশেষে ভাগ যায়, সেগুলোকে একটি ভেরিয়েবলে যোগ করব। লুপ শেষে যদি যোগফল আমাদের আসল সংখ্যার সমান হয়, তবে সেটি Perfect Number।

অ্যালগরিদম

Step 1: সংখ্যা num ইনপুট নেওয়া।

Step 2: sum_divisors = 0 রাখা।

Step 3: ১ থেকে num // 2 পর্যন্ত লুপ চালিয়ে ভাজকগুলো বের করা ও যোগ করা।

Step 4: if sum_divisors == num হলে Perfect Number প্রিন্ট করা।

Python Code

```
# Checking Perfect Number
num = int(input("Enter a positive integer: "))

if num <= 0:
    print("Please enter a positive number greater than 0")
else:
    sum_divisors = 0
    for i in range(1, (num // 2) + 1):
        if num % i == 0:
            sum_divisors += i

    if sum_divisors == num:
        print(num, "is a Perfect Number")
    else:
        print(num, "is NOT a Perfect Number")
```

Built-in Function / Shortcut Method

```
# Comprehension sum
# is_perfect = sum(i for i in range(1, num) if num % i == 0) == num
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a positive integer: 28	28 is a Perfect Number

ব্যাখ্যা

২৮ এর প্রকৃত গুণনীয়কগুলো হলো ১, ২, ৪, ৭ এবং ১৪। এদের যোগফল (১+২+৪+৭+১৪) = ২৮, যা আসল সংখ্যার সমান। তাই এটি Perfect Number।

Common Mistakes

- লুপ চালানোর সময় আসল সংখ্যাটি (`num`) নিজেও গুণনীয়ক হিসেবে যোগ করে ফেলা।
- ১ কে Perfect Number মনে করা।

অনুশীলনের জন্য

1. ১ থেকে ১০০০ এর মধ্যে সব Perfect Number খুঁজে বের করুন (দেখবেন শুধু ৬ এবং ২৮ আসবে)।

2. পরবর্তী Perfect Number ৪৯৬ যাচাই করুন।

3. প্রকৃত ভাজকদের যোগফল মূল সংখ্যার চেয়ে বেশি হলে তাকে Abundant Number বলা হয়—তা যাচাই করুন।

সমস্যা 82: একটি সংখ্যা Armstrong Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি Armstrong Number কিনা তা নির্ণয় করতে হবে। কোনো সংখ্যার প্রতিটি ডিজিটের ওপর মোট ডিজিট সংখ্যার পাওয়ার বা ঘাত দিয়ে যোগ করলে যদি যোগফল মূল সংখ্যার সমান হয়, তবে তাকে Armstrong Number বলে (যেমন 153 একটি ৩ ডিজিটের সংখ্যা এবং $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$)।

কী কী জানা থাকতে হবে?

- Digit separation
- Exponentiation (**)
- Digit count

কীভাবে চিন্তা করব?

প্রথমে আমাদের দেখতে হবে সংখ্যাটিতে কয়টি ডিজিট আছে (ধরি `p` টি)। এরপর প্রতিটি ডিজিট আলাদা করে তার পাওয়ার `p` বের করতে হবে এবং সবগুলোর যোগফল করতে হবে। যোগফল আসল সংখ্যার সমান হলে Armstrong।

অ্যালগরিদম

Step 1: সংখ্যাটি ইনপুট নেওয়া।

Step 2: ডিজিট সংখ্যা p = len(str(num)) বের করা।

Step 3: while লুপ চালিয়ে প্রতিটি ডিজিট আলাদা করা এবং digit ** p যোগ করা।

Step 4: যোগফল আসল সংখ্যার সমান কিনা যাচাই করা।

Python Code

```
# Checking Armstrong Number
num = int(input("Enter a number: "))

num_str = str(num)
power = len(num_str)
total_sum = 0

temp = num
while temp > 0:
    digit = temp % 10
    total_sum += digit ** power
    temp = temp // 10
```

```
if total_sum == num:
    print(num, "is an Armstrong Number")
else:
    print(num, "is NOT an Armstrong Number")
```

Built-in Function / Shortcut Method

```
# Pythonic one-liner shortcut
# is_armstrong = sum(int(d)**len(str(num)) for d in str(num)) == num
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 153	153 is an Armstrong Number

ব্যাখ্যা

১৫৩ এর ৩টি ডিজিটের কিউব ($১^3+৫^3+৩^3$ = ১+১২৫+২৭ = ১৫৩) আসল সংখ্যার সমান হয়েছে।

Common Mistakes

- সব সংখ্যার ক্ষেত্রে পাওয়ার ৩ ধরে হিসাব করা (৪ ডিজিটের সংখ্যার যেমন 1634 এর ক্ষেত্রে পাওয়ার ৪ হবে, ৩ নয়)।
- আসল `num` ভেরিয়েবলটিকেই লুপে ভাগ করে শূন্য করে ফেলা।

অনুশীলনের জন্য

1. ১ থেকে ১০০০ এর মধ্যে সব Armstrong Number প্রিন্ট করুন।

2. ৪ ডিজিটের Armstrong Number (যেমন ১৬৩৪) ইনপুট দিয়ে যাচাই করুন।

3. ১ ডিজিটের সব সংখ্যা যে Armstrong তা কোড চালিয়ে প্রমাণ করুন।

সমস্যা 83: একটি সংখ্যা Strong Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি Strong Number (বা Krishnamurthy Number) কিনা তা যাচাই করতে হবে। কোনো সংখ্যার প্রতিটি ডিজিটের ফ্যাক্টোরিয়ালের যোগফল যদি মূল সংখ্যার সমান হয়, তবে তাকে Strong Number বলে (যেমন 145 = 1! + 4! + 5! = 1 + 24 + 120 = 145)।

কী কী জানা থাকতে হবে?

- Factorial logic
- Digit separation

কীভাবে চিন্তা করব?

প্রতিটি ডিজিট আলাদা করতে হবে, সেই ডিজিটের ফ্যাক্টোরিয়াল বের করতে হবে এবং যোগফলের সাথে যুক্ত করতে হবে। যোগফল আসল সংখ্যার সমান হলে Strong Number।

অ্যালগরিদম

Step 1: সংখ্যা ইনপুট নেওয়া।

Step 2: while লুপ দিয়ে প্রতিটি ডিজিট বের করা।

Step 3: ভিতরে আরেকটি লুপ বা math.factorial দিয়ে ডিজিটের ফ্যাক্টোরিয়াল বের করে যোগ করা।

Step 4: যোগফল ও আসল সংখ্যা তুলনা করা।

Python Code

```
# Checking Strong Number
import math

num = int(input("Enter a number: "))
temp = num
total_sum = 0

while temp > 0:
    digit = temp % 10
    total_sum += math.factorial(digit)
    temp = temp // 10

if total_sum == num:
    print(num, "is a Strong Number")
else:
    print(num, "is NOT a Strong Number")
```

Built-in Function / Shortcut Method

```
# Shortcut using list comprehension
# print("Strong" if sum(math.factorial(int(d)) for d in str(num)) == num else
"Not Strong")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 145	145 is a Strong Number

ব্যাখ্যা

১৪৫ এর ডিজিটগুলোর ফ্যাক্টোরিয়াল (১! = ১, ৪! = ২৪, ৫! = ১২০) যোগ করলে ১৪৫ হয়।

Common Mistakes

- ফ্যাক্টোরিয়াল হিসাব করতে ভুল করা।
- ০ এর ফ্যাক্টোরিয়াল ১ তা মনে না রাখা।

অনুশীলনের জন্য

1. ১ থেকে ৫০০০০ এর মধ্যে সব Strong Number খুঁজে বের করুন (দেখবেন ১, ২, ১৪৫ এবং ৪০৫৮৫ আসবে)।

2. math.factorial না ব্যবহার করে ম্যানুয়াল লুপ দিয়ে Strong Number লিখুন।

3. ৪০৫৮৫ সংখ্যাটি Strong Number কিনা যাচাই করুন।

সমস্যা 84: একটি সংখ্যা Palindrome Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা ইনপুট নিয়ে সেটি Palindrome কিনা তা নির্ণয় করতে হবে। যে সংখ্যাকে উল্টো করে লিখলেও একই সংখ্যা থাকে, তাকে Palindrome বলে (যেমন 121, 55, 343 ইত্যাদি)।

কী কী জানা থাকতে হবে?

- Number Reversal Logic

কীভাবে চিন্তা করব?

আমরা প্রথমে সংখ্যাটিকে উল্টো (Reverse) করব। এরপর আমরা চেক করব `original_num == reversed_num` কিনা। সমান হলে Palindrome।

অ্যালগরিদম

Step 1: সংখ্যা ইনপুট নেওয়া।

Step 2: while লুপ দিয়ে রিভার্স সংখ্যা তৈরি করা।

Step 3: আসল সংখ্যা ও রিভার্স সংখ্যা তুলনা করা।

Python Code

```
# Checking Palindrome Number
num = int(input("Enter a number: "))
temp = num
rev_num = 0
```

```
while temp > 0:
    digit = temp % 10
    rev_num = (rev_num * 10) + digit
    temp = temp // 10

if num == rev_num:
    print(num, "is a Palindrome Number")
else:
    print(num, "is NOT a Palindrome Number")
```

Built-in Function / Shortcut Method

```
# String comparison shortcut
# print("Palindrome" if str(num) == str(num)[::-1] else "Not Palindrome")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 12321	12321 is a Palindrome Number

ব্যাখ্যা

১২৩২১ কে বাম থেকে বা ডান থেকে যেদিক থেকেই পড়ুন না কেন, মান একই থাকে।

Common Mistakes

- আসল ভেরিয়েবল `num` কে লুপে ভাগ করে শূন্য করে ফেলা, যার ফলে শেষে `0 == rev_num` তুলনা হয়ে False আসে।
- ঋণাত্মক সংখ্যার ক্ষেত্রে হিসাব না করা।

অনুশীলনের জন্য

1. স্ট্রিং রিভার্স `[::-1]` ব্যবহার করে Palindrome যাচাই করুন।

2. ১ থেকে ৫০০ এর মধ্যে সব Palindrome সংখ্যা প্রিন্ট করুন।

3. একটি স্ট্রিং বা শব্দ Palindrome কিনা যাচাই করুন।

সমস্যা 85: একটি সংখ্যা Automorphic Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা Automorphic কিনা তা নির্ণয় করতে হবে। কোনো সংখ্যার বর্গ (Square) করলে যদি সেই বর্গের শেষ ডিজিটগুলো মূল সংখ্যাটির সমান হয়, তবে তাকে Automorphic Number বলে (যেমন 5 এর বর্গ 25 যার শেষে 5 আছে, 25 এর বর্গ 625 যার শেষে 25 আছে)।

কী কী জানা থাকতে হবে?

- Square calculation
- Modulus by 10^digits বা string endswith()

কীভাবে চিন্তা করব?

প্রথমে আমরা সংখ্যার বর্গ বের করব `sq = num ** 2`। এরপর সবচেয়ে সহজ উপায় হলো স্ট্রিংয়ে রূপান্তর করে দেখা বর্গের শেষের অংশ আসল সংখ্যার সমান কিনা: `str(sq).endswith(str(num))`। অথবা গাণিতিক উপায়ে `sq % (10 ** digits) == num`।

অ্যালগরিদম

Step 1: সংখ্যা ইনপুট নেওয়া।

Step 2: বর্গ নির্ণয় করা।

Step 3: str(square).endswith(str(num)) পরীক্ষা করা।

Step 4: ফলাফল প্রদর্শন করা।

Python Code

```
# Checking Automorphic Number
num = int(input("Enter a number: "))
square = num ** 2

# Check if square ends with the original number
if str(square).endswith(str(num)):
    print(num, "is an Automorphic Number (Square:", square, ")")
else:
    print(num, "is NOT an Automorphic Number (Square:", square, ")")
```

Built-in Function / Shortcut Method

```
# Mathematical logic
# digits = len(str(num))
# is_auto = (num**2) % (10**digits) == num
```

Sample Input / Output Table

Sample Input	Sample Output

Enter a number: 25	25 is an Automorphic Number (Square: 625)

ব্যাখ্যা

২৫ এর বর্গ ৬২৫ যার শেষ দুটি অংক ২৫, যা আমাদের মূল সংখ্যার সমান।

Common Mistakes

- শুধু শেষ ১টি ডিজিট চেক করা (যেমন ২৫ এর ক্ষেত্রে শুধু ৫ চেক করা। পুরো ২৫-ই শেষে থাকতে হবে)।
- বর্গ করতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. ১ থেকে ১০০০ এর মধ্যে সব Automorphic Number প্রিন্ট করুন (যেমন ১, ৫, ৬, ২৫, ৭৬, ৩৭৬)।

2. ৭৬ এর বর্গ ৫৭৭৬ দিয়ে কোড যাচাই করুন।

3. endswith() মেথড ছাড়া গাণিতিক মডুলাস দিয়ে Automorphic বের করুন।

সমস্যা 86: একটি সংখ্যা Neon Number কিনা নির্ণয় করুন।

সমস্যা

একটি সংখ্যা Neon Number কিনা তা নির্ণয় করতে হবে। কোনো সংখ্যার বর্গের সকল ডিজিট যোগ করলে যদি যোগফল মূল সংখ্যাটির সমান হয়, তবে তাকে Neon Number বলে (যেমন 9 এর বর্গ 81 এবং 8+1 = 9)।

কী কী জানা থাকতে হবে?

- Square calculation
- Sum of digits

কীভাবে চিন্তা করব?

প্রথমে সংখ্যার বর্গ বের করব। তারপর বর্গের প্রতিটি ডিজিট আলাদা করে যোগ করব। যোগফল আসল সংখ্যার সমান হলে Neon Number।

অ্যালগরিদম

Step 1: সংখ্যা ইনপুট নেওয়া।

Step 2: square = num ** 2 বের করা।

Step 3: square এর ডিজিটগুলোর যোগফল বের করা।

Step 4: যোগফল ও আসল সংখ্যা তুলনা করা।

Python Code

```
# Checking Neon Number
num = int(input("Enter a number: "))
square = num ** 2

temp = square
sum_digits = 0

while temp > 0:
    digit = temp % 10
    sum_digits += digit
    temp = temp // 10

if sum_digits == num:
    print(num, "is a Neon Number")
else:
    print(num, "is NOT a Neon Number")
```

Built-in Function / Shortcut Method

```
# Shortcut comprehension
# is_neon = sum(int(d) for d in str(num**2)) == num
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a number: 9	9 is a Neon Number

ব্যাখ্যা

৯ এর বর্গ ৮১ এবং ৮১ এর ডিজিটগুলোর যোগফল ৮+১ = ৯, যা আসল সংখ্যার সমান।

Common Mistakes

- বর্গ না করেই আসল সংখ্যার ডিজিট যোগ করে ফেলা।
- ০ এবং ১ যে Neon Number তা যাচাই না করা।

অনুশীলনের জন্য

1. ১ থেকে ১০০০০ এর মধ্যে সব Neon Number বের করুন (দেখবেন শুধু ০, ১ এবং ৯ আসবে)।

2. ১ সংখ্যাটি Neon কিনা যাচাই করুন।

3. লিস্ট কম্প্রিহেনশন দিয়ে এক লাইনে Neon Number যাচাই করুন।

Topic: Series & Patterns

সমস্যা 87: 1+2+3+...+N ধারার যোগফল নির্ণয় করুন।

সমস্যা

স্বাভাবিক সংখ্যার ধারা 1 + 2 + 3 + ... + N এর যোগফল লুপ ব্যবহার করে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- for loop
- Accumulator variable

কীভাবে চিন্তা করব?

১ থেকে N পর্যন্ত প্রতিটি সংখ্যা লুপ চালিয়ে `total` এর সাথে যুক্ত করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(1, N + 1) চালিয়ে total += i করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of series 1 + 2 + 3 + ... + N
N = int(input("Enter N: "))
total = 0

for i in range(1, N + 1):
    total += i

print("Sum of the series is:", total)
```

Built-in Function / Shortcut Method

```
# Mathematical shortcut
# print("Sum:", N * (N + 1) // 2)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 10	Sum of the series is: 55

ব্যাখ্যা

১ থেকে ১০ পর্যন্ত স্বাভাবিক সংখ্যাগুলোর যোগফল ৫৫।

Common Mistakes

- range(1, N) দেওয়া।

অনুশীলনের জন্য

1. বীজগাণিতিক সূত্র প্রয়োগ করে যোগফল বের করুন।

2. while লুপ ব্যবহার করে ধারার যোগফল বের করুন।

3. ১ থেকে ১০০ পর্যন্ত ধারার যোগফল বের করুন।

সমস্যা 88: 2+4+6+...+N ধারার যোগফল নির্ণয় করুন।

সমস্যা

জোড় সংখ্যার ধারা 2 + 4 + 6 + ... + N এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(2, N+1, 2)

কীভাবে চিন্তা করব?

২ থেকে শুরু করে ২ ঘর করে লাফ দিয়ে লুপ চালিয়ে সংখ্যাগুলো যোগ করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(2, N + 1, 2) চালিয়ে total += i করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of even series 2 + 4 + 6 + ... + N
N = int(input("Enter N: "))
total = 0

for i in range(2, N + 1, 2):
    total += i

print("Sum of even series is:", total)
```

Built-in Function / Shortcut Method

```
# If n is number of terms: n * (n + 1)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 10	Sum of even series is: 30

ব্যাখ্যা

২+৪+৬+৮+১০ = ৩০।

Common Mistakes

- step হিসেবে 2 না দেওয়া।

অনুশীলনের জন্য

1. ২ থেকে ১০০ পর্যন্ত জোড় ধারার যোগফল বের করুন।

2. if শর্ত দিয়ে জোড় ধারার যোগফল বের করুন।

3. জোড় ধারার পদসংখ্যা কত তা বের করুন।

সমস্যা 89: 1+3+5+...+N ধারার যোগফল নির্ণয় করুন।

সমস্যা

বিজোড় সংখ্যার ধারা 1 + 3 + 5 + ... + N এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(1, N+1, 2)

কীভাবে চিন্তা করব?

১ থেকে শুরু করে ২ ঘর করে লাফ দিয়ে লুপ চালিয়ে বিজোড় সংখ্যাগুলো যোগ করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(1, N + 1, 2) চালিয়ে total += i করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of odd series 1 + 3 + 5 + ... + N
N = int(input("Enter N: "))
total = 0

for i in range(1, N + 1, 2):
    total += i

print("Sum of odd series is:", total)
```

Built-in Function / Shortcut Method

```
# Sum of first k odd numbers = k^2
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 9	Sum of odd series is: 25

ব্যাখ্যা

১+৩+৫+৭+৯ = ২৫।

Common Mistakes

- লুপ ২ থেকে শুরু করা।

অনুশীলনের জন্য

1. ১ থেকে ৫০ পর্যন্ত বিজোড় ধারার যোগফল বের করুন।

2. while লুপ ব্যবহার করে বিজোড় ধারার যোগফল বের করুন।

3. বিজোড় ধারার গড় নির্ণয় করুন।

সমস্যা 90: $1^2+2^2+3^2+...+N^2$ ধারার যোগফল নির্ণয় করুন।

সমস্যা

স্বাভাবিক সংখ্যার বর্গের ধারা $1^2 + 2^2 + 3^2 + ... + N^2$ এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Exponentiation (** 2)

কীভাবে চিন্তা করব?

১ থেকে N পর্যন্ত প্রতিটি সংখ্যার বর্গ (`i ** 2`) বের করে `total` এর সাথে যুক্ত করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(1, N + 1) চালিয়ে total += i ** 2 করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of square series 1^2 + 2^2 + ... + N^2
N = int(input("Enter N: "))
total = 0

for i in range(1, N + 1):
    total += i ** 2

print("Sum of square series is:", total)
```

Built-in Function / Shortcut Method

```
# Formula: N * (N + 1) * (2*N + 1) // 6
# print("Sum:", N * (N + 1) * (2*N + 1) // 6)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 5	Sum of square series is: 55

ব্যাখ্যা

১²+২²+৩²+৪²+৫² = ১+৪+৯+১৬+২৫ = ৫৫।

Common Mistakes

- `i * 2` করে ফেলা (বর্গ করতে হবে `i ** 2`)।

অনুশীলনের জন্য

1. বীজগাণিতিক সূত্র ব্যবহার করে বর্গের ধারার যোগফল বের করুন।

2. ১ থেকে ১০ পর্যন্ত বর্গের ধারার যোগফল বের করুন।

3. বর্গের ধারার পদগুলো স্ক্রিনে প্রিন্ট করুন।

সমস্যা 91: $1^3+2^3+3^3+...+N^3$ ধারার যোগফল নির্ণয় করুন।

সমস্যা

স্বাভাবিক সংখ্যার ঘনের ধারা $1^3 + 2^3 + 3^3 + ... + N^3$ এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Exponentiation (** 3)

কীভাবে চিন্তা করব?

১ থেকে N পর্যন্ত প্রতিটি সংখ্যার কিউব (`i ** 3`) বের করে যোগ করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(1, N + 1) চালিয়ে total += i ** 3 করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of cube series 1^3 + 2^3 + ... + N^3
N = int(input("Enter N: "))
total = 0

for i in range(1, N + 1):
    total += i ** 3

print("Sum of cube series is:", total)
```

Built-in Function / Shortcut Method

```
# Formula: ((N * (N + 1)) // 2) ** 2
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 4	Sum of cube series is: 100

ব্যাখ্যা

১³+২³+৩³+৪³ = ১+৮+২৭+৬৪ = ১০০।

Common Mistakes

- পাওয়ার ৩ এর জায়গায় ২ দেওয়া।

অনুশীলনের জন্য

1. ঘনের ধারার যোগফলের সূত্রের সাথে কোডের ফলাফল মিলিয়ে দেখুন।

2. ১ থেকে ১০ পর্যন্ত ঘনের ধারার যোগফল বের করুন।

3. প্রতিটি পদের কিউব মান কমা দিয়ে প্রিন্ট করুন।

সমস্যা 92: $3^2+6^2+9^2+...+99^2$ ধারার যোগফল নির্ণয় করুন।

সমস্যা

৩ এর গুণিতকগুলোর বর্গের ধারা $3^2 + 6^2 + 9^2 + ... + 99^2$ এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(3, 100, 3)

কীভাবে চিন্তা করব?

৩ থেকে শুরু করে ৩ ঘর লাফ দিয়ে ৯৯ পর্যন্ত লুপ চালাব এবং প্রতিটি সংখ্যার বর্গ যোগ করব।

অ্যালগরিদম

Step 1: total = 0 রাখা।

Step 2: for i in range(3, 100, 3) লুপ চালানো।

Step 3: total += i ** 2 করা।

Step 4: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of series 3^2 + 6^2 + ... + 99^2
total = 0

for i in range(3, 100, 3):
    total += i ** 2

print("Sum of the series 3^2 + ... + 99^2 is:", total)
```

Built-in Function / Shortcut Method

```
# One-liner
# print("Sum:", sum(x**2 for x in range(3, 100, 3)))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Sum of the series 3^2 + ... + 99^2 is: 306405

ব্যাখ্যা

৩, ৬, ৯ থেকে ৯৯ পর্যন্ত মোট ৩৩টি পদের বর্গ যোগ করে ৩০৬৪০৫ নির্ণয় করা হয়েছে।

Common Mistakes

- range এর শেষ মান ১০০ না দিয়ে ৯৯ দেওয়া (যার ফলে ৯৯ বাদ পড়ে যায়)।

অনুশীলনের জন্য

1. $5^2 + 10^2 + ... + 100^2$ ধারার যোগফল বের করুন।

2. 3 + 6 + 9 + ... + 99 ধারার যোগফল বের করুন।

3. ৩ এর গুণিতকের ঘনের ধারার যোগফল বের করুন।

সমস্যা 93: $2^2+4^2+6^2+...+N^2$ ধারার যোগফল নির্ণয় করুন।

সমস্যা

জোড় সংখ্যার বর্গের ধারা $2^2 + 4^2 + 6^2 + ... + N^2$ এর যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- range(2, N+1, 2)

কীভাবে চিন্তা করব?

২ থেকে N পর্যন্ত ২ ঘর করে লাফ দিয়ে প্রতিটি সংখ্যার বর্গ যোগ করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: total = 0 রাখা।

Step 3: for i in range(2, N + 1, 2) লুপ চালানো।

Step 4: total += i ** 2 করা।

Step 5: যোগফল প্রদর্শন করা।

Python Code

```
# Sum of even square series 2^2 + 4^2 + ... + N^2
N = int(input("Enter N: "))
total = 0

for i in range(2, N + 1, 2):
    total += i ** 2

print("Sum of even square series is:", total)
```

Built-in Function / Shortcut Method

```
# One-liner
# print("Sum:", sum(x**2 for x in range(2, N+1, 2)))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter N: 6	Sum of even square series is: 56

ব্যাখ্যা

২²+৪²+৬² = ৪+১৬+৩৬ = ৫৬।

Common Mistakes

- step হিসেবে 2 না দেওয়া।

অনুশীলনের জন্য

1. ১² + ৩² + ৫² + ... + N² (বিজোড় বর্গের ধারা) বের করুন।

2. ২² + ৪² + ... + ২০² বের করুন।

3. জোড় বর্গের ধারার গড় বের করুন।

সমস্যা 94: Fibonacci Series-এর প্রথম Nটি সংখ্যা প্রদর্শন করুন।

সমস্যা

ফিবোনাচ্চি ধারার প্রথম Nটি পদ স্ক্রিনে কমা দিয়ে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Fibonacci logic

কীভাবে চিন্তা করব?

a = 0 এবং b = 1 নিয়ে লুপ চালিয়ে প্রতিবার a প্রিন্ট করব এবং `a, b = b, a + b` আপডেট করব।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: a, b = 0, 1 সেট করা।

Step 3: N বার লুপ চালানো।

Step 4: a প্রিন্ট করা এবং a, b = b, a + b করা।

Python Code

```
# Printing first N Fibonacci numbers
N = int(input("Enter number of terms (N): "))

a, b = 0, 1
print("Fibonacci Series:")
for i in range(N):
    print(a, end=" ")
    a, b = b, a + b
```

Built-in Function / Shortcut Method

```
# Generator approach
```

Sample Input / Output Table

Sample Input	Sample Output
Enter number of terms (N): 8	Fibonacci Series: 0 1 1 2 3 5 8 13

ব্যাখ্যা

পরপর দুটি পদের যোগফল পরবর্তী পদ তৈরি করে ৮টি পদ প্রদর্শন করা হয়েছে।

Common Mistakes

- লুপের ভেতরে a প্রিন্ট না করে b প্রিন্ট করা।

অনুশীলনের জন্য

1. প্রথম ১০টি ফিবোনাচ্চি সংখ্যার যোগফল বের করুন।

2. ১০০ পর্যন্ত ফিবোনাচ্চি সংখ্যাগুলো প্রিন্ট করুন।

3. লিস্টে ফিবোনাচ্চি সংখ্যাগুলো সংরক্ষণ করুন।

সমস্যা 95: Factorial Series-এর প্রথম Nটি মান প্রদর্শন করুন।

সমস্যা

১ থেকে N পর্যন্ত প্রতিটি সংখ্যার ফ্যাক্টোরিয়াল (1!, 2!, 3!, ...) স্ক্রিনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- Accumulator multiplication inside loop

কীভাবে চিন্তা করব?

প্রতিবার লুপে আগের ফ্যাক্টোরিয়ালের সাথে বর্তমান সংখ্যা i গুণ করলেই নতুন ফ্যাক্টোরিয়াল পাওয়া যায়।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: fact = 1 রাখা।

Step 3: ১ থেকে N পর্যন্ত লুপ চালানো।

Step 4: fact *= i করে প্রিন্ট করা।

Python Code

```
# Printing first N Factorial values
N = int(input("Enter N: "))
fact = 1

print("Factorial Series up to", N, ":")
for i in range(1, N + 1):
    fact *= i
    print(f"{i}! = {fact}")
```

Built-in Function / Shortcut Method

```
# Using math.factorial
# import math
# for i in range(1, N+1): print(f"{i}! = {math.factorial(i)}")
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter N: 5                              Factorial Series up to 5 :
                                        1! = 1
                                        2! = 2
                                        3! = 6
                                        4! = 24
                                        5! = 120
```

ব্যাখ্যা

প্রতিটি ধাপে সংখ্যাটির ফ্যাক্টোরিয়াল হিসাব করে প্রদর্শন করা হয়েছে।

Common Mistakes

- প্রতিবার লুপে `fact = 1` করে ফেলা।

অনুশীলনের জন্য

1. 1! + 2! + 3! + ... + N! ধারার যোগফল বের করুন।

2. 1/1! + 1/2! + 1/3! ধারার যোগফল বের করুন।

3. প্রথম ৭টি ফ্যাক্টোরিয়াল মান কমা দিয়ে প্রিন্ট করুন।

সমস্যা 96: একটি Square Pattern প্রিন্ট করুন।

সমস্যা

তারকা চিহ্ন (*) ব্যবহার করে N শারি ও N কলামের একটি বর্গাকার (Square) প্যাটার্ন তৈরি করতে হবে।

কী কী জানা থাকতে হবে?

- Nested for loop / String multiplication (*)

কীভাবে চিন্তা করব?

N টি লাইনের প্রতিটিতে N টি করে তারকা প্রিন্ট করতে হবে। পাইথনে `print('* ' * N)` দিয়ে খুব সহজেই করা যায়।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: N বার লুপ চালানো।

Step 3: প্রতিবার '* ' * N প্রিন্ট করা।

Python Code

```
# Printing a Square Pattern
N = int(input("Enter size of square: "))
```

```
for i in range(N):
    print("* " * N)
```

Built-in Function / Shortcut Method

```
# Nested loop approach
# for i in range(N):
#     for j in range(N): print("*", end=" ")
#     print()
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter size of square: 4`	`* * * *` `* * * *` `* * * *` `* * * *`

ব্যাখ্যা

৪ সারি ও ৪ কলামের একটি সুন্দর বর্গাকার প্যাটার্ন তৈরি হয়েছে।

Common Mistakes

- নতুন লাইনে যাওয়ার জন্য লুপের শেষে print() না দেওয়া (যদি nested loop ব্যবহার করা হয়)।

অনুশীলনের জন্য

1. # চিহ্ন দিয়ে ৫x৫ বর্গ তৈরি করুন।

2. ফাঁকা বর্গ (Hollow square) তৈরি করুন।

3. সংখ্যার বর্গ প্যাটার্ন তৈরি করুন।

সমস্যা 97: একটি Rectangle Pattern প্রিন্ট করুন।

সমস্যা

নির্দিষ্ট সারি (Rows) ও কলাম (Columns) সংখ্যা ইনপুট নিয়ে একটি আয়তাকার (Rectangle) প্যাটার্ন তৈরি করুন।

কী কী জানা থাকতে হবে?

- String multiplication

কীভাবে চিন্তা করব?

সারি সংখ্যা rows এবং কলাম সংখ্যা cols নিয়ে লুপ চালাব rows বার এবং প্রতিবার `cols` টি তারকা প্রিন্ট করব।

অ্যালগরিদম

Step 1: rows এবং cols ইনপুট নেওয়া।

Step 2: for i in range(rows) চালানো।

Step 3: print('* ' * cols) করা।

Python Code

```
# Printing a Rectangle Pattern
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

for i in range(rows):
    print("* " * cols)
```

Built-in Function / Shortcut Method

```
# Direct print
# print(("* " * cols + "\n") * rows)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter number of rows: 3 Enter number of columns: 6	* * * * * * * * * * * * * * * * * *

ব্যাখ্যা

৩ সারি ও ৬ কলাম বিশিষ্ট আয়তক্ষেত্র প্যাটার্ন প্রিন্ট হয়েছে।

Common Mistakes

- rows এবং cols গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. ৪ সারি ও ৮ কলামের আয়তক্ষেত্র তৈরি করুন।

2. $ চিহ্ন দিয়ে আয়তক্ষেত্র বানান।

3. Hollow rectangle তৈরি করুন।

সমস্যা 98: একটি Left Triangle Pattern প্রিন্ট করুন।

সমস্যা

বাম দিকে সমকোণযুক্ত একটি ত্রিভুজ (Left Triangle / Right-angled triangle aligned left) প্যাটার্ন তৈরি করুন।

কী কী জানা থাকতে হবে?

- for i in range(1, N+1)
- print('* ' * i)

কীভাবে চিন্তা করব?

১ম লাইনে ১টি তারকা, ২য় লাইনে ২টি, ৩য় লাইনে ৩টি... এভাবে N তম লাইনে N টি তারকা প্রিন্ট করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for i in range(1, N + 1) লুপ চালানো।

Step 3: print('* ' * i) করা।

Python Code

```
# Printing Left Triangle Pattern
N = int(input("Enter height: "))

for i in range(1, N + 1):
    print("* " * i)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
Enter height: 5	* * * * * * * * * *

```
* * * * *
```

ব্যাখ্যা

প্রতি লাইনে লাইনের নম্বরের সমান সংখ্যক তারকা প্রিন্ট হয়েছে।

Common Mistakes

- range(N) দিয়ে শুরু করা যার ফলে প্রথম লাইন ফাঁকা আসে।

অনুশীলনের জন্য

1. ১০ লাইনের ত্রিভুজ বানান।

2. তারকার বদলে @ চিহ্ন ব্যবহার করুন।

3. প্রতি লাইনে একই সংখ্যা প্রিন্ট করে ত্রিভুজ বানান।

সমস্যা 99: একটি Right Triangle Pattern প্রিন্ট করুন।

সমস্যা

ডান দিকে সমকোণযুক্ত ত্রিভুজ (Right-aligned Triangle) প্যাটার্ন প্রিন্ট করতে হবে।

কী কী জানা থাকতে হবে?

- String formatting / spaces

কীভাবে চিন্তা করব?

ডানদিকে এলাইন করার জন্য তারকার আগে কিছু স্পেস দিতে হবে। i তম লাইনে স্পেস থাকবে `(N - i)` টি এবং তারকা থাকবে `i` টি।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: ১ থেকে N পর্যন্ত লুপ চালানো।

Step 3: spaces = ' ' * (N - i) এবং stars = '* ' * i প্রিন্ট করা।

Python Code

```
# Printing Right-aligned Triangle Pattern
N = int(input("Enter height: "))

for i in range(1, N + 1):
    print("  " * (N - i) + "* " * i)
```

Built-in Function / Shortcut Method

```
# Using rjust()
# for i in range(1, N+1): print(("* " * i).rjust(N * 2))
```

Sample Input / Output Table

Sample Input	Sample Output

Enter height: 5	`        *`
	`      * *`
	`    * * *`
	`  * * * *`
	`* * * * *`

ব্যাখ্যা

সামনে স্পেস যুক্ত করার কারণে ত্রিভুজটি ডানদিকে সুন্দরভাবে সরে গেছে।

Common Mistakes

- স্পেসের সংখ্যা ঠিকমতো হিসাব না করা।

অনুশীলনের জন্য

1. ডানদিকে এলাইন করা সংখ্যার ত্রিভুজ বানান।

2. ৬ লাইনের Right triangle বানান।

3. # চিহ্ন দিয়ে বানান।

সমস্যা 100: একটি Inverted Triangle Pattern প্রিন্ট করুন।

সমস্যা

উল্টো ত্রিভুজ (Inverted Left Triangle) প্যাটার্ন প্রিন্ট করতে হবে যেখানে প্রথম লাইনে N টি তারকা এবং শেষ লাইনে ১টি তারকা থাকবে।

কী কী জানা থাকতে হবে?

- range(N, 0, -1)

কীভাবে চিন্তা করব?

N থেকে শুরু করে নিচের দিকে ১ পর্যন্ত লুপ চালিয়ে প্রতিবার `i` টি তারকা প্রিন্ট করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for i in range(N, 0, -1) চালানো।

Step 3: print('* ' * i) করা।

Python Code

```
# Printing Inverted Triangle Pattern
N = int(input("Enter height: "))

for i in range(N, 0, -1):
    print("* " * i)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
Enter height: 5	* * * * * * * * * * * * * * *

ব্যাখ্যা

উপর থেকে নিচের দিকে তারকার সংখ্যা ১ করে কমে উল্টো ত্রিভুজ তৈরি হয়েছে।

Common Mistakes

- step হিসেবে -1 না দেওয়া।

অনুশীলনের জন্য

1. ডানদিকে এলাইন করা উল্টো ত্রিভুজ বানান।

2. সংখ্যার উল্টো ত্রিভুজ বানান।

3. ৭ লাইনের উল্টো ত্রিভুজ বানান।

সমস্যা 101: একটি Pyramid Pattern প্রিন্ট করুন।

সমস্যা

কেন্দ্রবিন্দু বরাবর সুষম পিরামিড (Pyramid / Triangle) প্যাটার্ন তৈরি করতে হবে।

কী কী জানা থাকতে হবে?

- Spaces and Stars combination

কীভাবে চিন্তা করব?

প্রতি লাইনে স্পেস থাকবে `N - i` টি এবং তারকা থাকবে `2 * i - 1` টি (অথবা প্রতিটি তারকার পর স্পেস দিলে দেখতে সুন্দর পিরামিড হয় `(N - i)` টি স্পেস এবং `i` টি তারকা)।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: ১ থেকে N পর্যন্ত লুপ চালানো।

Step 3: print(' ' * (N - i) + '* ' * i) করা।

Python Code

```
# Printing a Pyramid Pattern
N = int(input("Enter height of pyramid: "))

for i in range(1, N + 1):
    print(" " * (N - i) + "* " * i)
```

Built-in Function / Shortcut Method

```
# Using center alignment
# for i in range(1, N+1): print(("* "*i).center(N*2))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter height of pyramid: 5	

ব্যাখ্যা

সামনে পরিমিত স্পেস দেওয়ার কারণে তারকাগুলো মাঝ বরাবর এসে সুন্দর পিরামিড তৈরি করেছে।

Common Mistakes

- স্পেস ডাবল দিয়ে ফেলা যার ফলে ডানদিকে সরে যায়।

অনুশীলনের জন্য

1. বিজোড় সংখ্যক তারকা (`2*i - 1`) দিয়ে পিরামিড বানান।

2. ১০ লাইনের পিরামিড বানান।

3. সংখ্যার পিরামিড বানান।

সমস্যা 102: একটি Reverse Pyramid Pattern প্রিন্ট করুন।

সমস্যা

উল্টো সুষম পিরামিড (Reverse Pyramid) প্যাটার্ন প্রিন্ট করতে হবে।

কী কী জানা থাকতে হবে?

- Reverse range(N, 0, -1)

কীভাবে চিন্তা করব?

N থেকে ১ পর্যন্ত লুপ চালিয়ে স্পেস `N - i` এবং তারকা `i` প্রিন্ট করতে হবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for i in range(N, 0, -1) চালানো।

Step 3: print(' ' * (N - i) + '* ' * i) করা।

Python Code

```
# Printing Reverse Pyramid Pattern
N = int(input("Enter height: "))

for i in range(N, 0, -1):
    print(" " * (N - i) + "* " * i)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter height: 5
```

```
* * * * *
 * * * *
  * * *
   * *
    *
```

ব্যাখ্যা

উপর থেকে নিচে তারকা কমে এবং স্পেস বেড়ে উল্টো পিরামিড তৈরি হয়েছে।

Common Mistakes

- লুপের সীমানা ভুল করা।

অনুশীলনের জন্য

1. সোজা পিরামিড ও উল্টো পিরামিড একসাথে জোড়া দিয়ে ডায়মন্ড (Diamond) প্যাটার্ন বানান।

2. ৬ লাইনের উল্টো পিরামিড বানান।

3. # দিয়ে উল্টো পিরামিড বানান।

সমস্যা 103: একটি Number Triangle Pattern প্রিন্ট করুন।

সমস্যা

১ থেকে শুরু করে সংখ্যা দিয়ে একটি ত্রিভুজ প্যাটার্ন তৈরি করুন (১ম লাইনে 1, ২য় লাইনে 1 2 ইত্যাদি)।

কী কী জানা থাকতে হবে?

- Nested loop for columns

কীভাবে চিন্তা করব?

বাইরের লুপ i চলবে ১ থেকে N পর্যন্ত। ভিতরের লুপ j চলবে ১ থেকে i পর্যন্ত এবং j প্রিন্ট করবে।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: for i in range(1, N + 1) চালানো।

Step 3: ভিতরে for j in range(1, i + 1) চালিয়ে j প্রিন্ট করা।

Step 4: প্রতি লাইনের শেষে print() দিয়ে নতুন লাইনে যাওয়া।

Python Code

```
# Printing Number Triangle Pattern
N = int(input("Enter height: "))
```

```
for i in range(1, N + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Built-in Function / Shortcut Method

```
# Using join
# for i in range(1, N+1): print(" ".join(map(str, range(1, i+1))))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter height: 5	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5

ব্যাখ্যা

প্রতি লাইনে ১ থেকে শুরু হয়ে ওই লাইনের নম্বর পর্যন্ত সংখ্যাগুলো প্রিন্ট হয়েছে।

Common Mistakes

- print() দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. প্রতি লাইনে একই সংখ্যা (যেমন ১ম লাইনে 1, ২য় লাইনে 2 2) প্রিন্ট করুন।

2. উল্টো সংখ্যার ত্রিভুজ বানান।

3. বিজোড় সংখ্যার ত্রিভুজ বানান।

সমস্যা 104: একটি Floyd Triangle Pattern প্রিন্ট করুন।

সমস্যা

ধারাবাহিক সংখ্যা (1, 2, 3, 4...) ব্যবহার করে ফ্লয়েডের ত্রিভুজ (Floyd's Triangle) তৈরি করতে হবে।

কী কী জানা থাকতে হবে?

- Independent counter variable

কীভাবে চিন্তা করব?

একটি ভেরিয়েবল `num = 1` লুপের বাইরে রাখব। ত্রিভুজের প্রতি ঘরে `num` প্রিন্ট করব এবং `num += 1` করব।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া এবং num = 1 রাখা।

Step 2: i এর লুপ ১ থেকে N পর্যন্ত চালানো।

Step 3: j এর লুপ ১ থেকে i পর্যন্ত চালিয়ে num প্রিন্ট ও num += 1 করা।

Step 4: নতুন লাইনে যাওয়া।

Python Code

```
# Printing Floyd's Triangle Pattern
N = int(input("Enter height: "))
num = 1

for i in range(1, N + 1):
    for j in range(1, i + 1):
        print(num, end=" ")
        num += 1
    print()
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
Enter height: 4	1 2 3 4 5 6 7 8 9 10

ব্যাখ্যা

সংখ্যাগুলো প্রতি ঘরে ১ করে বেড়ে সুন্দর ফ্লয়েডের ত্রিভুজ গঠন করেছে।

Common Mistakes

- num কে লুপের ভেতরে 1 সেট করা।

অনুশীলনের জন্য

1. ৫ লাইনের Floyd's Triangle বানান।

2. শুধু জোড় সংখ্যা দিয়ে Floyd's Triangle বানান।

3. ০ এবং ১ দিয়ে বাইনারি ত্রিভুজ বানান।

সমস্যা 105: একটি Alphabet Triangle Pattern প্রিন্ট করুন।

সমস্যা

ইংরেজি বর্ণমালা (A, B, C...) ব্যবহার করে একটি ত্রিভুজ প্যাটার্ন তৈরি করতে হবে।

কী কী জানা থাকতে হবে?

- chr() এবং ord() ফাংশন

কীভাবে চিন্তা করব?

ASCII কোডে 'A' এর মান 65। আমরা `chr(65)` দিলে 'A' পাই। প্রতি লাইনে বা প্রতি কলামে ASCII মান বাড়িয়ে অক্ষর প্রিন্ট করা যায়।

অ্যালগরিদম

Step 1: N ইনপুট নেওয়া।

Step 2: ১ থেকে N পর্যন্ত লুপ চালানো।

Step 3: for j in range(i) চালিয়ে chr(65 + j) প্রিন্ট করা।

Step 4: নতুন লাইনে যাওয়া।

Python Code

```
# Printing Alphabet Triangle Pattern
N = int(input("Enter height: "))

for i in range(1, N + 1):
    for j in range(i):
        char = chr(65 + j)
        print(char, end=" ")
    print()
```

Built-in Function / Shortcut Method

```
# Using string slicing
# import string
# for i in range(1, N+1): print(" ".join(string.ascii_uppercase[:i]))
```

Sample Input / Output Table

Sample Input	Sample Output

`Enter height: 5`	`A`
	`A B`
	`A B C`
	`A B C D`
	`A B C D E`

ব্যাখ্যা

`chr(65 + j)` ব্যবহারের ফলে প্রতি কলামে A থেকে শুরু করে পরবর্তী অক্ষরগুলো প্রদর্শিত হয়েছে।

Common Mistakes

- ASCII কোড 65 এর জায়গায় অন্য কিছু দেওয়া।

অনুশীলনের জন্য

1. প্রতি লাইনে একই অক্ষর (A, B B, C C C) প্রিন্ট করুন।

2. ছোট হাতের অক্ষর দিয়ে ত্রিভুজ বানান।

3. ধারাবাহিক অক্ষর দিয়ে Floyd's Triangle (A, B C, D E F) বানান।

Topic: String Manipulation

সমস্যা 106: ব্যবহারকারীর কাছ থেকে একটি String ইনপুট গ্রহণ করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে কীবোর্ডের মাধ্যমে একটি টেক্সট বা বাক্য (String) ইনপুট গ্রহণ করে স্ক্রিনে প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- input() ফাংশন

কীভাবে চিন্তা করব?

পাইথনের `input()` ফাংশন ডিফল্টভাবেই যেকোনো ইনপুটকে String হিসেবে গ্রহণ করে। তাই কোনো Type Casting ছাড়াই সরাসরি ভেরিয়েবলে রাখা যায়।

অ্যালগরিদম

Step 1: input() ফাংশন দিয়ে ব্যবহারকারীর কাছ থেকে টেক্সট নেওয়া।

Step 2: সেটি একটি ভেরিয়েবলে রাখা।

Step 3: প্রিন্ট করা।

Python Code

```
# Taking a string input from user
user_text = input("Enter a sentence or text: ")
print("You entered:", user_text)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a sentence or text: Learning Python is awesome!	You entered: Learning Python is awesome!

ব্যাখ্যা

`input()` ফাংশনটি পুরো বাক্যটিকে string হিসেবে user_text ভেরিয়েবলে সংরক্ষণ করেছে।

Common Mistakes

- স্ট্রিং নেওয়ার জন্য str(input()) লেখা (যা অপ্রয়োজনীয়)।

অনুশীলনের জন্য

1. আপনার নাম ইনপুট নিয়ে একটি স্বাগত বার্তা প্রিন্ট করুন।

2. একটি প্যারাগ্রাফ ইনপুট নিন।

3. ইনপুট করা স্ট্রিংয়ের ডাটা টাইপ পরীক্ষা করুন।

সমস্যা 107: একটি String-এর দৈর্ঘ্য নির্ণয় করুন।

সমস্যা

একটি String-এ মোট কতটি ক্যারেক্টার (স্পেস ও চিহ্নসহ) আছে তা লুপ বা বিল্ট-ইন ফাংশন দিয়ে নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- len() ফাংশন
- for loop counter

কীভাবে চিন্তা করব?

স্ট্রিংয়ের দৈর্ঘ্য বের করার জন্য আমরা একটি লুপ চালিয়ে প্রতিটি ক্যারেক্টারের জন্য কাউন্টার ১ করে বাড়াতে পারি। আর সবচেয়ে সহজ উপায় হলো পাইথনের বিল্ট-ইন `len()` ফাংশন ব্যবহার করা।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: count = 0 রাখা।

Step 3: for char in text চালিয়ে count += 1 করা।

Step 4: count প্রিন্ট করা।

Python Code

```
# Finding length of a string without len()
text = input("Enter a string: ")
count = 0

for char in text:
    count += 1

print("Length of string is:", count)
```

Built-in Function / Shortcut Method

```
# Using built-in len() function
# print("Length:", len(text))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a string: Hello World	Length of string is: 11

ব্যাখ্যা

'Hello World' এ ৫টি অক্ষর + ১টি স্পেস + ৫টি অক্ষর = মোট ১১টি ক্যারেক্টার আছে।

Common Mistakes

- স্পেসকে ক্যারেক্টার হিসেবে না গোনা।

অনুশীলনের জন্য

1. len() ব্যবহার করে আপনার নামের দৈর্ঘ্য বের করুন।

2. স্পেস বাদে শুধুমাত্র অক্ষরগুলোর দৈর্ঘ্য বের করুন।

3. while লুপ ব্যবহার করে স্ট্রিংয়ের দৈর্ঘ্য বের করুন।

সমস্যা 108: একটি String উল্টো (Reverse) করে প্রদর্শন করুন।

সমস্যা

একটি বাক্য বা শব্দ ইনপুট নিয়ে তার অক্ষরগুলোকে উল্টো করে প্রদর্শন করতে হবে (যেমন 'Python' হবে 'nohtyP')।

কী কী জানা থাকতে হবে?

- String Slicing [::-1]
- Loop concatenation

কীভাবে চিন্তা করব?

একটি ফাঁকা স্ট্রিং `rev_text = ''` রাখব। লুপ চালিয়ে প্রতিটি ক্যারেক্টারকে `rev_text` এর সামনে যুক্ত করব (`rev_text = char + rev_text`)। আর শর্টকাট হলো স্লাইসিং `text[::-1]`।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: rev_text = '' রাখা।

Step 3: for char in text চালিয়ে rev_text = char + rev_text করা।

Step 4: rev_text প্রিন্ট করা।

Python Code

```
# Reversing a string using loop
text = input("Enter a string: ")
rev_text = ""

for char in text:
    rev_text = char + rev_text

print("Reversed string is:", rev_text)
```

Built-in Function / Shortcut Method

```
# Using string slicing [::-1]
# print("Reversed:", text[::-1])
```

Sample Input / Output Table

Sample Input	Sample Output
`Enter a string: Python`	`Reversed string is: nohtyP`

ব্যাখ্যা

লুপে প্রতিটি নতুন অক্ষর আগের অক্ষরের বামে যুক্ত হয়ে সম্পূর্ণ শব্দটিকে উল্টে দিয়েছে।

Common Mistakes

- `rev_text = rev_text + char` লেখা (যা সোজা শব্দই তৈরি করে। উল্টাতে হলে `char + rev_text` লিখতে হবে)।

অনুশীলনের জন্য

1. স্লাইসিং `[::-1]` ব্যবহার করে আপনার নাম রিভার্স করুন।

2. একটি বাক্যের শব্দগুলোর ক্রম ঠিক রেখে প্রতিটি শব্দ আলাদাভাবে রিভার্স করুন।

3. reversed() ফাংশন ও join ব্যবহার করে স্ট্রিং রিভার্স করুন।

সমস্যা 109: একটি String Palindrome কিনা নির্ণয় করুন।

সমস্যা

একটি শব্দ বা স্ট্রিং Palindrome কিনা তা নির্ণয় করতে হবে (যেমন 'madam', 'radar')।

কী কী জানা থাকতে হবে?

- String equality check

কীভাবে চিন্তা করব?

শব্দটিকে ছোট হাতের অক্ষরে রূপান্তর করে তার রিভার্সের সাথে তুলনা করব। সমান হলে Palindrome।

অ্যালগরিদম

Step 1: শব্দ ইনপুট নিয়ে lower() করা।

Step 2: rev_text = text[::-1] বের করা।

Step 3: if text == rev_text হলে Palindrome প্রিন্ট করা।

Python Code

```
# Checking string Palindrome
text = input("Enter a word: ").lower()
rev_text = text[::-1]
```

```
if text == rev_text:
    print("It is a Palindrome")
else:
    print("It is NOT a Palindrome")
```

Built-in Function / Shortcut Method

```
# One liner
# print("Palindrome" if text == text[::-1] else "Not Palindrome")
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a word: Radar	It is a Palindrome

ব্যাখ্যা

'Radar' কে lower করলে 'radar' হয়, যা উল্টালেও 'radar' থাকে।

Common Mistakes

- বড় হাতের ও ছোট হাতের অক্ষরের পার্থক্য দূর করার জন্য lower() না করা।

অনুশীলনের জন্য

1. 'madam' এবং 'level' শব্দ দিয়ে যাচাই করুন।

2. লুপ চালিয়ে স্ট্রিংয়ের দুই প্রান্ত থেকে অক্ষর মিলিয়ে Palindrome যাচাই করুন।

3. স্পেস বাদ দিয়ে পুরো বাক্য Palindrome কিনা যাচাই করুন।

সমস্যা 110: একটি String-এ মোট কতটি Vowel আছে তা নির্ণয় করুন।

সমস্যা

একটি বাক্যে বা শব্দে মোট কতটি স্বরবর্ণ (a, e, i, o, u) আছে তা গণনা করতে হবে।

কী কী জানা থাকতে হবে?

- in operator
- for loop

কীভাবে চিন্তা করব?

লুপ চালিয়ে স্ট্রিংয়ের প্রতিটি অক্ষর দেখব সেটি ভাওয়েল লিস্টে আছে কিনা। থাকলে count বাড়াব।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নিয়ে lower() করা।

Step 2: vowel_count = 0 রাখা।

Step 3: for char in text চালিয়ে if char in 'aeiou': vowel_count += 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Counting vowels in a string
text = input("Enter a string: ").lower()
vowels = "aeiou"
count = 0

for char in text:
    if char in vowels:
        count += 1

print("Total vowels:", count)
```

Built-in Function / Shortcut Method

```
# Comprehension sum
# print("Vowels:", sum(1 for c in text if c in "aeiou"))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a string: Programming in Python	Total vowels: 5

ব্যাখ্যা

বাক্যটিতে o, a, i, i, o মোট ৫টি ভাওয়েল রয়েছে।

Common Mistakes

- lower() না করার কারণে বড় হাতের ভাওয়েল বাদ পড়া।

অনুশীলনের জন্য

1. প্রতিটি ভাওয়েল (a, e, i, o, u) আলাদাভাবে কোনটা কয়বার আছে তা বের করুন।

2. ভাওয়েলগুলো বাদ দিয়ে বাকি বাক্যটি প্রিন্ট করুন।

3. একটি ফাইলে ভাওয়েল গণনা করুন।

সমস্যা 111: একটি String-এ মোট কতটি Consonant আছে তা নির্ণয় করুন।

সমস্যা

একটি স্ট্রিংয়ে মোট কতটি ব্যঞ্জনবর্ণ (Consonant) আছে তা গণনা করতে হবে।

কী কী জানা থাকতে হবে?

- isalpha() method
- not in operator

কীভাবে চিন্তা করব?

অক্ষরটি যদি বর্ণমালা হয় (`char.isalpha()`) এবং ভাওয়েল না হয় (`char not in 'aeiou'`), তবে সেটি Consonant।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নিয়ে lower() করা।

Step 2: count = 0 রাখা।

Step 3: লুপ চালিয়ে if char.isalpha() and char not in 'aeiou': count += 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Counting consonants in a string
text = input("Enter a string: ").lower()
vowels = "aeiou"
consonant_count = 0

for char in text:
    if char.isalpha() and char not in vowels:
        consonant_count += 1

print("Total consonants:", consonant_count)
```

Built-in Function / Shortcut Method

```
# Comprehension sum
# print("Consonants:", sum(1 for c in text if c.isalpha() and c not in
"aeiou"))
```

Sample Input / Output Table

Sample Input	Sample Output

```
Enter a string: Hello World        Total consonants: 7
```

ব্যাখ্যা

H, l, l, W, r, l, d এই ৭টি Consonant রয়েছে।

Common Mistakes

- isalpha() চেক না করার কারণে স্পেস বা চিহ্নকেও Consonant মনে করা।

অনুশীলনের জন্য

1. Vowel এবং Consonant এর অনুপাত বা পার্থক্য বের করুন।

2. একটি স্ট্রিং থেকে সব Consonant মুছে ফেলুন।

3. Consonant গুলো বড় হাতের অক্ষরে রূপান্তর করুন।

সমস্যা 112: একটি String-এ মোট কতটি Digit আছে তা নির্ণয় করুন।

সমস্যা

একটি বাক্যের ভেতর কতটি সংখ্যা বা অংক (0-9) আছে তা গণনা করতে হবে।

কী কী জানা থাকতে হবে?

- isdigit() method

কীভাবে চিন্তা করব?

লুপ চালিয়ে প্রতিটি ক্যারেক্টারে `.isdigit()` চেক করব। সত্য হলে কাউন্টার বাড়াব।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: digit_count = 0 রাখা।

Step 3: for char in text চালিয়ে if char.isdigit(): digit_count += 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Counting digits in a string
text = input("Enter a string with numbers: ")
digit_count = 0

for char in text:
```

```
    if char.isdigit():
        digit_count += 1

print("Total digits in the string:", digit_count)
```

Built-in Function / Shortcut Method

```
# Comprehension sum
# print("Digits:", sum(1 for c in text if c.isdigit()))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a string with numbers: My phone number is 01712345678	Total digits in the string: 11

ব্যাখ্যা

বাক্যটিতে ১১ ডিজিটের ফোন নম্বরটি সফলভাবে গণনা করা হয়েছে।

Common Mistakes

- isdigit() মেথডে ব্র্যাকেট না দেওয়া।

অনুশীলনের জন্য

1. বাক্যের ভেতরের সব ডিজিটের যোগফল বের করুন।

2. বাক্য থেকে ডিজিটগুলো আলাদা করে একটি নতুন স্ট্রিং বানান।

3. ডিজিট বাদে বাকি টেক্সট প্রিন্ট করুন।

সমস্যা 113: একটি String-এ মোট কতটি Space আছে তা নির্ণয় করুন।

সমস্যা

একটি বাক্যে কতটি ফাঁকা স্থান বা Space (' ') আছে তা গণনা করতে হবে।

কী কী জানা থাকতে হবে?

- Comparison char == ' ' বা isspace()

কীভাবে চিন্তা করব?

লুপ চালিয়ে দেখব অক্ষরটি স্পেস কিনা (`char == ' '`)। অথবা `text.count(' ')` ব্যবহার করব।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: space_count = 0 রাখা।

Step 3: for char in text চালিয়ে if char == ' ': space_count += 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Counting spaces in a string
text = input("Enter a sentence: ")
space_count = 0

for char in text:
    if char == ' ':
        space_count += 1

print("Total white spaces:", space_count)
```

Built-in Function / Shortcut Method

```
# Using count() method
# print("Spaces:", text.count(' '))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a sentence: Python is very easy to learn	Total white spaces: 5

ব্যাখ্যা

বাক্যটিতে ৬টি শব্দের মাঝে ৫টি স্পেস রয়েছে।

Common Mistakes

- কোটেশনের মাঝে স্পেস না দেওয়া (`''` দেওয়া)।

অনুশীলনের জন্য

1. count() মেথড ব্যবহার করে স্পেস গুনুন।

2. একাধিক স্পেস থাকলে তা কমিয়ে ১টি স্পেস বানান।

3. স্পেসের জায়গায় আন্ডারস্কোর (_) বসান।

সমস্যা 114: একটি String-এ মোট কতটি Word আছে তা নির্ণয় করুন।

সমস্যা

একটি বাক্যে মোট কতটি শব্দ (Words) আছে তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- split() method

কীভাবে চিন্তা করব?

বাক্যটিকে স্পেসের ভিত্তিতে আলাদা করতে পাইথনের `.split()` মেথড ব্যবহার করা হয়, যা শব্দগুলোর একটি List তৈরি করে। এরপর সেই লিস্টের দৈর্ঘ্য `len()` মেপেই শব্দ সংখ্যা পাওয়া যায়।

অ্যালগরিদম

Step 1: বাক্য ইনপুট নেওয়া।

Step 2: words = text.split() করে শব্দ আলাদা করা।

Step 3: len(words) প্রিন্ট করা।

Python Code

```
# Counting total words in a string
text = input("Enter a sentence: ")

words = text.split()
word_count = len(words)

print("Total words:", word_count)
```

Built-in Function / Shortcut Method

```
# One-liner
# print("Words:", len(text.split()))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a sentence: I love programming in Python	Total words: 5

ব্যাখ্যা

`.split()` বাক্যটিকে ৫টি শব্দের একটি লিস্ট বানিয়ে দিয়েছে।

Common Mistakes

- শুধু স্পেস সংখ্যা + ১ ধরে হিসাব করা (যা মাঝে একাধিক স্পেস থাকলে ভুল দেয়। split() সবচেয়ে নির্ভুল)।

অনুশীলনের জন্য

1. সবচেয়ে বড় শব্দটি খুঁজে বের করুন।

2. প্রতিটি শব্দের প্রথম অক্ষর প্রিন্ট করুন।

3. ৫ অক্ষরের বেশি বড় শব্দগুলো প্রিন্ট করুন।

সমস্যা 115: একটি String-এর প্রতিটি Character কতবার এসেছে তা নির্ণয় করুন।

সমস্যা

একটি স্ট্রিংয়ে কোন অক্ষরটি কতবার (Frequency) ব্যবহার করা হয়েছে তা নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Dictionary বা count() method

কীভাবে চিন্তা করব?

একটি ফাঁকা ডিকশনারি `freq = {}` নেব। লুপ চালিয়ে প্রতিটি অক্ষরের জন্য ডিকশনারিতে কাউন্ট বাড়াব।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: freq = {} রাখা।

Step 3: for char in text চালিয়ে if char in freq: freq[char] += 1 else: freq[char] = 1 করা।

Step 4: ডিকশনারি প্রিন্ট করা।

Python Code

```
# Finding character frequency in a string
text = input("Enter a string: ")
freq = {}

for char in text:
    if char in freq:
        freq[char] += 1
    else:
        freq[char] = 1

print("Character Frequencies:")
```

```
for char, count in freq.items():
    print(f"'{char}' : {count}")
```

Built-in Function / Shortcut Method

```
# Using collections.Counter
# from collections import Counter
# print(Counter(text))
```

Sample Input / Output Table

Sample Input	Sample Output

`Enter a string: apple`	`Character Frequencies:`
	`'a' : 1`
	`'p' : 2`
	`'l' : 1`
	`'e' : 1`

ব্যাখ্যা

'apple' শব্দে p দুইবার এবং বাকিরা একবার করে এসেছে।

Common Mistakes

- স্পেসকেও অক্ষর হিসেবে গণনা করা (চাইলে if char != ' ' দিয়ে স্পেস বাদ দেওয়া যায়)।

অনুশীলনের জন্য

1. collections.Counter ব্যবহার করে ফ্রিকোয়েন্সি বের করুন।

2. সবচেয়ে বেশিবার আসা অক্ষরটি বের করুন।

3. শুধু ভাওয়েলগুলোর ফ্রিকোয়েন্সি বের করুন।

সমস্যা 116: একটি String-কে সম্পূর্ণ Uppercase-এ রূপান্তর করুন।

সমস্যা

একটি স্ট্রিংয়ের সকল অক্ষর বড় হাতের (Uppercase) অক্ষরে রূপান্তর করতে হবে।

কী কী জানা থাকতে হবে?

- upper() method

কীভাবে চিন্তা করব?

পাইথনের স্ট্রিং মেথড `.upper()` যেকোনো বাক্যের সব ছোট হাতের অক্ষরকে বড় হাতের অক্ষরে রূপান্তর করে।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: upper_text = text.upper() করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Converting string to Uppercase
text = input("Enter text: ")

upper_text = text.upper()
print("Uppercase Text:", upper_text)
```

Built-in Function / Shortcut Method

```
# Direct print
# print(input("Text: ").upper())
```

Sample Input / Output Table

Sample Input	Sample Output
Enter text: python programming	Uppercase Text: PYTHON PROGRAMMING

ব্যাখ্যা

`.upper()` মেথড সব অক্ষরকে ক্যাপিটাল লেটারে রূপান্তর করেছে।

Common Mistakes

- upper মেথডে ব্র্যাকেট না দেওয়া।

অনুশীলনের জন্য

1. ASCII মান থেকে ৩২ বিয়োগ করে ম্যানুয়াল লুপ দিয়ে upper করুন।

2. শুধু শব্দের প্রথম অক্ষর বড় হাতের (`title()`) করুন।

3. বাক্যের অর্ধেক অক্ষর upper করুন।

সমস্যা 117: একটি String-কে সম্পূর্ণ Lowercase-এ রূপান্তর করুন।

সমস্যা

একটি স্ট্রিংয়ের সকল অক্ষর ছোট হাতের (Lowercase) অক্ষরে রূপান্তর করতে হবে।

কী কী জানা থাকতে হবে?

- lower() method

কীভাবে চিন্তা করব?

পাইথনের `.lower()` মেথড ব্যবহার করে সব বড় হাতের অক্ষরকে ছোট হাতের অক্ষরে রূপান্তর করা যায়।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: lower_text = text.lower() করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Converting string to Lowercase
text = input("Enter text: ")

lower_text = text.lower()
print("Lowercase Text:", lower_text)
```

Built-in Function / Shortcut Method

```
# Direct print
# print(input("Text: ").lower())
```

Sample Input / Output Table

Sample Input	Sample Output
Enter text: PYTHON PROGRAMMING	Lowercase Text: python programming

ব্যাখ্যা

`.lower()` সব অক্ষরকে ছোট হাতের করে দিয়েছে।

Common Mistakes

- lower এবং upper গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. swapcase() মেথড ব্যবহার করে বড়কে ছোট ও ছোটকে বড় করুন।

2. ASCII মান থেকে ৩২ যোগ করে ম্যানুয়ালি lower করুন।

3. ইনপুট নেওয়ার সাথেই lower() করুন।

সমস্যা 118: একটি String-এর নির্দিষ্ট অংশ Replace করুন।

সমস্যা

একটি বাক্যের ভেতরের কোনো নির্দিষ্ট শব্দ বা অক্ষর পরিবর্তন করে নতুন শব্দ বা অক্ষর বসাতে হবে।

কী কী জানা থাকতে হবে?

- replace(old, new) method

কীভাবে চিন্তা করব?

পাইথনের `.replace(old_word, new_word)` মেথড দিয়ে সহজেই বাক্যের নির্দিষ্ট অংশ পরিবর্তন করা যায়।

অ্যালগরিদম

Step 1: মূল বাক্য ইনপুট নেওয়া।

Step 2: পুরনো শব্দ এবং নতুন শব্দ ইনপুট নেওয়া।

Step 3: new_text = text.replace(old, new) করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Replacing a part of a string
text = "I like Java programming"
print("Original text:", text)

new_text = text.replace("Java", "Python")
print("Updated text:", new_text)
```

Built-in Function / Shortcut Method

```
# Direct replace
# print(text.replace("Java", "Python"))
```

Sample Input / Output Table

Sample Input	Sample Output

None	Original text: I like Java programming Updated text: I like Python programming

ব্যাখ্যা

`replace("Java", "Python")` মেথডটি Java শব্দের জায়গায় Python বসিয়ে দিয়েছে।

Common Mistakes

- replace করার পর নতুন ভেরিয়েবলে না রেখে আগের স্ট্রিং প্রিন্ট করা (স্ট্রিং Immutable, তাই নতুন ভেরিয়েবলে রাখতে হয়)।

অনুশীলনের জন্য

1. বাক্যের সব স্পেসকে হাইফেন (-) দিয়ে replace করুন।

2. replace() ব্যবহার করে বাক্য থেকে সব Vowel মুছে ফেলুন (`replace(vowel, '')`)।

3. replace এর ৩য় প্যারামিটার ব্যবহার করে শুধুমাত্র প্রথম ২টি শব্দ পরিবর্তন করুন।

সমস্যা 119: একটি String থেকে Slice ব্যবহার করে নির্দিষ্ট অংশ বের করুন।

সমস্যা

স্ট্রিং স্লাইসিং (Slicing `[start:stop:step]`) ব্যবহার করে একটি বড় স্ট্রিং থেকে নির্দিষ্ট অংশ বা সাবস্ট্রিং কেটে বের করতে হবে।

কী কী জানা থাকতে হবে?

- String slicing [start:stop]

কীভাবে চিন্তা করব?

স্লাইসিংয়ের নিয়ম হলো `text[start:stop]`। start ইনডেক্স থেকে শুরু হয়ে stop এর আগের ইনডেক্স পর্যন্ত অংশ কেটে আনে। ইনডেক্স 0 থেকে শুরু হয়।

অ্যালগরিদম

Step 1: একটি স্ট্রিং text = 'Python Programming' নেওয়া।

Step 2: text[0:6] দিয়ে প্রথম শব্দ বের করা।

Step 3: text[7:] দিয়ে দ্বিতীয় শব্দ বের করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Demonstrating String Slicing
text = "Python Programming"

first_word = text[0:6]   # From index 0 to 5
second_word = text[7:]   # From index 7 to end
last_chars = text[-4:]   # Last 4 characters

print("Original Text:", text)
print("First Word [0:6]:", first_word)
print("Second Word [7:]:", second_word)
print("Last 4 Chars [-4:]:", last_chars)
```

Built-in Function / Shortcut Method

```
# Slicing with step
# print("Every second char:", text[::2])
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original Text: Python Programming First Word [0:6]: Python Second Word [7:]: Programming Last 4 Chars [-4:]: ming

ব্যাখ্যা

`text[0:6]` প্রথম ৬টি অক্ষর 'Python' কেটে এনেছে। ঋণাত্মক ইনডেক্স `[-4:]` শেষের ৪টি অক্ষর এনেছে।

Common Mistakes

- stop ইনডেক্স ১ বেশি দিতে হয় তা ভুলে যাওয়া।

অনুশীলনের জন্য

1. স্লাইসিং দিয়ে শব্দের মাঝখানের ৩টি অক্ষর বের করুন।

2. `text[::2]` দিয়ে জোড় ইনডেক্সের অক্ষরগুলো প্রিন্ট করুন।

3. ঋণাত্মক স্লাইসিং অনুশীলন করুন।

Topic: List Operations

সমস্যা 120: একটি List তৈরি করে বিভিন্ন মান সংরক্ষণ করুন।

সমস্যা

থার্ড ব্র্যাকেট `[]` ব্যবহার করে একটি List তৈরি করুন এবং তাতে সংখ্যা, স্ট্রিং ও ফ্লোটসহ বিভিন্ন ধরনের মান সংরক্ষণ করে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- List creation `[]`

কীভাবে চিন্তা করব?

পাইথনে লিস্ট তৈরি করা হয় থার্ড ব্র্যাকেট `[]` এর ভেতর কমা দিয়ে মান লিখে। এতে যেকোনো ধরনের ডাটা একসাথে রাখা যায়।

অ্যালগরিদম

Step 1: my_list = [10, 20.5, 'Python', True] তৈরি করা।

Step 2: print() দিয়ে লিস্ট প্রদর্শন করা।

Python Code

```
# Creating a mixed list
my_list = [10, 25.5, "Python", True, "Abdur Rahman"]

print("The list is:", my_list)
print("Data type:", type(my_list))
```

Built-in Function / Shortcut Method

```
# Using list() constructor
# num_list = list(range(1, 6))
```

Sample Input / Output Table

Sample Input	Sample Output
None	The list is: [10, 25.5, 'Python', True, 'Abdur Rahman'] Data type: <class 'list'>

ব্যাখ্যা

থার্ড ব্র্যাকেটের ভেতর কমা দিয়ে একাধিক ডাটা টাইপ একসাথে সংরক্ষণ করা হয়েছে।

Common Mistakes

- থার্ড ব্র্যাকেটের বদলে ফার্স্ট ব্র্যাকেট `()` ব্যবহার করা (যা Tuple তৈরি করে)।

অনুশীলনের জন্য

1. ৫টি ফলের নাম দিয়ে একটি লিস্ট তৈরি করুন।

2. একটি ফাঁকা লিস্ট তৈরি করে append() দিয়ে মান যোগ করুন।

3. লিস্টের ইনডেক্স 0 এবং 2 এর মান প্রিন্ট করুন।

সমস্যা 121: একটি List-এর প্রতিটি Element Loop ব্যবহার করে প্রদর্শন করুন।

সমস্যা

for loop ব্যবহার করে একটি List-এর প্রতিটি উপাদান আলাদা আলাদা লাইনে বা পাশাপাশি প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- for item in list

কীভাবে চিন্তা করব?

`for item in my_list:` লুপ চালালে লুপটি নিজে থেকেই লিস্টের প্রতিটি উপাদান একে একে `item` ভেরিয়েবলে নিয়ে আসে।

অ্যালগরিদম

Step 1: fruits = ['Apple', 'Banana', 'Mango'] তৈরি করা।

Step 2: for fruit in fruits লুপ চালানো।

Step 3: fruit প্রিন্ট করা।

Python Code

```
# Displaying list elements using loop
fruits = ["Apple", "Banana", "Mango", "Orange"]

print("Fruit List:")
for item in fruits:
    print("-", item)
```

Built-in Function / Shortcut Method

```
# Using range and len
# for i in range(len(fruits)): print(f"Index {i}: {fruits[i]}")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Fruit List: - Apple - Banana - Mango - Orange

ব্যাখ্যা

লুপটি লিস্টের ৪টি উপাদানের জন্য ৪ বার ঘুরে প্রতিটি ফলের নাম প্রদর্শন করেছে।

Common Mistakes

- লুপে লিস্টের নাম না দিয়ে ভুল ভেরিয়েবল দেওয়া।

অনুশীলনের জন্য

1. enumerate() ব্যবহার করে ইনডেক্সসহ লিস্ট প্রিন্ট করুন।

2. while লুপ ব্যবহার করে লিস্ট প্রিন্ট করুন।

3. লিস্টের উপাদানগুলো কমা দিয়ে একই লাইনে প্রিন্ট করুন।

সমস্যা 122: একটি List-এর সকল সংখ্যার যোগফল নির্ণয় করুন।

সমস্যা

একটি সংখ্যার লিস্ট থেকে লুপ চালিয়ে সকল সংখ্যার যোগফল নির্ণয় করতে হবে।

কী কী জানা থাকতে হবে?

- Loop summation
- sum() function

কীভাবে চিন্তা করব?

`total = 0` রেখে লুপ চালিয়ে লিস্টের প্রতিটি সংখ্যা `total += num` করব। অথবা বিল্ট-ইন `sum(numbers)` ব্যবহার করব।

অ্যালগরিদম

Step 1: numbers = [10, 20, 30, 40, 50] তৈরি করা।

Step 2: total = 0 রাখা।

Step 3: for num in numbers চালিয়ে total += num করা।

Step 4: total প্রিন্ট করা।

Python Code

```
# Sum of all elements in a list
numbers = [15, 25, 35, 45, 55]
total = 0

for num in numbers:
    total += num

print("Sum of list elements:", total)
```

Built-in Function / Shortcut Method

```
# Using built-in sum()
# print("Sum:", sum(numbers))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Sum of list elements: 175

ব্যাখ্যা

লিস্টের ৫টি সংখ্যা যোগ হয়ে ১৭5 হয়েছে।

Common Mistakes

- total = 0 লুপের ভেতর রাখা।

অনুশীলনের জন্য

1. sum() ফাংশন ব্যবহার করে যোগফল বের করুন।

2. ব্যবহারকারীর কাছ থেকে ৫টি সংখ্যা ইনপুট নিয়ে লিস্টে রেখে যোগফল বের করুন।

3. লিস্টের জোড় সংখ্যাগুলোর যোগফল বের করুন।

সমস্যা 123: একটি List-এর সংখ্যাগুলোর Average নির্ণয় করুন।

সমস্যা

একটি সংখ্যার লিস্টের গড় (Average) নির্ণয় করতে হবে। গড় = মোট যোগফল / মোট উপাদান সংখ্যা।

কী কী জানা থাকতে হবে?

- len() function
- sum() function

কীভাবে চিন্তা করব?

লিস্টের যোগফলকে লিস্টের দৈর্ঘ্য `len(numbers)` দিয়ে ভাগ করলেই গড় পাওয়া যায়।

অ্যালগরিদম

Step 1: লিস্ট তৈরি করা।

Step 2: total = sum(numbers) বের করা।

Step 3: avg = total / len(numbers) করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Average of list elements
numbers = [10, 20, 30, 40, 50]

total = sum(numbers)
count = len(numbers)

average = total / count
print("Average is:", average)
```

Built-in Function / Shortcut Method

```
# Using statistics module
# import statistics
# print("Average:", statistics.mean(numbers))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Average is: 30.0

ব্যাখ্যা

১৫০ কে ৫ দ্বারা ভাগ করে ৩০.০ গড় বের করা হয়েছে।

Common Mistakes

- ফাঁকা লিস্টের গড় বের করতে গিয়ে ZeroDivisionError করা।

অনুশীলনের জন্য

1. লুপ ব্যবহার করে যোগফল ও দৈর্ঘ্য মেপে গড় বের করুন।

2. ফ্লোট সংখ্যার লিস্টের গড় বের করুন।

3. লিস্টের গড়ের চেয়ে বড় সংখ্যাগুলো আলাদা করুন।

সমস্যা 124: একটি List থেকে সবচেয়ে বড় সংখ্যাটি নির্ণয় করুন।

সমস্যা

লুপ চালিয়ে একটি List-এর সবচেয়ে বড় সংখ্যাটি (Maximum) খুঁজে বের করতে হবে।

কী কী জানা থাকতে হবে?

- max() function
- Comparison inside loop

কীভাবে চিন্তা করব?

লিস্টের প্রথম উপাদানটিকে আমরা আপাতত সবচেয়ে বড় `max_num = numbers[0]` ধরে নেব। এরপর লুপ চালিয়ে দেখব কোনো সংখ্যা `max_num` এর চেয়ে বড় কিনা। বড় হলে `max_num` আপডেট করব।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া।

Step 2: max_val = numbers[0] ধরা।

Step 3: for num in numbers চালিয়ে if num > max_val: max_val = num করা।

Step 4: max_val প্রিন্ট করা।

Python Code

```
# Finding maximum in a list
numbers = [45, 89, 12, 96, 54, 23]
max_val = numbers[0]

for num in numbers:
    if num > max_val:
        max_val = num
```

```
print("Maximum number is:", max_val)
```

Built-in Function / Shortcut Method

```
# Using built-in max()
# print("Max:", max(numbers))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Maximum number is: 96

ব্যাখ্যা

লুপে প্রতিটি সংখ্যা যাচাই করে ৯৬ কে সবচেয়ে বড় হিসেবে চিহ্নিত করা হয়েছে।

Common Mistakes

- `max_val = 0` ধরে শুরু করা (যা ঋণাত্মক সংখ্যার লিস্টে ভুল উত্তর দেবে। সবসময় `numbers[0]` ধরতে হয়)।

অনুশীলনের জন্য

1. max() ফাংশন ব্যবহার করুন।

2. দ্বিতীয় সর্বোচ্চ (Second largest) সংখ্যাটি বের করুন।

3. সবচেয়ে বড় সংখ্যাটির ইনডেক্স কত তা বের করুন।

সমস্যা 125: একটি List থেকে সবচেয়ে ছোট সংখ্যাটি নির্ণয় করুন।

সমস্যা

লুপ চালিয়ে একটি List-এর সবচেয়ে ছোট সংখ্যাটি (Minimum) খুঁজে বের করতে হবে।

কী কী জানা থাকতে হবে?

- min() function
- Comparison inside loop (<)

কীভাবে চিন্তা করব?

প্রথম সংখ্যাটিকে `min_val = numbers[0]` ধরে লুপ চালিয়ে দেখব কোনো সংখ্যা তার চেয়ে ছোট কিনা। ছোট হলে `min_val` আপডেট করব।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া।

Step 2: min_val = numbers[0] ধরা।

Step 3: for num in numbers চালিয়ে if num < min_val: min_val = num করা।

Step 4: min_val প্রিন্ট করা।

Python Code

```
# Finding minimum in a list
numbers = [45, 89, 12, 96, 54, 23]
min_val = numbers[0]

for num in numbers:
    if num < min_val:
        min_val = num

print("Minimum number is:", min_val)
```

Built-in Function / Shortcut Method

```
# Using built-in min()
# print("Min:", min(numbers))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Minimum number is: 12

ব্যাখ্যা

লুপে যাচাই করে ১২ কে সবচেয়ে ছোট হিসেবে পাওয়া গেছে।

Common Mistakes

- `min_val = 99999` ধরা (এর বদলে `numbers[0]` ধরা নিরাপদ)।

অনুশীলনের জন্য

1. min() ফাংশন ব্যবহার করুন।

2. সবচেয়ে ছোট ও সবচেয়ে বড় সংখ্যার পার্থক্য বের করুন।

3. দ্বিতীয় সর্বনিম্ন সংখ্যাটি বের করুন।

সমস্যা 126: একটি List থেকে সব জোড় সংখ্যা আলাদা করে প্রদর্শন করুন।

সমস্যা

একটি লিস্ট থেকে শুধুমাত্র জোড় সংখ্যাগুলো আলাদা করে নতুন লিস্টে সংরক্ষণ ও প্রদর্শন করতে হবে।

কী কী জানা থাকতে হবে?

- List append() method
- Modulus (%)

কীভাবে চিন্তা করব?

একটি ফাঁকা লিস্ট `even_list = []` নেব। মূল লিস্টে লুপ চালিয়ে যে সংখ্যাগুলো ২ দিয়ে বিভাজ্য (`num % 2 == 0`), সেগুলোকে `even_list.append(num)` করব।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া এবং evens = [] রাখা।

Step 2: for num in numbers চালিয়ে if num % 2 == 0: evens.append(num) করা।

Step 3: evens লিস্ট প্রিন্ট করা।

Python Code

```
# Separating even numbers from list
numbers = [12, 15, 18, 21, 24, 27, 30]
evens = []

for num in numbers:
    if num % 2 == 0:
        evens.append(num)

print("Original list:", numbers)
print("Even numbers:", evens)
```

Built-in Function / Shortcut Method

```
# List comprehension
# evens = [x for x in numbers if x % 2 == 0]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original list: [12, 15, 18, 21, 24, 27, 30] Even numbers: [12, 18, 24, 30]

ব্যাখ্যা

১২, ১৮, ২৪ এবং ৩০ জোড় হওয়ায় তারা evens লিস্টে যুক্ত হয়েছে।

Common Mistakes

- append() এর বদলে `=` ব্যবহার করা।

অনুশীলনের জন্য

1. লিস্ট কম্প্রিহেনশন ব্যবহার করে জোড় আলাদা করুন।

2. জোড় সংখ্যাগুলোর যোগফল বের করুন।

3. জোড় সংখ্যা কয়টি তা লুপ দিয়ে গুনুন।

সমস্যা 127: একটি List থেকে সব বিজোড় সংখ্যা আলাদা করে প্রদর্শন করুন।

সমস্যা

একটি লিস্ট থেকে শুধুমাত্র বিজোড় সংখ্যাগুলো আলাদা করে নতুন লিস্টে রাখতে হবে।

কী কী জানা থাকতে হবে?

- num % 2 != 0
- List append()

কীভাবে চিন্তা করব?

যেসব সংখ্যা ২ দ্বারা বিভাজ্য নয় (`num % 2 != 0`), সেগুলোকে `odds.append(num)` করব।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া এবং odds = [] রাখা।

Step 2: for num in numbers চালিয়ে if num % 2 != 0: odds.append(num) করা।

Step 3: odds প্রিন্ট করা।

Python Code

```
# Separating odd numbers from list
numbers = [12, 15, 18, 21, 24, 27, 30]
odds = []

for num in numbers:
    if num % 2 != 0:
        odds.append(num)

print("Odd numbers:", odds)
```

Built-in Function / Shortcut Method

```
# List comprehension
# odds = [x for x in numbers if x % 2 != 0]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Odd numbers: [15, 21, 27]

ব্যাখ্যা

১৫, ২১ এবং ২৭ বিজোড় হওয়ায় তারা আলাদা লিস্টে সংরক্ষিত হয়েছে।

Common Mistakes

- `% 2 == 1` এর বদলে ভুল শর্ত দেওয়া।

অনুশীলনের জন্য

1. বিজোড় সংখ্যাগুলোকে ২ গুণ করে নতুন লিস্ট বানান।

2. জোড় ও বিজোড় সংখ্যাকে একই লুপে দুটি আলাদা লিস্টে ভাগ করুন।

3. বিজোড় লিস্টের গড় বের করুন।

সমস্যা 128: একটি List থেকে সব ধনাত্মক সংখ্যা আলাদা করুন।

সমস্যা

একটি লিস্টে থাকা ধনাত্মক ও ঋণাত্মক সংখ্যার মধ্য থেকে শুধুমাত্র ধনাত্মক সংখ্যাগুলো (Positive numbers > 0) আলাদা করতে হবে।

কী কী জানা থাকতে হবে?

- num > 0 condition

কীভাবে চিন্তা করব?

লুপ চালিয়ে `if num > 0:` হলে নতুন লিস্টে `append()` করব।

অ্যালগরিদম

Step 1: numbers = [-10, 25, -5, 40, 0, 15] নেওয়া।

Step 2: positives = [] রাখা।

Step 3: for num in numbers চালিয়ে if num > 0: positives.append(num) করা।

Step 4: positives প্রিন্ট করা।

Python Code

```
# Separating positive numbers
numbers = [-15, 20, -8, 35, -2, 50, 0]
positives = []

for num in numbers:
    if num > 0:
        positives.append(num)

print("Positive numbers:", positives)
```

Built-in Function / Shortcut Method

```
# Comprehension
# positives = [x for x in numbers if x > 0]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Positive numbers: [20, 35, 50]

ব্যাখ্যা

শূন্যের চেয়ে বড় সংখ্যাগুলো আলাদা করা হয়েছে।

Common Mistakes

- শূন্য (0) কে ধনাত্মক মনে করে `>= 0` দেওয়া।

অনুশীলনের জন্য

1. কম্প্রিহেনশন দিয়ে ধনাত্মক আলাদা করুন।

2. ধনাত্মক সংখ্যাগুলোর যোগফল বের করুন।

3. লিস্ট থেকে সব ঋণাত্মক সংখ্যা remove() করুন।

সমস্যা 129: একটি List থেকে সব ঋণাত্মক সংখ্যা আলাদা করুন।

সমস্যা

লিস্ট থেকে শুধুমাত্র ঋণাত্মক সংখ্যাগুলো (Negative numbers < 0) আলাদা করতে হবে।

কী কী জানা থাকতে হবে?

- num < 0 condition

কীভাবে চিন্তা করব?

লুপ চালিয়ে `if num < 0:` হলে `negatives.append(num)` করব।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া।

Step 2: negatives = [] রাখা।

Step 3: for num in numbers চালিয়ে if num < 0: negatives.append(num) করা।

Step 4: negatives প্রিন্ট করা।

Python Code

```
# Separating negative numbers
numbers = [-15, 20, -8, 35, -2, 50, 0]
negatives = []

for num in numbers:
    if num < 0:
        negatives.append(num)

print("Negative numbers:", negatives)
```

Built-in Function / Shortcut Method

```
# Comprehension
# negatives = [x for x in numbers if x < 0]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Negative numbers: [-15, -8, -2]

ব্যাখ্যা

শূন্যের চেয়ে ছোট সংখ্যাগুলো আলাদা হয়েছে।

Common Mistakes

- শর্তে দিক ভুল করা।

অনুশীলনের জন্য

1. ঋণাত্মক সংখ্যাগুলোকে abs() দিয়ে ধনাত্মক বানিয়ে লিস্টে রাখুন।

2. ঋণাত্মক সংখ্যা কয়টি তা গুনুন।

3. ধনাত্মক ও ঋণাত্মক আলাদা লিস্টে রাখুন।

সমস্যা 130: একটি List Ascending Order-এ Sort করুন।

সমস্যা

একটি এলোমেলো সংখ্যার লিস্টকে ছোট থেকে বড় (Ascending Order) ক্রমে সাজাতে হবে।

কী কী জানা থাকতে হবে?

- sort() method বা sorted() function

কীভাবে চিন্তা করব?

লিস্টের নিজস্ব মেথড `.sort()` মূল লিস্টটিকে ছোট থেকে বড় সাজিয়ে দেয়। আর `sorted(my_list)` নতুন একটি সাজানো লিস্ট দেয়।

অ্যালগরিদম

Step 1: numbers = [45, 12, 89, 23, 5] নেওয়া।

Step 2: numbers.sort() কল করা।

Step 3: numbers প্রিন্ট করা।

Python Code

```
# Sorting list in Ascending Order
numbers = [54, 23, 89, 12, 67, 34]
print("Original list:", numbers)

numbers.sort()
print("Sorted list (Ascending):", numbers)
```

Built-in Function / Shortcut Method

```
# Using sorted() function
# sorted_list = sorted(numbers)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original list: [54, 23, 89, 12, 67, 34] Sorted list (Ascending): [12, 23, 34, 54, 67, 89]

ব্যাখ্যা

`.sort()` মেথড সংখ্যাগুলোকে ছোট থেকে বড় ক্রমে সাজিয়ে দিয়েছে।

Common Mistakes

- `new_list = numbers.sort()` করা (sort() কোনো মান রিটার্ন করে না, None দেয়। তাই সরাসরি `numbers.sort()` লিখতে হয়)।

অনুশীলনের জন্য

1. Bubble Sort অ্যালগরিদম ব্যবহার করে লুপ দিয়ে লিস্ট সাজান।

2. স্ট্রিংয়ের লিস্টকে বর্ণানুক্রমে (A-Z) সাজান।

3. sorted() ফাংশন ব্যবহার করুন।

সমস্যা 131: একটি List Descending Order-এ Sort করুন।

সমস্যা

একটি লিস্টকে বড় থেকে ছোট (Descending Order) ক্রমে সাজাতে হবে।

কী কী জানা থাকতে হবে?

- sort(reverse=True)

কীভাবে চিন্তা করব?

`.sort(reverse=True)` লিখলে লিস্টটি বড় থেকে ছোট ক্রমে সাজানো হয়।

অ্যালগরিদম

Step 1: numbers লিস্ট নেওয়া।

Step 2: numbers.sort(reverse=True) কল করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Sorting list in Descending Order
```

```
numbers = [54, 23, 89, 12, 67, 34]

numbers.sort(reverse=True)
print("Sorted list (Descending):", numbers)
```

Built-in Function / Shortcut Method

```
# Using sorted()
# desc_list = sorted(numbers, reverse=True)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Sorted list (Descending): [89, 67, 54, 34, 23, 12]

ব্যাখ্যা

`reverse=True` প্যারামিটার দেওয়ার কারণে বড় থেকে ছোট ক্রমে সাজানো হয়েছে।

Common Mistakes

- `reverse=true` ছোট হাতের লেখা।

অনুশীলনের জন্য

1. আগে sort() করে পরে reverse() করে বড় থেকে ছোট করুন।

2. স্ট্রিং লিস্টকে Z থেকে A ক্রমে সাজান।

3. লুপ দিয়ে Descending sort করুন।

সমস্যা 132: একটি List থেকে Duplicate Element অপসারণ করুন।

সমস্যা

একটি লিস্টে একই সংখ্যা একাধিকবার থাকলে (Duplicate), তা সরিয়ে প্রতিটি মান একবার করে রেখে নতুন লিস্ট তৈরি করুন।

কী কী জানা থাকতে হবে?

- set() conversion বা loop checking

কীভাবে চিন্তা করব?

সবচেয়ে সহজ উপায় হলো লিস্টকে `set()` এ রূপান্তর করা কারণ Set কখনো ডুপ্লিকেট রাখে না। তবে ক্রম ঠিক রাখতে চাইলে লুপ চালিয়ে `if item not in unique_list:` হলে যোগ করা ভালো।

অ্যালগরিদম

Step 1: numbers = [10, 20, 10, 30, 20, 40] নেওয়া।

Step 2: unique_list = [] রাখা।

Step 3: লুপ চালিয়ে if num not in unique_list: unique_list.append(num) করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Removing duplicates while preserving order
numbers = [10, 25, 10, 30, 25, 40, 50, 30]
unique_list = []

for num in numbers:
    if num not in unique_list:
        unique_list.append(num)

print("Original list:", numbers)
print("Without duplicates:", unique_list)
```

Built-in Function / Shortcut Method

```
# Using set (order may change)
# print("Unique:", list(set(numbers)))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original list: [10, 25, 10, 30, 25, 40, 50, 30] Without duplicates: [10, 25, 30, 40, 50]

ব্যাখ্যা

লুপে যাচাই করে শুধুমাত্র নতুন উপাদানগুলো unique_list এ রাখা হয়েছে।

Common Mistakes

- set() করার পর লিস্টের ক্রম পরিবর্তন হওয়া নিয়ে অবাক হওয়া।

অনুশীলনের জন্য

1. set() ব্যবহার করে ডুপ্লিকেট দূর করুন।

2. কোন কোন উপাদান ডুপ্লিকেট ছিল তাদের তালিকা করুন।

3. স্ট্রিং লিস্ট থেকে ডুপ্লিকেট শব্দ সরান।

সমস্যা 133: একটি List-এ নির্দিষ্ট Element খুঁজে বের করুন।

সমস্যা

ব্যবহারকারীর ইনপুট করা কোনো নির্দিষ্ট সংখ্যা বা শব্দ একটি লিস্টে আছে কিনা এবং থাকলে তা কোন Index-এ আছে তা খুঁজে বের করুন (Linear Search)।

কী কী জানা থাকতে হবে?

- in operator / index() method

কীভাবে চিন্তা করব?

লুপ চালিয়ে প্রতিটি ইনডেক্স যাচাই করব `if numbers[i] == target:`। পেলে ইনডেক্স প্রিন্ট করে break করব। অথবা `in` অপারেটর ও `index()` মেথড ব্যবহার করব।

অ্যালগরিদম

Step 1: লিস্ট তৈরি ও target ইনপুট নেওয়া।

Step 2: found = False রাখা।

Step 3: for i in range(len(numbers)) চালিয়ে if numbers[i] == target হলে found = True ও ইনডেক্স প্রিন্ট করা।

Step 4: না পেলে Not found প্রিন্ট করা।

Python Code

```
# Searching an element in a list (Linear Search)
numbers = [15, 28, 35, 42, 59, 64]
target = int(input("Enter number to search: "))

found = False
for i in range(len(numbers)):
    if numbers[i] == target:
        print(f"Number {target} found at index {i}")
        found = True
        break

if not found:
    print(f"Number {target} is NOT in the list")
```

Built-in Function / Shortcut Method

```
# Using in and index()
# if target in numbers: print("Found at index:", numbers.index(target))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter number to search: 42	Number 42 found at index 3

ব্যাখ্যা

৪২ সংখ্যাটি লিস্টের ৪ নম্বর স্থানে অর্থাৎ ইনডেক্স ৩ এ পাওয়া গেছে।

Common Mistakes

- index() মেথডে লিস্টে না থাকা মান দিলে ValueError হওয়া।

অনুশীলনের জন্য

1. index() মেথড ব্যবহার করে উপাদান খুঁজুন।

2. একাধিকবার থাকলে সবগুলোর ইনডেক্স প্রিন্ট করুন।

3. Binary Search অ্যালগরিদম দিয়ে খুঁজুন।

সমস্যা 134: একটি List-এ নির্দিষ্ট Element কতবার আছে তা গণনা করুন।

সমস্যা

একটি নির্দিষ্ট উপাদান লিস্টের মধ্যে মোট কতবার এসেছে তা count() মেথড বা লুপ দিয়ে গণনা করুন।

কী কী জানা থাকতে হবে?

- count() method

কীভাবে চিন্তা করব?

লুপ চালিয়ে `if item == target:` হলে কাউন্ট বাড়াব। অথবা `numbers.count(target)` ব্যবহার করব।

অ্যালগরিদম

Step 1: লিস্ট তৈরি করা।

Step 2: target ইনপুট নেওয়া।

Step 3: count = numbers.count(target) করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Counting occurrences of an element
numbers = [10, 20, 10, 30, 10, 40, 20]
target = int(input("Enter number to count: "))

count = 0
for num in numbers:
    if num == target:
        count += 1

print(f"The number {target} appears {count} times")
```

Built-in Function / Shortcut Method

```
# Using built-in count()
# print("Count:", numbers.count(target))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter number to count: 10	The number 10 appears 3 times

ব্যাখ্যা

১০ সংখ্যাটি লিস্টে মোট ৩ বার উপস্থিত আছে।

Common Mistakes

- count() মেথডের নাম ভুল করা।

অনুশীলনের জন্য

1. count() মেথড ব্যবহার করুন।

2. লিস্টের প্রতিটি উপাদানের ফ্রিকোয়েন্সি বের করুন।

3. সবচেয়ে বেশিবার আসা সংখ্যাটি বের করুন।

সমস্যা 135: একটি List উল্টো (Reverse) করুন।

সমস্যা

একটি লিস্টের উপাদানগুলোর ক্রম সম্পূর্ণ উল্টো (Reverse) করে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- reverse() method / slicing [::-1]

কীভাবে চিন্তা করব?

লিস্টের নিজস্ব মেথড `.reverse()` মূল লিস্টটিকে উল্টে দেয়। আর স্লাইসিং `my_list[::-1]` নতুন উল্টো লিস্ট দেয়।

অ্যালগরিদম

Step 1: numbers = [10, 20, 30, 40, 50] নেওয়া।

Step 2: numbers.reverse() কল করা।

Step 3: numbers প্রিন্ট করা।

Python Code

```
# Reversing a list
numbers = [10, 20, 30, 40, 50]
print("Original list:", numbers)

numbers.reverse()
print("Reversed list:", numbers)
```

Built-in Function / Shortcut Method

```
# Using slicing
# rev_list = numbers[::-1]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original list: [10, 20, 30, 40, 50] Reversed list: [50, 40, 30, 20, 10]

ব্যাখ্যা

`.reverse()` মেথড শেষের উপাদানকে প্রথমে এবং প্রথমটিকে শেষে এনে লিস্ট উল্টে দিয়েছে।

Common Mistakes

- `reversed()` ফাংশন ব্যবহার করার পর `list()` এ রূপান্তর না করা।

অনুশীলনের জন্য

1. স্লাইসিং `[::-1]` ব্যবহার করে উল্টান।

2. লুপ চালিয়ে একটি নতুন লিস্টে উল্টো করে রাখুন।

3. reversed() ফাংশন ব্যবহার করুন।

Topic: Tuple & Set Operations

সমস্যা 136: একটি Tuple তৈরি করুন।

সমস্যা

ফার্স্ট ব্র্যাকেট `()` ব্যবহার করে একটি Tuple তৈরি করুন এবং প্রদর্শন করুন। Tuple হলো Immutable (যার মান পরিবর্তন করা যায় না)।

কী কী জানা থাকতে হবে?

- Tuple syntax `()`

কীভাবে চিন্তা করব?

লিস্টে থার্ড ব্র্যাকেট ব্যবহৃত হয় আর টাপলে ফার্স্ট ব্র্যাকেট `()` ব্যবহৃত হয়।

অ্যালগরিদম

Step 1: my_tuple = (10, 20, 30, 40) তৈরি করা।

Step 2: প্রিন্ট করা।

Python Code

```
# Creating a simple Tuple
my_tuple = (10, 20, 30, 40, 50)

print("The Tuple is:", my_tuple)
print("Data type:", type(my_tuple))
```

Built-in Function / Shortcut Method

```
# Tuple without brackets (Packing)
# my_tuple = 10, 20, 30
```

Sample Input / Output Table

Sample Input	Sample Output

```
None                                    The Tuple is: (10, 20, 30, 40,
                                        50)
                                        Data type: <class 'tuple'>
```

ব্যাখ্যা

ফার্স্ট ব্র্যাকেটের ভেতর মান রেখে একটি টাপল তৈরি করা হয়েছে।

Common Mistakes

- একটিমাত্র উপাদানের টাপল বানানোর সময় কমা না দেওয়া (যেমন `(10)` দিলে তা int হয়, `(10,)` লিখতে হয়)।

অনুশীলনের জন্য

1. ১টি উপাদান বিশিষ্ট টাপল `(5,)` তৈরি করে টাইপ দেখুন।

2. ব্র্যাকেট ছাড়া কমা দিয়ে টাপল তৈরি করুন।

3. টাপলের কোনো উপাদান পরিবর্তনের চেষ্টা করে এরর দেখুন।

সমস্যা 137: একটি Empty Tuple তৈরি করুন।

সমস্যা

কোনো উপাদান ছাড়া একটি খালি বা Empty Tuple তৈরি করুন।

কী কী জানা থাকতে হবে?

- Empty `()` বা tuple() constructor

কীভাবে চিন্তা করব?

শুধুমাত্র দুটি ফার্স্ট ব্র্যাকেট `()` দিয়ে বা `tuple()` লিখে খালি টাপল তৈরি করা যায়।

অ্যালগরিদম

Step 1: empty_tuple = () তৈরি করা।

Step 2: প্রিন্ট করা ও টাইপ দেখা।

Python Code

```
# Creating an Empty Tuple
empty_tuple = ()

print("Empty tuple:", empty_tuple)
print("Length:", len(empty_tuple))
print("Type:", type(empty_tuple))
```

Built-in Function / Shortcut Method

```
# Using tuple() constructor
# empty_tuple = tuple()
```

Sample Input / Output Table

Sample Input	Sample Output
None	Empty tuple: () Length: 0 Type: <class 'tuple'>

ব্যাখ্যা

খালি ব্র্যাকেট দিয়ে 0 দৈর্ঘ্যের টাপল তৈরি হয়েছে।

Common Mistakes

- খালি টাপলে append করার চেষ্টা করা (টাপল Immutable)।

অনুশীলনের জন্য

1. tuple() কনস্ট্রাক্টর ব্যবহার করে খালি টাপল বানান।

2. খালি লিস্ট ও খালি টাপলের পার্থক্য দেখুন।

3. if not empty_tuple: লিখে যাচাই করুন।

সমস্যা 138: বিভিন্ন Data Type ব্যবহার করে একটি Mixed Tuple তৈরি করুন।

সমস্যা

Integer, Float, String এবং Boolean মান একসাথে রেখে একটি Mixed Tuple তৈরি করুন।

কী কী জানা থাকতে হবে?

- Mixed data types inside tuple

কীভাবে চিন্তা করব?

টাপলে যেকোনো ধরনের ডাটা রাখা যায়।

অ্যালগরিদম

Step 1: mixed_tuple = (100, 3.14, 'Hello', True) তৈরি করা।

Step 2: প্রিন্ট করা।

Python Code

```
# Creating a Mixed Tuple
mixed_tuple = (101, 99.5, "Abdur Rahman", True)

print("Mixed Tuple:", mixed_tuple)
for item in mixed_tuple:
    print(f"Value: {item} | Type: {type(item)}")
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Mixed Tuple: (101, 99.5, 'Abdur Rahman', True) Value: 101 | Type: <class 'int'> Value: 99.5 | Type: <class 'float'> Value: Abdur Rahman | Type: <class 'str'> Value: True | Type: <class 'bool'>

ব্যাখ্যা

ভিন্ন ভিন্ন ডাটা টাইপ একই টাপলে সংরক্ষিত হয়েছে।

Common Mistakes

- ডাটা টাইপ গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. টাপলের ভেতর একটি লিস্ট রাখুন।

2. টাপলের ভেতরের লিস্টের মান পরিবর্তন করে দেখুন (টাপলের ভেতরের লিস্ট Mutable হয়)।

3. মিক্সড টাপলের দৈর্ঘ্য বের করুন।

সমস্যা 139: একটি Nested Tuple তৈরি করুন।

সমস্যা

একটি টাপলের ভেতরে এক বা একাধিক টাপল রেখে Nested Tuple তৈরি ও প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- Nested structure `((), ())`

কীভাবে চিন্তা করব?

টাপলের উপাদান হিসেবে অন্য টাপল রাখা যায়। অ্যাক্সেস করতে ডাবল ইনডেক্স `[i][j]` লাগে।

অ্যালগরিদম

Step 1: nested = ((1, 2), (3, 4), (5, 6)) তৈরি করা।

Step 2: প্রিন্ট করা।

Python Code

```
# Creating a Nested Tuple
nested_tuple = ((10, 20), (30, 40), (50, 60))

print("Nested Tuple:", nested_tuple)
print("First inner tuple:", nested_tuple[0])
print("Element 40 inside second inner tuple:", nested_tuple[1][1])
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Nested Tuple: ((10, 20), (30, 40), (50, 60)) First inner tuple: (10, 20) Element 40 inside second inner tuple: 40

ব্যাখ্যা

`nested_tuple[1][1]` দিয়ে ২য় টাপলের ২য় উপাদান ৪০ বের করা হয়েছে।

Common Mistakes

- ইনডেক্সিং ভুল করা।

অনুশীলনের জন্য

1. ৩ স্তরের নেস্টেড টাপল তৈরি করুন।

2. লুপ চালিয়ে নেস্টেড টাপলের সব মান প্রিন্ট করুন।

3. ম্যাট্রিক্স আকারে টাপল সাজান।

সমস্যা 140: একটি Tuple-এর প্রতিটি Element Loop ব্যবহার করে প্রদর্শন করুন।

সমস্যা

for loop ব্যবহার করে টাপলের প্রতিটি উপাদান প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- for item in tuple

কীভাবে চিন্তা করব?

লিস্টের মতোই টাপলে `for item in my_tuple:` লুপ চলে।

অ্যালগরিদম

Step 1: colors = ('Red', 'Green', 'Blue') তৈরি করা।

Step 2: লুপ চালিয়ে প্রিন্ট করা।

Python Code

```
# Looping through a Tuple
colors = ("Red", "Green", "Blue", "Yellow")

print("Color Tuple Elements:")
for color in colors:
    print("->", color)
```

Built-in Function / Shortcut Method

```
# Using range and len
# for i in range(len(colors)): print(colors[i])
```

Sample Input / Output Table

Sample Input	Sample Output

```
None                                    Color Tuple Elements:
                                        -> Red
                                        -> Green
                                        -> Blue
                                        -> Yellow
```

ব্যাখ্যা

লুপটি টাপলের ৪টি রঙের নাম প্রিন্ট করেছে।

Common Mistakes

- টাপলকে লিস্ট মনে করে append করার চেষ্টা করা।

অনুশীলনের জন্য

1. while লুপ দিয়ে টাপল প্রিন্ট করুন।

2. enumerate() দিয়ে ইনডেক্সসহ প্রিন্ট করুন।

3. উল্টো লুপ চালিয়ে টাপল প্রিন্ট করুন।

সমস্যা 141: একটি Tuple-এর নির্দিষ্ট Index-এর মান প্রদর্শন করুন।

সমস্যা

ইনডেক্স নম্বর ব্যবহার করে টাপলের নির্দিষ্ট উপাদান (যেমন প্রথম বা তৃতীয় উপাদান) অ্যাক্সেস করুন।

কী কী জানা থাকতে হবে?

- Indexing `my_tuple[i]`

কীভাবে চিন্তা করব?

ইনডেক্স 0 থেকে শুরু হয়। `my_tuple[2]` দিলে ৩য় উপাদান পাওয়া যায়।

অ্যালগরিদম

Step 1: numbers = (100, 200, 300, 400) তৈরি করা।

Step 2: numbers[0] এবং numbers[2] প্রিন্ট করা।

Python Code

```
# Accessing tuple elements by Index
numbers = (100, 200, 300, 400, 500)

print("First element (index 0):", numbers[0])
print("Third element (index 2):", numbers[2])
print("Last element (index -1):", numbers[-1])
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	First element (index 0): 100 Third element (index 2): 300 Last element (index -1): 500

ব্যাখ্যা

ইনডেক্সিংয়ের মাধ্যমে টাপলের উপাদানগুলো নির্ভুলভাবে বের করা হয়েছে।

Common Mistakes

- টাপলের সীমানার বাইরের ইনডেক্স দিয়ে IndexError করা।

অনুশীলনের জন্য

1. ঋণাত্মক ইনডেক্স ব্যবহার করে শেষের ২টি উপাদান বের করুন।

2. ব্যবহারকারীর ইনপুট করা ইনডেক্সের মান প্রিন্ট করুন।

3. মাঝখানের উপাদান বের করুন।

সমস্যা 142: Tuple Slicing ব্যবহার করে নির্দিষ্ট অংশ বের করুন।

সমস্যা

স্লাইসিং `[start:stop]` ব্যবহার করে টাপলের একটি অংশ বা সাব-টাপল বের করুন।

কী কী জানা থাকতে হবে?

- Tuple slicing

কীভাবে চিন্তা করব?

লিস্ট ও স্ট্রিংয়ের মতোই টাপলে স্লাইসিং কাজ করে।

অ্যালগরিদম

Step 1: data = (10, 20, 30, 40, 50, 60) নেওয়া।

Step 2: data[1:4] কেটে নেওয়া।

Step 3: প্রিন্ট করা।

Python Code

```
# Demonstrating Tuple Slicing
data = (10, 20, 30, 40, 50, 60, 70)

sub_tuple = data[1:5]  # From index 1 to 4
print("Original Tuple:", data)
print("Sliced Tuple [1:5]:", sub_tuple)
```

Built-in Function / Shortcut Method

```
# Slicing with step
# print("Every second item:", data[::2])
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original Tuple: (10, 20, 30, 40, 50, 60, 70) Sliced Tuple [1:5]: (20, 30, 40, 50)

ব্যাখ্যা

ইনডেক্স ১ থেকে ৪ পর্যন্ত ৪টি উপাদান নিয়ে নতুন টাপল তৈরি হয়েছে।

Common Mistakes

- stop মান ১ বেশি ধরতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. `data[:3]` এবং `data[3:]` বের করুন।

2. স্লাইসিং দিয়ে টাপল রিভার্স করুন।

3. জোড় ইনডেক্সের মানগুলো স্লাইস করুন।

সমস্যা 143: Tuple Unpacking ব্যবহার করে প্রতিটি মান আলাদা Variable-এ সংরক্ষণ করুন।

সমস্যা

Tuple Unpacking পদ্ধতির মাধ্যমে একটি টাপলের ভেতরের মানগুলোকে এক লাইনে আলাদা আলাদা ভেরিয়েবলে সংরক্ষণ করুন।

কী কী জানা থাকতে হবে?

- Tuple Unpacking `a, b, c = my_tuple`

কীভাবে চিন্তা করব?

যদি টাপলে ৩টি মান থাকে, তবে বামপাশে ৩টি ভেরিয়েবল কমা দিয়ে লিখলে প্রতিটি মান স্বয়ংক্রিয়ভাবে আলাদা ভেরিয়েবলে বসে যায়।

অ্যালগরিদম

Step 1: person = ('Abdur Rahman', 22, 'Dhaka') তৈরি করা।

Step 2: name, age, city = person লেখা।

Step 3: ভেরিয়েবলগুলো প্রিন্ট করা।

Python Code

```
# Demonstrating Tuple Unpacking
person = ("Abdur Rahman", 22, "Dhaka")

# Unpacking into separate variables
name, age, city = person

print("Name:", name)
print("Age:", age)
print("City:", city)
```

Built-in Function / Shortcut Method

```
# Using asterisk (*) for remaining items
# first, *rest = (1, 2, 3, 4, 5)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Name: Abdur Rahman Age: 22

```
City: Dhaka
```

ব্যাখ্যা

টাপলের ৩টি মান এক লাইনেই name, age ও city ভেরিয়েবলে আনপ্যাক করা হয়েছে।

Common Mistakes

- বামপাশের ভেরিয়েবল সংখ্যা ও টাপলের উপাদান সংখ্যা সমান না হলে ValueError হওয়া।

অনুশীলনের জন্য

1. `*` (Asterisk) ব্যবহার করে প্রথম উপাদান রেখে বাকিগুলো একটি লিস্টে আনপ্যাক করুন।

2. ফাংশন থেকে একাধিক মান রিটার্ন করে টাপল আনপ্যাকিং করুন।

3. ৪টি ভেরিয়েবলে আনপ্যাক করুন।

সমস্যা 144: Tuple ব্যবহার করে দুটি Variable-এর মান Swap করুন।

সমস্যা

Tuple Unpacking ব্যবহার করে কোনো Temporary ভেরিয়েবল ছাড়াই দুটি ভেরিয়েবলের মান অদলবদল (Swap) করুন।

কী কী জানা থাকতে হবে?

- Tuple swapping `x, y = y, x`

কীভাবে চিন্তা করব?

পাইথনে `a, b = b, a` লিখলে ডানপাশে একটি টাপল তৈরি হয় এবং তা বামপাশের ভেরিয়েবলে আনপ্যাক হয়ে মান অদলবদল করে।

অ্যালগরিদম

Step 1: x = 50 এবং y = 100 নেওয়া।

Step 2: x, y = y, x লেখা।

Step 3: প্রিন্ট করা।

Python Code

```
# Swapping values using Tuple Unpacking
x = 10
y = 20
print("Before Swap: x =", x, ", y =", y)

# Swapping
```

```
x, y = y, x
print("After Swap: x =", x, ", y =", y)
```

Built-in Function / Shortcut Method

```
# Same as pythonic way
```

Sample Input / Output Table

Sample Input	Sample Output
None	Before Swap: x = 10 , y = 20 After Swap: x = 20 , y = 10

ব্যাখ্যা

টাপল প্যাকিং ও আনপ্যাকিংয়ের মাধ্যমে খুব সহজে মান অদলবদল হয়েছে।

Common Mistakes

- ক্রম ভুল করা।

অনুশীলনের জন্য

1. ৩টি ভেরিয়েবলের মান চক্রাকারে Swap করুন।

2. দুটি স্ট্রিং Swap করুন।

3. লিস্টের দুটি ইনডেক্সের মান এক লাইনে Swap করুন (`list[i], list[j] = list[j], list[i]`)।

সমস্যা 145: একটি Set তৈরি করুন।

সমস্যা

সেকেন্ড ব্র্যাকেট `{}` ব্যবহার করে একটি Set তৈরি করুন। Set হলো Unordered (ক্রমহীন) এবং এতে কোনো Duplicate মান থাকে না।

কী কী জানা থাকতে হবে?

- Set syntax `{}`

কীভাবে চিন্তা করব?

সেকেন্ড ব্র্যাকেট `{}` দিয়ে সেট তৈরি করা হয়। মনে রাখবেন খালি `{}` ডিকশনারি তৈরি করে, খালি সেটের জন্য `set()` লিখতে হয়।

অ্যালগরিদম

Step 1: my_set = {10, 20, 30, 40} তৈরি করা।

Step 2: প্রিন্ট করা।

Python Code

```
# Creating a simple Set
my_set = {10, 20, 30, 40, 50}

print("The Set is:", my_set)
print("Data type:", type(my_set))
```

Built-in Function / Shortcut Method

```
# Creating set from list
# s = set([1, 2, 3])
```

Sample Input / Output Table

Sample Input	Sample Output
None	The Set is: {40, 10, 50, 20, 30} Data type: <class 'set'>

ব্যাখ্যা

সেটের আউটপুটে উপাদানগুলোর ক্রম এলোমেলো দেখা যেতে পারে কারণ সেট Unordered।

Common Mistakes

- খালি সেকেন্ড ব্র্যাকেট `{}` দিয়ে খালি সেট তৈরি করা (সেটি dict হয়ে যায়)।

অনুশীলনের জন্য

1. set() কনস্ট্রাক্টর দিয়ে খালি সেট বানান।

2. add() মেথড দিয়ে সেটে মান যুক্ত করুন।

3. remove() দিয়ে মান সরান।

সমস্যা 146: একটি Collection থেকে Duplicate Value সরিয়ে Set তৈরি করুন।

সমস্যা

একাধিক ডুপ্লিকেট মানযুক্ত একটি List-কে Set-এ রূপান্তর করে সকল ডুপ্লিকেট মান নিমেষেই অপসারণ করুন।

কী কী জানা থাকতে হবে?

- set(list) conversion

কীভাবে চিন্তা করব?

যেহেতু Set কখনো ডুপ্লিকেট রাখে না, তাই `set(my_list)` করলেই সব ডুপ্লিকেট গায়েব হয়ে যায়।

অ্যালগরিদম

Step 1: numbers = [1, 2, 2, 3, 3, 3, 4, 4] তৈরি করা।

Step 2: unique_set = set(numbers) করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Removing duplicates by converting List to Set
numbers_list = [10, 20, 10, 30, 20, 40, 50, 30]
print("Original List with duplicates:", numbers_list)

unique_set = set(numbers_list)
print("After converting to Set:", unique_set)
```

Built-in Function / Shortcut Method

```
# Converting back to list
# unique_list = list(set(numbers_list))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original List with duplicates: [10, 20, 10, 30, 20, 40, 50, 30] After converting to Set: {40, 10, 50, 20, 30}

ব্যাখ্যা

সেটে রূপান্তরিত হওয়ার সাথে সাথেই সব ডুপ্লিকেট ১০, ২০ ও ৩০ মুছে গেছে।

Common Mistakes

- সেটে রূপান্তর করার পর ইনডেক্সিং (`s[0]`) করার চেষ্টা করা (সেটে ইনডেক্সিং চলে না)।

অনুশীলনের জন্য

1. স্ট্রিংয়ের অক্ষরগুলো থেকে ডুপ্লিকেট সরাতে `set('hello')` করুন।

2. সেট থেকে আবার লিস্টে রূপান্তর করুন।

3. দুটি লিস্টের সংযোগ ঘটিয়ে ডুপ্লিকেট সরান।

সমস্যা 147: দুটি Set-এর Union নির্ণয় করুন।

সমস্যা

দুটি সেটের Union (`|` অপারেটর বা `.union()` মেথড) নির্ণয় করে উভয় সেটের সকল উপাদান একত্র করুন (ডুপ্লিকেট ছাড়া)।

কী কী জানা থাকতে হবে?

- Set Union `|` বা union()

কীভাবে চিন্তা করব?

Union মানে হলো দুটি সেটের সব উপাদান একসাথে নেওয়া। সাধারণ উপাদান একবারই আসবে।

অ্যালগরিদম

Step 1: set1 = {1, 2, 3} এবং set2 = {3, 4, 5} নেওয়া।

Step 2: union_set = set1 | set2 করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding Union of two Sets
set1 = {10, 20, 30}
set2 = {30, 40, 50}

union_set = set1 | set2
print("Set 1:", set1)
print("Set 2:", set2)
print("Union (set1 | set2):", union_set)
```

Built-in Function / Shortcut Method

```
# Using union() method
# print("Union:", set1.union(set2))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set 1: {10, 20, 30} Set 2: {40, 50, 30} Union (set1 | set2): {40, 10, 50, 20, 30}

ব্যাখ্যা

উভয় সেটের মান একত্রিত হয়ে নতুন সেট গঠিত হয়েছে এবং সাধারণ মান ৩০ একবারই এসেছে।

Common Mistakes

- `+` চিহ্ন দিয়ে সেট যোগ করার চেষ্টা করা (সেটে `+` কাজ করে না, `|` দিতে হয়)।

অনুশীলনের জন্য

1. ৩টি সেটের Union বের করুন।

2. union() মেথড ব্যবহার করুন।

3. একটি সেটের সাথে অন্য সেটের union করে আপডেট (`update()`) করুন।

সমস্যা 148: দুটি Set-এর Intersection নির্ণয় করুন।

সমস্যা

দুটি সেটের Intersection (`&` অপারেটর বা `.intersection()` মেথড) নির্ণয় করে শুধুমাত্র সাধারণ (Common) উপাদানগুলো বের করুন।

কী কী জানা থাকতে হবে?

- Set Intersection `&` বা intersection()

কীভাবে চিন্তা করব?

Intersection দুটি সেটের মধ্যে শুধুমাত্র মিল থাকা বা কমন উপাদানগুলো আলাদা করে।

অ্যালগরিদম

Step 1: set1 এবং set2 তৈরি করা।

Step 2: common = set1 & set2 করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding Intersection of two Sets
```

```
set1 = {10, 20, 30, 40}
set2 = {30, 40, 50, 60}

common_set = set1 & set2
print("Set 1:", set1)
print("Set 2:", set2)
print("Intersection (set1 & set2):", common_set)
```

Built-in Function / Shortcut Method

```
# Using intersection() method
# print("Common:", set1.intersection(set2))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set 1: {40, 10, 20, 30} Set 2: {40, 50, 60, 30} Intersection (set1 & set2): {40, 30}

ব্যাখ্যা

উভয় সেটের মধ্যে ৩০ এবং ৪০ কমন থাকায় তারা Intersection হিসেবে এসেছে।

Common Mistakes

- `and` কিওয়ার্ড দিয়ে করার চেষ্টা করা।

অনুশীলনের জন্য

1. intersection_update() ব্যবহার করুন।

2. ৩টি লিস্টের মধ্যে কমন সংখ্যাগুলো বের করুন।

3. কোনো কমন উপাদান না থাকলে তা Disjoint কিনা (`isdisjoint()`) যাচাই করুন।

সমস্যা 149: দুটি Set-এর Difference নির্ণয় করুন।

সমস্যা

দুটি সেটের Difference (`-` অপারেটর বা `.difference()` মেথড) নির্ণয় করে প্রথম সেটে আছে কিন্তু দ্বিতীয় সেটে নেই এমন উপাদানগুলো বের করুন।

কী কী জানা থাকতে হবে?

- Set Difference `-` বা difference()

কীভাবে চিন্তা করব?

`set1 - set2` করলে set1 থেকে set2 এর সাথে মিলে যাওয়া মানগুলো বাদ দিয়ে বাকিগুলো দেয়।

অ্যালগরিদম

Step 1: set1 এবং set2 তৈরি করা।

Step 2: diff = set1 - set2 করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding Difference between two Sets
set1 = {10, 20, 30, 40}
set2 = {30, 40, 50, 60}

diff_set = set1 - set2
print("Set 1:", set1)
print("Set 2:", set2)
print("Difference (set1 - set2):", diff_set)
```

Built-in Function / Shortcut Method

```
# Using difference() method
# print("Diff:", set1.difference(set2))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set 1: {40, 10, 20, 30} Set 2: {40, 50, 60, 30} Difference (set1 - set2): {10, 20}

ব্যাখ্যা

set1 এর ৩০ ও ৪০ set2 তে থাকায় তারা বাদ গিয়ে শুধু ১০ ও ২০ অবশিষ্ট রয়েছে।

Common Mistakes

- `set2 - set1` এবং `set1 - set2` একই মনে করা।

অনুশীলনের জন্য

1. `set2 - set1` বের করে পার্থক্য দেখুন।

2. difference_update() ব্যবহার করুন।

3. দুটি দলের খেলোয়াড়দের মধ্যে কারা শুধু প্রথম দলে খেলে তা বের করুন।

সমস্যা 150: দুটি Set-এর Symmetric Difference নির্ণয় করুন।

সমস্যা

Symmetric Difference (`^` অপারেটর বা `.symmetric_difference()`) নির্ণয় করে উভয় সেটের সাধারণ (Common) উপাদান বাদে বাকি সকল উপাদান বের করুন।

কী কী জানা থাকতে হবে?

- Symmetric Difference `^`

কীভাবে চিন্তা করব?

এটি মূলত Union থেকে Intersection বাদ দেওয়ার সমান। যা কমন তা বাদ যাবে, বাকিরা আসবে।

অ্যালগরিদম

Step 1: set1 এবং set2 তৈরি করা।

Step 2: sym_diff = set1 ^ set2 করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding Symmetric Difference of two Sets
set1 = {10, 20, 30, 40}
set2 = {30, 40, 50, 60}

sym_diff = set1 ^ set2
print("Set 1:", set1)
print("Set 2:", set2)
print("Symmetric Difference (set1 ^ set2):", sym_diff)
```

Built-in Function / Shortcut Method

```
# Using symmetric_difference() method
# print("Sym Diff:", set1.symmetric_difference(set2))
```

Sample Input / Output Table

Sample Input	Sample Output

None	Set 1: {40, 10, 20, 30} Set 2: {40, 50, 60, 30} Symmetric Difference (set1 ^ set2): {10, 50, 20, 60}

ব্যাখ্যা

কমন ৩০ ও ৪০ বাদ গিয়ে উভয় সেটের অনন্য উপাদান ১০, ২০, ৫০ ও ৬০ এসেছে।

Common Mistakes

- `^` চিহ্নকে পাওয়ার বা সূচক মনে করা।

অনুশীলনের জন্য

1. symmetric_difference_update() ব্যবহার করুন।

2. `(set1 | set2) - (set1 & set2)` করে একই ফলাফল পান।

3. ৩টি সেটের ওপর প্রয়োগ করুন।

সমস্যা 151: একটি নির্দিষ্ট Value Set-এর মধ্যে আছে কিনা যাচাই করুন।

সমস্যা

Membership Operator (`in`) ব্যবহার করে কোনো নির্দিষ্ট সংখ্যা বা শব্দ একটি সেটের মধ্যে উপস্থিত আছে কিনা তা দ্রুত যাচাই করুন।

কী কী জানা থাকতে হবে?

- in operator in Set

কীভাবে চিন্তা করব?

লিস্টের চেয়ে সেটে `in` অপারেটর দিয়ে খোঁজা (Hash table lookup) হাজার গুণ দ্রুত কাজ করে।

অ্যালগরিদম

Step 1: সেট তৈরি ও target ইনপুট নেওয়া।

Step 2: if target in my_set যাচাই করা।

Step 3: ফলাফল প্রিন্ট করা।

Python Code

```
# Checking item presence in a Set
fruits = {"Apple", "Banana", "Mango", "Orange"}
```

```
target = input("Enter fruit name to check: ")

if target in fruits:
    print(target, "is available in the set")
else:
    print(target, "is NOT available in the set")
```

Built-in Function / Shortcut Method

```
# Direct boolean print
# print(target in fruits)
```

Sample Input / Output Table

Sample Input	Sample Output
Enter fruit name to check: Mango	Mango is available in the set

ব্যাখ্যা

সেটের ভেতর 'Mango' থাকায় `True` হয়েছে।

Common Mistakes

- Case sensitivity ভুল করা।

অনুশীলনের জন্য

1. `not in` ব্যবহার করুন।

2. ১ লাখ সংখ্যার সেটে কোনো সংখ্যা খোঁজার গতি পরীক্ষা করুন।

3. সেট থেকে কোনো উপাদান আছে কিনা যাচাই করে remove() করুন।

সমস্যা 152: একটি Set-এর মোট Element সংখ্যা নির্ণয় করুন।

সমস্যা

বিল্ট-ইন `len()` ফাংশন বা লুপ ব্যবহার করে একটি সেটে মোট কতটি অনন্য উপাদান আছে তা নির্ণয় করুন।

কী কী জানা থাকতে হবে?

- len() function

কীভাবে চিন্তা করব?

`len(my_set)` সেটের মোট উপাদান সংখ্যা প্রদান করে।

অ্যালগরিদম

Step 1: সেট তৈরি করা।

Step 2: count = len(my_set) করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Counting total elements in a Set
my_set = {10, 20, 30, 40, 50}

count = len(my_set)
print("Set elements:", my_set)
print("Total elements in the set:", count)
```

Built-in Function / Shortcut Method

```
# Loop count
# print(sum(1 for _ in my_set))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set elements: {40, 10, 50, 20, 30} Total elements in the set: 5

ব্যাখ্যা

সেটে ৫টি উপাদান থাকায় ৫ এসেছে।

Common Mistakes

- ডুপ্লিকেটসহ লিস্টকে সেটে নিয়ে len মেপে আসল লিস্টের দৈর্ঘ্য মনে করা।

অনুশীলনের জন্য

1. লুপ চালিয়ে সেটের দৈর্ঘ্য মেপে দেখুন।

2. লিস্ট থেকে ডুপ্লিকেট বাদ দেওয়ার পর কয়টি উপাদান কমলো তা হিসাব করুন।

3. খালি সেটের দৈর্ঘ্য মেপে দেখুন।

সমস্যা 153: একটি Set থেকে সবচেয়ে বড় মান নির্ণয় করুন।

সমস্যা

বিল্ট-ইন `max()` ফাংশন বা লুপ ব্যবহার করে সেটের সবচেয়ে বড় সংখ্যাটি খুঁজে বের করুন।

কী কী জানা থাকতে হবে?

- max() function

কীভাবে চিন্তা করব?

লিস্টের মতোই সেটেও `max(my_set)` সবচেয়ে বড় মানটি রিটার্ন করে।

অ্যালগরিদম

Step 1: numbers = {45, 89, 12, 96, 54} নেওয়া।

Step 2: max_num = max(numbers) করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding maximum in a Set
numbers = {45, 89, 12, 96, 54, 23}

max_val = max(numbers)
print("Set elements:", numbers)
print("Maximum value is:", max_val)
```

Built-in Function / Shortcut Method

```
# Using loop
# max_val = pop() and loop comparison
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set elements: {96, 45, 12, 89, 54, 23} Maximum value is: 96

ব্যাখ্যা

৯৬ সেটের বৃহত্তম সংখ্যা।

Common Mistakes

- খালি সেটে max() চালিয়ে ValueError করা।

অনুশীলনের জন্য

1. লুপ চালিয়ে সেটের সর্বোচ্চ মান বের করুন।

2. স্ট্রিং সেটের সর্বোচ্চ (বর্ণানুক্রমিক শেষ) শব্দ বের করুন।

3. সেট থেকে সর্বোচ্চ মানটি remove() করুন।

সমস্যা 154: একটি Set থেকে সবচেয়ে ছোট মান নির্ণয় করুন।

সমস্যা

বিল্ট-ইন `min()` ফাংশন ব্যবহার করে সেটের সবচেয়ে ছোট সংখ্যাটি নির্ণয় করুন।

কী কী জানা থাকতে হবে?

- min() function

কীভাবে চিন্তা করব?

`min(my_set)` সেটের ক্ষুদ্রতম মান দেয়।

অ্যালগরিদম

Step 1: numbers সেট নেওয়া।

Step 2: min_val = min(numbers) করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Finding minimum in a Set
numbers = {45, 89, 12, 96, 54, 23}

min_val = min(numbers)
print("Set elements:", numbers)
print("Minimum value is:", min_val)
```

Built-in Function / Shortcut Method

```
# Direct print
# print("Min:", min(numbers))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Set elements: {96, 45, 12, 89, 54, 23} Minimum value is: 12

ব্যাখ্যা

১২ ক্ষুদ্রতম সংখ্যা হিসেবে বের হয়েছে।

Common Mistakes

- min এর বানান ভুল করা।

অনুশীলনের জন্য

1. লুপ চালিয়ে সেটের সর্বনিম্ন মান বের করুন।

2. সর্বোচ্চ ও সর্বনিম্ন মানের যোগফল বের করুন।

3. min() ও max() একসাথে ব্যবহার করুন।

Topic: Dictionary Operations

সমস্যা 155: একটি Dictionary তৈরি করে Key-Value Pair সংরক্ষণ করুন।

সমস্যা

সেকেন্ড ব্র্যাকেট `{}` এর ভেতর কোলন (`:`) দিয়ে Key ও Value জোড়া (Key-Value Pair) আকারে একটি Dictionary তৈরি ও প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- Dictionary syntax `{key: value}`

কীভাবে চিন্তা করব?

ডিকশনারি হলো এমন একটি ডেটা স্ট্রাকচার যেখানে প্রতিটি মানের (Value) সাথে একটি নাম বা চাবি (Key) যুক্ত থাকে। যেমন `{'name': 'Abdur Rahman', 'age': 22}`।

অ্যালগরিদম

Step 1: student = {'name': 'Rahim', 'age': 20, 'grade': 'A'} তৈরি করা।

Step 2: প্রিন্ট করা।

Python Code

```
# Creating a simple Dictionary
student = {
    "name": "Abdur Rahman",
    "age": 22,
    "district": "Dhaka"
}

print("Student Dictionary:", student)
print("Data type:", type(student))
```

Built-in Function / Shortcut Method

```
# Using dict() constructor
# student = dict(name="Abdur Rahman", age=22)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Student Dictionary: {'name': 'Abdur Rahman', 'age': 22, 'district': 'Dhaka'} Data type: <class 'dict'>

ব্যাখ্যা

Key এবং Value কোলন দিয়ে যুক্ত হয়ে ডিকশনারি গঠন করেছে।

Common Mistakes

- Key হিসেবে Mutable অবজেক্ট যেমন List ব্যবহার করা (Key সবসময় Immutable হতে হয়)।

অনুশীলনের জন্য

1. dict() কনস্ট্রাক্টর ব্যবহার করে ডিকশনারি বানান।

2. খালি ডিকশনারি তৈরি করে টাইপ দেখুন।

3. `student['name']` লিখে মান বের করুন।

সমস্যা 156: একটি Dictionary-তে নতুন Item যোগ করুন।

সমস্যা

একটি বিদ্যমান ডিকশনারিতে নতুন Key-Value pair (যেমন নতুন কোনো ফিল্ড বা তথ্য) যোগ করুন।

কী কী জানা থাকতে হবে?

- my_dict[new_key] = value

কীভাবে চিন্তা করব?

ডিকশনারিতে নতুন মান যোগ করার নিয়ম হলো `my_dict['new_key'] = value`। Key আগে থেকে না থাকলে সেটি নতুন করে যোগ হয়ে যায়।

অ্যালগরিদম

Step 1: student ডিকশনারি তৈরি করা।

Step 2: student['phone'] = '01712345678' যোগ করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Adding a new item to Dictionary
student = {"name": "Abdur Rahman", "age": 22}
print("Before adding:", student)

# Adding new key-value pair
student["department"] = "Computer Science"
student["cgpa"] = 3.85

print("After adding:", student)
```

Built-in Function / Shortcut Method

```
# Using update() method
# student.update({"phone": "017xxxx"})
```

Sample Input / Output Table

Sample Input	Sample Output
None	Before adding: {'name': 'Abdur Rahman', 'age': 22} After adding: {'name': 'Abdur Rahman', 'age': 22, 'department': 'Computer Science', 'cgpa': 3.85}

ব্যাখ্যা

নতুন দুটি Key 'department' এবং 'cgpa' ডিকশনারিতে সফলভাবে যুক্ত হয়েছে।

Common Mistakes

- Key লেখার সময় কোটেশন দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. update() মেথড ব্যবহার করে একসাথে ৩টি তথ্য যোগ করুন।

2. ব্যবহারকারীর কাছ থেকে ইনপুট নিয়ে নতুন তথ্য ডিকশনারিতে যোগ করুন।

3. লুপ চালিয়ে ডিকশনারিতে ১ থেকে ৫ এর বর্গ যোগ করুন (`{1: 1, 2: 4...}`)।

সমস্যা 157: একটি Dictionary-এর বিদ্যমান Item Update করুন।

সমস্যা

ডিকশনারিতে আগে থেকেই আছে এমন একটি Key-এর মান পরিবর্তন বা Update করুন।

কী কী জানা থাকতে হবে?

- my_dict[existing_key] = new_value

কীভাবে চিন্তা করব?

যদি Key আগে থেকেই ডিকশনারিতে থাকে, তবে `my_dict['key'] = new_value` দিলে পুরনো মানটি মুছে নতুন মান বসে যায়।

অ্যালগরিদম

Step 1: student = {'name': 'Rahim', 'age': 20} নেওয়া।

Step 2: student['age'] = 21 করে বয়স আপডেট করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Updating an existing item in Dictionary
student = {"name": "Abdur Rahman", "age": 22, "district": "Comilla"}
print("Original Dictionary:", student)

# Updating age and district
student["age"] = 23
student["district"] = "Dhaka"

print("Updated Dictionary:", student)
```

Built-in Function / Shortcut Method

```
# Using update()
# student.update({"age": 23})
```

Sample Input / Output Table

Sample Input	Sample Output
None	Original Dictionary: {'name': 'Abdur Rahman', 'age': 22, 'district': 'Comilla'} Updated Dictionary: {'name': 'Abdur Rahman', 'age': 23, 'district': 'Dhaka'}

ব্যাখ্যা

বয়স ২২ থেকে ২৩ এবং জেলা Comilla থেকে Dhaka আপডেট হয়েছে।

Common Mistakes

- Key এর বানান ভুল করা (যার ফলে পুরনো মান আপডেট না হয়ে নতুন Key তৈরি হয়ে যায়)।

অনুশীলনের জন্য

1. পণ্যের দাম ১০% বাড়িয়ে ডিকশনারি আপডেট করুন।

2. update() মেথড ব্যবহার করুন।

3. শর্ত দিয়ে যাচাই করে মান আপডেট করুন।

সমস্যা 158: একটি Dictionary থেকে নির্দিষ্ট Item Delete করুন।

সমস্যা

del কি-ওয়ার্ড বা pop() মেথড ব্যবহার করে ডিকশনারি থেকে নির্দিষ্ট কোনো Key-Value জোড়া মুছে ফেলুন।

কী কী জানা থাকতে হবে?

- del my_dict[key]
- pop(key) method

কীভাবে চিন্তা করব?

`del my_dict['key']` সরাসরি মুছে দেয়। আর `my_dict.pop('key')` মুছে দেওয়ার পাশাপাশি সেই মানটি রিটার্নও করে।

অ্যালগরিদম

Step 1: student ডিকশনারি নেওয়া।

Step 2: del student['age'] বা student.pop('district') করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Deleting items from a Dictionary
student = {"name": "Abdur Rahman", "age": 22, "district": "Dhaka", "grade":
"A"}
print("Before deletion:", student)

# Using del keyword
del student["grade"]

# Using pop() method
removed_val = student.pop("age")

print("After deleting 'grade' and 'age':", student)
print("Removed age value:", removed_val)
```

Built-in Function / Shortcut Method

```
# Removing last inserted item using popitem()
# student.popitem()
```

Sample Input / Output Table

Sample Input	Sample Output
None	Before deletion: {'name': 'Abdur Rahman', 'age': 22, 'district': 'Dhaka', 'grade': 'A'} After deleting 'grade' and 'age': {'name': 'Abdur Rahman', 'district': 'Dhaka'} Removed age value: 22

ব্যাখ্যা

grade এবং age ফিল্ড দুটি সফলভাবে ডিকশনারি থেকে মুছে ফেলা হয়েছে।

Common Mistakes

- না থাকা Key ডিলিট করতে গিয়ে KeyError হওয়া (তাই আগে `if key in dict:` চেক করা বা `pop(key, None)` দেওয়া নিরাপদ)।

অনুশীলনের জন্য

1. popitem() মেথডের কাজ পরীক্ষা করুন।

2. clear() মেথড দিয়ে পুরো ডিকশনারি খালি করুন।

3. KeyError এড়াতে pop এর ডিফল্ট প্যারামিটার ব্যবহার করুন।

সমস্যা 159: একজন Student-এর তথ্য Dictionary-তে সংরক্ষণ ও প্রদর্শন করুন।

সমস্যা

ব্যবহারকারীর কাছ থেকে একজন শিক্ষার্থীর নাম, রোল নম্বর এবং জিপিএ ইনপুট নিয়ে একটি ডিকশনারিতে সংরক্ষণ ও সুন্দরভাবে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- Dynamic dict insertion

কীভাবে চিন্তা করব?

ইনপুট নিয়ে ভেরিয়েবলে রাখব এবং ডিকশনারিতে Key বসিয়ে মানগুলো সংরক্ষণ করব।

অ্যালগরিদম

Step 1: name, roll এবং gpa ইনপুট নেওয়া।

Step 2: student = {'Name': name, 'Roll': roll, 'GPA': gpa} তৈরি করা।

Step 3: লুপ চালিয়ে সুন্দরভাবে প্রিন্ট করা।

Python Code

```
# Storing and displaying student info using dictionary
name = input("Enter Student Name: ")
roll = int(input("Enter Roll Number: "))
gpa = float(input("Enter GPA: "))

student_info = {
    "Name": name,
    "Roll": roll,
    "GPA": gpa
}
```

```
print("\n--- Student Bio-Data ---")
for key, val in student_info.items():
    print(f"{key} : {val}")
```

Built-in Function / Shortcut Method

```
# Direct construction inside dict()
```

Sample Input / Output Table

Sample Input	Sample Output
Enter Student Name: Tanvir Ahmed Enter Roll Number: 101 Enter GPA: 3.92	--- Student Bio-Data --- Name : Tanvir Ahmed Roll : 101 GPA : 3.92

ব্যাখ্যা

ইনপুট করা তথ্যগুলো ডিকশনারিতে সংরক্ষিত হয়ে প্রফেশনাল আকারে প্রদর্শিত হয়েছে।

Common Mistakes

- রোল ও জিপিএ কে int বা float না করা।

অনুশীলনের জন্য

1. শিক্ষার্থীর ৫টি বিষয়ের নম্বর ডিকশনারিতে লিস্ট হিসেবে রাখুন।

2. ঠিকানা ও ফোন নম্বর যুক্ত করুন।

3. get() মেথড ব্যবহার করে তথ্য পড়ুন।

সমস্যা 160: একাধিক Student-এর Result Dictionary ব্যবহার করে সংরক্ষণ ও প্রদর্শন করুন।

সমস্যা

রোল নম্বরকে Key এবং পরীক্ষার নম্বর বা জিপিএকে Value হিসেবে ধরে একাধিক শিক্ষার্থীর রেজাল্ট সংরক্ষণ ও অনুসন্ধান করুন।

কী কী জানা থাকতে হবে?

- Nested Dict বা Dict of Students

কীভাবে চিন্তা করব?

ডিকশনারির Key হবে রোল নম্বর এবং Value হবে শিক্ষার্থীর নাম ও জিপিএ এর আরেকটি ডিকশনারি বা লিস্ট।

অ্যালগরিদম

Step 1: results = {101: 3.8, 102: 3.5, 103: 4.0} তৈরি করা।

Step 2: রোল ইনপুট নিয়ে রেজাল্ট প্রদর্শন করা।

Python Code

```
# Storing and searching multiple student results
results = {
    101: {"name": "Rahim", "gpa": 3.85},
    102: {"name": "Karim", "gpa": 3.60},
    103: {"name": "Sadiya", "gpa": 4.00}
}

search_roll = int(input("Enter Roll Number to check result (101-103): "))

if search_roll in results:
    info = results[search_roll]
    print(f"Name: {info['name']} | GPA: {info['gpa']}")
else:
    print("Result not found for this Roll Number!")
```

Built-in Function / Shortcut Method

```
# Using get()
# print(results.get(search_roll, "Not found"))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter Roll Number to check result (101-103): 103	Name: Sadiya | GPA: 4.0

ব্যাখ্যা

রোল ১০৩ অনুসন্ধান করে সাদিয়ার জিপিএ ৪.০০ পাওয়া গেছে।

Common Mistakes

- নেস্টেড ডিকশনারি অ্যাক্সেস করতে গিয়ে ইনডেক্স ভুল করা।

অনুশীলনের জন্য

1. নতুন শিক্ষার্থীর রেজাল্ট ডিকশনারিতে যোগ করুন।

2. সব শিক্ষার্থীর গড় জিপিএ বের করুন।

3. সবচেয়ে বেশি জিপিএ পাওয়া শিক্ষার্থীর নাম বের করুন।

সমস্যা 161: বিভিন্ন Product-এর Price Dictionary-তে সংরক্ষণ ও প্রদর্শন করুন।

সমস্যা

পণ্যের নামকে Key এবং মূল্যকে Value ধরে একটি Price List তৈরি করুন এবং ব্যবহারকারীর ইনপুট করা পণ্যের দাম প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- dict lookup

কীভাবে চিন্তা করব?

`price_list = {'Laptop': 65000, 'Mouse': 500}` রেখে পণ্যের নাম দিয়ে দাম বের করব।

অ্যালগরিদম

Step 1: price_table তৈরি করা।

Step 2: পণ্যের নাম ইনপুট নেওয়া।

Step 3: if product in price_table হলে দাম দেখানো।

Python Code

```
# Storing and fetching product prices
price_table = {
    "Laptop": 65000,
    "Keyboard": 1200,
    "Mouse": 450,
    "Monitor": 14500
}

product = input("Enter product name (Laptop/Keyboard/Mouse/Monitor): ")

if product in price_table:
    print(f"Price of {product} is: {price_table[product]} BDT")
else:
    print("Product out of stock or unavailable.")
```

Built-in Function / Shortcut Method

```
# Using get() with fallback
# print("Price:", price_table.get(product, "Not Available"))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter product name (Laptop/Keyboard/Mouse/Monitor): Keyboard	Price of Keyboard is: 1200 BDT

ব্যাখ্যা

'Keyboard' কী (Key) দিয়ে তার মূল্য ১২০০ টাকা বের করা হয়েছে।

Common Mistakes

- Case sensitivity (যেমন 'laptop' ছোট হাতের দিলে না পাওয়া)।

অনুশীলনের জন্য

1. সব পণ্যের মোট মূল্য যোগ করুন।

2. ১০% ডিসকাউন্ট দিয়ে সব পণ্যের দাম আপডেট করুন।

3. সবচেয়ে দামি পণ্যটি খুঁজে বের করুন।

সমস্যা 162: একটি String-এর Character Frequency Dictionary ব্যবহার করে নির্ণয় করুন।

সমস্যা

একটি শব্দের বা বাক্যের প্রতিটি অক্ষর কতবার এসেছে তা ডিকশনারি ব্যবহার করে গণনা করুন (আগে Problem 115 এ দেখানো লজিকের বিস্তারিত রূপ)।

কী কী জানা থাকতে হবে?

- dict frequency counting

কীভাবে চিন্তা করব?

অক্ষরটি ডিকশনারির Key হবে। আগে থেকে থাকলে মান ১ বাড়বে, না থাকলে ১ দিয়ে শুরু হবে।

অ্যালগরিদম

Step 1: স্ট্রিং ইনপুট নেওয়া।

Step 2: freq = {} রাখা।

Step 3: লুপ চালিয়ে freq[char] = freq.get(char, 0) + 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Character frequency calculation using get()
text = input("Enter a word: ")
char_freq = {}

for char in text:
    char_freq[char] = char_freq.get(char, 0) + 1

print("Character Frequencies:")
for char, count in char_freq.items():
    print(f"'{char}' occurred {count} times")
```

Built-in Function / Shortcut Method

```
# Using collections.Counter
# from collections import Counter
# print(Counter(text))
```

Sample Input / Output Table

Sample Input	Sample Output
Enter a word: banana	Character Frequencies: 'b' occurred 1 times 'a' occurred 3 times 'n' occurred 2 times

ব্যাখ্যা

`dict.get(char, 0) + 1` মেথডটি অত্যন্ত সহজে ফ্রিকোয়েন্সি হিসাব করেছে।

Common Mistakes

- get() মেথডের ডিফল্ট মান 0 না দেওয়া।

অনুশীলনের জন্য

1. collections.Counter ব্যবহার করুন।

2. স্পেস ও চিহ্ন বাদে শুধু বর্ণমালার ফ্রিকোয়েন্সি বের করুন।

3. যে অক্ষরটি মাত্র ১ বার এসেছে তা বের করুন।

সমস্যা 163: একটি বাক্যের Word Frequency Dictionary ব্যবহার করে নির্ণয় করুন।

সমস্যা

একটি বাক্যের প্রতিটি শব্দ কতবার ব্যবহার করা হয়েছে তা ডিকশনারি দিয়ে নির্ণয় করুন।

কী কী জানা থাকতে হবে?

- split() method
- dict frequency logic

কীভাবে চিন্তা করব?

বাক্যটিকে `.split()` করে শব্দের লিস্ট বানাব। তারপর লুপ চালিয়ে প্রতিটি শব্দের ফ্রিকোয়েন্সি ডিকশনারিতে রাখব।

অ্যালগরিদম

Step 1: বাক্য ইনপুট নিয়ে split() করা।

Step 2: word_freq = {} রাখা।

Step 3: লুপ চালিয়ে word_freq[word] = word_freq.get(word, 0) + 1 করা।

Step 4: প্রিন্ট করা।

Python Code

```
# Word frequency calculation
sentence = input("Enter a sentence: ").lower()
words = sentence.split()

word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1

print("Word Frequencies:")
for word, count in word_freq.items():
    print(f"{word} : {count}")
```

Built-in Function / Shortcut Method

```
# Using Counter
# from collections import Counter
# print(Counter(sentence.split()))
```

Sample Input / Output Table

Sample Input	Sample Output

`Enter a sentence: Python is great and Python is easy`	`Word Frequencies:`
	`python : 2`
	`is : 2`
	`great : 1`
	`and : 1`
	`easy : 1`

ব্যাখ্যা

'python' এবং 'is' শব্দ দুটি ২ বার করে এসেছে।

Common Mistakes

- lower() না করার কারণে 'Python' এবং 'python' কে আলাদা শব্দ মনে করা।

অনুশীলনের জন্য

1. সবচেয়ে বেশি ব্যবহৃত শব্দটি বের করুন।

2. বাক্য থেকে যতিচিহ্ন (Punctuation) সরিয়ে শব্দ গুনুন।

3. ৩ বার বা তার বেশি আসা শব্দগুলো প্রিন্ট করুন।

সমস্যা 164: একটি Dictionary-এর প্রতিটি Key ও Value Loop ব্যবহার করে প্রদর্শন করুন।

সমস্যা

for loop এবং `.items()` মেথড ব্যবহার করে ডিকশনারির সকল Key এবং Value আলাদাভাবে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- dict.items() method

কীভাবে চিন্তা করব?

`for key, value in my_dict.items():` লিখলে লুপে চাবি ও মান উভয়ই একসাথে পাওয়া যায়।

অ্যালগরিদম

Step 1: country_capitals তৈরি করা।

Step 2: for k, v in country_capitals.items() লুপ চালানো।

Step 3: k এবং v প্রিন্ট করা।

Python Code

```
# Looping through a Dictionary keys and values
capitals = {
    "Bangladesh": "Dhaka",
    "India": "New Delhi",
    "Japan": "Tokyo",
    "UK": "London"
}

print("Country and Capital List:")
for country, capital in capitals.items():
    print(f"The capital of {country} is {capital}")
```

Built-in Function / Shortcut Method

```
# Only keys: for k in capitals.keys()
# Only values: for v in capitals.values()
```

Sample Input / Output Table

Sample Input	Sample Output
None	Country and Capital List: The capital of Bangladesh is Dhaka The capital of India is New Delhi The capital of Japan is Tokyo The capital of UK is London

ব্যাখ্যা

`.items()` মেথড প্রতিটি জোড়াকে টাপল আকারে লুপে প্রদান করেছে।

Common Mistakes

- `.items()` না লিখে সরাসরি লুপ চালালে শুধু Key আসে, Value আসে না।

অনুশীলনের জন্য

1. শুধু keys() প্রিন্ট করুন।

2. শুধু values() প্রিন্ট করুন।

3. Key অনুসারে ডিকশনারি সর্ট করে প্রিন্ট করুন।

Topic: Functions

সমস্যা 165: কোনো Parameter ছাড়া একটি Simple Function তৈরি করে কল করুন।

সমস্যা

def কি-ওয়ার্ড ব্যবহার করে কোনো প্যারামিটার ছাড়া একটি সাধারণ ফাংশন তৈরি করুন এবং সেটি Call করে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- def keyword
- Function calling

কীভাবে চিন্তা করব?

ফাংশন তৈরি করার জন্য `def function_name():` লিখতে হয়। ফাংশন তৈরি করলেই কোড চলে না, তাকে নাম ধরে ডাকতে বা কল করতে হয় `function_name()`।

অ্যালগরিদম

Step 1: def greet(): তৈরি করা।

Step 2: ভেতরে print('Welcome to Python Functions!') লেখা।

Step 3: ফাংশনটি কল করা।

Python Code

```
# Defining and calling a simple function without parameters
def greet():
    print("Welcome to Python Programming!")
    print("Functions make code reusable.")

# Calling the function
greet()
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Welcome to Python Programming! Functions make code reusable.

ব্যাখ্যা

`greet()` লিখে কল করার সাথে সাথেই ফাংশনের ভেতরের লাইনগুলো এক্সিকিউট হয়েছে।

Common Mistakes

- ফাংশন সংজ্ঞায়িত (define) করার পর কল করতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. আপনার নাম ও ঠিকানা প্রিন্ট করার একটি ফাংশন তৈরি করুন।

2. একই ফাংশন ৩ বার কল করুন।

3. বর্তমান সময় দেখানোর একটি ফাংশন তৈরি করুন।

সমস্যা 166: Parameter গ্রহণ করে একটি Function তৈরি করুন।

সমস্যা

একটি ফাংশন তৈরি করুন যা ব্যবহারকারীর নাম আর্গুমেন্ট (Parameter) হিসেবে গ্রহণ করে তাকে স্বাগত জানাবে।

কী কী জানা থাকতে হবে?

- Function arguments/parameters

কীভাবে চিন্তা করব?

ব্র্যাকেটের ভেতর ভেরিয়েবলের নাম দিলে তাকে Parameter বলে, যেমন `def welcome(name):`। কল করার সময় ভ্যালু পাস করতে হয়।

অ্যালগরিদম

Step 1: def welcome(user_name): তৈরি করা।

Step 2: ভেতরে স্বাগত বার্তা প্রিন্ট করা।

Step 3: welcome('Rahim') কল করা।

Python Code

```
# Function accepting parameters
def welcome(user_name):
    print(f"Hello {user_name}, welcome to our Python course!")

# Calling with arguments
welcome("Abdur Rahman")
welcome("Sadiya")
```

Built-in Function / Shortcut Method

```
# Default parameters
# def welcome(name="Guest"): print(f"Hello {name}")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Hello Abdur Rahman, welcome to our Python course! Hello Sadiya, welcome to our Python course!

ব্যাখ্যা

ফাংশনে পাস করা আর্গুমেন্টটি user_name ভেরিয়েবলে বসে প্রিন্ট হয়েছে।

Common Mistakes

- প্যারামিটার যুক্ত ফাংশন কল করার সময় আর্গুমেন্ট না দেওয়া (TypeError হওয়া)।

অনুশীলনের জন্য

1. দুটি প্যারামিটার (নাম এবং বয়স) গ্রহণ করে প্রদর্শন করার ফাংশন বানান।

2. ডিফল্ট প্যারামিটার (`name="Guest"`) ব্যবহার করুন।

3. কীওয়ার্ড আর্গুমেন্ট দিয়ে ফাংশন কল করুন।

সমস্যা 167: Return Value ব্যবহার করে একটি Function তৈরি করুন।

সমস্যা

return কি-ওয়ার্ড ব্যবহার করে একটি ফাংশন তৈরি করুন যা কোনো হিসাবের ফলাফল প্রিন্ট না করে কলকারী স্থানে ফেরত (Return) পাঠাবে।

কী কী জানা থাকতে হবে?

- return keyword

কীভাবে চিন্তা করব?

`print()` শুধু স্ক্রিনে দেখায়, আর `return` ফলাফলটিকে ফাংশনের বাইরে পাঠিয়ে দেয় যা ভেরিয়েবলে রাখা যায় বা অন্য কাজে লাগানো যায়।

অ্যালগরিদম

Step 1: def square(x): তৈরি করা।

Step 2: return x ** 2 লেখা।

Step 3: result = square(6) করে মান সংরক্ষণ ও প্রিন্ট করা।

Python Code

```
# Function using return statement
def get_square(number):
    return number ** 2

# Storing returned value in a variable
res = get_square(7)
print("The square of 7 is:", res)
```

Built-in Function / Shortcut Method

```
# Inline call inside print
# print("Square:", get_square(5))
```

Sample Input / Output Table

Sample Input	Sample Output
None	The square of 7 is: 49

ব্যাখ্যা

`return number ** 2` হিসাবটি ৪৯ ফেরত পাঠিয়েছে যা res ভেরিয়েবলে জমা হয়েছে।

Common Mistakes

- return এর নিচে কোনো কোড লেখা (return এক্সিকিউট হলে ফাংশন সাথে সাথে বন্ধ হয়ে যায়)।

অনুশীলনের জন্য

1. একাধিক মান (যেমন যোগফল ও বিয়োগফল) টাপল আকারে রিটার্ন করুন।

2. কোনো সংখ্যার ঘন (Cube) রিটার্ন করুন।

3. True বা False রিটার্ন করার ফাংশন লিখুন।

সমস্যা 168: দুটি সংখ্যার যোগফল নির্ণয়ের জন্য একটি Function তৈরি করুন।

সমস্যা

দুটি সংখ্যা আর্গুমেন্ট হিসেবে গ্রহণ করে তাদের যোগফল রিটার্ন করার একটি ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- Function with multiple parameters

কীভাবে চিন্তা করব?

`def add_numbers(a, b): return a + b`।

অ্যালগরিদম

Step 1: def add_numbers(num1, num2): তৈরি করা।

Step 2: return num1 + num2 করা।

Step 3: ফাংশন কল করে প্রিন্ট করা।

Python Code

```
# Function to add two numbers
def add_numbers(num1, num2):
    sum_res = num1 + num2
    return sum_res

ans = add_numbers(25, 40)
print("Sum is:", ans)
```

Built-in Function / Shortcut Method

```
# Lambda shortcut
# add = lambda a, b: a + b
# print(add(25, 40))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Sum is: 65

ব্যাখ্যা

২৫ এবং ৪০ যোগ হয়ে ৬৫ রিটার্ন হয়েছে।

Common Mistakes

- num1 এবং num2 পাস করতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. ৩টি সংখ্যার যোগফলের ফাংশন লিখুন।

2. `*args` ব্যবহার করে অসংখ্য সংখ্যার যোগফল বের করার ফাংশন লিখুন।

3. ব্যবহারকারীর ইনপুট নিয়ে এই ফাংশনে পাস করুন।

সমস্যা 169: দুটি সংখ্যার বিয়োগফল নির্ণয়ের জন্য একটি Function তৈরি করুন।

সমস্যা

দুটি সংখ্যা প্যারামিটার হিসেবে গ্রহণ করে প্রথমটি থেকে দ্বিতীয়টির বিয়োগফল রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- Subtraction in function

কীভাবে চিন্তা করব?

`def subtract(a, b): return a - b`।

অ্যালগরিদম

Step 1: def subtract(a, b): তৈরি করা।

Step 2: return a - b করা।

Step 3: কল করে ফলাফল দেখা।

Python Code

```
# Function to subtract two numbers
def subtract(a, b):
    return a - b

diff = subtract(80, 35)
print("Difference is:", diff)
```

Built-in Function / Shortcut Method

```
# Lambda shortcut
# sub = lambda a, b: a - b
```

Sample Input / Output Table

Sample Input	Sample Output
None	`Difference is: 45`

ব্যাখ্যা

৮০ থেকে ৩৫ বিয়োগ করে ৪৫ রিটার্ন হয়েছে।

Common Mistakes

- আর্গুমেন্টের ক্রম উল্টে দেওয়া।

অনুশীলনের জন্য

1. `subtract(b=35, a=80)` কিওয়ার্ড আর্গুমেন্ট দিয়ে কল করুন।

2. বড় সংখ্যা থেকে ছোট সংখ্যা বিয়োগ করার লজিক ফাংশনে দিন।

3. ৩টি সংখ্যার বিয়োগফল বের করুন।

সমস্যা 170: দুটি সংখ্যার গুণফল নির্ণয়ের জন্য একটি Function তৈরি করুন।

সমস্যা

দুটি সংখ্যার গুণফল নির্ণয় করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- Multiplication in function

কীভাবে চিন্তা করব?

`def multiply(x, y): return x * y`।

অ্যালগরিদম

Step 1: def multiply(x, y): তৈরি করা।

Step 2: return x * y করা।

Step 3: কল করে প্রিন্ট করা।

Python Code

```
# Function to multiply two numbers
def multiply(x, y):
    return x * y

product = multiply(12, 5)
```

```
print("Product is:", product)
```

Built-in Function / Shortcut Method

```
# Lambda shortcut
# mul = lambda x, y: x * y
```

Sample Input / Output Table

Sample Input	Sample Output
None	Product is: 60

ব্যাখ্যা

১২ x ৫ = ৬০ রিটার্ন হয়েছে।

Common Mistakes

- গুণের জন্য x ব্যবহার করা।

অনুশীলনের জন্য

1. লিস্টের সব সংখ্যার গুণফল বের করার ফাংশন লিখুন।

2. দশমিক সংখ্যার গুণফল ফাংশন দিয়ে বের করুন।

3. একটি সংখ্যার সাথে ১০ গুণের ডিফল্ট ফাংশন লিখুন।

সমস্যা 171: দুটি সংখ্যার ভাগফল নির্ণয়ের জন্য একটি Function তৈরি করুন।

সমস্যা

দুটি সংখ্যা গ্রহণ করে ভাগফল রিটার্ন করার ফাংশন লিখুন এবং শূন্য দ্বারা ভাগের (ZeroDivision) সতর্কতা রাখুন।

কী কী জানা থাকতে হবে?

- Division inside function
- Zero check

কীভাবে চিন্তা করব?

ভাজক শূন্য কিনা তা আগে চেক করতে হবে `if b == 0: return 'Cannot divide by zero'`।

অ্যালগরিদম

Step 1: def divide(a, b): তৈরি করা।

Step 2: if b == 0 হলে এরর বার্তা রিটার্ন করা।

Step 3: else return a / b করা।

Python Code

```
# Function to divide two numbers with Zero check
def divide(a, b):
    if b == 0:
        return "Error! Division by zero is not allowed."
    return a / b

print("Division Result:", divide(50, 4))
print("Zero Division Test:", divide(20, 0))
```

Built-in Function / Shortcut Method

```
# Using ternary return
# def div(a, b): return a/b if b != 0 else "Error"
```

Sample Input / Output Table

Sample Input	Sample Output
None	Division Result: 12.5 Zero Division Test: Error! Division by zero is not allowed.

ব্যাখ্যা

শূন্য দ্বারা ভাগের এররটি ফাংশন সুন্দরভাবে হ্যান্ডেল করেছে।

Common Mistakes

- শূন্য চেক না করে সরাসরি ভাগ করা।

অনুশীলনের জন্য

1. Floor division (//) ও ভাগশেষ (%) একসাথে রিটার্ন করুন।

2. try-except ব্লক ব্যবহার করে ফাংশনটি লিখুন।

3. গড় বের করার ফাংশন লিখুন।

সমস্যা 172: একটি সংখ্যার Factorial নির্ণয়ের জন্য Function তৈরি করুন।

সমস্যা

একটি সংখ্যা আর্গুমেন্ট হিসেবে গ্রহণ করে লুপ বা রিকার্সন দিয়ে তার Factorial রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- Factorial logic inside function

কীভাবে চিন্তা করব?

`def get_factorial(n):` তৈরি করে ভেতরে লুপ চালিয়ে গুণফল রিটার্ন করব।

অ্যালগরিদম

Step 1: def get_factorial(n): তৈরি করা।

Step 2: fact = 1 রেখে লুপ চালিয়ে fact *= i করা।

Step 3: return fact করা।

Python Code

```
# Function to find factorial of a number
def get_factorial(n):
    if n < 0:
        return "Invalid input"
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

print("Factorial of 6 is:", get_factorial(6))
```

Built-in Function / Shortcut Method

```
# Using recursion
# def fact(n): return 1 if n<=1 else n*fact(n-1)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Factorial of 6 is: 720

ব্যাখ্যা

৬ এর ফ্যাক্টোরিয়াল ৭২০ রিটার্ন হয়েছে।

Common Mistakes

- রিকার্সন ব্যবহার করলে Base case দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. রিকার্সন (Recursion) ব্যবহার করে ফ্যাক্টোরিয়াল ফাংশন লিখুন।

2. math.factorial ব্যবহার করে ফাংশন লিখুন।

3. ১ থেকে ১০ পর্যন্ত সংখ্যার ফ্যাক্টোরিয়াল এই ফাংশন কল করে বের করুন।

সমস্যা 173: একটি সংখ্যা Prime কিনা যাচাই করার Function তৈরি করুন।

সমস্যা

একটি সংখ্যা গ্রহণ করে সেটি মৌলিক (Prime) হলে True এবং না হলে False রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- Prime logic in function
- Boolean return

কীভাবে চিন্তা করব?

`if n <= 1: return False`। ২ থেকে বর্গমূল পর্যন্ত লুপ চালিয়ে ভাগ গেলে `return False`। লুপ শেষে `return True`।

অ্যালগরিদম

Step 1: def is_prime(n): তৈরি করা।

Step 2: n <= 1 হলে return False করা।

Step 3: ২ থেকে int(n**0.5)+1 পর্যন্ত লুপ চালিয়ে n % i == 0 হলে return False করা।

Step 4: লুপ শেষে return True করা।

Python Code

```
# Function to check Prime number
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

num = 37
if is_prime(num):
    print(num, "is a Prime Number")
```

```
else:
    print(num, "is NOT a Prime Number")
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	37 is a Prime Number

ব্যাখ্যা

ফাংশনটি True রিটার্ন করায় if শর্তটি সত্য হয়ে Prime বার্তা দিয়েছে।

Common Mistakes

- লুপের ভেতরেই প্রথমবারেই return True করে দেওয়া।

অনুশীলনের জন্য

1. এই ফাংশন ব্যবহার করে ১ থেকে ১০০ এর সব মৌলিক সংখ্যা প্রিন্ট করুন।

2. পরবর্তী মৌলিক সংখ্যা বের করার ফাংশন লিখুন।

3. জোড় মৌলিক সংখ্যা যাচাই করুন।

সমস্যা 174: Fibonacci Series তৈরি করার Function লিখুন।

সমস্যা

N পদ পর্যন্ত ফিবোনাচ্চি ধারার সংখ্যাগুলো একটি List আকারে রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- List generation in function

কীভাবে চিন্তা করব?

`fib_list = [0, 1]` নিয়ে লুপ চালিয়ে নতুন পদ যোগ করব এবং শেষে লিস্ট রিটার্ন করব।

অ্যালগরিদম

Step 1: def generate_fibonacci(n): তৈরি করা।

Step 2: n <= 0 হলে ফাঁকা লিস্ট, n == 1 হলে [0] রিটার্ন করা।

Step 3: fib = [0, 1] রেখে লুপ চালিয়ে fib.append(fib[-1] + fib[-2]) করা।

Step 4: return fib করা।

Python Code

```
# Function to generate Fibonacci series as a list
def generate_fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    fib_series = [0, 1]
    for i in range(2, n):
        next_val = fib_series[-1] + fib_series[-2]
        fib_series.append(next_val)
    return fib_series

print("First 10 Fibonacci numbers:", generate_fibonacci(10))
```

Built-in Function / Shortcut Method

```
# Generator yield approach
# def fib(n): a,b=0,1; ... yield a
```

Sample Input / Output Table

Sample Input	Sample Output
None	First 10 Fibonacci numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

ব্যাখ্যা

লিস্ট আকারে প্রথম ১০টি ফিবোনাচ্চি পদ সফলভাবে রিটার্ন হয়েছে।

Common Mistakes

- n=1 এর ক্ষেত্রে [0, 1] দিয়ে দেওয়া।

অনুশীলনের জন্য

1. yield ব্যবহার করে জেনারেটর ফাংশন লিখুন।

2. N-তম ফিবোনাচ্চি সংখ্যা রিটার্ন করার ফাংশন লিখুন।

3. রিকার্সন দিয়ে ফিবোনাচ্চি লিখুন।

সমস্যা 175: একাধিক সংখ্যার মধ্যে সবচেয়ে বড় সংখ্যা নির্ণয়ের Function লিখুন।

সমস্যা

*args (Arbitrary Arguments) ব্যবহার করে ইচ্ছামতো যত খুশি সংখ্যার মধ্যে সবচেয়ে বড় সংখ্যাটি রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- *args parameter
- max() logic inside function

কীভাবে চিন্তা করব?

`def find_max(*numbers):` লিখলে সবগুলো আর্গুমেন্ট একটি টাপল হিসেবে `numbers` এ চলে আসে।

অ্যালগরিদম

Step 1: def find_max(*nums): তৈরি করা।

Step 2: nums ফাঁকা হলে None রিটার্ন করা।

Step 3: max_val = nums[0] রেখে লুপ চালিয়ে সর্বোচ্চ বের করা।

Step 4: return max_val করা।

Python Code

```
# Function to find maximum among arbitrary numbers using *args
def find_max(*numbers):
    if not numbers:
        return None
    max_val = numbers[0]
    for num in numbers:
        if num > max_val:
            max_val = num
    return max_val

print("Max among (10, 50, 25):", find_max(10, 50, 25))
print("Max among (5, 99, 12, 105, 88):", find_max(5, 99, 12, 105, 88))
```

Built-in Function / Shortcut Method

```
# Using built-in max inside function
# def find_max(*nums): return max(nums)
```

Sample Input / Output Table

Sample Input	Sample Output
None	`Max among (10, 50, 25): 50` `Max among (5, 99, 12, 105, 88): 105`

ব্যাখ্যা

`*numbers` যেকোনো সংখ্যক আর্গুমেন্ট গ্রহণ করে তাদের মধ্যে ১০৫ কে বৃহত্তম হিসেবে বের করেছে।

Common Mistakes

- `*` দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. `*args` ব্যবহার করে সর্বনিম্ন সংখ্যা বের করুন।

2. `*args` এর সব সংখ্যার গড় বের করুন।

3. `**kwargs` ব্যবহার করে ডিকশনারি আর্গুমেন্ট নিয়ে কাজ করুন।

সমস্যা 176: একটি List-এর সকল সংখ্যার যোগফল নির্ণয়ের Function লিখুন।

সমস্যা

একটি সংখ্যার List প্যারামিটার হিসেবে গ্রহণ করে তার ভেতরের সকল উপাদানের যোগফল রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- List parameter in function

কীভাবে চিন্তা করব?

`def sum_list(lst):` তৈরি করে লুপে যোগফল বের করে রিটার্ন করব।

অ্যালগরিদম

Step 1: def sum_list(numbers_list): তৈরি করা।

Step 2: total = 0 রাখা।

Step 3: for item in numbers_list চালিয়ে total += item করা।

Step 4: return total করা।

Python Code

```
# Function to find sum of list elements
```

```
def sum_list(my_list):
    total = 0
    for num in my_list:
        total += num
    return total

sample_list = [10, 20, 30, 40, 50]
print("Sum of list elements:", sum_list(sample_list))
```

Built-in Function / Shortcut Method

```
# Using built-in sum()
# def sum_list(lst): return sum(lst)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Sum of list elements: 150

ব্যাখ্যা

ফাংশনটি লিস্টের উপাদানগুলো যোগ করে ১৫০ রিটার্ন করেছে।

Common Mistakes

- লিস্ট পাস করার সময় ব্র্যাকেট না দেওয়া।

অনুশীলনের জন্য

1. লিস্টের উপাদানগুলোর গুণফল রিটার্ন করার ফাংশন লিখুন।

2. লিস্ট থেকে জোড় সংখ্যাগুলো আলাদা করে নতুন লিস্ট রিটার্ন করার ফাংশন লিখুন।

3. লিস্টের প্রতিটি উপাদান দ্বিগুণ করার ফাংশন লিখুন।

সমস্যা 177: একটি String Palindrome কিনা যাচাই করার Function লিখুন।

সমস্যা

একটি String আর্গুমেন্ট হিসেবে গ্রহণ করে সেটি Palindrome হলে True এবং না হলে False রিটার্ন করার ফাংশন লিখুন।

কী কী জানা থাকতে হবে?

- String manipulation inside function

কীভাবে চিন্তা করব?

`clean_text = text.lower()` করে `return clean_text == clean_text[::-1]`।

অ্যালগরিদম

Step 1: def is_palindrome(text): তৈরি করা।

Step 2: clean = text.lower() করা।

Step 3: return clean == clean[::-1] করা।

Python Code

```
# Function to check Palindrome string
def is_palindrome(text):
    clean_text = text.lower()
    return clean_text == clean_text[::-1]

word = "Madam"
if is_palindrome(word):
    print(word, "is a Palindrome")
else:
    print(word, "is NOT a Palindrome")
```

Built-in Function / Shortcut Method

```
# One-liner lambda
# pal = lambda s: s.lower() == s.lower()[::-1]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Madam is a Palindrome

ব্যাখ্যা

ফাংশনটি 'Madam' কে 'madam' বানিয়ে রিভার্সের সাথে মিলিয়ে True রিটার্ন করেছে।

Common Mistakes

- lower() না করা।

অনুশীলনের জন্য

1. স্পেস বাদ দিয়ে পুরো বাক্য Palindrome কিনা যাচাই করার ফাংশন লিখুন।

2. শব্দের লিস্ট পাঠিয়ে কোনগুলো Palindrome তা ফিল্টার করার ফাংশন লিখুন।

3. লুপ ব্যবহার করে Palindrome যাচাইয়ের ফাংশন লিখুন।

Topic: File Handling

সমস্যা 178: একটি নতুন File তৈরি করুন।

সমস্যা

open() ফাংশন এবং 'w' (write) বা 'x' (create exclusive) মোড ব্যবহার করে পাইথনের মাধ্যমে একটি নতুন Text File তৈরি করুন।

কী কী জানা থাকতে হবে?

- open() function
- 'w' mode
- with statement

কীভাবে চিন্তা করব?

ফাইল তৈরি করার জন্য `open('filename.txt', 'w')` ব্যবহার করা হয়। `with` ব্লক ব্যবহার করলে কাজ শেষে ফাইলটি স্বয়ংক্রিয়ভাবে বন্ধ (`close()`) হয়ে যায়।

অ্যালগরিদম

Step 1: with open('sample.txt', 'w') as f: তৈরি করা।

Step 2: f.write('Hello File!') লিখে ফাইল সেভ করা।

Python Code

```
# Creating a new text file using 'w' mode
with open("my_file.txt", "w") as file:
    file.write("Welcome to Python File Handling!\n")
    file.write("This file was created programmatically.")

print("File 'my_file.txt' created successfully!")
```

Built-in Function / Shortcut Method

```
# Using 'x' mode (throws error if file already exists)
# open("new_file.txt", "x").close()
```

Sample Input / Output Table

Sample Input	Sample Output
None	File 'my_file.txt' created

```
                                                    successfully!
```

ব্যাখ্যা

`open()` ফাংশনটি 'my_file.txt' নামে একটি নতুন ফাইল তৈরি করে তাতে দুটি লাইন টেক্সট লিখে দিয়েছে।

Common Mistakes

- ফাইল open করার পর close না করা (with ব্লক ব্যবহার করলে এই ভুল হয় না)।

অনুশীলনের জন্য

1. আপনার নাম দিয়ে একটি ফাইল তৈরি করুন।

2. 'x' মোড ব্যবহার করে ফাইল তৈরির চেষ্টা করুন।

3. একাধিক লাইন সেভ করুন।

সমস্যা 179: একটি File থেকে সকল তথ্য Read করে প্রদর্শন করুন।

সমস্যা

একটি টেক্সট ফাইল থেকে 'r' (read) মোড ব্যবহার করে ভেতরের সকল লেখা পড়ে স্ক্রিনে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- file.read() method

কীভাবে চিন্তা করব?

ফাইল পড়ার জন্য `open('filename.txt', 'r')` ব্যবহার করা হয়। এরপর `.read()` মেথড পুরো ফাইলের লেখা স্ট্রিং আকারে নিয়ে আসে।

অ্যালগরিদম

Step 1: with open('my_file.txt', 'r') as file: খোলা।

Step 2: content = file.read() করা।

Step 3: print(content) করা।

Python Code

```
# Reading entire content from a file
try:
    with open("my_file.txt", "r") as file:
        content = file.read()
    print("--- File Content ---")
    print(content)
except FileNotFoundError:
    print("Error: The specified file does not exist!")
```

Built-in Function / Shortcut Method

```
# Reading line by line using readlines()
# lines = open("my_file.txt").readlines()
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- File Content --- Welcome to Python File Handling! This file was created programmatically.

ব্যাখ্যা

`.read()` মেথড ফাইলের সম্পূর্ণ টেক্সট পড়ে স্ক্রিনে প্রদর্শন করেছে। FileNotFoundError হ্যান্ডেল করার জন্য try-except দেওয়া হয়েছে।

Common Mistakes

- না থাকা ফাইল read মোডে খুলতে গিয়ে এরর খাওয়া।

অনুশীলনের জন্য

1. readline() ব্যবহার করে শুধু প্রথম লাইন পড়ুন।

2. readlines() ব্যবহার করে লিস্ট আকারে পড়ুন।

3. লুপ চালিয়ে লাইন বাই লাইন পড়ুন।

সমস্যা 180: একটি File-এ নতুন তথ্য Write করুন।

সমস্যা

বিদ্যমান বা নতুন একটি টেক্সট ফাইলে 'w' মোড ব্যবহার করে নতুন লেখা লিখুন (মনে রাখবেন 'w' মোড আগের সব লেখা মুছে নতুন করে লেখে)।

কী কী জানা থাকতে হবে?

- file.write() method

কীভাবে চিন্তা করব?

`write()` মেথড ফাইলে স্ট্রিং সেভ করে। 'w' মোডে খুললে পুরনো ডেটা মুছে যায় (Overwritten)।

অ্যালগরিদম

Step 1: ফাইল 'w' মোডে খোলা।

Step 2: file.write('New Data') করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Writing new data into a file (Overwriting existing content)
with open("my_file.txt", "w") as file:
    file.write("This is completely new text.\n")
    file.write("Previous contents have been overwritten.")

print("New data written successfully.")
```

Built-in Function / Shortcut Method

```
# Using print() function with file parameter
# print("Hello using print!", file=open("my_file.txt", "w"))
```

Sample Input / Output Table

Sample Input	Sample Output
None	New data written successfully.

ব্যাখ্যা

'my_file.txt' এর আগের লেখা মুছে গিয়ে নতুন এই দুটি লাইন লেখা হয়েছে।

Common Mistakes

- পুরনো লেখা রাখতে চাইলে 'w' মোড ব্যবহার করা (আগের লেখা রাখতে চাইলে 'a' মোড লাগে)।

অনুশীলনের জন্য

1. print() ফাংশনের `file=` প্যারামিটার দিয়ে ফাইলে লিখুন।

2. writelines() মেথড ব্যবহার করে লিস্টের সব স্ট্রিং ফাইলে লিখুন।

3. ১ থেকে ১০ পর্যন্ত সংখ্যা ফাইলে লিখুন।

সমস্যা 181: একটি বিদ্যমান File-এর শেষে নতুন তথ্য Append করুন।

সমস্যা

'a' (append) মোড ব্যবহার করে বিদ্যমান ফাইলের আগের লেখা নষ্ট না করে একদম শেষে নতুন তথ্য যুক্ত করুন।

কী কী জানা থাকতে হবে?

- 'a' append mode

কীভাবে চিন্তা করব?

`open('filename.txt', 'a')` মোডে খুললে ফাইলের কার্সার একদম শেষে চলে যায়। ফলে `write()` করলে আগের লেখার নিচে নতুন লেখা যুক্ত হয়।

অ্যালগরিদম

Step 1: ফাইল 'a' মোডে খোলা।

Step 2: file.write('\nAppended Line') করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Appending data to the end of a file
with open("my_file.txt", "a") as file:
    file.write("\nThis line is appended at the end.")
    file.write("\nWe preserved the old text!")

print("Data appended successfully.")
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Data appended successfully.

ব্যাখ্যা

'a' মোডের কারণে আগের লেখার নিচে নতুন দুটি লাইন যুক্ত হয়েছে।

Common Mistakes

- নতুন লাইনে যাওয়ার জন্য `\n` না দেওয়া (যার ফলে আগের লাইনের শেষ শব্দের সাথে লেগে যায়)।

অনুশীলনের জন্য

1. লগ ফাইলের মতো বর্তমান তারিখ ও সময় append করুন।

2. ব্যবহারকারীর ইনপুট নেওয়া বাক্য ফাইলে append করুন।

3. append করার পর read করে সম্পূর্ণ ফাইল দেখুন।

সমস্যা 182: একাধিক Student-এর তথ্য একটি File-এ সংরক্ষণ করুন।

সমস্যা

লুপ ব্যবহার করে ৩ জন শিক্ষার্থীর নাম, রোল ও জিপিএ একটি Text ফাইলে লাইন বাই লাইন সংরক্ষণ করুন।

কী কী জানা থাকতে হবে?

- Loop writing to file

কীভাবে চিন্তা করব?

লুপ চালিয়ে প্রতিটি শিক্ষার্থীর তথ্য একটি ফরম্যাটেড স্ট্রিং বানিয়ে ফাইলে `write()` করব।

অ্যালগরিদম

Step 1: students লিস্ট তৈরি করা।

Step 2: students.txt ফাইল 'w' মোডে খোলা।

Step 3: লুপ চালিয়ে প্রতিটি তথ্য write করা।

Python Code

```
# Storing multiple student records in a file
students = [
    {"roll": 1, "name": "Rahim", "gpa": 3.80},
    {"roll": 2, "name": "Karim", "gpa": 3.90},
    {"roll": 3, "name": "Sadiya", "gpa": 4.00}
]

with open("students.txt", "w") as file:
    file.write("Roll | Name   | GPA\n")
    file.write("---------------------\n")
    for s in students:
        file.write(f"{s['roll']}    | {s['name']:<6} | {s['gpa']}\n")

print("Student records saved to 'students.txt'.")
```

Built-in Function / Shortcut Method

```
# Using csv module for tabular data
```

```
# import csv ...
```

Sample Input / Output Table

Sample Input	Sample Output
None	Student records saved to 'students.txt'.

ব্যাখ্যা

টেবিল আকারে ৩ জন শিক্ষার্থীর ডেটা ফাইলে সংরক্ষিত হয়েছে।

Common Mistakes

- `\n` দিতে ভুলে যাওয়া।

অনুশীলনের জন্য

1. ব্যবহারকারীর কাছ থেকে লুপ চালিয়ে ইনপুট নিয়ে ফাইলে রাখুন।

2. csv মডিউল ব্যবহার করে ডেটা রাখুন।

3. ফাইলটি পড়ে আবার স্ক্রিনে দেখান।

সমস্যা 183: একটি Text File-এ মোট Word সংখ্যা নির্ণয় করুন।

সমস্যা

একটি টেক্সট ফাইল পড়ে তার ভেতর মোট কতটি শব্দ (Words) আছে তা `.split()` ব্যবহার করে গণনা করুন।

কী কী জানা থাকতে হবে?

- file.read().split()

কীভাবে চিন্তা করব?

ফাইল পড়ে পুরো টেক্সটকে `.split()` করলে শব্দের লিস্ট হয়, তার দৈর্ঘ্যই মোট শব্দ।

অ্যালগরিদম

Step 1: ফাইল read মোডে খোলা।

Step 2: words = file.read().split() করা।

Step 3: len(words) প্রিন্ট করা।

Python Code

```
# Counting total words in a file
```

```
with open("my_file.txt", "w") as f:
    f.write("Python is a powerful and easy programming language.")

with open("my_file.txt", "r") as file:
    content = file.read()
    words = content.split()
    print("Total words in file:", len(words))
```

Built-in Function / Shortcut Method

```
# One-liner
# print("Words:", len(open("my_file.txt").read().split()))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Total words in file: 8

ব্যাখ্যা

ফাইলে থাকা ৮টি শব্দ সফলভাবে গণনা করা হয়েছে।

Common Mistakes

- ফাইল বন্ধ হওয়ার আগেই split করার চেষ্টা করা।

অনুশীলনের জন্য

1. সবচেয়ে বড় শব্দটি বের করুন।

2. প্রতিটি শব্দের ফ্রিকোয়েন্সি বের করুন।

3. নির্দিষ্ট কোনো শব্দ কতবার আছে তা গুনুন।

সমস্যা 184: একটি Text File-এ মোট Line সংখ্যা নির্ণয় করুন।

সমস্যা

একটি টেক্সট ফাইলে মোট কতটি লাইন (Lines) আছে তা `readlines()` মেথড বা লুপ দিয়ে গণনা করুন।

কী কী জানা থাকতে হবে?

- file.readlines() method

কীভাবে চিন্তা করব?

`.readlines()` প্রতিটি লাইনকে লিস্টের এক একটি উপাদান বানায়। তার দৈর্ঘ্যই লাইন সংখ্যা।

অ্যালগরিদম

Step 1: ফাইল খোলা।

Step 2: lines = file.readlines() করা।

Step 3: len(lines) প্রিন্ট করা।

Python Code

```
# Counting lines in a file
with open("sample_lines.txt", "w") as f:
    f.write("Line 1: Hello\nLine 2: Python\nLine 3: World")

with open("sample_lines.txt", "r") as file:
    lines = file.readlines()
    print("Total lines in file:", len(lines))
```

Built-in Function / Shortcut Method

```
# Loop generator count
# print("Lines:", sum(1 for line in open("sample_lines.txt")))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Total lines in file: 3

ব্যাখ্যা

ফাইলে ৩টি আলাদা লাইন রয়েছে।

Common Mistakes

- বড় ফাইলের ক্ষেত্রে readlines() মেমোরি ভর্তি করতে পারে, সেক্ষেত্রে লুপ দিয়ে গোনা ভালো।

অনুশীলনের জন্য

1. ফাঁকা লাইনগুলো বাদ দিয়ে শুধু লেখা থাকা লাইন গুনুন।

2. প্রতি লাইনে কতটি শব্দ আছে তা দেখান।

3. লুপ চালিয়ে লাইন গুনুন।

সমস্যা 185: একটি Text File-এ মোট Character সংখ্যা নির্ণয় করুন।

সমস্যা

একটি টেক্সট ফাইলে মোট কতটি ক্যারেক্টার (অক্ষর, স্পেস ও চিহ্ন) আছে তা নির্ণয় করুন।

কী কী জানা থাকতে হবে?

- len(file.read())

কীভাবে চিন্তা করব?

`len(file.read())` সরাসরি ফাইলের মোট ক্যারেক্টার সংখ্যা দেয়।

অ্যালগরিদম

Step 1: ফাইল খোলা।

Step 2: char_count = len(file.read()) করা।

Step 3: প্রিন্ট করা।

Python Code

```
# Counting characters in a file
with open("my_file.txt", "r") as file:
    content = file.read()
    print("Total characters in file:", len(content))
```

Built-in Function / Shortcut Method

```
# Direct print
# print("Chars:", len(open("my_file.txt").read()))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Total characters in file: 51

ব্যাখ্যা

ফাইলের সব অক্ষর ও স্পেস মিলে ৫১টি ক্যারেক্টার রয়েছে।

Common Mistakes

- স্পেস বা নিউলাইন (
) গোনা হবে কিনা তা বুঝতে না পারা (len সব গুনবে)।

অনুশীলনের জন্য

1. স্পেস ও নিউলাইন বাদে শুধু অক্ষর গুনুন।

2. বড় হাতের অক্ষর কয়টি তা গুনুন।

3. ডিজিট কয়টি তা গুনুন।

সমস্যা 186: একটি Text File-এ মোট Vowel সংখ্যা নির্ণয় করুন।

সমস্যা

একটি টেক্সট ফাইল পড়ে তার ভেতর মোট কতটি স্বরবর্ণ (Vowel) আছে তা গণনা করুন।

কী কী জানা থাকতে হবে?

- Looping through file text

কীভাবে চিন্তা করব?

ফাইল পড়ে `char in 'aeiou'` চেক করে কাউন্ট বাড়াব।

অ্যালগরিদম

Step 1: ফাইল পড়ে lower() করা।

Step 2: vowel_count = 0 রাখা।

Step 3: লুপ চালিয়ে ভাওয়েল গুনুন।

Python Code

```
# Counting vowels in a file
with open("my_file.txt", "r") as file:
    text = file.read().lower()

vowels = "aeiou"
vowel_count = sum(1 for char in text if char in vowels)

print("Total vowels in file:", vowel_count)
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Total vowels in file: 17

ব্যাখ্যা

ফাইলটির টেক্সটে মোট ১৭টি ভাওয়েল পাওয়া গেছে।

Common Mistakes

- lower() না করা।

অনুশীলনের জন্য

1. Consonant কয়টি তা গুনুন।

2. কোন ভাওয়েল কয়বার আছে তা ডিকশনারি দিয়ে দেখান।

3. ভাওয়েলগুলো ফাইলে বড় হাতের করে সেভ করুন।

সমস্যা 187: একটি File-এর কপি (Copy) তৈরি করুন।

সমস্যা

একটি বিদ্যমান ফাইলের সকল তথ্য Read করে নতুন আরেকটি ফাইলে Write করার মাধ্যমে ফাইলের কপি (Backup) তৈরি করুন।

কী কী জানা থাকতে হবে?

- Simultaneous read and write

কীভাবে চিন্তা করব?

প্রথম ফাইল 'r' মোডে এবং দ্বিতীয় ফাইল 'w' মোডে খুলব। প্রথমটির `read()` দ্বিতীয়টিতে `write()` করব।

অ্যালগরিদম

Step 1: source.txt থেকে read করা।

Step 2: backup.txt তে write করা।

Python Code

```
# Copying one file to another
with open("my_file.txt", "r") as src:
    data = src.read()

with open("copy_file.txt", "w") as dest:
    dest.write(data)

print("File copied successfully to 'copy_file.txt'.")
```

Built-in Function / Shortcut Method

```
# Using shutil module
# import shutil
# shutil.copy("my_file.txt", "copy_file.txt")
```

Sample Input / Output Table

Sample Input	Sample Output
None	File copied successfully to 'copy_file.txt'.

ব্যাখ্যা

'my_file.txt' এর সম্পূর্ণ হুবহু কপি 'copy_file.txt' নামে তৈরি হয়েছে।

Common Mistakes

- source এবং destination ফাইলের নাম উল্টে দেওয়া।

অনুশীলনের জন্য

1. shutil.copy() ব্যবহার করুন।

2. বড় ফাইল কপি করার সময় লাইন বাই লাইন কপি করুন।

3. কপি করার পর উভয় ফাইলের সাইজ সমান কিনা যাচাই করুন।

সমস্যা 188: দুটি আলাদা File-এর তথ্য একত্র করে একটি নতুন File তৈরি করুন।

সমস্যা

file1.txt এবং file2.txt এর তথ্য পড়ে একত্র (Merge) করে merged.txt ফাইলে সংরক্ষণ করুন।

কী কী জানা থাকতে হবে?

- Multiple file reading and writing

কীভাবে চিন্তা করব?

উভয় ফাইল পড়ে তাদের স্ট্রিং যোগ করে নতুন ফাইলে লিখে দেব।

অ্যালগরিদম

Step 1: file1 তৈরি ও read করা।

Step 2: file2 তৈরি ও read করা।

Step 3: merged ফাইলে write করা।

Python Code

```
# Merging two files into a third file
with open("f1.txt", "w") as f: f.write("Part 1: Hello from file 1.\n")
with open("f2.txt", "w") as f: f.write("Part 2: Welcome from file 2.\n")

# Reading both
with open("f1.txt", "r") as f1, open("f2.txt", "r") as f2:
    data1 = f1.read()
    data2 = f2.read()

# Merging
with open("merged.txt", "w") as m:
    m.write(data1 + data2)

print("Files merged successfully into 'merged.txt'.")
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Files merged successfully into 'merged.txt'.

ব্যাখ্যা

দুটি ফাইলের ডেটা একত্র করে 'merged.txt' তৈরি হয়েছে।

Common Mistakes

- মাঝখানে নিউলাইন না দেওয়ার কারণে দুই ফাইলের লেখা একসাথে লেগে যাওয়া।

অনুশীলনের জন্য

1. ৩টি ফাইল মার্জ করুন।

2. প্রথম ফাইলের শেষে দ্বিতীয় ফাইলের তথ্য append করুন।

3. মার্জ করার পর মোট লাইন সংখ্যা প্রদর্শন করুন।

সমস্যা 189: Student Result একটি File-এ সংরক্ষণ ও প্রদর্শন করুন।

সমস্যা

শিক্ষার্থীদের পরীক্ষার নম্বর ও গ্রেড একটি Text ফাইলে সেভ করুন এবং পরবর্তীতে তা ফাইল থেকে Read করে টেবিলে প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- Complete CRUD logic with files

কীভাবে চিন্তা করব?

প্রথমে রেজাল্ট ফাইলে সেভ করব। তারপর readlines() দিয়ে পড়ে লুপ চালিয়ে প্রদর্শন করব।

অ্যালগরিদম

Step 1: results.txt তে ৩ জনের রেজাল্ট write করা।

Step 2: results.txt read মোডে খুলে প্রদর্শন করা।

Python Code

```
# Saving and Reading Student Results from File
# Step 1: Saving results
with open("results.txt", "w") as file:
    file.write("101,Abdur Rahman,A+\n")
    file.write("102,Sadiya Islam,A\n")
    file.write("103,Tanvir Ahmed,A+\n")

# Step 2: Reading and formatting results
print("--- Result Sheet ---")
print("Roll | Name          | Grade")
print("----------------------------")
with open("results.txt", "r") as file:
    for line in file:
        roll, name, grade = line.strip().split(",")
        print(f"{roll:<4} | {name:<13} | {grade}")
```

Built-in Function / Shortcut Method

```
# Using csv module for clean table view
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Result Sheet --- Roll | Name | Grade ----------

```
101  | Abdur Rahman  | A+
102  | Sadiya Islam  | A
103  | Tanvir Ahmed  | A+
```

ব্যাখ্যা

কমা (,) দিয়ে সেভ করা ডেটা `.split(',')` করে সুন্দর টেবিল আকারে দেখানো হয়েছে।

Common Mistakes

- `.strip()` না করার কারণে প্রতি লাইনের শেষে অতিরিক্ত স্পেস বা নিউলাইন আসা।

অনুশীলনের জন্য

1. নির্দিষ্ট রোলের রেজাল্ট ফাইল থেকে খুঁজে বের করুন।

2. ফাইলে নতুন শিক্ষার্থীর রেজাল্ট append করুন।

3. ফাইল থেকে পড়ে জিপিএ গড় বের করুন।

সমস্যা 190: একটি File-এ নির্দিষ্ট Word বা Text অনুসন্ধান (Search) করুন।

সমস্যা

একটি টেক্সট ফাইলের ভেতর নির্দিষ্ট কোনো শব্দ (Search Word) আছে কিনা এবং থাকলে তা কোন কোন লাইনে আছে তা খুঁজে বের করুন।

কী কী জানা থাকতে হবে?

- in operator inside loop over lines

কীভাবে চিন্তা করব?

ফাইলটি লাইন বাই লাইন পড়ব (`for idx, line in enumerate(file, 1):`)। যদি `search_word in line` হয়, তবে লাইনের নম্বরসহ প্রিন্ট করব।

> **[Congratulations]** অভিনন্দন! আপনি পাইথনের ১৯০টি সমস্যার সম্পূর্ণ প্র্যাকটিস বুকটি সফলভাবে সম্পন্ন করেছেন!

অ্যালগরিদম

Step 1: ফাইল তৈরি ও খোলা।

Step 2: search_word ইনপুট নেওয়া।

Step 3: enumerate লুপ চালিয়ে শব্দ খোঁজা।

Step 4: ফলাফল প্রদর্শন করা।

Python Code

```
# Searching a word inside a file line by line
with open("story.txt", "w") as f:
    f.write("Python is very popular.\nMany developers love Python.\nJava is
also widely used.\nPython makes automation easy.")

search_word = "Python"
found_lines = []

with open("story.txt", "r") as file:
    for line_num, line in enumerate(file, 1):
        if search_word.lower() in line.lower():
            found_lines.append((line_num, line.strip()))

print(f"Search results for '{search_word}':")
if found_lines:
    for num, text in found_lines:
        print(f"Line {num}: {text}")
else:
    print("Word not found in the file.")
```

Built-in Function / Shortcut Method

```
# None needed
```

Sample Input / Output Table

Sample Input	Sample Output
None	Search results for 'Python': Line 1: Python is very popular. Line 2: Many developers love Python. Line 4: Python makes automation easy.

ব্যাখ্যা

৩ নম্বর লাইনে Java থাকায় তা বাদে ১, ২ ও ৪ নম্বর লাইনে 'Python' শব্দটি সফলভাবে খুঁজে বের করা হয়েছে। এটিই আমাদের বইয়ের ১৯০তম এবং শেষ সমস্যা!

Common Mistakes

- Case sensitivity চেক না করা (`lower()` না করা)।

অনুশীলনের জন্য

1. শব্দটি মোট কতবার আছে তা count() দিয়ে গুনুন।

2. ব্যবহারকারীর কাছ থেকে অনুসন্ধানের শব্দ ইনপুট নিয়ে খুঁজুন।

3. শব্দটি replace করে নতুন ফাইলে সেভ করুন।

□ Part 2: Basic Application Development in Python (Syllabus Aligned)

> [ভিত্তি] Diploma in Engineering-এর Subject Code 28531 — 'Basic Application Development in Python' (Theory 60 + Practical 25) সিলেবাস অনুসরণ করে এই অংশ সাজানো হয়েছে।

সিলেবাসের সাথে বইয়ের ম্যাপিং (Syllabus Mapping)

ইউনিট	সিলেবাস টপিক	ক্লাস	নম্বর	বইয়ের সমস্যা নম্বর
১	PYTHON FUNCTIONS	৪	৬	১৬৫–১৭৭, ১৯১–২০০
২	FILE OPERATION IN PYTHON	২	৪	১৭৮–১৯০, ২১১–২২০
৩	MODULE, PACKAGE & APPLICATION SOFTWARE	২	৪	২০১–২১০, ২৯৯–৩০২
৪	BASICS OF OOP	৩	৬	২২১–২৩২
৫	FOUR PILLARS OF OOP	৫	৯	২৩৩–২৫০
৬	ITERATOR, GENERATOR & DECORATORS	২	৪	২৫১–২৬০
৭	EXCEPTION & ERROR HANDLING	৪	৭	২৬১–২৭২
৮	LOGGING IN PYTHON	৩	৫	২৭৩–২৭৯

৯	UNIT TESTING	৩	৬	২৮০–২৮৬
১০	PYTHON REGEX	২	৪	২৮৭–২৯৮
১১	APPLICATION SOFTWARE	২	৫	২৯৯–৩০২
—	Practical / Experiment ১–৬	১৬	২৫	প্রজেক্ট ১–৬ (শেষে)

Topic: Python Functions (Advanced) — Unit 1

সমস্যা 191: Library Function এবং User-defined Function-এর পার্থক্য বুঝুন ও ব্যবহার করুন।

সমস্যা

সিলেবাসের 1.2 অনুযায়ী — Library (Built-in) Function এবং User-defined Function-এর পার্থক্য জানতে হবে এবং উভয় ধরনের ফাংশন ব্যবহার করে একটি প্রোগ্রাম লিখতে হবে।

কী কী জানা থাকতে হবে?

- print(), len(), type() ইত্যাদি built-in function
- def keyword
- Function call

কীভাবে চিন্তা করব?

Library function হলো পাইথন আগে থেকেই তৈরি করে দেয় — যেমন `len()`, `sum()`, `max()`। User-defined function আমরা নিজেরা `def` দিয়ে তৈরি করি যখন বিল্ট-ইন ফাংশন দিয়ে কাজ হয় না। প্রোগ্রামে দুটোই একসাথে ব্যবহার করবো।

> **[Best Practice]** Function-এর নাম সবসময় কাজ বোঝায় এমন দিন, যেমন `calculate_tax()`, `get_average()`। `func1()`, `abc()` এড়িয়ে চলুন।

অ্যালগরিদম

Step 1: একটি list তৈরি করা।

Step 2: Library function `len()` ও `sum()` ব্যবহার করে দৈর্ঘ্য ও যোগফল বের করা।

Step 3: `def average_of_list()` নামে নিজের একটি function তৈরি করা।

Step 4: main অংশে দুটোই কল করে ফলাফল প্রিন্ট করা।

Python Code

```
# Library function vs User-defined function

marks = [78, 85, 92, 65, 70]

# 1. Library (built-in) functions - Python already provides these
total_marks = sum(marks)
number_of_subjects = len(marks)
highest = max(marks)

print("Using Library Functions:")
```

```
print("Total marks =", total_marks)
print("Number of subjects =", number_of_subjects)
print("Highest marks =", highest)



# 2. User-defined function - we create it ourselves using 'def'
def average_of_list(values):
    total = sum(values)
    count = len(values)
    return total / count



def grade(avg):
    if avg >= 80:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 60:
        return "B"
    else:
        return "Fail"



average = average_of_list(marks)
print("\nUsing User-defined Functions:")
print("Average marks =", average)
print("Grade =", grade(average))
```

Built-in Function / Shortcut Method

```
# Difference at a glance
# Library function    : print(), len(), sum(), max(), min(), sorted(), type()
# User-defined function: created with 'def' keyword by the programmer
```

Sample Input / Output Table

Sample Input	Sample Output
None	Using Library Functions: Total marks = 390 Number of subjects = 5 Highest marks = 92 Using User-defined Functions: Average marks = 78.0

```
Grade = A
```

ব্যাখ্যা

`sum()`, `len()`, `max()` হলো Library Function — পাইথন নিজেই এগুলো দিয়েছে। আর `average_of_list()` ও `grade()` আমরা নিজেরা `def` দিয়ে বানিয়েছি, তাই এগুলো User-defined Function। বড় প্রোগ্রামে নিজের ফাংশন বানালে কোড বারবার লিখতে হয় না।

Common Mistakes

- Function define করার আগে কল করা (`NameError`)।
- Function ের variable-কে ভিতরে সরাসরি ব্যবহার করে ফেলা — ভালো প্র্যাকটিস হলো parameter হিসেবে পাঠানো।
- Library function-এর নামে নিজের variable রাখা (যেমন `list = [1,2,3]` লিখলে `list()` ফাংশন নষ্ট হয়ে যায়)।

অনুশীলনের জন্য

1. `min()` এবং `max()` ব্যবহার করে সবচেয়ে ছোট ও বড় সংখ্যা বের করুন।

2. নিজের একটি function বানান যা কোনো list-এর শুধু জোড় সংখ্যাগুলো ফেরত দেয়।

3. একটি function বানান যা ইনপুট নেওয়া বছরকে লিপ ইয়ার কিনা তা বলে।

সমস্যা 192: Function-এ Default Argument ব্যবহার করুন।

সমস্যা

সিলেবাসের 1.5 অনুযায়ী — Function Argument-এর একটি প্রকার হলো Default Argument। এমন একটি function তৈরি করুন যেখানে argument না পাঠালে একটি ডিফল্ট মান ব্যবহার হবে।

কী কী জানা থাকতে হবে?

- Function definition
- Parameter & Argument

কীভাবে চিন্তা করব?

Function define করার সময় `def greet(name="Guest")` লিখলে `name`-এর ডিফল্ট মান 'Guest' হয়ে যায়। কল করার সময় মান দিলে সেটি ব্যবহৃত হবে, না দিলে ডিফল্ট মান ব্যবহৃত হবে।

> **[Note]** নিয়ম মনে রাখুন: `def function_name(simple_args..., default_args...)` — ডিফল্ট argument সবসময় শেষে বসে।

অ্যালগরিদম

Step 1: `def greet(name="Guest"):` লিখে function তৈরি করা।

Step 2: প্রথমবার argument সহ কল করা।

Step 3: দ্বিতীয়বার argument ছাড়া কল করা।

Step 4: দুই ক্ষেত্রেই আউটপুট প্রিন্ট করা।

Python Code

```
# Default argument example

def greet(name="Guest"):
    print("Hello", name + "! Welcome to Python class.")



# Calling with argument
greet("Rahim")
greet("Karim")

# Calling without argument -> default value is used
greet()



# Another example: default argument in calculation
def calculate_price(quantity, unit_price=50):
    total = quantity * unit_price
    print("Quantity:", quantity, "| Unit price:", unit_price, "| Total:",
total)



calculate_price(4, 60)
calculate_price(3)   # unit_price will be 50 (default)
```

Built-in Function / Shortcut Method

```
# Multiple default arguments
# def student_info(name, department="CSE", shift="1st"):
#     print(name, department, shift)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Hello Rahim! Welcome to Python class.

```
Hello Karim! Welcome to Python
class.
Hello Guest! Welcome to Python
class.
Quantity: 4 | Unit price: 60 |
Total: 240
Quantity: 3 | Unit price: 50 |
Total: 150
```

ব্যাখ্যা

`greet()` কল করার সময় কোনো নাম দেওয়া হয়নি, তাই ডিফল্ট মান 'Guest' ব্যবহার হয়েছে। `calculate_price(3)`-তেও unit_price ডিফল্ট ৫০ ধরা হয়েছে।

Common Mistakes

- Default argument-কে non-default argument-এর আগে রাখা (যেমন `def f(a=1, b)` — এটি SyntaxError দেয়)।
- Default মান হিসেবে পরিবর্তনযোগ্য (mutable) জিনিস যেমন list ব্যবহার করা — `def f(items=[])` ভুল প্র্যাকটিস।

অনুশীলনের জন্য

1. একটি function বানান যার ডিফল্ট shift মান '1st'।

2. ডিফল্ট মান ১০ রেখে একটি power function বানান।

3. তিনটি parameter-এর মধ্যে দুটিতে ডিফল্ট মান দিন।

সমস্যা 193: Keyword Argument ব্যবহার করে Function কল করুন।

সমস্যা

সিলেবাসের 1.5 অনুযায়ী — Keyword Argument ব্যবহার করে function-এর parameter-এর নাম ধরে মান পাঠানোর প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Function with multiple parameters

কীভাবে চিন্তা করব?

সাধারণত argument-এর ক্রম (position) অনুযায়ী মান বসে। কিন্তু `name="Rahim"` লিখে পাঠালে ক্রম মনে রাখতে হয় না — parameter-এর নাম দিয়েই মান চলে যায়। একে Keyword Argument বলে।

অ্যালগরিদম

Step 1: তিনটি parameter-যুক্ত একটি function তৈরি করা।

Step 2: Positional argument দিয়ে কল করা।

Step 3: Keyword argument দিয়ে কল করা (ক্রম বদলে)।

Step 4: উভয় মিশ্রিত (mixed) করে কল করা।

Python Code

```
# Keyword argument example

def student_info(name, roll, department):
    print("Name:", name)
    print("Roll:", roll)
    print("Department:", department)
    print("---")



# 1. Positional argument (order matters)
student_info("Rahim", 101, "CSE")

# 2. Keyword argument (order does NOT matter)
student_info(roll=102, department="EEE", name="Karim")

# 3. Mixed: positional first, then keyword
student_info("Sadia", department="Civil", roll=103)
```

Built-in Function / Shortcut Method

```
# Keyword arguments make code self-explanatory
# student_info(name="Rahim", roll=101, department="CSE")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Name: Rahim Roll: 101 Department: CSE --- Name: Karim Roll: 102 Department: EEE --- Name: Sadia

```
                                    Roll: 103
                                    Department: Civil
                                    ---
```

ব্যাখ্যা

Keyword argument দিলে ক্রম ভুল হলেও সমস্যা হয় না, কারণ মান parameter-এর নাম অনুযায়ী বসে। তবে mixed করলে positional argument আগে দিতে হয়।

Common Mistakes

- Keyword argument-এর পর positional argument দেওয়া (`f(name="A", 101)` — SyntaxError)।
- ভুল parameter নাম লেখা (`TypeError: got an unexpected keyword argument`)।

অনুশীলনের জন্য

1. চারটি parameter-এর function বানিয়ে keyword argument দিয়ে কল করুন।

2. একটি function বানান যা keyword argument দিয়ে রেক্ট্যাঙ্গেলের area বের করে।

3. Positional ও Keyword মিশিয়ে তিনভাবে কল করুন।

সমস্যা 194: *args ব্যবহার করে যেকোনো সংখ্যক Argument গ্রহণ করুন।

সমস্যা

সিলেবাসের 1.5 অনুযায়ী — Variable-length argument (`*args`) ব্যবহার করে এমন function তৈরি করুন যা যত খুশি তত argument নিতে পারবে।

কী কী জানা থাকতে হবে?

- Function definition
- Tuple concept
- for loop

কীভাবে চিন্তা করব?

কতগুলো argument আসবে আগে জানা না থাকলে parameter-এর আগে `*` দিই, যেমন `*args`। পাইথন সব argument-গুলো একটি tuple-এ জমা করে। তারপর loop চালিয়ে কাজ করা যায়।

> **[Note]** `args` নামটি জরুরি নয়, আসল জিনিস হলো তার আগের `*` চিহ্ন। `*numbers`, `*values` যেকোনো নাম চলবে।

অ্যালগরিদম

Step 1: `def total_marks(*args):` function তৈরি করা।

Step 2: args-এর ওপর for loop চালিয়ে যোগ করা।

Step 3: বিভিন্ন সংখ্যক argument দিয়ে কল করা।

Python Code

```
# Variable length argument using *args

def total_marks(*args):
    print("Received:", args)
    total = 0
    for m in args:
        total = total + m
    return total



print("Total (3 subjects):", total_marks(70, 80, 90))
print("Total (5 subjects):", total_marks(70, 80, 90, 65, 75))
print("Total (2 subjects):", total_marks(50, 60))



# Another example: find biggest of any number of values
def find_biggest(*numbers):
    biggest = numbers[0]
    for n in numbers:
        if n > biggest:
            biggest = n
    return biggest



print("\nBiggest:", find_biggest(12, 45, 7, 99, 23))
```

Built-in Function / Shortcut Method

```
# Shortcut using built-in functions
# def total_marks(*args):
#     return sum(args)
#
# def find_biggest(*numbers):
#     return max(numbers)
```

Sample Input / Output Table

Sample Input	Sample Output

None	Received: (70, 80, 90) Total (3 subjects): 240 Received: (70, 80, 90, 65, 75) Total (5 subjects): 380 Received: (50, 60) Total (2 subjects): 110 Biggest: 99

ব্যাখ্যা

`*args` সব argument-কে একটি tuple `(70, 80, 90)` হিসেবে রাখে। তাই loop বা `sum()`/`max()` দিয়ে সহজে কাজ করা যায়। argument সংখ্যা আগে থেকে জানা থাকলে `*args` লাগে না।

Common Mistakes

- `*args`-এর বদলে `args` লেখা (তাহলে শুধু একটিই argument নেবে)।
- `*args`-এর পর আবার normal positional parameter রাখা।
- args-কে list ভেবে `args.append()` করা (এটি tuple, অপরিবর্তনীয়)।

অনুশীলনের জন্য

1. `*args` দিয়ে যেকোনো সংখ্যক সংখ্যার গড় বের করুন।

2. `*args` দিয়ে সব argument-এর গুণফল বের করুন।

3. একটি function বানান যা `*args` নিয়ে শুধু জোড় সংখ্যাগুলো প্রিন্ট করে।

সমস্যা 195: **kwargs ব্যবহার করে Keyword Argument-এর সেট গ্রহণ করুন।

সমস্যা

সিলেবাসের 1.5 অনুযায়ী — `**kwargs` ব্যবহার করে যেকোনো সংখ্যক keyword argument গ্রহণ করে সেগুলো dictionary আকারে প্রসেস করুন।

কী কী জানা থাকতে হবে?

- Dictionary
- Keyword argument
- for loop with .items()

কীভাবে চিন্তা করব?

`*args` tuple বানায়, আর `**kwargs` (double star) একটি dictionary বানায় যেখানে key হলো parameter-এর নাম এবং value হলো পাঠানো মান। student-এর তথ্য যেমন name, roll, department — এগুলোর জন্য খুব উপযোগী।

অ্যালগরিদম

Step 1: `def show_profile(**kwargs):` function তৈরি করা।

Step 2: `kwargs.items()` দিয়ে loop চালিয়ে key-value প্রিন্ট করা।

Step 3: বিভিন্ন সংখ্যক keyword argument দিয়ে কল করা।

Python Code

```
# Variable length keyword argument using **kwargs

def show_profile(**kwargs):
    print("Type of kwargs:", type(kwargs))
    for key, value in kwargs.items():
        print(key, ":", value)
    print("---")



show_profile(name="Rahim", roll=101, department="CSE")
show_profile(name="Karim", shift="1st", session="2024-25", cgpa=3.75)



# Practical use: build a student record
def build_record(**kwargs):
    record = {}
    for key, value in kwargs.items():
        record[key] = value
    return record



student = build_record(name="Sadia", roll=103, department="Civil", shift="2nd")
print("\nFinal record:", student)
print("Roll number:", student["roll"])
```

Built-in Function / Shortcut Method

```
# Combining normal, *args and **kwargs
# def everything(name, *args, **kwargs):
#     print(name)
#     print(args)
#     print(kwargs)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Type of kwargs: <class 'dict'> name : Rahim roll : 101 department : CSE --- Type of kwargs: <class 'dict'> name : Karim shift : 1st session : 2024-25 cgpa : 3.75 --- Final record: {'name': 'Sadia', 'roll': 103, 'department': 'Civil', 'shift': '2nd'} Roll number: 103

ব্যাখ্যা

`**kwargs` সব keyword argument-কে dictionary-তে রূপ দেয়, তাই `.items()` দিয়ে সব key-value পড়া যায়। এটি ব্যবহার করে ডায়নামিক ডেটা (যেমন ফর্মের তথ্য) সহজে হ্যান্ডেল করা যায়।

Common Mistakes

- `**kwargs` parameter-কে function-এর শেষে না রাখা।
- kwargs-এর key-গুলো quote দিয়ে লেখা (`show_profile("name"=...)` ভুল)।
- dictionary-র মতো `kwargs[key]` দিয়ে না পড়ে লুপে `kwargs.key` লেখার চেষ্টা করা।

অনুশীলনের জন্য

1. `**kwargs` দিয়ে একটি function বানান যা শুধু numeric value-গুলোর যোগফল দেয়।

2. normal argument + `*args` + `**kwargs` একসাথে ব্যবহার করুন।

3. `**kwargs` দিয়ে একটি simple invoice (bill) তৈরি করুন।

সমস্যা 196: একটি Function থেকে একাধিক মান Return করুন।

সমস্যা

এমন একটি function তৈরি করুন যা একাধিক মান (যেমন যোগফল, গড়, সর্বোচ্চ ও সর্বনিম্ন) একসাথে ফেরত দেয়।

কী কী জানা থাকতে হবে?

- return statement
- Tuple unpacking

কীভাবে চিন্তা করব?

পাইথনে `return a, b, c` লিখলে আসলে একটি tuple ফেরত যায়। কল করার সময় `x, y, z = function()` দিয়ে আলাদা আলাদা ভ্যারিয়েবলে রাখা যায় — একে tuple unpacking বলে।

অ্যালগরিদম

Step 1: `return total, average, highest, lowest` দিয়ে function তৈরি করা।

Step 2: কল করার সময় চারটি ভ্যারিয়েবলে unpack করা।

Step 3: সব মান প্রিন্ট করা।

Python Code

```
# Returning multiple values from a function

def analyze_numbers(numbers):
    total = 0
    for n in numbers:
        total = total + n
    average = total / len(numbers)
    highest = max(numbers)
    lowest = min(numbers)
    return total, average, highest, lowest


marks = [70, 85, 90, 60, 75]

# Unpacking the returned tuple into 4 variables
total, average, highest, lowest = analyze_numbers(marks)

print("Marks   :", marks)
print("Total   :", total)
print("Average :", average)
print("Highest :", highest)
print("Lowest  :", lowest)
```

```
# We can also receive it as a single tuple
result = analyze_numbers(marks)
print("\nAs a tuple:", result)
print("Type of returned value:", type(result))
```

Built-in Function / Shortcut Method

```
# Using built-in functions directly
# total = sum(marks)
# average = sum(marks) / len(marks)
# highest = max(marks)
# lowest = min(marks)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Marks : [70, 85, 90, 60, 75] Total : 380 Average : 76.0 Highest : 90 Lowest : 60 As a tuple: (380, 76.0, 90, 60) Type of returned value: <class 'tuple'>

ব্যাখ্যা

functionটি চারটি মান একসাথে একটি tuple আকারে ফেরত দিয়েছে। `total, average, highest, lowest = ...` দিয়ে সেগুলো আলাদা ভ্যারিয়েবলে রাখা হয়েছে।

Common Mistakes

- return-এর পরের লাইনে কোড লেখা (return-এর পর function শেষ হয়ে যায়, কোড রান হয় না)।
- return করা মানের সংখ্যা ও গ্রহণকারী ভ্যারিয়েবলের সংখ্যা না মেলানো (`ValueError`)।
- return কথাটি লুপের ভিতরে রেখে দেওয়া, ফলে প্রথম ধাপেই function বের হয়ে যায়।

অনুশীলনের জন্য

1. একটি function বানান যা ইনপুট নেওয়া সংখ্যার square ও cube একসাথে return করে।

2. একটি function বানান যা নাম ও বয়স return করে।

3. rectangle-এর area ও perimeter একসাথে return করুন।

সমস্যা 197: Lambda Function ব্যবহার করুন।

সমস্যা

Anonymous function বা Lambda function ব্যবহার করে ছোট এক লাইনের function তৈরি করুন (যেমন দুইটি সংখ্যার যোগ, বড় সংখ্যা নির্ণয়)।

কী কী জানা থাকতে হবে?

- def function
- return statement

কীভাবে চিন্তা করব?

Lambda হলো নামহীন (anonymous) ছোট function। গঠন: `lambda arguments : expression`। এটি শুধু একটিমাত্র expression লিখতে পারে, তাই জটিল কাজের জন্য সাধারণ `def` ব্যবহার করাই ভালো।

> **[Best Practice]** ল্যাম্বডা শুধু ছোট, এক লাইনের কাজে ব্যবহার করুন। ২ লাইনের বেশি লজিক হলে সবসময় `def` ব্যবহার করুন — কোড পড়তে সহজ হবে।

অ্যালগরিদম

Step 1: `add = lambda a, b: a + b` দিয়ে lambda তৈরি করা।

Step 2: `add(10, 20)` কল করে ফলাফল প্রিন্ট করা।

Step 3: `bigger = lambda a, b: a if a > b else b` দিয়ে conditional lambda বানানো।

Step 4: list-এর সাথে `sorted(..., key=lambda ...)` ব্যবহার করা।

Python Code

```
# Lambda (anonymous) function examples

# 1. Simple lambda to add two numbers
add = lambda a, b: a + b
print("Addition:", add(10, 20))

# 2. Lambda with condition
bigger = lambda a, b: a if a > b else b
print("Bigger number:", bigger(45, 30))

# 3. Lambda to square a number
square = lambda x: x * x
print("Square of 7:", square(7))
```

```
# 4. Very common use: sorting with a custom key
students = [("Rahim", 3.75), ("Karim", 3.20), ("Sadia", 3.95)]
students_sorted = sorted(students, key=lambda student: student[1])
print("\nSorted by CGPA:", students_sorted)

# 5. Lambda without storing in a variable (IIFE style)
result = (lambda x, y: x * y)(6, 7)
print("Multiplication:", result)
```

Built-in Function / Shortcut Method

```
# Normal def version of the same lambda
# def add(a, b):
#     return a + b
#
# def bigger(a, b):
#     return a if a > b else b
```

Sample Input / Output Table

Sample Input	Sample Output
None	Addition: 30 Bigger number: 45 Square of 7: 49 Sorted by CGPA: [('Karim', 3.2), ('Rahim', 3.75), ('Sadia', 3.95)] Multiplication: 42

ব্যাখ্যা

Lambda function-এর কোনো নাম থাকে না, তাই একে anonymous function বলে। এটি সাধারণত `sorted()`, `map()`, `filter()`-এর ভিতরে key হিসেবে ব্যবহার করা হয়। বড় কাজের জন্য `def` ব্যবহার করবেন।

Common Mistakes

- lambda-এর ভিতরে একাধিক statement লেখার চেষ্টা করা (lambda শুধু একটি expression নেয়)।
- lambda-এর ভিতরে `return` লেখা (লাগবে না, expression নিজেই ফেরত যায়)।
- জটিল লজিক lambda-এ ঢুকিয়ে কোড অপাঠযোগ্য করে ফেলা।

অনুশীলনের জন্য

1. lambda দিয়ে একটি function বানান যা সংখ্যা জোড় কিনা তা বলে।

2. lambda দিয়ে তিনটি সংখ্যার গড় বের করুন।

3. `sorted()` ও lambda ব্যবহার করে নামের দৈর্ঘ্য অনুযায়ী list সাজান।

সমস্যা 198: map() এবং filter() Function ব্যবহার করুন।

সমস্যা

`map()` ব্যবহার করে একটি list-এর সব উপাদানের উপর একই কাজ প্রয়োগ করুন এবং `filter()` ব্যবহার করে শর্ত মান্য করে এমন উপাদান বাছাই করুন।

কী কী জানা থাকতে হবে?

- List
- Function
- lambda

কীভাবে চিন্তা করব?

`map(function, list)` প্রতিটি উপাদানে function চালিয়ে নতুন তালিকা দেয়। `filter(function, list)` শুধু সেই উপাদানগুলো রাখে যেগুলোর জন্য function `True` ফেরত দেয়। দুটিই iterator ফেরত দেয়, তাই `list()` দিয়ে রূপান্তর করতে হয়।

অ্যালগরিদম

Step 1: একটি number list তৈরি করা।

Step 2: `map()` দিয়ে সবার square বের করে list-এ রূপান্তর করা।

Step 3: `filter()` দিয়ে শুধু জোড় সংখ্যা বাছাই করা।

Step 4: সাধারণ function দিয়েও (lambda ছাড়া) map/filter ব্যবহার করা।

Python Code

```
# map() and filter() examples

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# 1. map() with lambda: square of every number
squares = list(map(lambda x: x * x, numbers))
print("Squares:", squares)

# 2. map() with a normal function
def to_fahrenheit(celsius):
    return (celsius * 9 / 5) + 32
```

```
temperatures = [0, 20, 30, 37, 100]
fahrenheit = list(map(to_fahrenheit, temperatures))
print("Fahrenheit:", fahrenheit)

# 3. filter() with lambda: keep only even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", evens)

# 4. filter() with a normal function: keep numbers greater than 50
marks = [45, 80, 33, 90, 55, 28]
def is_pass(mark):
    return mark >= 50



passed = list(filter(is_pass, marks))
print("Passed marks:", passed)
```

Built-in Function / Shortcut Method

```
# List comprehension - the Pythonic shortcut
# squares = [x * x for x in numbers]
# evens   = [x for x in numbers if x % 2 == 0]
# passed  = [m for m in marks if m >= 50]
```

Sample Input / Output Table

Sample Input	Sample Output
None	Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] Fahrenheit: [32.0, 68.0, 86.0, 98.6, 212.0] Even numbers: [2, 4, 6, 8, 10] Passed marks: [80, 90, 55]

ব্যাখ্যা

`map()` সব উপাদানে একই কাজ প্রয়োগ করে, `filter()` শর্ত পূরণকারী উপাদান রাখে। দুটিই map/filter object ফেরত দেয়, তাই `list()` দিয়ে তালিকায় রূপান্তর করা হয়েছে।

Common Mistakes

- `list()` না লিখে সরাসরি print করা (তখন `<map object at 0x...>` দেখাবে)।

- filter-এর function-কে True/False না ফেরত দেওয়া (যেমন সংখ্যা ফেরত দিলে 0 ছাড়া সব True ধরা হয়)।
- map/filter-কে for loop-এর বদলে সব জায়গায় ব্যবহার করে কোড জটিল করা।

অনুশীলনের জন্য

1. `map()` দিয়ে একটি string list-এর সব নাম uppercase করুন।

2. `filter()` দিয়ে ১–৫০ মধ্যে ৫ দিয়ে বিভাজ্য সংখ্যা বের করুন।

3. `map()` দিয়ে ইনপুট নেওয়া তিনটি সংখ্যার বর্গ বের করুন।

সমস্যা 199: Recursion ব্যবহার করে Factorial বের করুন।

সমস্যা

এমন একটি function তৈরি করুন যা নিজেই নিজেকে কল (recursion) করে n! (factorial) বের করবে।

কী কী জানা থাকতে হবে?

- Function
- if-else
- return

কীভাবে চিন্তা করব?

Recursion মানে একটি function নিজেকেই কল করে। যেকোনো recursion-এ দুটো জিনিস লাগে: (১) Base case — যেখানে থেমে যাবে, (২) Recursive case — যেখানে নিজেকে কল করবে। factorial(5) = 5 × factorial(4), আর factorial(0) = 1 (base case)।

> **[Important]** Recursion লেখার সময় সবসময় আগে Base case লিখুন। Base case ছাড়া recursion হলো একটা অসীম লুপের মতো — প্রোগ্রাম ক্র্যাশ করবে।

অ্যালগরিদম

Step 1: `def factorial(n):` function তৈরি করা।

Step 2: Base case: যদি n == 0 বা n == 1 হয়, 1 ফেরত দেওয়া।

Step 3: Recursive case: `return n * factorial(n - 1)`।

Step 4: ইউজারের কাছ থেকে n ইনপুট নিয়ে ফলাফল প্রিন্ট করা।

Python Code

```
# Factorial using recursion
```

```
def factorial(n):
    # Base case: stop here
    if n == 0 or n == 1:
        return 1
    # Recursive case: function calls itself
    else:
        return n * factorial(n - 1)



num = int(input("Enter a number: "))

if num < 0:
    print("Factorial is not defined for negative numbers.")
else:
    result = factorial(num)
    print("Factorial of", num, "is:", result)

# Showing the steps
print("\nStep by step:")
for i in range(num + 1):
    print(i, "! =", factorial(i))
```

Built-in Function / Shortcut Method

```
# Shortcut: built-in factorial from math module
# import math
# print(math.factorial(num))
```

Sample Input / Output Table

Sample Input	Sample Output
5	Factorial of 5 is: 120 Step by step: 0 ! = 1 1 ! = 1 2 ! = 2 3 ! = 6 4 ! = 24 5 ! = 120

ব্যাখ্যা

`factorial(5)` কল হলে এটি `5 * factorial(4)` → `4 * factorial(3)` → ... → `factorial(1)` পর্যন্ত যায়। এরপর base case থেকে ফেরত আসতে আসতে 5×4×3×2×1 = 120 হয়।

Common Mistakes

- Base case না লেখা — তখন function কখনো থামবে না এবং `RecursionError` দেখাবে।
- `return n * factorial(n-1)`-এর বদলে শুধু `factorial(n-1)` লেখা (ফলাফল ফেরত যাবে না, None হবে)।
- খুব বড় সংখ্যায় (যেমন ২০০০) recursion কল করলে stack overflow হতে পারে — তখন loop ব্যবহার করুন।

অনুশীলনের জন্য

1. Recursion দিয়ে ১ থেকে n পর্যন্ত সংখ্যার যোগফল বের করুন।

2. Recursion দিয়ে Fibonacci সিরিজের n-তম পদ বের করুন।

3. Recursion দিয়ে একটি সংখ্যার অঙ্কগুলোর যোগফল বের করুন।

সমস্যা 200: Date and Time Function ব্যবহার করুন।

সমস্যা

সিলেবাসের 1.6 অনুযায়ী — `datetime` মডিউল ব্যবহার করে বর্তমান তারিখ ও সময় বের করা, নির্দিষ্ট ফরম্যাটে দেখানো এবং দুই তারিখের পার্থক্য বের করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- import module
- Function
- String formatting

কীভাবে চিন্তা করব?

`datetime` মডিউলের `datetime.now()` বর্তমান তারিখ-সময় দেয়। `strftime()` দিয়ে আমাদের পছন্দমতো ফরম্যাটে (যেমন %d-%m-%Y) দেখানো যায়। দুইটি date বিয়োগ করলে `timedelta` পাওয়া যায়, যার `.days` দিনের পার্থক্য দেয়।

> **[Best Practice]** বাংলাদেশে সাধারণত DD-MM-YYYY ফরম্যাট ব্যবহার হয়: `now.strftime("%d-%m-%Y")`। রিপোর্ট বা ইনভয়েসে এটি ব্যবহার করুন।

অ্যালগরিদম

Step 1: `from datetime import datetime, date` লিখে মডিউল আনা।

Step 2: `datetime.now()` দিয়ে বর্তমান সময় নেওয়া।

Step 3: `strftime('%d-%m-%Y %H:%M:%S')` দিয়ে সুন্দর ফরম্যাটে প্রিন্ট করা।

Step 4: দুইটি `date` বিয়োগ করে দিনের পার্থক্য বের করা।

Python Code

```
# Date and time functions using datetime module
from datetime import datetime, date

# 1. Current date and time
now = datetime.now()
print("Current date and time:", now)

# 2. Only today's date
today = date.today()
print("Today's date:", today)

# 3. Separate parts
print("Year :", now.year)
print("Month:", now.month)
print("Day  :", now.day)
print("Hour :", now.hour)
print("Min  :", now.minute)

# 4. Formatting with strftime()
formatted = now.strftime("%d-%m-%Y  %H:%M:%S")
print("\nFormatted (DD-MM-YYYY):", formatted)
print("Readable format:", now.strftime("%A, %d %B %Y"))

# 5. Difference between two dates
date1 = date(2025, 1, 1)
date2 = date(2025, 12, 31)
difference = date2 - date1
print("\nDays between 01-01-2025 and 31-12-2025:", difference.days, "days")
```

Built-in Function / Shortcut Method

```
# Common strftime codes
# %d = day, %m = month, %Y = year (4 digit)
# %H = hour (24h), %M = minute, %S = second
# %A = weekday name, %B = month name
```

Sample Input / Output Table

Sample Input	Sample Output

None	Current date and time: 2025-08-14 10:35:42.123456 Today's date: 2025-08-14 Year : 2025 Month: 8 Day : 14 Hour : 10 Min : 35 Formatted (DD-MM-YYYY): 14-08-2025 10:35:42 Readable format: Thursday, 14 August 2025 Days between 01-01-2025 and 31-12-2025: 364 days

ব্যাখ্যা

`datetime.now()` পুরো তারিখ-সময় দেয়, `date.today()` শুধু তারিখ। `strftime()` দিয়ে ফরম্যাট করা হয়। দুইটি date বিয়োগ করলে `timedelta` অবজেক্ট পাওয়া যায় যার `.days` দিয়ে দিন গোনা যায়।

Common Mistakes

- `datetime` মডিউল import না করে সরাসরি `datetime.now()` লেখা (`NameError`)।
- strftime-এ `%m` (মাস) ও `%M` (মিনিট) গুলিয়ে ফেলা।
- `date(2025, 1, 1)`-এ সাল/মাস/দিন string আকারে দেওয়া (`'2025'`) — এগুলো অবশ্যই integer হবে।

অনুশীলনের জন্য

1. বর্তমান সময় থেকে ব্যবহারকারীর জন্মতারিখ পর্যন্ত মোট কত দিন তা বের করুন।

2. বর্তমান বছরটি লিপ ইয়ার কিনা date function দিয়ে যাচাই করুন।

3. আজকের তারিখ একটি ফাইলে লিখে রাখুন।

Topic: Module, Package & Application Software — Unit 3

সমস্যা 201: math Module ব্যবহার করে গাণিতিক কাজ করুন।

সমস্যা

সিলেবাসের 3.4 ও 3.6 অনুযায়ী — Python-এর বিল্ট-ইন `math` module import করে বর্গমূল, ঘাত, পাই-এর মান, factorial ইত্যাদি বের করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- import statement
- Function call
- Variable

কীভাবে চিন্তা করব?

Module হলো একটি .py ফাইল যাতে আগে থেকে লেখা function/class থাকে। `import math` লিখলে ওই সব function আমাদের প্রোগ্রামে ব্যবহার করা যায়। ব্যবহারের সময় `math.sqrt(25)` — অর্থাৎ `module_name.function_name()`।

> **[Best Practice]** পুরো module import করাই নিরাপদ: `import math`। `from math import *` ব্যবহার করবেন না — নামের সংঘাত (conflict) হতে পারে।

অ্যালগরিদম

Step 1: `import math` দিয়ে মডিউল আনা।

Step 2: `math.sqrt()`, `math.pow()`, `math.pi`, `math.factorial()` ব্যবহার করা।

Step 3: প্রতিটির ফলাফল প্রিন্ট করা।

Python Code

```
# Using the built-in math module
import math

num = 25

print("Square root of", num, ":", math.sqrt(num))
print("Power 2^8:", math.pow(2, 8))
print("Value of pi:", math.pi)
print("Value of e:", math.e)
print("Factorial of 5:", math.factorial(5))

print("\nCeiling of 4.2:", math.ceil(4.2))
print("Floor of 4.8:", math.floor(4.8))
```

```
# Practical use: area of a circle
radius = float(input("\nEnter radius of a circle: "))
area = math.pi * radius * radius
print("Area of the circle:", round(area, 2))
```

Built-in Function / Shortcut Method

```
# Import only what you need
# from math import sqrt, pi, factorial
# print(sqrt(25), pi, factorial(5))
#
# Or with an alias (short name)
# import math as m
# print(m.sqrt(25))
```

Sample Input / Output Table

Sample Input	Sample Output
7	Square root of 25 : 5.0 Power 2^8: 256.0 Value of pi: 3.141592653589793 Value of e: 2.718281828459045 Factorial of 5: 120 Ceiling of 4.2: 5 Floor of 4.8: 4 Enter radius of a circle: 7 Area of the circle: 153.94

ব্যাখ্যা

`import math` করার পর `math.` লিখে ডট (.) দিয়ে মডিউলের সব function-access করা যায়। `math.pi` হলো একটি constant (ধ্রুবক), function নয় — তাই ব্র্যাকেট লাগে না।

Common Mistakes

- `import math` না লিখে `sqrt(25)` কল করা (`NameError`)।
- `from math import sqrt` করার পর `math.sqrt(25)` লেখা (তখন `NameError: name 'math' is not defined`)।
- `math.pow(2, 8)`-এর ফলাফল float (256.0) — integer চাইলে `2 ** 8` ব্যবহার করুন।

অনুশীলনের জন্য

1. `math` module দিয়ে একটি ত্রিভুজের hypotenuse বের করুন।

2. `math.gcd()` দিয়ে দুইটি সংখ্যার গ.সা.গু বের করুন।

3. `math.radians()` ও `math.sin()` দিয়ে ৩০ ডিগ্রির sine বের করুন।

সমস্যা 202: random Module ব্যবহার করে এলোমেলো সংখ্যা তৈরি করুন।

সমস্যা

`random` module ব্যবহার করে random সংখ্যা তৈরি, list থেকে এলোমেলোভাবে বাছাই এবং list এলোমেলো (shuffle) করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- import statement
- List
- for loop

কীভাবে চিন্তা করব?

`random.randint(a, b)` a থেকে b-এর মধ্যে একটি পূর্ণসংখ্যা দেয় (দুই প্রান্তসহ)। `random.choice(list)` তালিকা থেকে একটা বাছে, আর `random.shuffle(list)` তালিকার ক্রম এলোমেলো করে দেয়।

অ্যালগরিদম

Step 1: `import random` করা।

Step 2: `random.randint(1, 100)` দিয়ে ৫টি random সংখ্যা তৈরি করা।

Step 3: `random.choice()` দিয়ে list থেকে একটি নাম বাছাই করা।

Step 4: `random.shuffle()` দিয়ে list এলোমেলো করা।

Python Code

```
# Using the random module
import random

# 1. Random integer between 1 and 100 (both included)
print("Random number 1-100:", random.randint(1, 100))

# 2. Five random numbers using a loop
print("\nFive random numbers:")
for i in range(5):
    print(random.randint(1, 50))

# 3. Random float between 0 and 1
```

```
print("\nRandom float:", random.random())

# 4. Pick a random item from a list
students = ["Rahim", "Karim", "Sadia", "Tania", "Imran"]
lucky = random.choice(students)
print("\nRandomly selected student:", lucky)

# 5. Shuffle a list
print("Before shuffle:", students)
random.shuffle(students)
print("After shuffle :", students)

# 6. Pick 2 unique winners
winners = random.sample(students, 2)
print("Two winners:", winners)
```

Built-in Function / Shortcut Method

```
# Other useful random functions
# random.randrange(0, 100, 5)   -> random multiple of 5
# random.uniform(1.0, 10.0)     -> random float in range
# random.seed(10)               -> same random output every run (for testing)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Random number 1-100: 47 Five random numbers: 23 8 41 15 33 Random float: 0.7321458931 Randomly selected student: Sadia Before shuffle: ['Rahim', 'Karim', 'Sadia', 'Tania', 'Imran'] After shuffle : ['Tania', 'Imran', 'Rahim', 'Sadia',

```
                                        'Karim']
                                        Two winners: ['Karim', 'Tania']
```

ব্যাখ্যা

প্রতিবার রান করলে ভিন্ন ফলাফল আসবে — এটিই random-এর বৈশিষ্ট্য। `shuffle()` মূল list-টিকেই বদলে দেয় (কোনো নতুন list ফেরত দেয় না)।

Common Mistakes

- `random.shuffle(students)`-এর ফলাফল নতুন ভ্যারিয়েবলে রাখার চেষ্টা করা (এটি `None` ফেরত দেয়)।
- `random.randint(1, 10)`-কে float ভাবা — এটি সবসময় integer দেয়; float চাইলে `random.uniform()` ব্যবহার করুন।
- `random` নামে নিজের কোনো ফাইল (`random.py`) বানিয়ে ফেলা — তখন Python ভুল ফাইল import করবে।

অনুশীলনের জন্য

1. random দিয়ে একটি ছক্কা (1-6) ঘোরানোর প্রোগ্রাম বানান।

2. random দিয়ে সংখ্যা অনুমানের (Guess the number) গেম বানান।

3. ১ থেকে ১০ এর মধ্যে ৩টি ইউনিক random সংখ্যা তৈরি করুন।

সমস্যা 203: import করার বিভিন্ন পদ্ধতি (import, from-import, alias) ব্যবহার করুন।

সমস্যা

সিলেবাসের 3.4 অনুযায়ী — Python-এ module import করার তিনটি পদ্ধতি (`import x`, `from x import y`, `import x as z`) ব্যবহার করে দেখান।

কী কী জানা থাকতে হবে?

- Module concept
- math module

কীভাবে চিন্তা করব?

তিনটি পদ্ধতি আছে: (১) `import math` → `math.sqrt()` লিখতে হয়; (২) `from math import sqrt` → সরাসরি `sqrt()` লেখা যায়; (৩) `import math as m` → ছোট নামে `m.sqrt()`। বড় প্রজেক্টে alias খুব কাজে লাগে।

অ্যালগরিদম

Step 1: প্রথম পদ্ধতি: `import math` ব্যবহার করা।

Step 2: দ্বিতীয় পদ্ধতি: `from math import sqrt, pi` ব্যবহার করা।

Step 3: তৃতীয় পদ্ধতি: `import math as m` (alias) ব্যবহার করা।

Step 4: তিনটির আউটপুট প্রিন্ট করা।

Python Code

```
# Method 1: import the whole module
import math

print("Method 1 - import math")
print("Square root:", math.sqrt(144))
print("PI value:", math.pi)

# Method 2: import specific items only
from math import sqrt, pi, pow

print("\nMethod 2 - from math import sqrt, pi, pow")
print("Square root:", sqrt(144))
print("PI value:", pi)
print("2 power 10:", pow(2, 10))

# Method 3: import with an alias (short nickname)
import math as m

print("\nMethod 3 - import math as m")
print("Square root:", m.sqrt(144))

# Very common aliases used in projects
import datetime as dt
import random as rnd

print("\nCommon aliases in real projects:")
print("Today:", dt.date.today())
print("Random 1-10:", rnd.randint(1, 10))
```

Built-in Function / Shortcut Method

```
# Alias convention used by professionals
# import pandas as pd
# import numpy as np
# import matplotlib.pyplot as plt
# import tkinter as tk
```

Sample Input / Output Table

Sample Input	Sample Output
None	Method 1 - import math Square root: 12.0 PI value: 3.141592653589793 Method 2 - from math import sqrt, pi, pow Square root: 12.0 PI value: 3.141592653589793 2 power 10: 1024.0 Method 3 - import math as m Square root: 12.0 Common aliases in real projects: Today: 2025-08-14 Random 1-10: 6

ব্যাখ্যা

তিনটি পদ্ধতিতেই একই কাজ হয়। তবে প্রজেক্টে `import pandas as pd`, `import numpy as np` — এমন alias স্ট্যান্ডার্ড হিসেবে ব্যবহার করা হয়, যাতে কোড ছোট ও পড়তে সহজ হয়।

Common Mistakes

- `from math import sqrt` করার পর `math.sqrt()` লেখা — তখন `math` নামটি define-ই হয়নি।
- সবকিছুই `from module import *` দিয়ে আনা — কোডে কোন্ নাম কোথা থেকে এলো তা বোঝা যায় না।
- নিজের ফাইলের নাম বিল্ট-ইন module-এর নামে (যেমন `math.py`, `random.py`) রাখা।

অনুশীলনের জন্য

1. `datetime` module-কে `dt` alias দিয়ে import করে আজকের তারিখ দেখান।

2. `from math import *` দিয়ে ৩টি ভিন্ন function ব্যবহার করুন।

3. `statistics` module import করে একটি list-এর গড় (mean) বের করুন।

সমস্যা 204: dir() এবং help() Function দিয়ে Module অন্বেষণ করুন।

সমস্যা

`dir()` ব্যবহার করে কোনো module-এর সব function/class-এর তালিকা দেখুন এবং `help()` ব্যবহার করে নির্দিষ্ট function-এর ডকুমেন্টেশন পড়ুন।

কী কী জানা থাকতে হবে?

- import statement
- Module concept

কীভাবে চিন্তা করব?

কোনো module-এ কী কী আছে তা মনে রাখা সম্ভব নয়। `dir(module)` সব নামের তালিকা দেয়, আর `help(module.function)` বিস্তারিত ব্যাখ্যা দেখায়। এটি শেখার জন্য দারুণ একটি অভ্যাস।

> **[Best Practice]** নতুন কোনো module শেখার সময় প্রথমেই `dir(module)` ও `help(module)` চালান। নিজে মুখস্থ না করে ডকুমেন্টেশন পড়তে অভ্যস্ত হোন — এটিই প্রফেশনাল প্রোগ্রামারদের অভ্যাস।

অ্যালগরিদম

Step 1: `import math` করা।

Step 2: `dir(math)` দিয়ে সব নামের তালিকা প্রিন্ট করা।

Step 3: `help(math.sqrt)` দিয়ে ডকুমেন্টেশন দেখা।

Step 4: `math.__doc__` দিয়ে module-এর সারাংশ দেখা।

Python Code

```
# Exploring a module with dir() and help()
import math

# 1. List everything available inside the math module
names = dir(math)
print("Total items in math module:", len(names))
print("\nFirst 15 items:")
for item in names[:15]:
    print(" ", item)

# 2. Check whether a specific function exists
print("\n'factorial' exists in math?", "factorial" in dir(math))
print("'average' exists in math? ", "average" in dir(math))

# 3. Read the documentation of a function
print("\n--- Documentation of math.sqrt ---")
help(math.sqrt)
```

```
# 4. Module level description
print("\n--- Module description ---")
print(math.__doc__)
```

Built-in Function / Shortcut Method

```
# In interactive Python shell you can simply type:
# >>> dir(math)
# >>> help(math.sqrt)
# >>> math.sqrt.__doc__
```

Sample Input / Output Table

Sample Input	Sample Output
None	Total items in math module: 58 First 15 items: __doc__ __loader__ __name__ __package__ __spec__ acos acosh asin asinh atan atan2 atan3 ceil comb 'factorial' exists in math? True 'average' exists in math? False --- Documentation of math.sqrt --- Help on built-in function sqrt in module math: sqrt(x, /)

```
                                        Return the square root of x.

                                    --- Module description ---
                                    This module provides access to
                                    the mathematical functions
                                    defined by the C standard.
```

ব্যাখ্যা

`dir()` module-এর ভেতরের সব নামের একটি list দেয়। `'factorial' in dir(math)` দিয়ে যাচাই করা যায় কোনো নির্দিষ্ট function আছে কিনা। `help()` সেই function-এর ব্যবহারবিধি (signature) ও কাজ বাংলায়-ইংরেজিতে দেখায়।

Common Mistakes

- `dir(math)`-এর বদলে `dir('math')` লেখা (string দিলে string-এর method দেখাবে)।
- `help(math.sqrt())` লেখা — ব্র্যাকেটসহ লিখলে function কল হয়ে যায়; সঠিক হলো `help(math.sqrt)`।
- `dir()`-এর ফলাফলে `__name__` সহ underscore-যুক্ত নামগুলোকে সাধারণ function ভাবা (এগুলো special attribute)।

অনুশীলনের জন্য

1. `dir()` দিয়ে `random` module-এর সব function দেখুন।

2. `help()` দিয়ে `math.factorial`-এর ডকুমেন্টেশন পড়ুন।

3. `dir()` দিয়ে যাচাই করুন `math` module-এ `gcd` আছে কিনা।

সমস্যা 205: __name__ == "__main__" এর ব্যবহার বুঝুন।

সমস্যা

সিলেবাসের 3.6 অনুযায়ী — নিজের module তৈরির সময় `if __name__ == "__main__":` ব্লকের কাজ ও প্রয়োজনীয়তা একটি উদাহরণ দিয়ে বুঝান।

কী কী জানা থাকতে হবে?

- Function
- Module
- import

কীভাবে চিন্তা করব?

যখন আমরা কোনো .py ফাইল সরাসরি রান করি, তখন Python সেই ফাইলের `__name__` ভ্যারিয়েবলে `'__main__'` রাখে। কিন্তু ফাইলটি import করা হলে `__name__`-এ ফাইলের নাম থাকে। তাই `if __name__ == "__main__":` ব্লকের ভিতরের কোড শুধু সরাসরি রান করলে চলে — import করলে চলে না।

> **[Important]** যেকোনো module-এর সব function-এর নিচে `if __name__ == "__main__":` ব্লক রাখুন, আর সেখানে function-গুলোর টেস্ট কোড দিন। এতে module import করলে অপ্রয়োজনীয় আউটপুট আসবে না।

অ্যালগরিদম

Step 1: দুইটি ফাইল তৈরি করতে হবে: `mymodule.py` এবং `main.py`।

Step 2: `mymodule.py`-তে একটি function রাখা এবং নিচে `if __name__ == "__main__":` ব্লক রাখা।

Step 3: `main.py` থেকে `import mymodule` করা।

Step 4: দুইভাবে রান করে পার্থক্য দেখা।

Python Code

```
# ============================================================
# FILE 1: mymodule.py   (save this file first)
# ============================================================
def greet(name):
    return "Hello " + name + ", welcome!"


def add(a, b):
    return a + b


print("mymodule is loaded. __name__ =", __name__)

# This block runs ONLY when mymodule.py is executed directly
if __name__ == "__main__":
    print("Running mymodule.py directly")
    print(greet("Rahim"))
    print("Sum:", add(5, 3))



# ============================================================
# FILE 2: main.py   (run this file from the same folder)
# ============================================================
import mymodule
```

```
print(mymodule.greet("Karim"))
print("Sum:", mymodule.add(10, 20))
```

Built-in Function / Shortcut Method

```
# Why it matters:
# - Run mymodule.py directly  -> prints both lines + test code
# - Run main.py               -> prints ONLY the import message,
#                                the test block is skipped
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Running mymodule.py directly --- mymodule is loaded. __name__ = __main__ Running mymodule.py directly Hello Rahim, welcome! Sum: 8 --- Running main.py --- mymodule is loaded. __name__ = mymodule Hello Karim, welcome! Sum: 30

ব্যাখ্যা

`main.py` রান করার সময় `mymodule`-এর `__name__` হয় `'mymodule'`, তাই `if __name__ == "__main__":` ব্লকটি কাজ করেনি — অর্থাৎ টেস্ট কোড চলল না, শুধু function-গুলো ব্যবহার করা গেল। এটিই এই ব্লকের মূল উদ্দেশ্য।

Common Mistakes

- `__name__ == '__main__'` লিখতে গিয়ে underscore ভুল করা (দুই পাশে দুইটি করে underscore)।
- সব কোড (function-সহ) `if __name__ == "__main__":` ব্লকের ভিতরে ঢুকিয়ে দেওয়া — তখন import করলে function পাওয়া যাবে না।
- মনে করা এটি বাধ্যতামূলক — ছোট স্ক্রিপ্টে লাগবে না, তবে module বানানোর সময় অবশ্যই দেবেন।

অনুশীলনের জন্য

1. একটি module বানান যাতে area ও perimeter function থাকে এবং `if __name__ == "__main__":` ব্লকে টেস্ট কোড থাকে।

2. দুইটি ভিন্ন ফাইল থেকে একই module import করুন।

3. `__name__` ভ্যারিয়েবলের মান প্রিন্ট করে দুই ক্ষেত্রে পার্থক্য দেখুন।

সমস্যা 206: নিজের Module তৈরি করে অন্য ফাইল থেকে Import করুন।

সমস্যা

সিলেবাসের 3.6 অনুযায়ী — নিজে একটি module (যেমন `calculator.py`) তৈরি করুন যাতে যোগ, বিয়োগ, গুণ, ভাগের function থাকবে, তারপর অন্য একটি ফাইল থেকে সেটি import করে ব্যবহার করুন।

কী কী জানা থাকতে হবে?

- Function definition
- import
- __name__ == "__main__"

কীভাবে চিন্তা করব?

যেকোনো .py ফাইল-ই একটি module। তাই আমরা `calculator.py` নামে একটি ফাইল বানিয়ে তাতে কয়েকটি function লিখবো। এরপর একই ফোল্ডারে থাকা অন্য ফাইল থেকে `import calculator` লিখলে ওই function-গুলো ব্যবহার করা যাবে।

অ্যালগরিদম

Step 1: `calculator.py` ফাইলে ৪টি function (add, subtract, multiply, divide) লেখা।

Step 2: `if __name__ == "__main__":` ব্লকে টেস্ট কোড রাখা।

Step 3: `use_calculator.py` ফাইলে `import calculator` করা।

Step 4: function-গুলো কল করে ফলাফল প্রিন্ট করা।

Python Code

```
# ============================================================
# FILE 1: calculator.py     <- this is our own module
# ============================================================
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b
```

```
def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        return "Error: Cannot divide by zero"
    return a / b


if __name__ == "__main__":
    print("Testing calculator module")
    print(add(10, 5))
    print(subtract(10, 5))



# ============================================================
# FILE 2: use_calculator.py    <- run this file
# ============================================================
import calculator

print("Addition       :", calculator.add(20, 10))
print("Subtraction    :", calculator.subtract(20, 10))
print("Multiplication :", calculator.multiply(20, 10))
print("Division       :", calculator.divide(20, 10))
print("Divide by zero :", calculator.divide(20, 0))

# Importing specific functions only
from calculator import add, multiply

print("\nUsing imported names directly:")
print("Add:", add(5, 5))
print("Multiply:", multiply(5, 5))
```

Built-in Function / Shortcut Method

```
# After importing you can also read the file location:
# import calculator
# print(calculator.__file__)
# print(calculator.__name__)
```

Sample Input / Output Table

Sample Input	Sample Output

None	Addition : 30 Subtraction : 10 Multiplication : 200 Division : 2.0 Divide by zero : Error: Cannot divide by zero Using imported names directly: Add: 10 Multiply: 25

ব্যাখ্যা

`calculator.py` ফাইলটি-ই একটি module। `use_calculator.py` থেকে `import calculator` করার পর `calculator.add()` ব্যবহার করা গেছে। দুইটি ফাইল অবশ্যই একই ফোল্ডারে থাকতে হবে (অথবা Python path-এ থাকতে হবে)।

Common Mistakes

- দুইটি ফাইল আলাদা ফোল্ডারে রাখা — তখন `ModuleNotFoundError` হবে।
- module ফাইলের নামে স্পেস বা ড্যাশ (`-`) দেওয়া — নাম সবসময় ছোট হাতের, স্পেস ছাড়া (যেমন `my_module.py`)।
- module import করার পর function কল করতে গিয়ে module-এর নাম না লেখা (`add(2,3)` → `NameError`)।

অনুশীলনের জন্য

1. `geometry.py` module বানান যাতে circle/square/triangle-এর area function থাকে।

2. `student.py` module বানান যাতে grade ঠিক করার function থাকে।

3. `converter.py` module বানান যা Celsius-কে Fahrenheit-এ রূপান্তর করে।

সমস্যা 207: Python Package তৈরি করুন ও ব্যবহার করুন।

সমস্যা

সিলেবাসের 3.7 অনুযায়ী — `__init__.py` ফাইল ব্যবহার করে একটি package তৈরি করুন, তার ভেতরে কয়েকটি module রাখুন এবং সেখান থেকে import করুন।

কী কী জানা থাকতে হবে?

- Module

- Folder concept
- import

কীভাবে চিন্তা করব?

Package হলো এমন একটি ফোল্ডার যার ভেতরে একাধিক module (.py ফাইল) এবং একটি বিশেষ `__init__.py` ফাইল থাকে। `__init__.py` ফাইলটি Python-কে বুঝিয়ে দেয় যে এই ফোল্ডারটি একটি package। বড় প্রজেক্টে কোড গুছিয়ে রাখতে package ব্যবহার করা হয়।

> **[Note]** Module = একটি .py ফাইল। Package = একাধিক module-এর ফোল্ডার (ভেতরে `__init__.py` সহ)। Library = অনেকগুলো package-এর সমষ্টি।

অ্যালগরিদম

Step 1: `myapp` নামে একটি ফোল্ডার তৈরি করা।

Step 2: ফোল্ডারের ভেতরে খালি `__init__.py` ফাইল রাখা।

Step 3: একই ফোল্ডারে `math_tools.py` ও `string_tools.py` module রাখা।

Step 4: `main.py` থেকে `from myapp import math_tools` করে ব্যবহার করা।

Python Code

```
# ============================================================
# Folder structure of our package:
#
#   project/
#   |-- main.py
#   |-- myapp/
#       |-- __init__.py        (can be empty)
#       |-- math_tools.py
#       |-- string_tools.py
# ============================================================

# ---------- FILE: myapp/__init__.py ----------
print("myapp package is being imported")

# ---------- FILE: myapp/math_tools.py ----------
def square(n):
    return n * n


def cube(n):
    return n * n * n
```

```
# ---------- FILE: myapp/string_tools.py ----------
def to_upper(text):
    return text.upper()



def word_count(text):
    return len(text.split())



# ---------- FILE: main.py   (run this) ----------
from myapp import math_tools, string_tools

print("Square of 6:", math_tools.square(6))
print("Cube of 3  :", math_tools.cube(3))

print("Upper case :", string_tools.to_upper("hello python"))
print("Word count :", string_tools.word_count("Learn Python step by step"))

# Another way: import a single function from a module inside package
from myapp.math_tools import square

print("Direct import ->", square(9))
```

Built-in Function / Shortcut Method

```
# You can also expose things directly in __init__.py:
# myapp/__init__.py
#   from .math_tools import square, cube
#   from .string_tools import to_upper
#
# Then in main.py:
#   from myapp import square, to_upper
```

Sample Input / Output Table

Sample Input	Sample Output
None	myapp package is being imported Square of 6: 36 Cube of 3 : 27 Upper case : HELLO PYTHON Word count : 5 Direct import -> 81

ব্যাখ্যা

`myapp` ফোল্ডারের ভেতরে `__init__.py` থাকায় Python একে package হিসেবে চিনেছে। `from myapp import math_tools` লিখে ভেতরের module ব্যবহার করা গেছে। `__init__.py` import-এর সময় একবার রান হয় — তাই সেখানে initialization কোড রাখা হয়।

Common Mistakes

- `__init__.py` ফাইল না রাখা (Python 3.3+ এ namespace package হলেও, ভালো প্র্যাকটিস হলো রাখা)।
- `main.py`-কে package-এর ভেতরে রেখে রান করা — main ফাইল সবসময় package-এর বাইরে রাখুন।
- package import করতে গিয়ে `import myapp.math_tools.math` লেখা (সঠিক: `from myapp import math_tools`)।

অনুশীলনের জন্য

1. `school` package বানান যাতে `student.py` ও `teacher.py` module থাকবে।

2. `__init__.py`-তে from-import দিয়ে সরাসরি function expose করুন।

3. `utils` package বানিয়ে তার ভেতরে `file_utils.py` module রাখুন।

সমস্যা 208: Application Software-এর সংজ্ঞা, প্রকারভেদ ও ব্যবহার বুঝুন।

সমস্যা

সিলেবাসের 3.1, 3.2, 3.3 অনুযায়ী — Application Software কী, এটি কত প্রকার, এবং এর ব্যবহার কী — তা সারণি আকারে উপস্থাপন করুন।

কী কী জানা থাকতে হবে?

- Software concept
- Module/Package concept

কীভাবে চিন্তা করব?

Application Software হলো সেই সব সফটওয়্যার যা ব্যবহারকারীর নির্দিষ্ট কাজ সম্পন্ন করতে বানানো হয় — যেমন MS Word, Browser, Student ERP। সিলেবাসের Practical-এ আমরা যে Inventory, PDF Report, QR Code বানাবো — সেগুলো-ই Application Software।

> **[Important]** পরীক্ষায় Unit ১১ থেকে সাধারণত 'Application Software-এর প্রকারভেদ ও ব্যবহার' — সংজ্ঞা + ২/৩টি উদাহরণসহ লিখতে হয়। অবশ্যই বাংলাদেশি প্রেক্ষাপটে (যেমন স্কুল ERP) উদাহরণ দিন।

অ্যালগরিদম

Step 1: Application Software-এর সংজ্ঞা লেখা।

Step 2: প্রকারভেদ (Ready-made vs Customized, সেক্টরভিত্তিক) সারণিতে দেখানো।

Step 3: ব্যবহারের ক্ষেত্রগুলো তালিকা করা।

Step 4: পাইথন দিয়ে বানানো যায় এমন অ্যাপের উদাহরণ দেওয়া।

Python Code

```
# Application Software - classification stored in a dictionary
# and displayed as a report (a tiny console "application")

applications = {
    "Word Processing": ["MS Word", "Google Docs", "LibreOffice Writer"],
    "Spreadsheet": ["MS Excel", "Google Sheets"],
    "Database": ["MySQL Workbench", "MS Access", "SQLite Browser"],
    "Accounting": ["Tally", "QuickBooks"],
    "Inventory Control": ["Custom Python App", "Odoo"],
    "Communication": ["WhatsApp", "Zoom", "Gmail"],
}

print("=" * 55)
print("        APPLICATION SOFTWARE - CATEGORY REPORT")
print("=" * 55)

total_categories = 0
for category, software_list in applications.items():
    total_categories = total_categories + 1
    print("\nCategory:", category)
    for software in software_list:
        print("   -", software)

print("\n" + "=" * 55)
print("Total categories:", total_categories)

# A small example of a 'custom' application we can build with Python
print("\n--- Example: Custom Student Management (Python) ---")
students = []
students.append({"id": 1, "name": "Rahim", "department": "CSE"})
students.append({"id": 2, "name": "Karim", "department": "EEE"})

for s in students:
    print("ID:", s["id"], "| Name:", s["name"], "| Dept:", s["department"])
```

Built-in Function / Shortcut Method

```
# Types of Application Software at a glance:
# 1. Ready-made / General purpose : MS Word, Excel, Chrome
# 2. Customized / Tailor-made     : Hospital ERP, School ERP
# 3. Utility software             : Antivirus, Disk Cleaner
```

Sample Input / Output Table

Sample Input	Sample Output
None	========== ========== APPLICATION SOFTWARE - CATEGORY REPORT ========== ========== Category: Word Processing - MS Word - Google Docs - LibreOffice Writer Category: Spreadsheet - MS Excel - Google Sheets Category: Database - MySQL Workbench - MS Access - SQLite Browser Category: Accounting - Tally - QuickBooks Category: Inventory Control - Custom Python App - Odoo Category: Communication - WhatsApp - Zoom - Gmail

```
                                ================================
                                ====================
                                Total categories: 6

                                --- Example: Custom Student
                                Management (Python) ---
                                ID: 1 | Name: Rahim | Dept: CSE
                                ID: 2 | Name: Karim | Dept: EEE
```

ব্যাখ্যা

Application Software হলো এন্ড-ইউজারের কাজ সম্পন্ন করার সফটওয়্যার। সিলেবাসের মূল লক্ষ্য হলো Python দিয়ে এমন Customized Application Software (Inventory, PDF Report, QR, Weather App, Database App) বানানো।

Common Mistakes

- System Software (Windows, Linux) ও Application Software (MS Word, Python App) গুলিয়ে ফেলা।
- ভাবা যে Application Software মানেই বড় বাণিজ্যিক সফটওয়্যার — ছোট Python স্ক্রিপ্টও একটি application।
- সিলেবাসের ১১ নম্বর ইউনিটে শুধু সংজ্ঞা লিখে উদাহরণ না দেওয়া — পরীক্ষায় উদাহরণসহ লিখলে বেশি নম্বর পাওয়া যায়।

অনুশীলনের জন্য

1. ৫টি Application Software-এর নাম ও কাজ লিখুন।

2. System Software ও Application Software-এর পার্থক্য সারণি আকারে লিখুন।

3. আপনার পরিচিত একটি Customized Application Software-এর বর্ণনা দিন।

Topic: File Operation in Python (CSV, JSON & Advanced) — Unit 2

সমস্যা 209: File Opening Mode গুলো বুঝুন ও ব্যবহার করুন।

সমস্যা

সিলেবাসের 2.2 অনুযায়ী — Python-এর বিভিন্ন file opening mode (`r`, `w`, `a`, `r+`, `w+`, `a+`, `x`, `b`) বুঝে একটি প্রোগ্রামে প্রয়োগ করুন।

কী কী জানা থাকতে হবে?

- open() function

- File read/write
- with statement

কীভাবে চিন্তা করব?

ফাইল খোলার সময় দ্বিতীয় argument-এ mode দিতে হয়। `r` = পড়া, `w` = লেখা (পুরনো মুছে যায়), `a` = শেষে যোগ করা, `x` = নতুন ফাইল (থাকলে error), `b` = binary, `+` = পড়া+লেখা। ভুল mode ব্যবহার করলে ডেটা হারিয়ে যেতে পারে।

> **[Important]** সবচেয়ে বড় বিপদ: `'w'` mode। ইনভেন্টরি বা স্টুডেন্ট ডেটা `'w'` দিয়ে খুললে আগের সব তথ্য মুছে যাবে। ডেটা যোগ করতে সবসময় `'a'` বা `'a+'` ব্যবহার করুন।

অ্যালগরিদম

Step 1: 'w' mode দিয়ে একটি ফাইল তৈরি করে লেখা।

Step 2: 'a' mode দিয়ে শেষে নতুন লাইন যোগ করা।

Step 3: 'r' mode দিয়ে পুরো ফাইল পড়া।

Step 4: 'r+' mode দিয়ে পড়া ও লেখা একসাথে করা।

Python Code

```
# Understanding different file opening modes

# 1. 'w' mode -> write (creates file, deletes old content)
with open("demo.txt", "w") as file:
    file.write("Line 1: Hello Python\n")
    file.write("Line 2: File handling is easy\n")
print("Written using 'w' mode")

# 2. 'a' mode -> append (adds at the end, keeps old content)
with open("demo.txt", "a") as file:
    file.write("Line 3: This line was appended\n")
print("Appended using 'a' mode")

# 3. 'r' mode -> read
with open("demo.txt", "r") as file:
    content = file.read()
print("\n--- Content after 'r' mode ---")
print(content)

# 4. 'r+' mode -> read and write both
with open("demo.txt", "r+") as file:
    old_data = file.read()
    file.write("Line 4: Added by r+ mode\n")
print("--- Content after 'r+' mode ---")
```

```
with open("demo.txt", "r") as file:
    print(file.read())

# 5. Checking file mode properties
with open("demo.txt", "r") as file:
    print("File name   :", file.name)
    print("File mode   :", file.mode)
    print("Closed?     :", file.closed)
print("Closed after with-block? :", file.closed)
```

Built-in Function / Shortcut Method

```
# Mode summary table
# 'r'  -> read only        (file must exist)
# 'w'  -> write            (creates / truncates)
# 'a'  -> append           (keeps old, adds at end)
# 'x'  -> exclusive create (error if exists)
# 'r+' -> read + write
# 'w+' -> write + read     (truncates)
# 'a+' -> append + read
# 'rb' / 'wb' -> binary mode (image, pdf, audio)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Written using 'w' mode Appended using 'a' mode --- Content after 'r' mode --- Line 1: Hello Python Line 2: File handling is easy Line 3: This line was appended --- Content after 'r+' mode --- Line 1: Hello Python Line 2: File handling is easy Line 3: This line was appended Line 4: Added by r+ mode File name : demo.txt File mode : r Closed? : False Closed after with-block? : True

ব্যাখ্যা

'w' mode পুরনো কনটেন্ট মুছে দেয়, তাই সাবধানে ব্যবহার করতে হয়। 'a' mode শেষে যোগ করে। `with` ব্লক শেষ হলে ফাইল স্বয়ংক্রিয়ভাবে বন্ধ হয় — `file.closed` True হয়ে গেছে।

Common Mistakes

- 'r' mode-এ না থাকা ফাইল খোলা (`FileNotFoundError`)।
- 'w' mode ব্যবহার করে অসাবধানে পুরনো ডেটা মুছে ফেলা।
- `file.close()` না দেওয়া (with ব্লক ব্যবহার করলে এই সমস্যা হয় না)।
- mode-এর সাথে `+` ব্যবহার করে ভাবা ফাইল insert হবে — আসলে কার্সারের জায়গায় overwrite হয়।

অনুশীলনের জন্য

1. 'x' mode দিয়ে ফাইল তৈরি করুন এবং আগে থাকলে error দেখুন।

2. 'w+' mode ব্যবহার করে লিখে ও পড়ে দেখান।

3. 'rb' mode দিয়ে একটি ছবির ফাইল পড়ুন।

সমস্যা 210: readline() এবং readlines() দিয়ে লাইন বাই লাইন পড়ুন।

সমস্যা

সিলেবাসের 2.4 অনুযায়ী — `readline()` ও `readlines()` ব্যবহার করে ফাইলের লেখা লাইন বাই লাইন পড়ে লাইন নম্বরসহ প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- File read
- for loop
- List

কীভাবে চিন্তা করব?

`read()` পুরো ফাইল একসাথে পড়ে। `readline()` প্রতিবার এক লাইন পড়ে, আর `readlines()` সব লাইনকে একটি list-এ রাখে। বড় ফাইলে for লুপ দিয়ে সরাসরি iterate করাই সবচেয়ে memory-efficient উপায়।

অ্যালগরিদম

Step 1: একটি ফাইল তৈরি করে কয়েকটি লাইন লেখা।

Step 2: `readline()` দিয়ে প্রথম লাইন পড়া।

Step 3: `readlines()` দিয়ে সব লাইন list আকারে পড়া।

Step 4: for লুপ দিয়ে লাইন নম্বরসহ প্রিন্ট করা।

Python Code

```
# Reading a file line by line
```

```
# Step 1: create a sample file first
with open("notes.txt", "w") as file:
    file.write("Python is easy\n")
    file.write("File handling is important\n")
    file.write("CSV and JSON are useful\n")

# Step 2: read only the first line using readline()
with open("notes.txt", "r") as file:
    first_line = file.readline()
print("First line:", first_line)

# Step 3: read all lines into a list using readlines()
with open("notes.txt", "r") as file:
    all_lines = file.readlines()
print("\nAll lines as a list:", all_lines)

# Step 4: print with line numbers
print("\n--- File content with line numbers ---")
with open("notes.txt", "r") as file:
    lines = file.readlines()

line_number = 1
for line in lines:
    print(str(line_number) + ". " + line.rstrip())
    line_number = line_number + 1

# Step 5: best way - iterate directly (memory friendly)
print("\n--- Iterating directly ---")
count = 0
with open("notes.txt", "r") as file:
    for line in file:
        count = count + 1
        print("Line", count, ":", line.rstrip())
```

Built-in Function / Shortcut Method

```
# Remove the trailing newline neatly
# line.rstrip()      -> removes spaces & newline from right
# line.strip()       -> removes from both sides
# line.rstrip("\n")  -> removes only the newline character
```

Sample Input / Output Table

Sample Input	Sample Output

```
None                                    First line: Python is easy

                                        All lines as a list: ['Python is
                                        easy\n', 'File handling is
                                        important\n', 'CSV and JSON are
                                        useful\n']

                                        --- File content with line
                                        numbers ---
                                        1. Python is easy
                                        2. File handling is important
                                        3. CSV and JSON are useful

                                        --- Iterating directly ---
                                        Line 1 : Python is easy
                                        Line 2 : File handling is
                                        important
                                        Line 3 : CSV and JSON are useful
```

ব্যাখ্যা

`readlines()` প্রতিটি লাইনের শেষে `\n` (newline) রেখেই দেয়, তাই প্রিন্ট করার সময় `rstrip()` ব্যবহার করা হয়। সবচেয়ে ভালো পদ্ধতি হলো for লুপে সরাসরি file object iterate করা — এতে পুরো ফাইল মেমোরিতে লোড হয় না।

Common Mistakes

- `readlines()`-এর লাইনে `\n` থাকবে না ভেবে string তুলনা করা (হলে `rstrip()` লাগবে)।
- `readline()` (singular) ও `readlines()` (plural) গুলিয়ে ফেলা।
- লুপের ভিতরে বারবার `file.readlines()` কল করা — দ্বিতীয়বার থেকে ফাঁকা list পাওয়া যায়।

অনুশীলনের জন্য

1. একটি ফাইলে মোট কতটি লাইন আছে তা গোনার প্রোগ্রাম লিখুন।

2. ফাইলের শুধু জোড় নম্বরের লাইনগুলো প্রিন্ট করুন।

3. ফাইলের প্রতিটি লাইনের শব্দ সংখ্যা গুনুন।

সমস্যা 211: seek() এবং tell() দিয়ে File Pointer নিয়ন্ত্রণ করুন।

সমস্যা

`tell()` দিয়ে বর্তমান file pointer-এর অবস্থান জানুন এবং `seek()` দিয়ে pointer-কে নির্দিষ্ট স্থানে সরিয়ে পড়ুন/লিখুন।

কী কী জানা থাকতে হবে?

- File read/write
- File pointer concept

কীভাবে চিন্তা করব?

ফাইল পড়ার সময় একটি invisible 'কার্সার' বা pointer থাকে যা কোথা থেকে পড়া/লেখা হবে তা নির্ধারণ করে। `tell()` pointer-এর বর্তমান অবস্থান (byte-এ) দেয়, আর `seek(position)` pointer-কে সেই অবস্থানে নিয়ে যায়।

অ্যালগরিদম

Step 1: একটি ফাইল তৈরি করে লেখা।

Step 2: `tell()` দিয়ে pointer-এর অবস্থান দেখা।

Step 3: `seek(0)` দিয়ে শুরুতে ফিরে যাওয়া।

Step 4: `seek(10)` দিয়ে ১০ নম্বর byte থেকে পড়া।

Python Code

```
# Using seek() and tell() to control the file pointer

with open("pointer.txt", "w") as file:
    file.write("ABCDEFGHIJKLMNOPQRSTUVWXYZ")

with open("pointer.txt", "r") as file:
    print("Initial position :", file.tell())

    data = file.read(5)
    print("Read 5 chars     :", data)
    print("Position now     :", file.tell())

    data = file.read(5)
    print("Next 5 chars     :", data)
    print("Position now     :", file.tell())

    # Move pointer back to the beginning
    file.seek(0)
    print("\nAfter seek(0)    :", file.tell())
    print("Full content     :", file.read())
```

```
    # Move to the 10th byte and read from there
    file.seek(10)
    print("\nAfter seek(10)   :", file.read(5))

    # Go to the end of the file
    file.seek(0, 2)
    print("End position     :", file.tell())
```

Built-in Function / Shortcut Method

```
# seek(offset, from_what)
# from_what: 0 = beginning (default), 1 = current position, 2 = end of file
# file.seek(0)    -> go to start
# file.seek(0, 2) -> go to end
```

Sample Input / Output Table

Sample Input	Sample Output
None	Initial position : 0 Read 5 chars : ABCDE Position now : 5 Next 5 chars : FGHIJ Position now : 10 After seek(0) : 0 Full content : ABCDEFGHIJKLMNOPQRSTUVWXYZ After seek(10) : KLMNO End position : 26

ব্যাখ্যা

ফাইল খোলার সময় pointer 0-এ থাকে। ৫টি অক্ষর পড়ার পর pointer 5-এ চলে যায়। `seek(0)` দিয়ে আবার শুরুতে ফেরা যায়। `seek(0, 2)` শেষে নিয়ে যায় — ফাইলের মোট আকার জানা যায়।

Common Mistakes

- `seek()`-এ negative offset দেওয়া (text mode-এ অনুমোদিত নয়)।
- `tell()`-এর মানকে character সংখ্যা ভাবা — আসলে এটি byte position (Bangla/ইউনিকোড টেক্সটে আলাদা হতে পারে)।
- 'a' mode-এ `seek()` করেও লেখা যায় না (সবসময় শেষে যোগ হয়)।

অনুশীলনের জন্য

1. ফাইলের মাঝ থেকে শেষ পর্যন্ত অংশ পড়ার প্রোগ্রাম লিখুন।

2. `seek()` ব্যবহার করে ফাইলের শেষ ১০টি অক্ষর পড়ুন।

3. ফাইলের মোট byte আকার `tell()` দিয়ে বের করুন।

সমস্যা 212: CSV File-এ ডেটা লিখুন (csv.writer)।

সমস্যা

Practical-1 (Inventory Management) এর প্রস্তুতি হিসেবে `csv` module-এর `writer` ব্যবহার করে একটি CSV ফাইলে সারি (row) আকারে ডেটা লেখার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- import csv
- File open in 'w' mode
- List

কীভাবে চিন্তা করব?

CSV মানে Comma Separated Values — এক ধরনের টেবিল-জাতীয় টেক্সট ফাইল। `csv.writer(file)` একটি writer object দেয়, যার `writerow()` দিয়ে এক সারি এবং `writerows()` দিয়ে একাধিক সারি লেখা যায়। `newline=''` দেওয়া বাধ্যতামূলক, নাহলে ফাঁকা লাইন তৈরি হয়।

> **[Best Practice]** CSV ফাইল খোলার সময় সবসময় `newline=""` ব্যবহার করুন — এটি Python-এর অফিশিয়াল ডকুমেন্টেশনে বাধ্যতামূলক বলা হয়েছে।

অ্যালগরিদম

Step 1: `import csv` করা।

Step 2: `open('items.csv', 'w', newline='')` দিয়ে ফাইল খোলা।

Step 3: `csv.writer(file)` তৈরি করা।

Step 4: `writerow()` দিয়ে header লেখা।

Step 5: `writerows()` দিয়ে সব ডেটা লেখা।

Python Code

```
# Writing data into a CSV file using csv.writer
import csv

# Open the file in write mode (newline='' prevents blank lines)
```

```
with open("items.csv", "w", newline="") as file:
    writer = csv.writer(file)

    # Write the header row
    writer.writerow(["Item Name", "Unit Price", "Quantity", "Total Price"])

    # Write multiple data rows
    writer.writerows([
        ["Mango", 120, 5, 600],
        ["Apple", 250, 3, 750],
        ["Banana", 40, 12, 480],
        ["Orange", 180, 4, 720],
    ])

print("Data written to items.csv successfully!")

# Verify by reading the raw text
print("\n--- Raw CSV content ---")
with open("items.csv", "r") as file:
    print(file.read())
```

Built-in Function / Shortcut Method

```
# Writing a single row at a time
# with open("items.csv", "a", newline="") as file:
#     writer = csv.writer(file)
#     writer.writerow(["Grapes", 300, 2, 600])
```

Sample Input / Output Table

Sample Input	Sample Output
None	Data written to items.csv successfully! --- Raw CSV content --- Item Name,Unit Price,Quantity,Total Price Mango,120,5,600 Apple,250,3,750 Banana,40,12,480 Orange,180,4,720

ব্যাখ্যা

প্রতিটি row CSV ফাইলে কমা দিয়ে আলাদা হয়ে সেভ হয়েছে। `newline=''` না দিলে Windows-এ প্রতি সারির মাঝে একটি ফাঁকা লাইন যোগ হয়ে যায় — এটি সবচেয়ে common সমস্যা।

Common Mistakes

- `newline=''` না দেওয়া (ফাঁকা লাইন তৈরি হয়)।
- CSV ফাইল Excel-এ খুলে save করে দেওয়া (ফরম্যাট নষ্ট হয়)।
- `writerow()`-এ list-এর বদলে আলাদা করে value দেওয়া — অবশ্যই একটিমাত্র list/argument দিতে হবে।

অনুশীলনের জন্য

1. ইউজারের কাছ থেকে ৩টি ফলের নাম, দাম ও পরিমাণ নিয়ে CSV-তে লিখুন।

2. 'a' mode ব্যবহার করে CSV-তে নতুন row যোগ করুন।

3. ১০ জন শিক্ষার্থীর নাম ও রোল CSV-তে লিখুন।

সমস্যা 213: CSV File থেকে ডেটা পড়ুন (csv.reader)।

সমস্যা

`csv.reader` ব্যবহার করে CSV ফাইলের সব সারি পড়ুন, header আলাদা করুন এবং মোট যোগফল হিসাব করুন।

কী কী জানা থাকতে হবে?

- csv.writer
- for loop
- List
- int() conversion

কীভাবে চিন্তা করব?

`csv.reader(file)` প্রতিটি সারিকে একটি list হিসেবে দেয়। প্রথম সারিটি সাধারণত header। `next(reader)` দিয়ে header বাদ দেওয়া যায়। CSV-এর সব মান string আকারে আসে, তাই হিসাব করতে `int()`/`float()` দিয়ে রূপান্তর করতে হয়।

অ্যালগরিদম

Step 1: `open('items.csv', 'r')` দিয়ে ফাইল খোলা।

Step 2: `csv.reader(file)` তৈরি করা।

Step 3: `next(reader)` দিয়ে header আলাদা করা।

Step 4: লুপ চালিয়ে প্রতিটি row পড়া ও total যোগ করা।

Python Code

```
# Reading data from a CSV file using csv.reader
import csv

with open("items.csv", "r") as file:
    reader = csv.reader(file)

    # Read and separate the header row
    header = next(reader)
    print("Header:", header)
    print("-" * 50)

    grand_total = 0
    row_count = 0

    for row in reader:
        name = row[0]
        unit_price = int(row[1])
        quantity = int(row[2])
        total_price = int(row[3])

        grand_total = grand_total + total_price
        row_count = row_count + 1

        print(name, "| Price:", unit_price, "| Qty:", quantity, "| Total:",
total_price)

    print("-" * 50)
    print("Number of items :", row_count)
    print("Grand total     :", grand_total)
```

Built-in Function / Shortcut Method

```
# Convert the whole CSV into a list of lists
# with open("items.csv", "r") as file:
#     rows = list(csv.reader(file))
# print(rows)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Header: ['Item Name', 'Unit Price', 'Quantity', 'Total Price']

```
                                    --------------------------------
                                    -----------------
                                    Mango | Price: 120 | Qty: 5 |
                                    Total: 600
                                    Apple | Price: 250 | Qty: 3 |
                                    Total: 750
                                    Banana | Price: 40 | Qty: 12 |
                                    Total: 480
                                    Orange | Price: 180 | Qty: 4 |
                                    Total: 720
                                    --------------------------------
                                    -----------------
                                    Number of items : 4
                                    Grand total     : 2550
```

ব্যাখ্যা

`next(reader)` প্রথম সারি (header) নিয়ে নেয়, ফলে লুপে শুধু ডেটা আসে। CSV-এর সব মান string, তাই `int(row[1])` দিয়ে সংখ্যায় রূপান্তর করা হয়েছে — না করলে যোগ করলে string concatenate হয়ে যাবে।

Common Mistakes

- CSV-এর string মানকে সংখ্যা ভেবে সরাসরি যোগ করা (`'120' + '5'` = `'1205'`)।
- `next(reader)` না দিয়ে header-কেও ডেটা হিসেবে গোনা।
- ভুল index ব্যবহার করা (`row[4]` → `IndexError`)।

অনুশীলনের জন্য

1. CSV থেকে শুধু সেই সব item প্রিন্ট করুন যাদের total 500-এর বেশি।

2. CSV পড়ে সবচেয়ে দামি item-এর নাম বের করুন।

3. CSV পড়ে একটি summary report টেক্সট ফাইলে লিখুন।

সমস্যা 214: DictWriter ব্যবহার করে Header সহ CSV লিখুন।

সমস্যা

সিলেবাসের Practical-2 অনুযায়ী — `csv.DictWriter` ব্যবহার করে dictionary আকারে শিক্ষার্থীর তথ্য (Student_Id, Name, Roll, Semester, Shift, Department, Date of birth, Phone no.) CSV ফাইলে লিখুন।

কী কী জানা থাকতে হবে?

- Dictionary

- csv module
- File write

কীভাবে চিন্তা করব?

`DictWriter` dictionary ব্যবহার করে CSV লেখে — তাই index মনে রাখতে হয় না, key দিয়েই কাজ হয়। `fieldnames` দিয়ে কলামের নাম ঠিক করতে হয় এবং `writeheader()` দিয়ে header লিখতে হয়। এটি Practical-2-এর মূল কাজ।

> **[Important]** Practical-2-এ CSV-তে header সহ লেখা বাধ্যতামূলক। `DictWriter` + `writeheader()` এই কাজটি সবচেয়ে সহজ করে দেয়।

অ্যালগরিদম

Step 1: fieldnames list-এ ৮টি কলামের নাম রাখা।

Step 2: `csv.DictWriter(file, fieldnames=fieldnames)` তৈরি করা।

Step 3: `writeheader()` দিয়ে header লেখা।

Step 4: ইউজারের কাছ থেকে তথ্য নিয়ে dictionary বানিয়ে `writerow()` দিয়ে লেখা।

Python Code

```
# Writing student data into CSV using DictWriter
import csv

fields = ["Student_Id", "Name", "Roll", "Semester", "Shift",
          "Department", "Date_of_Birth", "Phone_no"]

# Take input from the user
print("Enter student information:")
student = {
    "Student_Id": input("Student ID: "),
    "Name": input("Name: "),
    "Roll": input("Roll: "),
    "Semester": input("Semester: "),
    "Shift": input("Shift: "),
    "Department": input("Department: "),
    "Date_of_Birth": input("Date of Birth (DD-MM-YYYY): "),
    "Phone_no": input("Phone number: ")
}

# Write into CSV using 'a' mode so previous records are kept
with open("students_info.csv", "a", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=fields)

    # Write header only if the file is empty
```

```
    file.seek(0, 2)
    if file.tell() == 0:
        writer.writeheader()

    writer.writerow(student)

print("\nStudent data saved to students_info.csv successfully!")
```

Built-in Function / Shortcut Method

```
# Writing multiple students at once
# writer.writerows([student1, student2, student3])
```

Sample Input / Output Table

Sample Input	Sample Output
2025-CSE-01 Rahim Ahmed 101 3rd 1st CSE 15-03-2004 01712345678	Enter student information: Student ID: 2025-CSE-01 Name: Rahim Ahmed Roll: 101 Semester: 3rd Shift: 1st Department: CSE Date of Birth (DD-MM-YYYY): 15-03-2004 Phone number: 01712345678 Student data saved to students_info.csv successfully!

ব্যাখ্যা

`DictWriter` dictionary-এর key-কে fieldnames-এর সাথে মিলিয়ে সঠিক কলামে মান বসিয়ে দেয়। এখানে 'a' mode ব্যবহার করা হয়েছে যাতে বারবার রান করলে আগের রেকর্ড না মুছে যায়।

Common Mistakes

- `writeheader()` না দেওয়া — তখন CSV-তে কলামের নাম থাকবে না।
- dictionary-র key ও fieldnames না মিললে `ValueError` হয়।
- বারবার রান করলে header বারবার লেখা হওয়া (তাই `file.tell() == 0` চেক করা হয়েছে)।

অনুশীলনের জন্য

1. ৫ জন শিক্ষার্থীর তথ্য একসাথে `writerows()` দিয়ে লিখুন।

2. ইনভেন্টরি item DictWriter দিয়ে CSV-তে লিখুন।

3. CSV-তে লেখার আগে validate করুন phone number ১১ ডিজিট কিনা।

সমস্যা 215: DictReader ব্যবহার করে CSV পড়ুন ও রিপোর্ট তৈরি করুন।

সমস্যা

`csv.DictReader` ব্যবহার করে students_info.csv পড়ুন এবং সুন্দর টেবিল আকারে সামারি রিপোর্ট প্রদর্শন করুন।

কী কী জানা থাকতে হবে?

- DictWriter
- Dictionary
- String formatting

কীভাবে চিন্তা করব?

`DictReader` প্রতিটি সারিকে dictionary হিসেবে দেয় যেখানে key হলো header-এর কলাম নাম। তাই `row['Name']` দিয়ে সরাসরি নাম পাওয়া যায় — index মনে রাখার ঝামেলা নেই।

অ্যালগরিদম

Step 1: `csv.DictReader(file)` তৈরি করা।

Step 2: লুপ চালিয়ে প্রতিটি row-কে dictionary হিসেবে পড়া।

Step 3: নির্দিষ্ট কলাম বেছে নিয়ে টেবিল আকারে প্রিন্ট করা।

Step 4: মোট রেকর্ড সংখ্যা গোনা।

Python Code

```
# Reading CSV with DictReader and printing a summary report
import csv

try:
    with open("students_info.csv", "r") as file:
        reader = csv.DictReader(file)

        print("=" * 70)
        print("                    STUDENT INFORMATION REPORT")
        print("=" * 70)
        print("ID           Name           Roll   Dept   Shift   Semester")
        print("-" * 70)
```

```
        count = 0
        for row in reader:
            count = count + 1
            print(row["Student_Id"].ljust(14),
                  row["Name"].ljust(16),
                  row["Roll"].ljust(6),
                  row["Department"].ljust(6),
                  row["Shift"].ljust(7),
                  row["Semester"])

        print("-" * 70)
        print("Total students:", count)
        print("=" * 70)

except FileNotFoundError:
    print("Error: students_info.csv file not found. Please add data first.")
```

Built-in Function / Shortcut Method

```
# Convert CSV into a list of dictionaries
# with open("students_info.csv", "r") as file:
#     data = list(csv.DictReader(file))
# print(data[0]["Name"])
```

Sample Input / Output Table

Sample Input	Sample Output
None	========== ========== ==== STUDENT INFORMATION REPORT ========== ========== ==== ID Name Roll Dept Shift Semester ---------- ---------- ---- 2025-CSE-01 Rahim Ahmed 101 CSE 1st 3rd 2025-EEE-02 Karim Hossain 102

```
EEE    2nd     3rd
2025-CIV-03   Sadia Islam     103
CIV    1st     5th
--------------------------------
--------------------------------
----
Total students: 3
================================
================================
====
```

ব্যাখ্যা

`DictReader` header-কে key হিসেবে ব্যবহার করে, তাই `row['Name']` দিয়ে সরাসরি মান পাওয়া গেছে। `ljust()` দিয়ে কলামগুলো সারিবদ্ধ (aligned) করা হয়েছে। `try-except` দিয়ে ফাইল না থাকার সমস্যা হ্যান্ডেল করা হয়েছে।

Common Mistakes

- CSV-তে header না থাকলে `DictReader` প্রথম ডেটা সারিকেই header ধরে নেয়।
- key-এর বানান ভুল করা (`row['name']` vs `row['Name']` → `KeyError`)।
- `try-except` না দিয়ে ফাইল না থাকলে প্রোগ্রাম ক্র্যাশ করা।

অনুশীলনের জন্য

1. CSV থেকে শুধু CSE বিভাগের শিক্ষার্থীদের তালিকা প্রিন্ট করুন।

2. CSV পড়ে প্রতিটি বিভাগে কতজন শিক্ষার্থী আছে তা গুনুন।

3. CSV-এর তথ্য থেকে প্রতিটি শিক্ষার্থীর জন্য একটি ID card তথ্য প্রিন্ট করুন।

সমস্যা 216: JSON File-এ ডেটা সংরক্ষণ করুন (json.dump)।

সমস্যা

`json` module ব্যবহার করে একটি Python dictionary/list-কে JSON ফাইলে সংরক্ষণ করুন (Practical-5 Weather Report-এর ডেটা সেভ করার মতো কাজ)।

কী কী জানা থাকতে হবে?

- Dictionary
- List
- File write
- json module

কীভাবে চিন্তা করব?

JSON (JavaScript Object Notation) হলো ডেটা আদান-প্রদানের আন্তর্জাতিক ফরম্যাট — API থেকে ডেটা সবসময় JSON-এ আসে। `json.dump(data, file)` Python object-কে JSON ফাইলে লেখে, আর `json.dumps(data)` string আকারে দেয়।

> **[Note]** মনে রাখার সহজ নিয়ম: `dump()` = file-এ লেখে, `dumps()` = s মানে string। ঠিক তেমনি `load()` = file থেকে পড়ে, `loads()` = string থেকে পড়ে।

অ্যালগরিদম

Step 1: `import json` করা।

Step 2: একটি nested dictionary তৈরি করা (student records)।

Step 3: `json.dump(data, file, indent=4)` দিয়ে সুন্দর ফরম্যাটে সেভ করা।

Step 4: `json.dumps()` দিয়ে string আকারে দেখা।

Python Code

```
# Saving Python data into a JSON file
import json

# A Python dictionary (this is what we will save)
student_data = {
    "institute": "Bangladesh Polytechnic",
    "department": "Computer Science",
    "students": [
        {"id": 1, "name": "Rahim", "cgpa": 3.75},
        {"id": 2, "name": "Karim", "cgpa": 3.40},
        {"id": 3, "name": "Sadia", "cgpa": 3.95}
    ]
}

# 1. Write into a JSON file (indent=4 makes it readable)
with open("data.json", "w") as file:
    json.dump(student_data, file, indent=4)

print("Data saved to data.json")

# 2. Convert to a JSON formatted string (for API / printing)
json_string = json.dumps(student_data, indent=2)
print("\n--- JSON as string ---")
print(json_string)

# 3. Check Python vs JSON data types
print("\nType of Python object :", type(student_data))
```

```
print("Type of JSON string   :", type(json_string))
```

Built-in Function / Shortcut Method

```
# Quick one-liner save
# json.dump(data, open("data.json", "w"), indent=4)
```

Sample Input / Output Table

Sample Input	Sample Output

None	Data saved to data.json
	
	--- JSON as string ---
	{
	  "institute": "Bangladesh Polytechnic",
	  "department": "Computer Science",
	  "students": [
	  {
	  "id": 1,
	  "name": "Rahim",
	  "cgpa": 3.75
	  },
	  {
	  "id": 2,
	  "name": "Karim",
	  "cgpa": 3.4
	  },
	  {
	  "id": 3,
	  "name": "Sadia",
	  "cgpa": 3.95
	  }
	  ]
	}
	
	Type of Python object : <class 'dict'>
	Type of JSON string : <class 'str'>

ব্যাখ্যা

`json.dump()` ফাইলে লেখে, `json.dumps()` string দেয় (শেষের 's' মানে string)। `indent=4` দিলে ফাইলটি indentation সহ সুন্দর পাঠযোগ্য হয় — রিপোর্ট ও ডিবাগিংয়ের জন্য জরুরি।

Common Mistakes

- `json.dump` ও `json.dumps` গুলিয়ে ফেলা (file vs string)।
- set বা custom object JSON-এ সরাসরি সেভ করার চেষ্টা করা (`TypeError: not JSON serializable`)।
- `indent` না দিয়ে এক লাইনে সব লেখা হওয়া — পড়া কঠিন হয়।

অনুশীলনের জন্য

1. একটি list of products JSON ফাইলে সেভ করুন।

2. Weather API-এর মতো একটি sample response JSON ফাইলে সেভ করুন।

3. ইউজারের কাছ থেকে ৩ জনের তথ্য নিয়ে JSON-এ সেভ করুন।

সমস্যা 217: JSON File থেকে ডেটা পড়ুন (json.load)।

সমস্যা

`json.load()` ব্যবহার করে JSON ফাইল পড়ুন এবং Python dictionary হিসেবে ব্যবহার করে রিপোর্ট তৈরি করুন।

কী কী জানা থাকতে হবে?

- json.dump
- Dictionary
- List
- for loop

কীভাবে চিন্তা করব?

`json.load(file)` JSON ফাইল পড়ে Python-এর dictionary/list-এ রূপান্তর করে। এরপর সাধারণ dictionary-র মতো `data['key']` দিয়ে ব্যবহার করা যায়। Practical-5 (Weather API)-এ এই পদ্ধতি ব্যবহার করা হয়।

অ্যালগরিদম

Step 1: `open('data.json', 'r')` দিয়ে ফাইল খোলা।

Step 2: `json.load(file)` দিয়ে dictionary-তে রূপান্তর করা।

Step 3: nested list-এ লুপ চালিয়ে প্রতিটি শিক্ষার্থীর তথ্য প্রিন্ট করা।

Step 4: গড় CGPA হিসাব করা।

Python Code

```
# Reading data from a JSON file
import json

try:
    with open("data.json", "r") as file:
        data = json.load(file)

    print("Institute :", data["institute"])
    print("Department:", data["department"])
    print("\n--- Student List ---")

    total_cgpa = 0
    for student in data["students"]:
        print("ID:", student["id"], "| Name:", student["name"], "| CGPA:",
student["cgpa"])
        total_cgpa = total_cgpa + student["cgpa"]

    count = len(data["students"])
    print("\nNumber of students:", count)
    print("Average CGPA:", round(total_cgpa / count, 2))

    print("\nType after loading:", type(data))
    print("Available keys:", list(data.keys()))

except FileNotFoundError:
    print("Error: data.json not found!")
except json.JSONDecodeError:
    print("Error: The file is not a valid JSON file!")
```

Built-in Function / Shortcut Method

```
# Reading JSON from a string (used with APIs)
# data = json.loads(response_text)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Institute : Bangladesh Polytechnic Department: Computer Science --- Student List --- ID: 1 | Name: Rahim | CGPA: 3.75

```
ID: 2 | Name: Karim | CGPA: 3.4
ID: 3 | Name: Sadia | CGPA: 3.95

Number of students: 3
Average CGPA: 3.7

Type after loading: <class
'dict'>
Available keys: ['institute',
'department', 'students']
```

ব্যাখ্যা

JSON ফাইল পড়ার পর এটি একটি সাধারণ Python dictionary হয়ে গেছে, তাই `data['students']` দিয়ে list পেয়ে লুপ চালানো গেছে। `json.JSONDecodeError` হ্যান্ডেল করায় ফাইলটি নষ্ট হলে প্রোগ্রাম ক্র্যাশ করবে না।

Common Mistakes

- `json.load()`-এর বদলে `json.loads()` ব্যবহার করা file object-এর সাথে (TypeError)।
- JSON-এর key string ছাড়া অন্য কিছু ভাবা — সব key সবসময় string।
- ফাঁকা বা অসম্পূর্ণ JSON ফাইল পড়তে গিয়ে `JSONDecodeError` না হ্যান্ডেল করা।

অনুশীলনের জন্য

1. JSON থেকে সবচেয়ে বেশি CGPA-র শিক্ষার্থীর নাম বের করুন।

2. JSON-এ নতুন একজন শিক্ষার্থী যোগ করে আবার সেভ করুন।

3. JSON ডেটা পড়ে একটি টেক্সট রিপোর্ট ফাইল তৈরি করুন।

সমস্যা 218: os Module দিয়ে File ও Folder পরিচালনা করুন।

সমস্যা

`os` module ব্যবহার করে ফাইল আছে কিনা যাচাই, ফাইল নাম পরিবর্তন (rename), মুছে ফেলা (remove) এবং ফোল্ডার তৈরি/তালিকা দেখার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- import os
- File handling
- try-except

কীভাবে চিন্তা করব?

`os` module অপারেটিং সিস্টেমের সাথে কাজ করে। `os.path.exists()` দিয়ে ফাইল আছে কিনা যাচাই করা হয়, `os.rename()` নাম বদলায়, `os.remove()` মুছে দেয়, `os.mkdir()` ফোল্ডার বানায়। ফাইল মুছে ফেলা বিপজ্জনক, তাই আগে অবশ্যই যাচাই করে নিতে হয়।

> **[Warning]** `os.remove()` কোনো confirm না চেয়েই ফাইল মুছে দেয় এবং Recycle Bin-এও যায় না। সবসময় `if os.path.exists()` দিয়ে যাচাই করার অভ্যাস করুন।

অ্যালগরিদম

Step 1: `import os` করা।

Step 2: `os.getcwd()` দিয়ে বর্তমান ফোল্ডার দেখা।

Step 3: `os.listdir()` দিয়ে ফাইল তালিকা দেখা।

Step 4: `os.path.exists()` দিয়ে যাচাই করা।

Step 5: `os.rename()` ও `os.remove()` ব্যবহার করা (try-except সহ)।

Python Code

```
# File and folder management using the os module
import os

# 1. Current working directory
print("Current folder:", os.getcwd())

# 2. List all files and folders
print("\nFiles and folders here:")
for item in os.listdir("."):
    print("  ", item)

# 3. Check whether a file exists
file_name = "demo.txt"
if os.path.exists(file_name):
    print("\n" + file_name, "exists. Size:", os.path.getsize(file_name),
"bytes")
else:
    print("\n" + file_name, "does not exist.")

# 4. Create a new folder (only if it does not exist)
folder_name = "reports"
if not os.path.exists(folder_name):
    os.mkdir(folder_name)
    print("Folder created:", folder_name)
else:
    print("Folder already exists:", folder_name)
```

```
# 5. Rename a file safely
try:
    if os.path.exists("notes.txt"):
        os.rename("notes.txt", "old_notes.txt")
        print("File renamed to old_notes.txt")
    else:
        print("notes.txt not found - nothing to rename.")
except PermissionError:
    print("Permission denied!")

# 6. Delete a file safely
try:
    if os.path.exists("old_notes.txt"):
        os.remove("old_notes.txt")
        print("File deleted: old_notes.txt")
except PermissionError:
    print("Cannot delete - file is open somewhere.")
```

Built-in Function / Shortcut Method

```
# Check if it is a file or a folder
# os.path.isfile("demo.txt")
# os.path.isdir("reports")
#
# Join paths safely (works on Windows + Linux)
# os.path.join("reports", "report.pdf")
```

Sample Input / Output Table

Sample Input	Sample Output
None	Current folder: /home/student/python_project Files and folders here: demo.txt notes.txt items.csv data.json demo.txt exists. Size: 78 bytes Folder created: reports File renamed to old_notes.txt

```
File deleted: old_notes.txt
```

ব্যাখ্যা

`os.path.exists()` দিয়ে আগে যাচাই না করে `os.remove()` চালালে ফাইল না থাকলে `FileNotFoundError` হবে। তাই সবসময় if-condition বা try-except ব্যবহার করতে হবে। `os.path.join()` ব্যবহার করলে Windows ও Linux দুটোতেই path ঠিকভাবে কাজ করে।

Common Mistakes

- `os.remove()` দিয়ে ফোল্ডার মুছতে যাওয়া (ফোল্ডারের জন্য `os.rmdir()` লাগে)।
- path-এ সরাসরি `'folder\\file.txt'` বা `'folder/file.txt'` লেখা (সঠিক: `os.path.join()`)।
- আগে যাচাই না করেই rename/remove করা।

অনুশীলনের জন্য

1. একটি ফোল্ডারের ভেতরে শুধু .csv ফাইলগুলোর নাম প্রিন্ট করুন।

2. নির্দিষ্ট ফোল্ডারে একটি backup ফোল্ডার তৈরি করুন।

3. সব .txt ফাইলের মোট আকার (byte-এ) হিসাব করুন।

Topic: Basics of OOP (Object Oriented Programming) — Unit 4

সমস্যা 219: Class ও Object তৈরি করুন।

সমস্যা

সিলেবাসের 4.4 অনুযায়ী — Class ও Object কী, তা বুঝে একটি সহজ class তৈরি করুন এবং সেখান থেকে object বানিয়ে ব্যবহার করুন।

কী কী জানা থাকতে হবে?

- Function
- Variable
- Dictionary (তুলনার জন্য)

কীভাবে চিন্তা করব?

Class হলো একটি নকশা (blueprint) বা ছাঁচ, আর Object হলো সেই নকশা থেকে তৈরি বাস্তব জিনিস। যেমন — 'Student' একটি class (নকশা), আর Rahim বা Karim হলো object (বাস্তব শিক্ষার্থী)। Class বানাতে `class` keyword ব্যবহার করা হয়।

> **[Note]** Class-এর নাম সবসময় Capitalized (প্রতি শব্দের প্রথম অক্ষর বড়) হয় — যেমন

> `StudentInfo`, `BankAccount`। এটি PEP-8 নিয়ম।

অ্যালগরিদম

Step 1: `class Student:` লিখে class তৈরি করা।

Step 2: class-এর ভেতরে একটি method (function) লেখা।

Step 3: `student1 = Student()` দিয়ে object তৈরি করা।

Step 4: `student1.method_name()` দিয়ে method কল করা।

Python Code

```
# Creating a class and an object

# 1. Define the class (blueprint)
class Student:
    # This is a method (a function inside a class)
    def display_message(self):
        print("This is a Student class.")



# 2. Create objects (real things from the blueprint)
student1 = Student()
student2 = Student()

# 3. Call the method using the object
student1.display_message()
student2.display_message()

# 4. Check types
print("\nType of student1:", type(student1))
print("Is student1 an object of Student?", isinstance(student1, Student))

# 5. Two objects are different even from the same class
print("\nstudent1 id:", id(student1))
print("student2 id:", id(student2))
print("Are both objects same?", student1 is student2)
```

Built-in Function / Shortcut Method

```
# Empty class (valid but useless)
# class Student:
#     pass
#
# student1 = Student()
```

Sample Input / Output Table

Sample Input	Sample Output
None	This is a Student class. This is a Student class. Type of student1: <class '__main__.Student'> Is student1 an object of Student? True student1 id: 140234567891234 student2 id: 140234567891456 Are both objects same? False

ব্যাখ্যা

`class Student:` দিয়ে একটি নকশা তৈরি করা হয়েছে। `Student()` লিখে (ব্র্যাকেটসহ) কল করলে একটি নতুন object তৈরি হয়। একই class থেকে যত খুশি তত object বানানো যায় এবং প্রতিটির নিজস্ব মেমোরি ঠিকানা (id) থাকে।

Common Mistakes

- `Student()` না লিখে `Student` লেখা — তখন object তৈরি হয় না, class নিজেই উল্লেখ করা হয়।
- class-এর ভেতরের function-এ `self` প্যারামিটার না দেওয়া।
- `class Student()` লেখার প্রয়োজন নেই (এটি function নয়), শুধু `class Student:` লিখলেই হয়।

অনুশীলনের জন্য

1. `Teacher` নামে একটি class বানিয়ে দুইটি object তৈরি করুন।

2. `Car` class বানিয়ে একটি method দিয়ে গাড়ির নাম প্রিন্ট করুন।

3. `Book` class বানিয়ে বইয়ের তথ্য প্রিন্ট করার method যোগ করুন।

সমস্যা 220: __init__ Constructor ব্যবহার করুন।

সমস্যা

সিলেবাসের 4.5 অনুযায়ী — class-এর গঠন বুঝে `__init__()` constructor ব্যবহার করে object তৈরির সময়ই ডেটা সেট করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- Function
- self parameter

কীভাবে চিন্তা করব?

`__init__()` একটি বিশেষ method যা object তৈরির সময় স্বয়ংক্রিয়ভাবে কল হয় — একে constructor বলে। এর কাজ হলো object-এর প্রাথমিক ডেটা (যেমন নাম, রোল) সেট করা। দুই পাশে দুইটি করে underscore থাকে (মোট ৪টি আগে, ৪টি পরে)।

> **[Important]** `__init__` কখনো manually কল করা হয় না — `Student(...)` লিখলেই Python নিজে এটিকে কল করে। এটিই constructor-এর বিশেষত্ব।

অ্যালগরিদম

Step 1: `class Student:` এর ভেতরে `def __init__(self, name, roll):` লেখা।

Step 2: `self.name = name` দিয়ে মান সংরক্ষণ করা।

Step 3: `Student("Rahim", 101)` দিয়ে object তৈরি করা।

Step 4: object-এর attribute প্রিন্ট করা।

Python Code

```
# Using the __init__ constructor

class Student:
    # Constructor - runs automatically when an object is created
    def __init__(self, name, roll, department):
        self.name = name
        self.roll = roll
        self.department = department
        print("New student object created:", self.name)

    def show_info(self):
        print("Name:", self.name)
        print("Roll:", self.roll)
        print("Department:", self.department)


# Creating objects - values are passed to __init__
student1 = Student("Rahim Ahmed", 101, "CSE")
student2 = Student("Karim Hossain", 102, "EEE")

print("\n--- Student 1 ---")
```

```
student1.show_info()

print("\n--- Student 2 ---")
student2.show_info()
```

Built-in Function / Shortcut Method

```
# Accessing attributes directly
# print(student1.name)
# print(student1.roll)
```

Sample Input / Output Table

Sample Input	Sample Output
None	New student object created: Rahim Ahmed New student object created: Karim Hossain --- Student 1 --- Name: Rahim Ahmed Roll: 101 Department: CSE --- Student 2 --- Name: Karim Hossain Roll: 102 Department: EEE

ব্যাখ্যা

`Student("Rahim Ahmed", 101, "CSE")` লিখার সাথে সাথে `__init__()` কল হয়ে নাম, রোল ও বিভাগ `self`-এর মাধ্যমে object-এ জমা হয়। ফলে আলাদা করে কোনো set function কল করতে হয় না।

Common Mistakes

- `__init__` লিখতে গিয়ে underscore কম-বেশি করা (সঠিক: দুই পাশে দুইটি করে)।
- `self.name = name` না লিখে শুধু `name = name` লেখা — তখন ডেটা object-এ সংরক্ষিত হবে না।
- constructor-এ parameter দেওয়া কিন্তু object তৈরির সময় না দেওয়া (`TypeError`)।

অনুশীলনের জন্য

1. `Rectangle` class বানান যার constructor-এ length ও width নেওয়া হয়।

2. `Employee` class বানান যাতে name, id ও salary থাকে।

3. constructor থেকে একটি welcome message প্রিন্ট করুন।

সমস্যা 221: self প্যারামিটার-এর কাজ বুঝুন।

সমস্যা

class-এর method-এ `self` কেন ব্যবহার করা হয় এবং এটি ছাড়া কী সমস্যা হয় — তা একটি উদাহরণ দিয়ে প্রমাণ করুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- Function argument

কীভাবে চিন্তা করব?

`self` হলো সেই object-এরই reference যেটি method-টি কল করছে। `student1.show_info()` লিখলে Python আসলে `Student.show_info(student1)` চালায় — অর্থাৎ object-টি নিজে থেকেই `self` হিসেবে পাস হয়। তাই method-এর প্রথম parameter-এ `self` লিখতে হয়।

অ্যালগরিদম

Step 1: class-এর method-এ `self` ব্যবহার করে attribute access করা।

Step 2: `student1.show_info()` ও `Student.show_info(student1)` — দুইভাবেই কল করে দেখা।

Step 3: `id(self)` প্রিন্ট করে প্রমাণ করা যে self-ই সেই object।

Python Code

```
# Understanding 'self'

class Student:
    def __init__(self, name):
        self.name = name

    def show_info(self):
        print("self name:", self.name)
        print("id of self inside method:", id(self))

    def greet(self, message):
        print(message, self.name)


student1 = Student("Rahim")
```

```
student2 = Student("Karim")

# Normal way of calling
print("--- Normal call ---")
student1.show_info()
print("id of student1 outside  :", id(student1))

# Proof that self IS the object - call through the class
print("\n--- Call using class name (same thing) ---")
Student.show_info(student1)

# self lets us use the object's own data
print("\n--- Using self inside another method ---")
student1.greet("Good morning,")
student2.greet("Good morning,")
```

Built-in Function / Shortcut Method

```
# Both lines below are exactly the same thing:
# student1.show_info()
# Student.show_info(student1)
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Normal call --- self name: Rahim id of self inside method: 140234567891234 id of student1 outside : 140234567891234 --- Call using class name (same thing) --- self name: Rahim --- Using self inside another method --- Good morning, Rahim Good morning, Karim

ব্যাখ্যা

method-এর ভিতরের `id(self)` আর বাইরের `id(student1)` একই — এর মানে `self` আর `student1` আসলে একই object। তাই `student1.show_info()` আর `Student.show_info(student1)` সম্পূর্ণ একই কাজ করে।

Common Mistakes

- method define করার সময় `self` না দেওয়া — তখন কল করলে `TypeError` হয়।
- `self` শব্দের বদলে অন্য নাম (যেমন `this`) ব্যবহার করা — Python-এ অবশ্যই `self` লিখতে হয়।
- method-এর ভিতরে `self` ছাড়া সরাসরি `name` লেখা (`NameError`)।

অনুশীলনের জন্য

1. দুইটি object বানিয়ে প্রমাণ করুন প্রতিটির `self` আলাদা।

2. `self` ব্যবহার করে একটি counter attribute বাড়ান।

3. class-এর method-এর ভিতর থেকে অন্য method কল করুন (`self.other_method()`)।

সমস্যা 222: Instance Variable ব্যবহার করুন।

সমস্যা

Instance Variable কী এবং কীভাবে প্রতিটি object-এর নিজস্ব আলাদা ডেটা থাকে — তা একটি প্রোগ্রামে দেখান।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- self

কীভাবে চিন্তা করব?

`self.name = name` দিয়ে তৈরি হওয়া ভ্যারিয়েবল-ই Instance Variable। প্রতিটি object-এর নিজস্ব কপি থাকে — এক object-এর মান বদলালে অন্যটির বদলায় না।

অ্যালগরিদম

Step 1: class-এর `__init__`-এ instance variable সেট করা।

Step 2: দুইটি object তৈরি করা।

Step 3: একটির মান পরিবর্তন করে অন্যটি অপরিবর্তিত থাকা প্রমাণ করা।

Step 4: `__dict__` দিয়ে object-এর সব variable দেখা।

Python Code

```
# Instance variables - each object has its own copy
```

```
class Student:
    def __init__(self, name, cgpa):
        self.name = name      # instance variable
        self.cgpa = cgpa      # instance variable


student1 = Student("Rahim", 3.75)
student2 = Student("Karim", 3.40)

print("Before change:")
print("student1:", student1.name, student1.cgpa)
print("student2:", student2.name, student2.cgpa)

# Change only student1
student1.cgpa = 3.90
student1.name = "Rahim Ahmed"

print("\nAfter changing student1 only:")
print("student1:", student1.name, student1.cgpa)
print("student2:", student2.name, student2.cgpa)

# Adding a new instance variable to just one object
student1.department = "CSE"

print("\nstudent1 variables:", student1.__dict__)
print("student2 variables:", student2.__dict__)
```

Built-in Function / Shortcut Method

```
# Access, update, delete instance variables
# student1.cgpa = 3.9        # update
# del student1.department    # delete
# getattr(student1, "name")  # safe read
```

Sample Input / Output Table

Sample Input	Sample Output
None	Before change: student1: Rahim 3.75 student2: Karim 3.4 After changing student1 only:

```
student1: Rahim Ahmed 3.9
student2: Karim 3.4

student1 variables: {'name':
'Rahim Ahmed', 'cgpa': 3.9,
'department': 'CSE'}
student2 variables: {'name':
'Karim', 'cgpa': 3.4}
```

ব্যাখ্যা

student1-এর মান বদলালেও student2-এর মান আগের মতোই আছে — কারণ instance variable প্রতিটি object-এর নিজস্ব। `__dict__` দিয়ে object-এর সব variable dictionary আকারে দেখা যায়।

Common Mistakes

- instance variable-কে function-এর ভিতরের local variable ভাবা।
- `self.` ছাড়া শুধু `name = name` লেখা (তখন local variable তৈরি হয়, object-এ থাকে না)।
- object তৈরির আগে instance variable access করার চেষ্টা করা (`AttributeError`)।

অনুশীলনের জন্য

1. তিনটি object বানিয়ে প্রতিটির আলাদা আলাদা মান রাখুন।

2. একটি object থেকে attribute মুছে (`del`) দেখান।

3. `getattr()` দিয়ে নিরাপদে attribute পড়ুন।

সমস্যা 223: Class Variable ব্যবহার করুন এবং Instance Variable-এর সাথে তুলনা করুন।

সমস্যা

Class Variable ও Instance Variable-এর পার্থক্য একটি প্রোগ্রাম ও তুলনামূলক সারণির মাধ্যমে বুঝান।

কী কী জানা থাকতে হবে?

- Instance variable
- Class & Object

কীভাবে চিন্তা করব?

Class Variable class-এর ভেতরে কিন্তু method-এর বাইরে লেখা হয় — এটি সব object-এর জন্য একই (shared)। Instance Variable `self.` দিয়ে method-এর ভিতরে থাকে এবং প্রতিটি object-এর জন্য আলাদা। যেমন — সব শিক্ষার্থীর institute একই (class variable), কিন্তু নাম আলাদা (instance variable)।

> **[Best Practice]** Class variable ব্যবহার করুন সেই সব ডেটার জন্য যা সব object-এর জন্যই এক — যেমন institute নাম, ট্যাক্স রেট, কোম্পানির নাম।

অ্যালগরিদম

Step 1: class-এর ভেতরে method-এর বাইরে `institute = "..."` লেখা।

Step 2: `__init__`-এ instance variable সেট করা।

Step 3: দুইটি object থেকে class variable access করা।

Step 4: `Student.institute = ...` দিয়ে class variable বদলে সব object-এ প্রভাব দেখা।

Python Code

```
# Class variable vs Instance variable

class Student:
    # Class variable - shared by all objects
    institute = "Bangladesh Polytechnic"
    total_students = 0

    def __init__(self, name, department):
        # Instance variables - separate for each object
        self.name = name
        self.department = department
        Student.total_students = Student.total_students + 1


student1 = Student("Rahim", "CSE")
student2 = Student("Karim", "EEE")

print("--- Accessing class variable ---")
print("Via class   :", Student.institute)
print("Via student1:", student1.institute)
print("Via student2:", student2.institute)

print("\n--- Instance variables (different for each) ---")
print("student1:", student1.name, student1.department)
print("student2:", student2.name, student2.department)

print("\nTotal students created:", Student.total_students)

# Changing the class variable affects all objects
Student.institute = "Sylhet Polytechnic"
print("\nAfter changing class variable:")
print("student1:", student1.institute)
```

```
print("student2:", student2.institute)
```

Built-in Function / Shortcut Method

```
# Comparison table
# Class variable    : declared inside class, outside methods, shared
# Instance variable : declared with self.<name>, unique per object
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Accessing class variable --- Via class : Bangladesh Polytechnic Via student1: Bangladesh Polytechnic Via student2: Bangladesh Polytechnic --- Instance variables (different for each) --- student1: Rahim CSE student2: Karim EEE Total students created: 2 After changing class variable: student1: Sylhet Polytechnic student2: Sylhet Polytechnic

ব্যাখ্যা

`institute` একটি class variable — তাই `Student.institute` বদলালে দুইটি object-ই নতুন মান দেখাল। কিন্তু `name` instance variable হওয়ায় প্রতিটি object-এর আলাদা। `total_students` দিয়ে কতগুলো object তৈরি হয়েছে তা গোনা হয়েছে।

Common Mistakes

- `Student.total_students = ...` না লিখে `self.total_students = ...` লেখা — তখন class variable না বেড়ে নতুন instance variable তৈরি হয়।
- class variable-কে constructor-এর ভিতরে লেখা (তখন সেটি instance variable হয়ে যায়)।
- class variable-এর নাম ও instance variable-এর নাম এক করে ফেলা (confusing bug)।

অনুশীলনের জন্য

1. `total_students` class variable দিয়ে মোট object গোনার প্রোগ্রাম লিখুন।

2. একটি class variable বদলে সব object-এ প্রভাব দেখান।

3. class variable ও instance variable-এর পার্থক্য ৩ লাইনে লিখুন।

সমস্যা 224: Class-এর ভেতরে বিভিন্ন ধরনের Method তৈরি করুন।

সমস্যা

একটি class-এর ভেতরে একাধিক method তৈরি করে দেখান কীভাবে একটি method অন্য method-কে কল করে এবং object-এর ডেটা প্রসেস করে।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- self
- if-else

কীভাবে চিন্তা করব?

Class-এর ভেতরের function-কেই method বলে। একটি class-এ যত খুশি method থাকতে পারে — যেমন তথ্য দেখানো, হিসাব করা, গ্রেড ঠিক করা। এক method থেকে অন্য method-কে `self.other_method()` দিয়ে কল করা যায়।

অ্যালগরিদম

Step 1: `BankAccount` class তৈরি করা।

Step 2: `deposit()` ও `withdraw()` method যোগ করা।

Step 3: `show_balance()` দিয়ে বর্তমান ব্যালেন্স দেখানো।

Step 4: method-গুলো কল করে কাজ যাচাই করা।

Python Code

```
# Multiple methods inside a class

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    # Method 1: deposit money
    def deposit(self, amount):
        if amount > 0:
```

```
            self.balance = self.balance + amount
            print(amount, "Taka deposited.")
        else:
            print("Deposit amount must be positive.")

    # Method 2: withdraw money
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance = self.balance - amount
            print(amount, "Taka withdrawn.")
        else:
            print("Insufficient balance!")

    # Method 3: show current balance
    def show_balance(self):
        print("Owner:", self.owner, "| Balance:", self.balance, "Taka")

    # Method 4: calling another method from inside a method
    def summary(self):
        print("---- Account Summary ----")
        self.show_balance()


account = BankAccount("Rahim", 1000)
account.show_balance()
account.deposit(500)
account.withdraw(300)
account.withdraw(5000)
account.summary()
```

Built-in Function / Shortcut Method

```
# Calling methods one after another (method chaining style)
# account.deposit(500).withdraw(300)   # works only if methods return self
```

Sample Input / Output Table

Sample Input	Sample Output
None	Owner: Rahim | Balance: 1000 Taka 500 Taka deposited. 300 Taka withdrawn. Insufficient balance! ---- Account Summary ---- Owner: Rahim | Balance: 1200 Taka

ব্যাখ্যা

চারটি method একই object-এর `self.balance` ডেটা নিয়ে কাজ করছে। `summary()` method-এর ভিতর থেকে `self.show_balance()` কল করা হয়েছে — এভাবে method-এর ভিতর থেকে অন্য method কল করা যায়।

Common Mistakes

- method কল করতে গিয়ে `self` argument পাঠিয়ে দেওয়া (`account.deposit(500)` সঠিক)।
- method-এর ভিতরে `self.` ছাড়া variable ব্যবহার করা।
- method define করতে গিয়ে function-এর মতো class-এর বাইরে লেখা।

অনুশীলনের জন্য

1. `Student` class-এ marks যোগ ও গড় বের করার method বানান।

2. `Calculator` class বানিয়ে add/subtract/multiply/divide method রাখুন।

3. `Shop` class বানিয়ে item যোগ ও মোট দাম বের করার method দিন।

সমস্যা 225: একই Class থেকে একাধিক Object তৈরি করে তালিকায় রাখুন।

সমস্যা

একই class থেকে একাধিক object তৈরি করে একটি list-এ রাখুন এবং লুপ চালিয়ে সব object-এর তথ্য প্রসেস করুন (Practical-1 Inventory-এর ভিত্তি)।

কী কী জানা থাকতে হবে?

- Class & Object
- List
- for loop
- __init__

কীভাবে চিন্তা করব?

Object-ও একটি মান — তাই list-এ রাখা যায়। ইউজারের কাছ থেকে বারবার ইনপুট নিয়ে object বানিয়ে list-এ append করলে একটি পূর্ণাঙ্গ ডেটা স্ট্রাকচার তৈরি হয়। এটিই Inventory/Student Management-এর মূল কৌশল।

> **[Important]** Practical-1 (Inventory Management) ও Practical-2 (Student Report) — দুটিতেই এই কৌশল লাগবে: class বানানো → object list-এ রাখা → লুপ চালিয়ে রিপোর্ট/CSV বানানো।

অ্যালগরিদম

Step 1: `Item` class তৈরি করা (name, price, quantity)।

Step 2: খালি `items = []` list তৈরি করা।

Step 3: লুপে ইউজারের ইনপুট নিয়ে object বানিয়ে list-এ append করা।

Step 4: list-এ লুপ চালিয়ে সব item ও মোট মূল্য প্রিন্ট করা।

Python Code

```
# Storing multiple objects in a list

class Item:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

    def total_price(self):
        return self.price * self.quantity


items = []

# Take input for 3 items
for i in range(3):
    print("\n--- Item", i + 1, "---")
    name = input("Item name: ")
    price = float(input("Unit price: "))
    quantity = int(input("Quantity: "))
    item = Item(name, price, quantity)
    items.append(item)

# Print all items
print("\n" + "=" * 50)
print("            INVENTORY SUMMARY")
print("=" * 50)
grand_total = 0
for item in items:
    print(item.name, "|", item.price, "x", item.quantity, "=",
item.total_price())
    grand_total = grand_total + item.total_price()

print("=" * 50)
print("Grand total:", grand_total, "Taka")
print("Total items stored:", len(items))
```

Built-in Function / Shortcut Method

```
# Creating objects directly in a list
# items = [Item("Mango", 120, 5), Item("Apple", 250, 3)]
```

Sample Input / Output Table

Sample Input	Sample Output
Mango 120 5 Apple 250 3 Banana 40 12	--- Item 1 --- Item name: Mango Unit price: 120 Quantity: 5 --- Item 2 --- Item name: Apple Unit price: 250 Quantity: 3 --- Item 3 --- Item name: Banana Unit price: 40 Quantity: 12 ========== ========== INVENTORY SUMMARY ========== ========== Mango | 120.0 x 5 = 600.0 Apple | 250.0 x 3 = 750.0 Banana | 40.0 x 12 = 480.0 ========== ========== Grand total: 1830.0 Taka Total items stored: 3

ব্যাখ্যা

প্রতিবার `Item(...)` কল করে নতুন object বানিয়ে `items.append()` দিয়ে list-এ রাখা হয়েছে। লুপে `for item in items:` চালিয়ে প্রতিটি object-এর method (`.total_price()`) কল করা হয়েছে — এটিই OOP-এর আসল শক্তি।

Common Mistakes

- list-এ object না রেখে শুধু মান রাখা (তখন OOP-এর সুবিধা পাওয়া যায় না)।
- loop-এর বাইরে object তৈরি করে বারবার append করা (সব item একই object হয়ে যায়)।
- `item.total_price` লেখা — method হলে ব্র্যাকেট `()` দিতে হবে।

অনুশীলনের জন্য

1. ৫ জন শিক্ষার্থীর object list বানিয়ে সবার গড় CGPA বের করুন।

2. list-এর object-গুলোর মধ্যে সবচেয়ে দামি item বের করুন।

3. object list থেকে CSV ফাইলে ডেটা লিখুন।

সমস্যা 226: __str__() Method দিয়ে Object-কে সুন্দরভাবে প্রিন্ট করুন।

সমস্যা

`__str__()` method ব্যবহার করে object-কে print করার সময় `<__main__.Student object at 0x...>`-এর বদলে অর্থপূর্ণ তথ্য দেখান।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- return statement

কীভাবে চিন্তা করব?

`print(student1)` করলে Python object-এর মেমোরি ঠিকানা দেখায়। কিন্তু class-এ `__str__()` method define করলে Python সেই method-এর return করা string-টি প্রিন্ট করে। এটি dunder (double underscore) method বা magic method।

অ্যালগরিদম

Step 1: class-এ `def __str__(self):` method যোগ করা।

Step 2: এর ভেতরে একটি সুন্দর formatted string return করা।

Step 3: `print(student1)` দিয়ে ফলাফল দেখা।

Step 4: `__str__` ছাড়া কী দেখায় তা তুলনা করা।

Python Code

```
# Using __str__() to print objects nicely

class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
```

```
        self.cgpa = cgpa

    # Without this method print() shows the memory address
    def __str__(self):
        return "Student[Name: " + self.name + ", Roll: " + str(self.roll) + ",
CGPA: " + str(self.cgpa) + "]"



student1 = Student("Rahim Ahmed", 101, 3.75)
student2 = Student("Karim Hossain", 102, 3.40)

print(student1)
print(student2)

# __str__ is also used automatically inside f-strings and str()
print("\nUsing str():", str(student1))
print("\nUsing f-string:", f"{student1}")

# Useful when printing a list of objects
students = [student1, student2]
print("\n--- List of students ---")
for s in students:
    print(s)
```

Built-in Function / Shortcut Method

```
# Modern Python way using f-strings inside __str__
# def __str__(self):
#     return f"Student[Name: {self.name}, Roll: {self.roll}, CGPA:
{self.cgpa}]"
```

Sample Input / Output Table

Sample Input	Sample Output
None	Student[Name: Rahim Ahmed, Roll: 101, CGPA: 3.75] Student[Name: Karim Hossain, Roll: 102, CGPA: 3.4] Using str(): Student[Name: Rahim Ahmed, Roll: 101, CGPA: 3.75] Using f-string: Student[Name:

```
Rahim Ahmed, Roll: 101, CGPA:
3.75]

--- List of students ---
Student[Name: Rahim Ahmed, Roll:
101, CGPA: 3.75]
Student[Name: Karim Hossain,
Roll: 102, CGPA: 3.4]
```

ব্যাখ্যা

`__str__()` method-টি string return করলে `print()`, `str()`, f-string — সব জায়গায় সেই string ব্যবহৃত হয়। `__str__` না থাকলে `<__main__.Student object at 0x7f...>` দেখাতো।

Common Mistakes

- `__str__`-এ `print()` ব্যবহার করে `return` না করা — অবশ্যই string return করতে হবে।
- `__str__` (string) ও `__repr__` (representation) গুলিয়ে ফেলা।
- return করার সময় number-কে `str()` দিয়ে রূপান্তর না করা (TypeError)।

অনুশীলনের জন্য

1. `Product` class-এ `__str__` যোগ করে সুন্দরভাবে প্রিন্ট করুন।

2. `Item` class-এ `__str__` যোগ করে inventory list প্রিন্ট করুন।

3. একটি class বানিয়ে `__str__` দিয়ে টেবিল আকারে তথ্য দেখান।

সমস্যা 227: del Keyword দিয়ে Object ও Attribute মুছুন।

সমস্যা

`del` keyword ব্যবহার করে object-এর attribute মুছে ফেলা এবং `__del__()` destructor-এর মাধ্যমে object ধ্বংস হওয়ার সময় কাজ করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- Instance variable
- __init__

কীভাবে চিন্তা করব?

`del object.attribute` দিয়ে নির্দিষ্ট attribute মুছে ফেলা যায়, আর `del object` দিয়ে পুরো object। object মেমোরি থেকে মুছে যাওয়ার আগে Python `__del__()` method কল করে — একে destructor বলে।

> **[Warning]** `__del__()` কখন কল হবে তা Python ঠিক করে — আপনি নিয়ন্ত্রণ করতে পারবেন না। ফাইল বন্ধ করা বা ডেটা সেভ করার মতো গুরুত্বপূর্ণ কাজের জন্য `with` ব্লক বা নিজস্ব `close()` method ব্যবহার করুন।

অ্যালগরিদম

Step 1: class-এ attribute সহ object তৈরি করা।

Step 2: `del object.attribute` দিয়ে একটি attribute মুছা।

Step 3: `__del__()` destructor যোগ করে মুছার সময় বার্তা দেখানো।

Step 4: `del object` দিয়ে object মুছা।

Python Code

```
# Deleting attributes and objects using del

class Student:
    def __init__(self, name, roll, department):
        self.name = name
        self.roll = roll
        self.department = department
        print("Created:", self.name)

    # Destructor - called when the object is destroyed
    def __del__(self):
        print("Destroyed:", self.name)



student1 = Student("Rahim", 101, "CSE")

print("\nBefore deletion:", student1.__dict__)

# Delete a single attribute
del student1.department
print("After deleting department:", student1.__dict__)

# Trying to access a deleted attribute causes AttributeError
print("\nName still available:", student1.name)

# Delete the whole object
print("\nDeleting the object now...")
del student1
print("Program finished.")
```

Built-in Function / Shortcut Method

```
# Safe way to delete an attribute only if it exists
# if hasattr(student1, "department"):
#     del student1.department
```

Sample Input / Output Table

Sample Input	Sample Output
None	Created: Rahim Before deletion: {'name': 'Rahim', 'roll': 101, 'department': 'CSE'} After deleting department: {'name': 'Rahim', 'roll': 101} Name still available: Rahim Deleting the object now... Destroyed: Rahim Program finished.

ব্যাখ্যা

`del student1.department` শুধু ওই attribute-টি মুছেছে, object-টি থেকেছে। `del student1` দিলে object ধ্বংস হয় এবং `__del__()` destructor কল হয়ে 'Destroyed' বার্তা এসেছে।

Common Mistakes

- মুছে ফেলা attribute access করার চেষ্টা করা (`AttributeError`)।
- `del student1` করার পর `student1` ব্যবহার করা (`NameError`)।
- `__del__()`-এর ওপর নির্ভর করে গুরুত্বপূর্ণ কাজ (যেমন ফাইল সেভ) রাখা — কখন কল হবে তা নিশ্চিত নয়, তাই `with` ব্লক ব্যবহার করুন।

অনুশীলনের জন্য

1. একটি object-এর দুইটি attribute মুছে `__dict__` দেখান।

2. `hasattr()` দিয়ে যাচাই করে নিরাপদে attribute মুছুন।

3. list-এ রাখা object-গুলো একসাথে মুছে ফেলুন।

সমস্যা 228: Class-এর মৌলিক গঠন (Basic Structure) ও Docstring ব্যবহার করুন।

সমস্যা

সিলেবাসের 4.5 অনুযায়ী — একটি class-এর পূর্ণাঙ্গ গঠন (docstring, class variable, constructor, methods) দেখান এবং `help()` দিয়ে ডকুমেন্টেশন পড়ুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- Methods
- Docstring

কীভাবে চিন্তা করব?

একটি প্রফেশনাল class-এর গঠন: (১) class declaration, (২) docstring (তিনটি উদ্ধৃতি চিহ্নের ভেতরে ব্যাখ্যা), (৩) class variable, (৪) `__init__` constructor, (৫) instance methods। `help(ClassName)` দিয়ে এই docstring পড়া যায়।

> **[Best Practice]** প্রতিটি class ও method-এর নিচে এক লাইনের docstring দেওয়ার অভ্যাস করুন। এটি প্রফেশনাল কোডিংয়ের অংশ এবং সিলেবাসের Unit 9 (Unit Testing)-এও কাজে লাগবে।

অ্যালগরিদম

Step 1: class-এর ঠিক নিচে triple-quoted docstring লেখা।

Step 2: class variable, constructor ও method যোগ করা।

Step 3: `__doc__` ও `help()` দিয়ে ডকুমেন্টেশন দেখা।

Step 4: class-এর attribute ও method-এর তালিকা `dir()` দিয়ে দেখা।

Python Code

```
# Basic structure of a professional Python class

class Student:
    """Student class stores and manages student information.

    Attributes:
        institute (str): Class variable, name of the institute.
        name (str): Name of the student.
        roll (int): Roll number of the student.
        cgpa (float): CGPA of the student.
    """

    # 1. Class variable
```

```
    institute = "Bangladesh Polytechnic"

    # 2. Constructor
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa

    # 3. Instance method
    def get_grade(self):
        """Return the grade based on CGPA."""
        if self.cgpa >= 3.75:
            return "A+"
        elif self.cgpa >= 3.00:
            return "A"
        elif self.cgpa >= 2.50:
            return "B"
        else:
            return "C"

    def show_info(self):
        """Display full information of the student."""
        print("Institute:", Student.institute)
        print("Name:", self.name)
        print("Roll:", self.roll)
        print("CGPA:", self.cgpa, "| Grade:", self.get_grade())


student1 = Student("Rahim Ahmed", 101, 3.80)
student1.show_info()

print("\n--- Class documentation ---")
print(Student.__doc__)

print("\n--- Method documentation ---")
print(Student.get_grade.__doc__)
```

Built-in Function / Shortcut Method

```
# Read documentation interactively
# help(Student)
# help(Student.get_grade)
```

Sample Input / Output Table

Sample Input	Sample Output

None	Institute: Bangladesh Polytechnic Name: Rahim Ahmed Roll: 101 CGPA: 3.8 | Grade: A+ --- Class documentation --- Student class stores and manages student information. Attributes: institute (str): Class variable, name of the institute. name (str): Name of the student. roll (int): Roll number of the student. cgpa (float): CGPA of the student. --- Method documentation --- Return the grade based on CGPA.

ব্যাখ্যা

class-এর ভেতরে docstring থাকলে `ClassName.__doc__` বা `help(ClassName)` দিয়ে সেটি পড়া যায়। বড় প্রজেক্টে docstring থাকা খুবই জরুরি — অন্য প্রোগ্রামার কোড বুঝতে পারে।

Common Mistakes

- docstring-কে comment (`#`) দিয়ে লেখা — এটি docstring হবে না।
- docstring-কে class-এর বাইরে বা method-এর নিচে রাখা (অবশ্যই ঠিক নিচে/উপরে হতে হবে)।
- class বানিয়ে কোনো docstring না লেখা — বড় প্রজেক্টে বুঝতে সমস্যা হয়।

অনুশীলনের জন্য

1. `BankAccount` class-এ docstring যোগ করে `help()` দিয়ে দেখুন।

2. একটি class বানিয়ে সব method-এ docstring দিন।

3. `dir()` দিয়ে class-এর method তালিকা বের করুন।

Topic: Four Pillars of OOP — Unit 5

সমস্যা 229: OOP-এর চারটি স্তম্ভ (Four Pillars) বুঝুন।

সমস্যা

সিলেবাসের 5.1 ও 5.2 অনুযায়ী — OOP-এর চারটি স্তম্ভ: Inheritance, Encapsulation, Polymorphism ও Abstraction — প্রতিটির সংজ্ঞা, গুরুত্ব ও একটি ছোট উদাহরণ সহ উপস্থাপন করুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- Methods

কীভাবে চিন্তা করব?

OOP-এর চারটি স্তম্ভ মনে রাখার সহজ উপায়: **I-E-P-A** = Inheritance (উত্তরাধিকার), Encapsulation (আবদ্ধকরণ), Polymorphism (বহুরূপতা), Abstraction (বিমূর্তকরণ)। সিলেবাসের ৯ নম্বরের মধ্যে এই ইউনিটেরই সবচেয়ে বেশি গুরুত্ব।

> **[Important]** পরীক্ষায় প্রশ্ন আসে: 'Mention the four pillars of OOP and explain any two.' — তাই চারটির নাম + যেকোনো দুটির বিস্তারিত উদাহরণ মুখস্থ রাখুন।

অ্যালগরিদম

Step 1: চারটি স্তম্ভের নাম ও সংজ্ঞা লেখা।

Step 2: প্রতিটির জন্য একটি করে ছোট কোড উদাহরণ দেওয়া।

Step 3: কেন দরকার — তার গুরুত্ব তালিকা করা।

Step 4: একটি demo class দিয়ে সবকিছু একসাথে দেখানো।

Python Code

```
# Four Pillars of OOP - one small demo for each

# 1. INHERITANCE: a child class acquires parent properties
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)
```

```
class Teacher(Person):        # Teacher inherits from Person
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject

    def show(self):
        print("Teacher:", self.name, "| Subject:", self.subject)



# 2. ENCAPSULATION: hiding data inside the class
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance       # private attribute

    def get_balance(self):
        return self.__balance          # controlled access



# 3. POLYMORPHISM: same method name, different behaviour
class Dog:
    def sound(self):
        print("Dog barks")



class Cat:
    def sound(self):
        print("Cat meows")



# 4. ABSTRACTION: hiding complexity, showing only what is needed
class EmailService:
    def send(self, to, message):
        self.__connect()
        print("Email sent to", to)

    def __connect(self):
        print("(Internal: connecting to server...)")



# ---- Demo of all four ----
print("=== INHERITANCE ===")
t = Teacher("Mr. Rahman", "Python")
t.show()

print("\n=== ENCAPSULATION ===")
```

```
acc = BankAccount(5000)
print("Balance via method:", acc.get_balance())

print("\n=== POLYMORPHISM ===")
for animal in [Dog(), Cat()]:
    animal.sound()

print("\n=== ABSTRACTION ===")
mail = EmailService()
mail.send("student@example.com", "Welcome!")
```

Built-in Function / Shortcut Method

```
# Remember: I - E - P - A
# I = Inheritance   : child gets parent's features
# E = Encapsulation : data + method together, data hidden
# P = Polymorphism  : one name, many forms
# A = Abstraction   : show what, hide how
```

Sample Input / Output Table

Sample Input	Sample Output
None	=== INHERITANCE === Teacher: Mr. Rahman | Subject: Python === ENCAPSULATION === Balance via method: 5000 === POLYMORPHISM === Dog barks Cat meows === ABSTRACTION === Email sent to student@example.com

ব্যাখ্যা

চারটি স্তম্ভই কোডকে পুনর্ব্যবহারযোগ্য, নিরাপদ ও সহজে রক্ষণাবেক্ষণযোগ্য করে। Inheritance দিয়ে কোড পুনর্ব্যবহার, Encapsulation দিয়ে ডেটা নিরাপত্তা, Polymorphism দিয়ে নমনীয়তা, আর Abstraction দিয়ে জটিলতা লুকানো হয়।

Common Mistakes

- Encapsulation ও Abstraction-কে একই জিনিস ভাবা (Encapsulation = ডেটা লুকানো, Abstraction = জটিলতা লুকানো)।
- Polymorphism বলতে শুধু method overriding বোঝা — overloading-ও এর অংশ।
- পরীক্ষায় সংজ্ঞা লিখে উদাহরণ না দেওয়া (উদাহরণ দিলে পূর্ণ নম্বর পাওয়া যায়)।

অনুশীলনের জন্য

1. চারটি স্তম্ভের প্রতিটির একটি করে বাস্তব জীবনের উদাহরণ লিখুন।

2. Inheritance ও Polymorphism-এর পার্থক্য লিখুন।

3. Encapsulation ব্যবহার করে একটি Student class বানান।

সমস্যা 230: Single Inheritance ব্যবহার করুন।

সমস্যা

সিলেবাসের 5.3 অনুযায়ী — একটি parent class থেকে একটি child class তৈরি (single inheritance) করে parent-এর সব বৈশিষ্ট্য পাওয়ার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- super()

কীভাবে চিন্তা করব?

Inheritance মানে এক class আরেক class-এর বৈশিষ্ট্য পাওয়া। `class Child(Parent):` লিখলেই Child, Parent-এর সব method ও attribute পেয়ে যায়। যে class দেয় তাকে Parent/Base class, যে নেয় তাকে Child/Derived class বলে।

অ্যালগরিদম

Step 1: `Person` নামে parent class তৈরি করা।

Step 2: `Student(Person)` নামে child class তৈরি করা।

Step 3: `super().__init__()` দিয়ে parent-এর constructor কল করা।

Step 4: child থেকে parent-এর method কল করা।

Python Code

```
# Single Inheritance: one parent, one child

# Parent (base) class
class Person:
```

```
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show_person(self):
        print("Name:", self.name)
        print("Age :", self.age)

    def greet(self):
        print("Hello, I am", self.name)


# Child (derived) class - inherits from Person
class Student(Person):
    def __init__(self, name, age, roll, department):
        super().__init__(name, age)     # call parent constructor
        self.roll = roll
        self.department = department

    def show_student(self):
        self.show_person()              # parent method
        print("Roll:", self.roll)
        print("Department:", self.department)


student1 = Student("Rahim Ahmed", 19, 101, "CSE")

print("--- Calling parent method from child object ---")
student1.greet()

print("\n--- Calling child method ---")
student1.show_student()

print("\n--- Inheritance check ---")
print("Is Student a subclass of Person?", issubclass(Student, Person))
```

Built-in Function / Shortcut Method

```
# Calling the parent constructor in another way
# Person.__init__(self, name, age)   # works, but super() is preferred
```

Sample Input / Output Table

Sample Input	Sample Output

```
None                                    --- Calling parent method from
                                        child object ---
                                        Hello, I am Rahim Ahmed

                                        --- Calling child method ---
                                        Name: Rahim Ahmed
                                        Age : 19
                                        Roll: 101
                                        Department: CSE

                                        --- Inheritance check ---
                                        Is Student a subclass of Person?
                                        True
```

ব্যাখ্যা

`Student(Person)` লেখার কারণে Student class Person-এর সব method পেয়েছে। তাই child object দিয়ে `greet()` (parent method) কল করা গেছে। `super().__init__(name, age)` দিয়ে parent-এর কাজটি parent-কেই করতে দেওয়া হয়েছে — কোড ডুপ্লিকেট হয়নি।

Common Mistakes

- `class Student(Person):` না লিখে `class Student:` লেখা (তখন inheritance হবে না)।
- child-এর constructor-এ `super().__init__()` কল না করা — তখন name/age সেট হবে না।
- parent-এর attribute-গুলো child-এ আবার লেখা (ডুপ্লিকেট কোড — inheritance-এর মানেই হলো এড়ানো)।

অনুশীলনের জন্য

1. `Animal` parent ও `Dog` child class বানিয়ে inheritance দেখান।

2. `Employee` parent থেকে `Manager` child বানান।

3. `Vehicle` parent থেকে `Car` child বানিয়ে নিজস্ব method যোগ করুন।

সমস্যা 231: Multilevel Inheritance ব্যবহার করুন।

সমস্যা

Multilevel inheritance ব্যবহার করে Person → Student → CSEStudent এই তিন স্তরের class তৈরি করুন এবং সবার বৈশিষ্ট্য একসাথে ব্যবহার করুন।

কী কী জানা থাকতে হবে?

- Single inheritance

- super()

কীভাবে চিন্তা করব?

Multilevel inheritance মানে ধাপে ধাপে inheritance: A → B → C। C তখন A ও B — দুজনেরই বৈশিষ্ট্য পাবে। `super().__init__()` চেইন হয়ে উপরের দিকে কল পৌঁছায়।

অ্যালগরিদম

Step 1: `Person` (স্তর ১), `Student` (স্তর ২), `CSEStudent` (স্তর ৩) class তৈরি করা।

Step 2: প্রতিটিতে `super().__init__()` দিয়ে উপরের স্তরের constructor কল করা।

Step 3: সব স্তরের method কল করে যাচাই করা।

Python Code

```
# Multilevel Inheritance: Person -> Student -> CSEStudent

class Person:                        # Level 1 (grandparent)
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("Person constructor called")

    def show_person(self):
        print("Name:", self.name, "| Age:", self.age)


class Student(Person):               # Level 2 (parent)
    def __init__(self, name, age, roll):
        super().__init__(name, age)
        self.roll = roll
        print("Student constructor called")

    def show_student(self):
        print("Roll:", self.roll)


class CSEStudent(Student):           # Level 3 (child)
    def __init__(self, name, age, roll, shift):
        super().__init__(name, age, roll)
        self.shift = shift
        self.department = "CSE"
        print("CSEStudent constructor called")

    def show_full_info(self):
        self.show_person()           # from Person
        self.show_student()          # from Student
        print("Shift:", self.shift, "| Department:", self.department)
```

```
print("--- Creating a CSEStudent object ---\n")
student1 = CSEStudent("Rahim Ahmed", 19, 101, "1st")

print("\n--- Full information (all levels) ---")
student1.show_full_info()
```

Built-in Function / Shortcut Method

```
# Check the method resolution order
# print(CSEStudent.__mro__)
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Creating a CSEStudent object --- Person constructor called Student constructor called CSEStudent constructor called --- Full information (all levels) --- Name: Rahim Ahmed | Age: 19 Roll: 101 Shift: 1st | Department: CSE

ব্যাখ্যা

object তৈরির সময় তিনটি constructor-ই ধারাবাহিকভাবে কল হয়েছে (Person → Student → CSEStudent)। ফলে সব স্তরের attribute একই object-এ জমা হয়েছে। `super()` চেইন এই কাজটি স্বয়ংক্রিয়ভাবে করে।

Common Mistakes

- মাঝের স্তরের constructor-এ `super().__init__()` না দেওয়া — তখন চেইন ভেঙে যায়।
- খুব বেশি স্তর (৪-৫ level) বানানো — কোড বুঝতে কঠিন হয়।
- `super()` কল করার 顺序 ভুল করা (সবসময় constructor-এর প্রথমে কল করুন)।

অনুশীলনের জন্য

1. `Vehicle → Car → ElectricCar` multilevel inheritance বানান।

2. `Employee → Manager → SeniorManager` class বানান।

3. `CSEStudent.__mro__` প্রিন্ট করে ক্রম দেখুন।

সমস্যা 232: Multiple Inheritance ব্যবহার করুন।

সমস্যা

Multiple inheritance ব্যবহার করে দুইটি parent class থেকে একটি child class তৈরি করুন এবং Method Resolution Order (MRO) বুঝুন।

কী কী জানা থাকতে হবে?

- Single inheritance
- Class & Object

কীভাবে চিন্তা করব?

Multiple inheritance মানে একাধিক parent: `class Child(Father, Mother):`। Child তখন দুজনেরই বৈশিষ্ট্য পায়। দুই parent-এ একই নামের method থাকলে Python বাম থেকে ডানে খোঁজে — একে MRO (Method Resolution Order) বলে।

> **[Best Practice]** Multiple inheritance যথাসম্ভব এড়িয়ে চলুন। বেশিরভাগ ক্ষেত্রেই composition (এক class-এর ভেতরে অন্য class-এর object রাখা) বেশিরভাগ সময় ভালো সমাধান।

অ্যালগরিদম

Step 1: দুইটি parent class (`Father`, `Mother`) তৈরি করা।

Step 2: `class Child(Father, Mother):` দিয়ে child বানানো।

Step 3: দুই parent-এর method আলাদা নামে রেখে কল করা।

Step 4: `Child.__mro__` দিয়ে খোঁজার ক্রম দেখা।

Python Code

```
# Multiple Inheritance: one child, two parents

class Father:
    def __init__(self):
        self.father_name = "Mr. Rahman"

    def skills(self):
```

```
        print("Father skill: Business")

    def common(self):
        print("This is Father's version")



class Mother:
    def __init__(self):
        self.mother_name = "Mrs. Rahman"

    def hobbies(self):
        print("Mother hobby: Cooking")

    def common(self):
        print("This is Mother's version")



# Child inherits from BOTH Father and Mother
class Child(Father, Mother):
    def __init__(self, name):
        Father.__init__(self)
        Mother.__init__(self)
        self.name = name

    def show_family(self):
        print("Child:", self.name)
        print("Father:", self.father_name)
        print("Mother:", self.mother_name)



child = Child("Rahim")
child.show_family()
child.skills()      # from Father
child.hobbies()     # from Mother

print("\n--- Which 'common' is used? ---")
child.common()

print("\n--- Method Resolution Order ---")
for cls in Child.__mro__:
    print(" ", cls.__name__)
```

Built-in Function / Shortcut Method

```
# MRO rule: Python searches left to right
```

```
# Child(Father, Mother) -> Father is checked first
```

Sample Input / Output Table

Sample Input	Sample Output
None	Child: Rahim Father: Mr. Rahman Mother: Mrs. Rahman Father skill: Business Mother hobby: Cooking --- Which 'common' is used? --- This is Father's version --- Method Resolution Order --- Child Father Mother object

ব্যাখ্যা

দুই parent-এই `common()` method থাকায় Python বাম দিকের (Father) method ব্যবহার করেছে। `Child.__mro__` দেখাচ্ছে খোঁজার ক্রম: Child → Father → Mother → object।

Common Mistakes

- দুই parent-এ `super().__init__()` ব্যবহার করে বিভ্রান্তি তৈরি করা — multiple inheritance-এ স্পষ্টভাবে `Father.__init__(self)` লেখাই নিরাপদ।
- দুই parent-এ একই নামের method রেখে কোনটি চলবে না বুঝে থাকা।
- অযথা multiple inheritance ব্যবহার করে কোড জটিল করা (Diamond problem)।

অনুশীলনের জন্য

1. `Teacher` ও `Researcher` class থেকে `Professor` বানান।

2. MRO প্রিন্ট করে ব্যাখ্যা করুন Python কোন class আগে খোঁজে।

3. দুই parent-এর common method-এর ভিন্ন ফলাফল দেখান।

সমস্যা 233: Method Overriding করুন।

সমস্যা

Parent class-এর method-কে child class-এ একই নামে নতুন করে লিখে (override) ভিন্ন আচরণ তৈরি করুন।

কী কী জানা থাকতে হবে?

- Inheritance
- Methods

কীভাবে চিন্তা করব?

Parent-এর কোনো method child-এর জন্য উপযুক্ত না হলে child-এ একই নামে নতুন method লিখি — একে Method Overriding বলে। object তৈরির পর কল করলে parent-এর নয়, child-এর method-ই চলে।

অ্যালগরিদম

Step 1: `Animal` parent class-এ `sound()` method রাখা।

Step 2: `Dog` ও `Cat` child class-এ একই নামে `sound()` override করা।

Step 3: দুইটি object বানিয়ে কল করে ভিন্ন ফলাফল দেখা।

Python Code

```
# Method Overriding

class Animal:                       # Parent class
    def sound(self):
        print("Animals make sounds")

    def sleep(self):
        print("Animal is sleeping")


class Dog(Animal):                  # Child overrides sound()
    def sound(self):
        print("Dog barks: Ghew Ghew")


class Cat(Animal):                  # Child overrides sound()
    def sound(self):
        print("Cat meows: Meow Meow")


class Cow(Animal):                  # Child does NOT override
```

```
    pass


print("--- Overridden methods ---")
dog = Dog()
dog.sound()

cat = Cat()
cat.sound()

print("\n--- Not overridden, uses parent version ---")
cow = Cow()
cow.sound()

print("\n--- Parent method still available ---")
dog.sleep()
cat.sleep()
```

Built-in Function / Shortcut Method

```
# To call the parent version from an overridden method:
# class Dog(Animal):
#     def sound(self):
#         super().sound()      # parent's sound
#         print("Dog barks")   # extra behaviour
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Overridden methods --- Dog barks: Ghew Ghew Cat meows: Meow Meow --- Not overridden, uses parent version --- Animals make sounds --- Parent method still available --- Animal is sleeping Animal is sleeping

ব্যাখ্যা

Dog ও Cat নিজেদের মতো `sound()` দিয়েছে, তাই তাদের নিজস্ব বার্তা এলো। Cow override করেনি, তাই parent-এর 'Animals make sounds' পাওয়া গেল। এই নমনীয়তাই Polymorphism-এর ভিত্তি।

Common Mistakes

- method-এর নাম child-এ ভিন্ন রাখা (স্পেলিং ভুল) — তখন override হবে না, parent-এর method চলবে।
- parent-এর method-এ parameter আছে কিন্তু child-এ কম/বেশি রাখা (`TypeError`)।
- override করার পরেও parent-এর method কল করতে চাইলে `super().method()` না জানা।

অনুশীলনের জন্য

1. `Shape` class-এ `area()` override করে Circle ও Rectangle বানান।

2. `Employee.bonus()` কে Manager-এ override করে বাড়তি বোনাস দিন।

3. override করার পর `super()` দিয়ে parent-এর method-ও কল করুন।

সমস্যা 234: super() Function ব্যবহার করুন।

সমস্যা

`super()` ব্যবহার করে child class থেকে parent class-এর constructor ও method কল করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Inheritance
- Method overriding
- __init__

কীভাবে চিন্তা করব?

`super()` parent class-কে নির্দেশ করে। এর দুটো প্রধান ব্যবহার: (১) parent-এর constructor কল করা যাতে কোড ডুপ্লিকেট না হয়, (২) override করা method-এর ভিতর থেকে parent-এর আসল method কল করা।

> **[Best Practice]** সবসময় `super().__init__(...)` ব্যবহার করুন, `ParentClass.__init__(self, ...)` নয়। `super()` multiple inheritance-এ MRO মেনে সঠিক parent খুঁজে নেয়।

অ্যালগরিদম

Step 1: `Employee` parent class তৈরি করা (name, salary)।

Step 2: `Manager` child class-এ `super().__init__()` দিয়ে parent constructor কল করা।

Step 3: override করা `show_info()` method-এ `super().show_info()` কল করা।

Python Code

```
# Using super() to call parent constructor and methods

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def show_info(self):
        print("Name:", self.name)
        print("Salary:", self.salary)

    def annual_salary(self):
        return self.salary * 12



class Manager(Employee):
    def __init__(self, name, salary, department):
        # 1. Call parent constructor using super()
        super().__init__(name, salary)
        self.department = department

    def show_info(self):
        # 2. Call parent method using super(), then add extra
        super().show_info()
        print("Department:", self.department)



manager1 = Manager("Mr. Karim", 50000, "IT")
manager1.show_info()

print("\nAnnual salary (parent method):", manager1.annual_salary())

# super() works with multilevel inheritance too
class SeniorManager(Manager):
    def __init__(self, name, salary, department, team_size):
        super().__init__(name, salary, department)
        self.team_size = team_size

    def show_info(self):
        super().show_info()
        print("Team size:", self.team_size)
```

```
print("\n--- Multilevel super() ---")
sm = SeniorManager("Mr. Rahman", 80000, "CSE", 10)
sm.show_info()
```

Built-in Function / Shortcut Method

```
# Inside a method you can write:
# super().__init__(...)   -> parent constructor
# super().method_name()   -> parent method
```

Sample Input / Output Table

Sample Input	Sample Output
None	Name: Mr. Karim Salary: 50000 Department: IT Annual salary (parent method): 600000 --- Multilevel super() --- Name: Mr. Rahman Salary: 80000 Department: CSE Team size: 10

ব্যাখ্যা

`super()`-এর কারণে parent-এর constructor ও method পুনরায় না লিখেও ব্যবহার করা গেছে, আর child শুধু নিজের অতিরিক্ত অংশ যোগ করেছে। এতে কোড ছোট ও রক্ষণাবেক্ষণ সহজ হয়।

Common Mistakes

- `super()`-এর বদলে `Employee.__init__(self, ...)` লেখা (কাজ করলেও multiple inheritance-এ ভুল হয়)।
- `super` কল করতে গিয়ে ব্র্যাকেট ভুলে যাওয়া (`super` নয়, `super()`)।
- `super().__init__()` constructor-এর শেষে কল করা (সবসময় প্রথমে কল করুন)।

অনুশীলনের জন্য

1. `Vehicle → Car` class-এ `super()` দিয়ে constructor কল করুন।

2. override করা method-এ `super()` দিয়ে parent-এর method আগে চালান।

3. multilevel inheritance-এ `super()` চেইন দেখান।

সমস্যা 235: isinstance() এবং issubclass() ব্যবহার করুন।

সমস্যা

`isinstance()` দিয়ে object কোন class-এর কিনা এবং `issubclass()` দিয়ে class-এর সম্পর্ক যাচাই করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Inheritance
- Class & Object

কীভাবে চিন্তা করব?

`isinstance(object, Class)` → object-টি ঐ class-এর কিনা (True/False)। `issubclass(Child, Parent)` → Child কি Parent-এর উত্তরসূরি কিনা। বড় প্রোগ্রামে object-এর ধরন যাচাই করে সিদ্ধান্ত নিতে এগুলো ব্যবহার হয়।

অ্যালগরিদম

Step 1: Person parent ও Student child class তৈরি করা।

Step 2: `isinstance()` দিয়ে object যাচাই করা।

Step 3: `issubclass()` দিয়ে class সম্পর্ক যাচাই করা।

Step 4: `type()`-এর সাথে পার্থক্য দেখানো।

Python Code

```
# isinstance() and issubclass()

class Person:
    pass


class Student(Person):
    pass


class Teacher(Person):
    pass


student1 = Student()
```

```
teacher1 = Teacher()

print("--- isinstance() : checks the OBJECT ---")
print("student1 is a Student?", isinstance(student1, Student))
print("student1 is a Person? ", isinstance(student1, Person))
print("student1 is a Teacher?", isinstance(student1, Teacher))

print("\n--- issubclass() : checks the CLASS ---")
print("Student is subclass of Person?", issubclass(Student, Person))
print("Teacher is subclass of Person?", issubclass(Teacher, Person))
print("Student is subclass of Teacher?", issubclass(Student, Teacher))

print("\n--- isinstance with built-in types ---")
value = 42
print("Is 42 an int?", isinstance(value, int))
print("Is 3.5 a float?", isinstance(3.5, float))
print("Is 'python' a str?", isinstance("python", str))

print("\n--- type() vs isinstance() ---")
print("type(student1) == Student :", type(student1) == Student)
print("type(student1) == Person  :", type(student1) == Person)
print("isinstance(student1, Person):", isinstance(student1, Person))
```

Built-in Function / Shortcut Method

```
# isinstance can check multiple types at once
# isinstance(value, (int, float))   -> True if int OR float
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- isinstance() : checks the OBJECT --- student1 is a Student? True student1 is a Person? True student1 is a Teacher? False --- issubclass() : checks the CLASS --- Student is subclass of Person? True Teacher is subclass of Person? True Student is subclass of Teacher?

```
False

--- isinstance with built-in
types ---
Is 42 an int? True
Is 3.5 a float? True
Is 'python' a str? True

--- isinstance with inheritance
---
type(student1) == Student : True
type(student1) == Person  : False
isinstance(student1, Person):
True
```

ব্যাখ্যা

`type()` সঠিক class-ই চায়, কিন্তু `isinstance()` inheritance মানে — Student object আসলে Person-ও। তাই inheritance-এর ক্ষেত্রে `isinstance()` ব্যবহার করাই সঠিক।

Common Mistakes

- `isinstance` ও `issubclass` উল্টাপাল্টা ব্যবহার করা (প্রথমটি object নেয়, দ্বিতীয়টি class)।
- `type(x) == Class` ব্যবহার করে inheritance-এর ক্ষেত্রে ভুল ফলাফল পাওয়া।
- `isinstance(Student, Person)` লেখা (class দিয়ে) — এটি সবসময় False, এখানে `issubclass` লাগবে।

অনুশীলনের জন্য

1. একটি list-এর সব object Student কিনা যাচাই করুন।

2. ইউজার ইনপুট number কিনা `isinstance()` দিয়ে যাচাই করুন।

3. `issubclass()` দিয়ে multilevel inheritance যাচাই করুন।

সমস্যা 236: Encapsulation: Public, Protected ও Private Member ব্যবহার করুন।

সমস্যা

সিলেবাসের 5.4 অনুযায়ী — Encapsulation বুঝে public, protected (`_`) ও private (`__`) attribute ব্যবহারের প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- Methods

কীভাবে চিন্তা করব?

Encapsulation মানে ডেটা ও সেই ডেটার ওপর কাজ করা method-কে একসাথে ক্যাপসুলের মতো বন্দি করে রাখা। পাইথনে তিন স্তর: public (`name`), protected (`_name` — মানে বাইরে ব্যবহার করবেন না), private (`__name` — Python নিজে name-mangling করে লুকায়)।

> **[Important]** Encapsulation-এর মূল উদ্দেশ্য হলো **validation**। set_cgpa()-এর মতো method দিয়ে যাচাই না করলে কেউ cgpa = 9.0 বসিয়ে দিতে পারতো — ডেটা ভুল হয়ে যেত।

অ্যালগরিদম

Step 1: class-এ public, protected ও private attribute রাখা।

Step 2: public attribute সরাসরি access করা।

Step 3: private attribute access করতে গিয়ে error দেখানো।

Step 4: private ডেটা method-এর মাধ্যমে নিরাপদে পড়া।

Python Code

```
# Encapsulation: public, protected and private members

class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name          # public
        self._roll = roll         # protected (convention: do not touch)
        self.__cgpa = cgpa        # private (name mangled)

    # Public method to read private data safely
    def get_cgpa(self):
        return self.__cgpa

    # Public method to update private data with validation
    def set_cgpa(self, value):
        if 0.0 <= value <= 4.0:
            self.__cgpa = value
            print("CGPA updated successfully.")
        else:
            print("Invalid CGPA! Must be between 0.0 and 4.0")

    def show_all(self):
```

```
        print("Name  :", self.name)
        print("Roll  :", self._roll)
        print("CGPA  :", self.__cgpa)



student1 = Student("Rahim", 101, 3.75)

print("--- Public access ---")
print("Name:", student1.name)

print("\n--- Reading private data through a method ---")
print("CGPA:", student1.get_cgpa())

print("\n--- Updating private data with validation ---")
student1.set_cgpa(3.90)
print("New CGPA:", student1.get_cgpa())
student1.set_cgpa(9.0)          # rejected

print("\n--- What happens if we try to access directly? ---")
print("Protected (_roll) still accessible:", student1._roll)
# print(student1.__cgpa)   -> AttributeError

print("\n--- Name mangling (how Python hides it) ---")
print("Real internal name:", student1._Student__cgpa)
```

Built-in Function / Shortcut Method

```
# Access levels at a glance
# self.name      -> public     : anyone can access
# self._name     -> protected  : "internal use only" (by convention)
# self.__name    -> private    : name-mangled to _ClassName__name
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Public access --- Name: Rahim --- Reading private data through a method --- CGPA: 3.75 --- Updating private data with

```
validation ---
CGPA updated successfully.
New CGPA: 3.9
Invalid CGPA! Must be between 0.0
and 4.0

--- What happens if we try to
access directly? ---
Protected (_roll) still
accessible: 101

--- Name mangling (how Python
hides it) ---
Real internal name: 3.9
```

ব্যাখ্যা

`__cgpa` private হওয়ায় বাইরে থেকে `student1.__cgpa` access করা যায় না। Python একে `_Student__cgpa` নামে লুকিয়ে রাখে (name mangling)। তবে `set_cgpa()` দিয়ে validation সহ নিরাপদে পরিবর্তন করা যায় — এটিই Encapsulation-এর আসল সুবিধা।

Common Mistakes

- private attribute-এর নামের আগে একটিমাত্র underscore দেওয়া (তখন protected হয়, private নয়)।
- `student1.__cgpa` access করতে গিয়ে `AttributeError` খাওয়া।
- সব attribute-ই private বানিয়ে ফেলা — সবসময় প্রয়োজন নেই, simple class-এ public যথেষ্ট।

অনুশীলনের জন্য

1. `BankAccount` class বানিয়ে balance-কে private রাখুন।

2. private attribute-এ validation সহ setter method বানান।

3. public, protected ও private-এর পার্থক্য ৩ লাইনে লিখুন।

সমস্যা 237: Getter ও Setter Method ব্যবহার করুন।

সমস্যা

Private attribute-এর জন্য getter ও setter method তৈরি করে নিরাপদ ডেটা access ও পরিবর্তনের প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Encapsulation
- Private attribute
- if-else

কীভাবে চিন্তা করব?

Getter হলো `get_xxx()` — private ডেটা পড়ার method। Setter হলো `set_xxx(value)` — private ডেটা লেখার method, যেখানে validation করা যায়। এটিই Encapsulation বাস্তবায়নের প্রথাগত (traditional) উপায়।

অ্যালগরিদম

Step 1: class-এ private attribute রাখা।

Step 2: `get_xxx()` method দিয়ে মান ফেরত দেওয়া।

Step 3: `set_xxx()` method-এ validation সহ মান সেট করা।

Step 4: ভুল মান দিয়ে validation পরীক্ষা করা।

Python Code

```
# Getter and Setter methods

class Student:
    def __init__(self, name, age, cgpa):
        self.name = name
        self.__age = age
        self.__cgpa = cgpa

    # ---------- Getter and Setter for age ----------
    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age > 0 and age < 120:
            self.__age = age
        else:
            print("Invalid age! Age must be between 1 and 119.")

    # ---------- Getter and Setter for cgpa ----------
    def get_cgpa(self):
        return self.__cgpa

    def set_cgpa(self, cgpa):
        if 0.0 <= cgpa <= 4.0:
            self.__cgpa = cgpa
        else:
            print("Invalid CGPA! Must be between 0.0 and 4.00")
```

```
    def show(self):
        print("Name:", self.name, "| Age:", self.__age, "| CGPA:", self.__cgpa)


student1 = Student("Rahim", 19, 3.75)
student1.show()

print("\n--- Using setters ---")
student1.set_age(20)
student1.set_cgpa(3.90)
student1.show()

print("\n--- Testing validation ---")
student1.set_age(200)
student1.set_cgpa(5.5)
student1.show()

print("\n--- Using getters ---")
print("Age :", student1.get_age())
print("CGPA:", student1.get_cgpa())
```

Built-in Function / Shortcut Method

```
# Pythonic way (preferred): use @property instead
# See the next problem (Problem 238)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Name: Rahim | Age: 19 | CGPA: 3.75 --- Using setters --- Name: Rahim | Age: 20 | CGPA: 3.9 --- Testing validation --- Invalid age! Age must be between 1 and 119. Invalid CGPA! Must be between 0.0 and 4.00 Name: Rahim | Age: 20 | CGPA: 3.9

```
--- Using getters ---
Age : 20
CGPA: 3.9
```

ব্যাখ্যা

setter method-এ validation থাকায় ভুল মান (age=200, cgpa=5.5) বার্তা দিয়ে বাতিল হয়ে গেছে — ডেটা অক্ষত রয়েছে। getter দিয়ে নিরাপদে মান পড়া গেছে।

Common Mistakes

- setter-এ validation না রাখা — তাহলে private করেই লাভ নেই।
- getter-এ `print()` করা (getter সবসময় `return` করে)।
- `getAge()`/`setAge()` camelCase লেখা — Python-এ `get_age()` snake_case ব্যবহার করুন।

অনুশীলনের জন্য

1. `Product` class-এ price-এর getter/setter বানান (নেগেটিভ হলে reject)।

2. `Employee` class-এ salary-এর getter/setter বানান।

3. setter-এ validation বার্তা ফেরত দিন (True/False)।

সমস্যা 238: @property Decorator ব্যবহার করুন।

সমস্যা

`@property` ও `@xxx.setter` decorator ব্যবহার করে getter/setter-কে attribute-এর মতো সহজভাবে ব্যবহার করুন।

কী কী জানা থাকতে হবে?

- Getter/Setter
- Decorator concept
- Private attribute

কীভাবে চিন্তা করব?

Getter/Setter ব্যবহার করতে `student.get_cgpa()` লিখতে হয় — কিন্তু `@property` দিলে `student.cgpa` লিখেই সেই method চলে। বাইরে থেকে দেখতে সাধারণ attribute মনে হয়, কিন্তু ভিতরে validation চলে। এটি Pythonic ও আধুনিক পদ্ধতি।

> **[Best Practice]** বাংলাদেশের সিলেবাসে getter/setter শেখানো হলেও, বাস্তব প্রজেক্টে `@property` ব্যবহার করাই Pythonic স্ট্যান্ডার্ড। দুটোই জানা থাকা জরুরি।

অ্যালগরিদম

Step 1: private attribute `__cgpa` রাখা।

Step 2: `@property` দিয়ে `def cgpa(self):` method বানানো।

Step 3: `@cgpa.setter` দিয়ে মান সেট করার method বানানো।

Step 4: `student.cgpa` ও `student.cgpa = 3.9` ব্যবহার করে দেখা।

Python Code

```
# Using @property decorator (Pythonic getter / setter)

class Student:
    def __init__(self, name, cgpa):
        self.name = name
        self.__cgpa = cgpa

    # Getter - accessed as student.cgpa
    @property
    def cgpa(self):
        print("(getter called)")
        return self.__cgpa

    # Setter - accessed as student.cgpa = value
    @cgpa.setter
    def cgpa(self, value):
        print("(setter called)")
        if 0.0 <= value <= 4.0:
            self.__cgpa = value
        else:
            print("Invalid CGPA! Value not changed.")

    # Read-only property: no setter, so it cannot be changed
    @property
    def grade(self):
        if self.__cgpa >= 3.75:
            return "A+"
        elif self.__cgpa >= 3.00:
            return "A"
        else:
            return "B"



student1 = Student("Rahim", 3.75)

print("Reading cgpa :", student1.cgpa)
student1.cgpa = 3.90
print("After update :", student1.cgpa)
```

```
print("\nTesting invalid value:")
student1.cgpa = 9.0
print("Value remains:", student1.cgpa)

print("\nRead-only property (grade):")
print("Grade:", student1.grade)
# student1.grade = "A"   -> AttributeError: can't set attribute
```

Built-in Function / Shortcut Method

```
# Remember the order:
# @property        -> defines the getter  (read)
# @cgpa.setter     -> defines the setter  (write)
# If you skip the setter, the property becomes read-only
```

Sample Input / Output Table

Sample Input	Sample Output
None	Reading cgpa : (getter called) 3.75 After update : (getter called) 3.9 Testing invalid value: (setter called) Invalid CGPA! Value not changed. (getter called) Value remains: 3.9 Read-only property (grade): Grade: A+

ব্যাখ্যা

`student1.cgpa` লিখলেই getter method চলছে, আর `student1.cgpa = 3.90` লিখলে setter চলছে — বাইরে থেকে সাধারণ attribute-এর মতো মনে হলেও ভিতরে validation হচ্ছে। `grade`-এর কোনো setter নেই, তাই এটি read-only।

Common Mistakes

- `@property` ও `@cgpa.setter`-এ method-এর নাম আলাদা রাখা (দুটির নাম অবশ্যই একই হবে)।

- setter method-এ `@property` লিখে দেওয়া (setter-এ `@name.setter` লাগে)।
- property-কে method মনে করে `student.cgpa()` লেখা (ব্র্যাকেট লাগবে না)।

অনুশীলনের জন্য

1. `Product` class-এ `@property` দিয়ে price validate করুন।

2. শুধুমাত্র read-only property (যেমন `grade`) বানান।

3. `Rectangle` class-এ `@property` দিয়ে area বের করুন।

সমস্যা 239: Polymorphism: Method Overriding দিয়ে বহুরূপতা তৈরি করুন।

সমস্যা

সিলেবাসের 5.5 অনুযায়ী — Polymorphism বুঝে একই method নাম দিয়ে বিভিন্ন class-এ ভিন্ন কাজ করানোর প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Inheritance
- Method overriding
- for loop
- List

কীভাবে চিন্তা করব?

Polymorphism মানে 'এক নাম, অনেক রূপ'। ভিন্ন ভিন্ন class-এ একই নামের method থাকলে আমরা object-এর ধরন না জেনেও সেই method কল করতে পারি। নিচের উদাহরণে `area()` method-টি Circle, Rectangle ও Triangle — তিনটিতেই আছে কিন্তু প্রতিটির হিসাব আলাদা।

> **[Important]** পরীক্ষায় Polymorphism-এর সংজ্ঞার সাথে অবশ্যই একটি উদাহরণ দিন: 'একই method name, ভিন্ন class, ভিন্ন আচরণ'।

অ্যালগরিদম

Step 1: `Shape` parent class-এ `area()` method রাখা।

Step 2: `Circle`, `Rectangle`, `Triangle` child class-এ `area()` override করা।

Step 3: সব object একট list-এ রাখা।

Step 4: লুপ চালিয়ে object-এর ধরন না জেনেই `area()` কল করা।

Python Code

```
# Polymorphism using method overriding
```

```
class Shape:
    def area(self):
        return 0



class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.1416 * self.radius * self.radius



class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width



class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height



# Store different objects in one list
shapes = [Circle(5), Rectangle(10, 4), Triangle(6, 8)]

# Same method call, different behaviour - this is polymorphism
for shape in shapes:
    print(type(shape).__name__, "area =", round(shape.area(), 2))

# A function that works with ANY shape
print("\n--- Using a common function ---")
def print_area(shape):
    print("Area:", round(shape.area(), 2))
```

```
print_area(Circle(7))
print_area(Rectangle(3, 9))
print_area(Triangle(4, 5))
```

Built-in Function / Shortcut Method

```
# Polymorphism checklist:
# 1. Many classes have a method with the SAME name
# 2. Each class implements it differently
# 3. The caller does not need to know the object type
```

Sample Input / Output Table

Sample Input	Sample Output
None	Circle area = 78.54 Rectangle area = 40 Triangle area = 24.0 --- Using a common function --- Area: 153.94 Area: 27 Area: 10.0

ব্যাখ্যা

লুপে `shape.area()` কল করা হয়েছে — কিন্তু প্রতিটি object নিজের নিজস্ব `area()` method ব্যবহার করেছে। আমাদের `if type == ...` লিখতে হয়নি। এটিই Polymorphism-এর শক্তি: কোড ছোট, নতুন shape যোগ করলেও লুপের কোড বদলাতে হবে না।

Common Mistakes

- Polymorphism-এর জন্য inheritance বাধ্যতামূলক মনে করা — না, একই নামের method থাকলেই হয় (duck typing)।
- parent class-এ method না রাখা — তখন নতুন object যোগ করলে ভুল হতে পারে।
- method-এর নাম ভিন্ন রাখা (তখন polymorphism হবে না)।

অনুশীলনের জন্য

1. `Bird`, `Fish`, `Snake` class-এ `move()` method override করুন।

2. `Bangla`, `English`, `Math` class-এ `total_marks()` override করুন।

3. নতুন একটি shape যোগ করে দেখুন লুপের কোড বদলাতে হয় না।

সমস্যা 240: Polymorphism: Duck Typing ও Built-in Function-এর বহুরূপতা।

সমস্যা

Duck Typing কী এবং কীভাবে `len()`, `+` অপারেটর বিভিন্ন ডেটা টাইপের জন্য ভিন্ন আচরণ করে — তা দেখান।

কী কী জানা থাকতে হবে?

- Polymorphism
- Built-in functions
- Class & Object

কীভাবে চিন্তা করব?

Duck Typing-এর নিয়ম: 'যদি কোনো কিছু হাঁসের মতো হাঁটে ও হাঁসের মতো ডাকে, তবে সেটি হাঁস'। অর্থাৎ object-এর class কী তা নয়, সে কী করতে পারে তাই জরুরি। `len()` string, list, dict — সবার ক্ষেত্রেই কাজ করে কিন্তু ভিন্নভাবে।

অ্যালগরিদম

Step 1: `len()` কে বিভিন্ন টাইপে ব্যবহার করা।

Step 2: `+` অপারেটর number ও string-এর ক্ষেত্রে দেখা।

Step 3: নিজের class-এ `__len__()` যোগ করে `len()` চালানো।

Step 4: duck typing-এর উদাহরণ হিসেবে ভিন্ন class-এর object-কে একই function-এ পাঠানো।

Python Code

```
# Polymorphism with built-in functions and duck typing

print("--- len() works differently for different types ---")
print("len('Python')      :", len("Python"))          # counts characters
print("len([1, 2, 3, 4]) :", len([1, 2, 3, 4]))      # counts items
print("len({'a': 1})     :", len({"a": 1}))          # counts keys

print("\n--- '+' operator behaves differently ---")
print("5 + 3        :", 5 + 3)                       # addition
print("'5' + '3'    :", "5" + "3")                   # concatenation
print("[1] + [2]    :", [1] + [2])                   # merging lists

print("\n--- Making our own class work with len() ---")

class Classroom:
    def __init__(self, students):
        self.students = students

    # Duck typing: because __len__ exists, len() works
    def __len__(self):
```

```
        return len(self.students)



room = Classroom(["Rahim", "Karim", "Sadia"])
print("Students in classroom:", len(room))

print("\n--- Duck typing: different classes, same behaviour ---")
class PdfReport:
    def generate(self):
        print("Generating a PDF report...")



class CsvReport:
    def generate(self):
        print("Generating a CSV report...")



class ExcelReport:
    def generate(self):
        print("Generating an Excel report...")



def create_report(report):
    # We do NOT check the type - we just call generate()
    report.generate()



for r in [PdfReport(), CsvReport(), ExcelReport()]:
    create_report(r)
```

Built-in Function / Shortcut Method

```
# Duck typing rule:
# "If it walks like a duck and quacks like a duck, it is a duck."
# Python cares about BEHAVIOUR (methods), not the class name.
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- len() works differently for different types --- len('Python') : 6 len([1, 2, 3, 4]) : 4

```
len({'a': 1})     : 1

--- '+' operator behaves
differently ---
5 + 3         : 8
'5' + '3'     : 53
[1] + [2]     : [1, 2]

--- Making our own class work
with len() ---
Students in classroom: 3

--- Duck typing: different
classes, same behaviour ---
Generating a PDF report...
Generating a CSV report...
Generating an Excel report...
```

ব্যাখ্যা

`create_report()` function-টি object-এর class যাচাই করছে না — শুধু `generate()` method আছে কিনা তা দেখছে। তিনটি ভিন্ন class-এর object-ই কাজ করছে। আমাদের নিজস্ব `Classroom` class-এ `__len__()` যোগ করায় `len(room)` কাজ করেছে।

Common Mistakes

- duck typing-এ `type()` দিয়ে যাচাই করা — এটি duck typing-এর মূলনীতির বিরুদ্ধে।
- method না থাকলে `AttributeError` — তাই `hasattr()` দিয়ে নিরাপদে যাচাই করা যায়।
- `len()`-কে শুধু list-এর জন্য ভাবা — এটি যেকোনো iterable-এ চলে।

অনুশীলনের জন্য

1. নিজের `Playlist` class-এ `__len__()` যোগ করুন।

2. `hasattr()` দিয়ে যাচাই করে duck typing নিরাপদ করুন।

3. তিনটি ভিন্ন class বানিয়ে একই function দিয়ে কল করুন।

সমস্যা 241: Operator Overloading করুন।

সমস্যা

`__add__()`, `__gt__()` ইত্যাদি dunder method ব্যবহার করে নিজের class-এর object-এর ওপর `+`, `>` অপারেটর চালানোর প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- __init__
- __str__
- Polymorphism

কীভাবে চিন্তা করব?

Operator Overloading মানে বিল্ট-ইন অপারেটরগুলোকে (`+`, `-`, `>`, `==`) নিজের class-এর জন্য নতুন করে define করা। `a + b` লিখলে Python আসলে `a.__add__(b)` কল করে। তাই আমরা `__add__()` method লিখলে `+` আমাদের object-এ কাজ করবে।

> **[Note]** Python-এ `__add__` মানেই 'যোগ' — যেকোনো কাস্টম class-এ এটি define করলে `+` অপারেটর সেখানে কাজ করবে। একে magic method বা dunder method বলে (double underscore)।

অ্যালগরিদম

Step 1: `Point` class তৈরি করা (x, y)।

Step 2: `__add__()` দিয়ে দুইটি Point যোগ করা।

Step 3: `__gt__()` দিয়ে তুলনা করা।

Step 4: `__str__()` দিয়ে সুন্দর প্রিন্ট করা।

Python Code

```
# Operator overloading with dunder methods

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # a + b  ->  a.__add__(b)
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    # a - b  ->  a.__sub__(b)
    def __sub__(self, other):
        return Point(self.x - other.x, self.y - other.y)

    # a > b  ->  a.__gt__(b)
    def __gt__(self, other):
        return (self.x + self.y) > (other.x + other.y)
```

```
    # a == b  ->  a.__eq__(b)
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return "(" + str(self.x) + ", " + str(self.y) + ")"


p1 = Point(2, 3)
p2 = Point(4, 5)

print("p1 :", p1)
print("p2 :", p2)
print("\np1 + p2 :", p1 + p2)
print("p1 - p2 :", p1 - p2)
print("\np2 > p1 :", p2 > p1)
print("p1 == p2:", p1 == p2)

# Practical example: adding two BankAccount balances
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def __add__(self, other):
        return BankAccount(self.balance + other.balance)

    def __str__(self):
        return "Balance: " + str(self.balance) + " Taka"


acc1 = BankAccount(5000)
acc2 = BankAccount(3000)
combined = acc1 + acc2
print("\nCombined account:", combined)
```

Built-in Function / Shortcut Method

```
# Common dunder methods for operators
# __add__  ->  +        __sub__   ->  -
# __mul__  ->  *        __truediv__ -> /
# __lt__   ->  <        __gt__    ->  >
# __eq__   ->  ==       __ne__    ->  !=
# __len__  ->  len()    __str__   ->  str() / print()
```

Sample Input / Output Table

Sample Input	Sample Output
None	p1 : (2, 3) p2 : (4, 5) p1 + p2 : (6, 8) p1 - p2 : (-2, -2) p2 > p1 : True p1 == p2: False Combined account: Balance: 8000 Taka

ব্যাখ্যা

`p1 + p2` লিখলে Python `p1.__add__(p2)` কল করেছে এবং নতুন একটি Point object ফেরত দিয়েছে। একইভাবে `>` ও `==` ও `+` BankAccount-এর ক্ষেত্রে কাজ করেছে। এটিই Operator Overloading।

Common Mistakes

- `__add__` method-এ নতুন object return না করা (তখন `None` আসে)।
- `other` object-এর attribute আছে কিনা যাচাই না করা (`AttributeError`)।
- `__str__` ছাড়া print করলে address দেখানো।

অনুশীলনের জন্য

1. `Vector` class বানিয়ে `__add__` ও `__mul__` overload করুন।

2. `Student` class-এ `__gt__` দিয়ে CGPA তুলনা করুন।

3. `Fraction` class বানিয়ে `__add__` overload করুন।

সমস্যা 242: Abstraction কী এবং কীভাবে করা হয় বুঝুন।

সমস্যা

সিলেবাসের 5.6 অনুযায়ী — Abstraction-এর সংজ্ঞা, গুরুত্ব এবং private method (`__method`) ব্যবহার করে জটিলতা লুকানোর প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Encapsulation
- Class & Object

- Methods

কীভাবে চিন্তা করব?

Abstraction মানে 'কী হচ্ছে' তা দেখানো, কিন্তু 'কীভাবে হচ্ছে' তা লুকানো। যেমন — আপনি `send_email()` কল করেন, কিন্তু server-এ কীভাবে connect হচ্ছে তা জানতে হয় না। পাইথনে private method (`__connect()`) দিয়ে এই জটিলতা লুকানো হয়।

অ্যালগরিদম

Step 1: একটি class তৈরি করা যার বাইরের method public।

Step 2: ভিতরের জটিল কাজগুলো private method (`__xxx`) হিসেবে লুকানো।

Step 3: public method থেকে private method কল করা।

Step 4: বাইরে থেকে private method access করতে গিয়ে ব্যর্থ হওয়া দেখানো।

Python Code

```
# Abstraction - hide complexity, show only what is needed

class EmailService:
    def __init__(self, username):
        self.username = username

    # PUBLIC method - this is what the user calls
    def send(self, to_address, message):
        self.__connect()
        self.__authenticate()
        self.__deliver(to_address, message)
        self.__disconnect()
        print("Email sent successfully!")

    # PRIVATE methods - internal complexity is hidden
    def __connect(self):
        print("(internal) Connecting to mail server...")

    def __authenticate(self):
        print("(internal) Authenticating user:", self.username)

    def __deliver(self, to_address, message):
        print("(internal) Delivering message to:", to_address)

    def __disconnect(self):
        print("(internal) Connection closed.")


# The user only needs to know about send()
mail = EmailService("teacher@polytechnic.gov.bd")
```

```
mail.send("student@example.com", "Your result is published")

print("\n--- Trying to call the private method ---")
# mail.__connect()   -> AttributeError
try:
    mail.__connect()
except AttributeError as error:
    print("AttributeError:", error)

print("\n--- Available public methods ---")
print([m for m in dir(mail) if not m.startswith("_")])
```

Built-in Function / Shortcut Method

```
# Real abstraction (with abc module) is shown in Problem 243
# Here we use private methods to hide internal steps.
```

Sample Input / Output Table

Sample Input	Sample Output
None	`--- Trying to call the private method ---`

ব্যাখ্যা

ইউজার শুধু `send()` জানে — server connect, authenticate, deliver, disconnect — এই জটিল ধাপগুলো private method-এ লুকানো আছে। বাইরে থেকে `__connect()` কল করতে গেলে `AttributeError` হয়েছে। এটিই Abstraction।

Common Mistakes

- Abstraction ও Encapsulation এক মনে করা (Abstraction = জটিলতা লুকানো, Encapsulation = ডেটা লুকানো)।
- সব method private বানিয়ে ফেলা — তখন class ব্যবহার করাই যায় না।
- private method-এর নামের আগে একটিমাত্র underscore দেওয়া (protected, private নয়)।

অনুশীলনের জন্য

1. `ATM` class বানিয়ে withdraw()-এর ভিতরের ধাপগুলো private করুন।

2. `FileUploader` class বানিয়ে internal process লুকান।

3. Abstraction ও Encapsulation-এর পার্থক্য ২ লাইনে লিখুন।

সমস্যা 243: abc Module দিয়ে Abstract Class ও Abstract Method তৈরি করুন।

সমস্যা

`abc` module ব্যবহার করে Abstract Base Class তৈরি করুন যাতে child class-গুলো নির্দিষ্ট method বাধ্যতামূলকভাবে implement করবে।

কী কী জানা থাকতে হবে?

- Inheritance
- Method overriding
- import abc

কীভাবে চিন্তা করব?

Abstract class হলো এমন class যা থেকে সরাসরি object বানানো যায় না — এটি শুধু নকশা (template)। এর ভেতরের `@abstractmethod` গুলো child class-কে অবশ্যই implement করতে হয়, নাহলে error দেয়। এটি Abstraction-এর প্রকৃত (formal) উপায়।

> **[Important]** Abstract class ব্যবহার করলে বড় প্রজেক্টে নিশ্চিত হওয়া যায় যে সব child class-এ নির্দিষ্ট method আছে। এটি Abstraction-এর আসল (formal) বাস্তবায়ন।

অ্যালগরিদম

Step 1: `from abc import ABC, abstractmethod` করা।

Step 2: `class Shape(ABC):` দিয়ে abstract class বানানো।

Step 3: ভেতরে `@abstractmethod` দিয়ে `area()` ও `perimeter()` declare করা।

Step 4: `Circle` ও `Rectangle` child class-এ দুটোই implement করা।

Step 5: abstract class থেকে object বানাতে গিয়ে error দেখানো।

Python Code

```
# Abstract class and abstract method using abc module
from abc import ABC, abstractmethod


class Shape(ABC):                   # Abstract Base Class
    def __init__(self, name):
        self.name = name

    @abstractmethod                 # child MUST implement this
    def area(self):
        pass

    @abstractmethod                 # child MUST implement this
```

```
    def perimeter(self):
        pass

    def show_name(self):          # normal (concrete) method
        print("Shape:", self.name)



class Circle(Shape):
    def __init__(self, radius):
        super().__init__("Circle")
        self.radius = radius

    def area(self):
        return 3.1416 * self.radius * self.radius

    def perimeter(self):
        return 2 * 3.1416 * self.radius



class Rectangle(Shape):
    def __init__(self, length, width):
        super().__init__("Rectangle")
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)



shapes = [Circle(5), Rectangle(10, 4)]

for shape in shapes:
    shape.show_name()
    print("  Area     :", round(shape.area(), 2))
    print("  Perimeter:", round(shape.perimeter(), 2))

print("\n--- Cannot create object of an abstract class ---")
try:
    s = Shape("Test")
except TypeError as error:
    print("TypeError:", error)
```

Built-in Function / Shortcut Method

```
# If a child class forgets to implement an abstract method:
# TypeError: Can't instantiate abstract class X with abstract method area
```

Sample Input / Output Table

Sample Input	Sample Output
None	Shape: Circle Area : 78.54 Perimeter: 31.42 Shape: Rectangle Area : 40 Perimeter: 28 --- Cannot create object of an abstract class --- TypeError: Can't instantiate abstract class Shape with abstract methods area, perimeter

ব্যাখ্যা

`Shape` একটি abstract class — এর থেকে object বানানো যায় না, error এসেছে। কিন্তু `Circle` ও `Rectangle` `area()` ও `perimeter()` implement করেছে বলে এদের object বানানো যায়। এটি নিশ্চিত করে যে সব child class-এ নির্দিষ্ট method থাকবেই।

Common Mistakes

- `class Shape(ABC):` না লিখে `@abstractmethod` ব্যবহার করা (তখন abstract method কাজ করবে না)।
- child class-এ সব abstract method implement না করা (`TypeError`)।
- abstract method-এর ভেতরে কোড লেখা (এটি শুধু declaration, body-তে `pass` থাকে)।

অনুশীলনের জন্য

1. `Payment` abstract class বানিয়ে Bkash ও Nagad child class তৈরি করুন।

2. `Employee` abstract class বানিয়ে `calculate_salary()` বাধ্যতামূলক করুন।

3. abstract class-এ একটি concrete method রেখে child-এ ব্যবহার করুন।

সমস্যা 244: Instance Method, Class Method ও Static Method-এর পার্থক্য বুঝুন।

সমস্যা

একই class-এ instance method (`self`), class method (`@classmethod`, `cls`) ও static method (`@staticmethod`) ব্যবহার করে এদের পার্থক্য দেখান।

কী কী জানা থাকতে হবে?

- Class & Object
- Class variable
- Decorator

কীভাবে চিন্তা করব?

তিন ধরনের method আছে: (১) **Instance method** — `self` পায়, object-এর ডেটা নিয়ে কাজ করে; (২) **Class method** — `@classmethod` দিয়ে হয়, `cls` পায়, class variable নিয়ে কাজ করে; (৩) **Static method** — `@staticmethod`, কোনো `self`/`cls` পায় না, শুধু utility কাজ করে।

> **[Best Practice]** যে method কখনো `self`/`cls` ব্যবহার করে না, সেটিকে `@staticmethod` বানান — এতে বোঝা যায় method-টি class-এর ডেটার ওপর নির্ভরশীল নয়।

অ্যালগরিদম

Step 1: class-এ class variable `total_students` রাখা।

Step 2: instance method দিয়ে object-এর তথ্য দেখানো।

Step 3: `@classmethod` দিয়ে class variable পরিবর্তন করা।

Step 4: `@staticmethod` দিয়ে সাধারণ utility কাজ (যেমন grade) করা।

Python Code

```
# Instance method vs Class method vs Static method

class Student:
    total_students = 0              # class variable
    institute = "Sylhet Polytechnic"

    def __init__(self, name, cgpa):
        self.name = name            # instance variable
        self.cgpa = cgpa
        Student.total_students = Student.total_students + 1

    # 1. INSTANCE METHOD - works with object data (self)
    def show_info(self):
```

```
        print("Name:", self.name, "| CGPA:", self.cgpa)

    # 2. CLASS METHOD - works with the class (cls), not the object
    @classmethod
    def total_count(cls):
        print("Total students:", cls.total_students)

    @classmethod
    def change_institute(cls, new_name):
        cls.institute = new_name
        print("Institute changed to:", cls.institute)

    # 3. STATIC METHOD - independent utility, no self / cls
    @staticmethod
    def calculate_grade(cgpa):
        if cgpa >= 3.75:
            return "A+"
        elif cgpa >= 3.00:
            return "A"
        elif cgpa >= 2.50:
            return "B"
        else:
            return "Fail"



s1 = Student("Rahim", 3.80)
s2 = Student("Karim", 3.20)

print("--- Instance method (needs an object) ---")
s1.show_info()
s2.show_info()

print("\n--- Class method (called on the class) ---")
Student.total_count()
Student.change_institute("Dhaka Polytechnic")

print("\n--- Static method (utility, no object needed) ---")
print("Grade of 3.80:", Student.calculate_grade(3.80))
print("Grade of 2.10:", Student.calculate_grade(2.10))
```

Built-in Function / Shortcut Method

```
# Quick comparison:
# instance method : def show(self)           -> uses object data
# class method    : @classmethod def f(cls)  -> uses/changes class data
# static method   : @staticmethod def f()    -> no self, no cls (utility)
```

Sample Input / Output Table

Sample Input	Sample Output

None	`--- Instance method (needs an object) ---`
	`Name: Rahim | CGPA: 3.8`
	`Name: Karim | CGPA: 3.2`
	
	`--- Class method (called on the class) ---`
	`Total students: 2`
	`Institute changed to: Dhaka Polytechnic`
	
	`--- Static method (utility, no object needed) ---`
	`Grade of 3.80: A+`
	`Grade of 2.10: Fail`

ব্যাখ্যা

instance method অবশ্যই object দিয়ে কল করতে হয় (`s1.show_info()`)। class method class-এর নামে কল করা যায় এবং class variable বদলাতে পারে। static method কোনো class/object ডেটা ছুঁয় না — শুধুই একটি helper function যা class-এর ভেতরে রাখা হয়েছে।

Common Mistakes

- class method-এ `cls` না লিখে `self` লেখা (তখন instance method হয়ে যায়)।
- static method-এ `self` বা `cls` লেখার চেষ্টা করা।
- class variable পরিবর্তনের জন্য instance method ব্যবহার করা (class method সঠিক)।

অনুশীলনের জন্য

1. `Employee` class-এ class method দিয়ে employee গোনা।

2. static method দিয়ে একটি temperature converter বানান।

3. তিন ধরনের method-এর পার্থক্য সারণি আকারে লিখুন।

Topic: Iterator, Generator & Decorators — Unit 6

সমস্যা 245: Iterator কী তা বুঝুন ও নিজে একটি Iterator তৈরি করুন।

সমস্যা

সিলেবাসের 6.1 ও 6.2 অনুযায়ী — Iterator-এর সংজ্ঞা বুঝে `__iter__()` ও `__next__()` ব্যবহার করে নিজের একটি custom Iterator class তৈরি করুন।

কী কী জানা থাকতে হবে?

- Class & Object
- for loop
- StopIteration

কীভাবে চিন্তা করব?

Iterable হলো এমন কিছু যাকে loop চালানো যায় (list, string)। Iterator হলো সেই object যা `next()` কল করলে একবারে একটি করে মান দেয় এবং শেষ হলে `StopIteration` তোলে। যেকোনো class-এ `__iter__()` (নিজেকে ফেরত দেয়) ও `__next__()` (পরের মান দেয়) থাকলে সেটি Iterator হয়ে যায়।

অ্যালগরিদম

Step 1: `class CountUpTo:` class তৈরি করা।

Step 2: `__iter__(self)` method-এ `return self` লেখা।

Step 3: `__next__(self)` method-এ পরের সংখ্যা ফেরত দেওয়া, শেষ হলে `StopIteration`।

Step 4: for loop ও `next()` দিয়ে ব্যবহার করা।

Python Code

```
# Creating a custom Iterator

class CountUpTo:
    def __init__(self, limit):
        self.limit = limit
        self.current = 1

    def __iter__(self):
        return self          # an iterator must return itself here

    def __next__(self):
        if self.current > self.limit:
            raise StopIteration      # signals "no more items"
        value = self.current
        self.current = self.current + 1
        return value
```

```
# Using our iterator with a for loop
print("Using for loop:")
counter = CountUpTo(5)
for number in counter:
    print(number)

# Using next() manually
print("\nUsing next() manually:")
counter2 = CountUpTo(3)
print(next(counter2))
print(next(counter2))
print(next(counter2))

try:
    print(next(counter2))     # no more items
except StopIteration:
    print("StopIteration: no more values!")

# Built-in iterators
print("\n--- Built-in iterator example ---")
my_list = [10, 20, 30]
list_iterator = iter(my_list)
print(type(list_iterator))
print(next(list_iterator))
print(next(list_iterator))
```

Built-in Function / Shortcut Method

```
# Everything iterable has an iterator behind the scenes
# it = iter([1, 2, 3])
# next(it) -> 1, next(it) -> 2, next(it) -> 3, next(it) -> StopIteration
```

Sample Input / Output Table

Sample Input	Sample Output
None	Using for loop: 1 2 3 4 5 Using next() manually:

```
1
2
3
StopIteration: no more values!

--- Built-in iterator example ---
<class 'list_iterator'>
10
20
```

ব্যাখ্যা

`for` loop ভিতরে ভিতরে `iter()` ও `next()` কল করে চলতে থাকে যতক্ষণ না `StopIteration` আসে। আমাদের `CountUpTo` class একই নিয়ম মেনে বানানো হয়েছে, তাই এটি সাধারণ list-এর মতোই for loop-এ ব্যবহার করা গেছে।

Common Mistakes

- `__next__()` কখনো `StopIteration` না তোলা — তখন অসীম লুপ চলবে।
- `__iter__()`-এ `return self` না লেখা।
- `__next__`-এর বদলে ভুলে `next` (আন্ডারস্কোর ছাড়া) লেখা।

অনুশীলনের জন্য

1. একটি Iterator বানান যা জোড় সংখ্যা দেয়।

2. একটি Iterator বানান যা string-এর অক্ষর একে একে দেয়।

3. `iter()` ও `next()` দিয়ে একটি tuple manually iterate করুন।

সমস্যা 246: Generator Function তৈরি করুন (yield keyword)।

সমস্যা

সিলেবাসের 6.3 অনুযায়ী — `yield` keyword ব্যবহার করে একটি Generator function তৈরি করুন যা মেমোরি সাশ্রয় করে ধারাবাহিকভাবে মান দেয়।

কী কী জানা থাকতে হবে?

- Function
- for loop
- Iterator ধারণা

কীভাবে চিন্তা করব?

Generator হলো `yield` ব্যবহার করা একটি সহজ function, যা প্রতিবার কল হলে থেমে যায় এবং পরের বার সেখান থেকেই চালু হয়। এটি স্বয়ংক্রিয়ভাবে Iterator তৈরি করে দেয় — `__iter__`/`__next__` নিজে লিখতে হয় না।

> **[Best Practice]** বড় ডেটাসেটে (যেমন লাখো রেকর্ডের ফাইল) generator ব্যবহার করুন — এটি একসাথে সব মেমোরিতে না রেখে একটি একটি করে দেয়, মেমোরি সাশ্রয় হয়।

অ্যালগরিদম

Step 1: `def count_up_to(limit):` function বানানো।

Step 2: `yield number` দিয়ে একটি একটি মান পাঠানো।

Step 3: for loop দিয়ে generator ব্যবহার করা।

Step 4: `next()` দিয়ে manual ব্যবহার দেখানো।

Python Code

```
# Generator function using yield

def count_up_to(limit):
    current = 1
    while current <= limit:
        yield current          # pauses here, remembers state
        current = current + 1


print("Using for loop:")
for number in count_up_to(5):
    print(number)

print("\nUsing next() manually:")
gen = count_up_to(3)
print(type(gen))
print(next(gen))
print(next(gen))
print(next(gen))

try:
    print(next(gen))
except StopIteration:
    print("StopIteration: generator finished!")

# Generator for even numbers (memory-friendly for large ranges)
def even_numbers(limit):
```

```
    for n in range(2, limit + 1, 2):
        yield n


print("\nEven numbers up to 10:")
for n in even_numbers(10):
    print(n, end=" ")
print()
```

Built-in Function / Shortcut Method

```
# Generator expression (shortcut, like list comprehension)
# squares = (x * x for x in range(1, 6))
# print(list(squares))
```

Sample Input / Output Table

Sample Input	Sample Output
None	Using for loop: 1 2 3 4 5 Using next() manually: <class 'generator'> 1 2 3 StopIteration: generator finished! Even numbers up to 10: 2 4 6 8 10

ব্যাখ্যা

`yield` কল হলেই function থেমে যায় এবং সেই মান ফেরত দেয়, কিন্তু পরের বার `next()` কল করলে ঠিক যেখানে থেমেছিল সেখান থেকেই চালু হয়। এটি Iterator বানানোর চেয়ে অনেক সহজ ও কম কোডে।

Common Mistakes

- `yield`-এর বদলে `return` লেখা (তখন এটি generator থাকে না, সাধারণ function হয়ে যায়)।
- generator একবার শেষ হয়ে গেলে আবার ব্যবহার করার চেষ্টা করা (নতুন করে কল করতে হবে)।
- generator-কে সরাসরি print করা (generator object দেখাবে, মান নয়)।

অনুশীলনের জন্য

1. একটি generator বানান যা Fibonacci সিরিজ দেয়।

2. একটি generator বানান যা 1 থেকে n পর্যন্ত সংখ্যার বর্গ দেয়।

3. generator expression দিয়ে জোড় সংখ্যার তালিকা বানান।

সমস্যা 247: Generator দিয়ে বড় ফাইল Line-by-Line প্রসেস করুন।

সমস্যা

Generator ব্যবহার করে একটি বড় ফাইলের লাইন ফিল্টার করে দেখানোর মেমোরি-সাশ্রয়ী প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Generator function
- File handling
- yield

কীভাবে চিন্তা করব?

ফাইল read করার সময় `readlines()` পুরো ফাইল মেমোরিতে লোড করে — বড় ফাইলে এটি বিপজ্জনক। Generator ব্যবহার করলে একবারে একটি লাইন প্রসেস হয়, মেমোরি বাঁচে। এটি বাস্তব প্রজেক্টে (লগ ফাইল বিশ্লেষণ) খুব দরকারি।

অ্যালগরিদম

Step 1: একটি sample ফাইল তৈরি করা।

Step 2: `def read_lines_containing(filename, keyword):` generator বানানো।

Step 3: প্রতিটি লাইনে keyword থাকলে `yield` করা।

Step 4: for loop দিয়ে ফলাফল প্রিন্ট করা।

Python Code

```
# Generator for memory-efficient file processing

# Step 1: create a sample log file
with open("activity_log.txt", "w") as file:
    file.write("INFO: System started\n")
    file.write("ERROR: File not found\n")
    file.write("INFO: User logged in\n")
```

```
    file.write("ERROR: Connection timeout\n")
    file.write("INFO: Task completed\n")



# Step 2: a generator that yields only matching lines
def read_lines_containing(filename, keyword):
    with open(filename, "r") as file:
        for line in file:
            if keyword in line:
                yield line.strip()



# Step 3: use the generator - file is read line by line, not all at once
print("--- ERROR lines only ---")
for error_line in read_lines_containing("activity_log.txt", "ERROR"):
    print(error_line)

print("\n--- INFO lines only ---")
for info_line in read_lines_containing("activity_log.txt", "INFO"):
    print(info_line)

# Count errors without loading everything into a list
error_count = sum(1 for _ in read_lines_containing("activity_log.txt",
"ERROR"))
print("\nTotal ERROR lines:", error_count)
```

Built-in Function / Shortcut Method

```
# Non-generator (memory heavy) alternative for comparison:
# with open("activity_log.txt") as f:
#     lines = f.readlines()          # loads EVERYTHING into memory
# errors = [l for l in lines if "ERROR" in l]
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- ERROR lines only --- ERROR: File not found ERROR: Connection timeout --- INFO lines only --- INFO: System started INFO: User logged in INFO: Task completed

```
                                        Total ERROR lines: 2
```

ব্যাখ্যা

`read_lines_containing()` generator একবারে একটি লাইন পড়ে, keyword মিললে `yield` করে দেয়। পুরো ফাইল একসাথে মেমোরিতে লোড হয় না — লাখো লাইনের ফাইলেও এটি কার্যকর।

Common Mistakes

- generator-এর ভিতরে `with open()` ব্যবহার না করে ফাইল খোলা রেখে দেওয়া।
- generator-কে বারবার একই ভ্যারিয়েবলে reuse করার চেষ্টা করা (নতুন কল দরকার)।
- `sum(1 for _ in generator)` না বুঝে generator-কে `len()` দিয়ে গোনার চেষ্টা করা (generator-এর `len()` নেই)।

অনুশীলনের জন্য

1. generator দিয়ে ফাইলের শুধু ফাঁকা লাইন বাদ দিয়ে পড়ুন।

2. generator দিয়ে CSV ফাইলের নির্দিষ্ট শর্তের row ফিল্টার করুন।

3. generator দিয়ে ফাইলের প্রতিটি লাইনের শব্দ গণনা করুন।

সমস্যা 248: Decorator কী তা বুঝুন ও একটি সাধারণ Decorator তৈরি করুন।

সমস্যা

সিলেবাসের 6.1 ও 6.4 অনুযায়ী — Decorator-এর সংজ্ঞা বুঝে একটি function-কে না বদলে তার আচরণ বাড়ানোর (যেমন লগ প্রিন্ট) প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- Function
- *args, **kwargs
- Function as argument

কীভাবে চিন্তা করব?

Decorator হলো এমন একটি function যা আরেকটি function-কে 'wrap' করে অতিরিক্ত কাজ যোগ করে, মূল function-এর কোড না বদলে। `@decorator_name` লিখে function-এর ঠিক উপরে বসালে সেটি স্বয়ংক্রিয়ভাবে প্রয়োগ হয়।

অ্যালগরিদম

Step 1: `def my_decorator(func):` function বানানো।

Step 2: ভিতরে `def wrapper(*args, **kwargs):` বানানো যা func-এর আগে-পরে কাজ করে।

Step 3: `return wrapper` করা।

Step 4: `@my_decorator` লিখে আসল function-এর উপরে বসানো।

Python Code

```
# Creating a simple decorator

def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before the function runs...")
        result = func(*args, **kwargs)
        print("After the function runs...")
        return result
    return wrapper



@my_decorator
def greet(name):
    print("Hello,", name)



greet("Rahim")

print("\n--- Without the @ syntax (same thing) ---")
def say_bye(name):
    print("Bye,", name)



decorated_bye = my_decorator(say_bye)
decorated_bye("Karim")

print("\n--- Decorator that returns a value ---")
@my_decorator
def add(a, b):
    return a + b



result = add(5, 10)
print("Result received:", result)
```

Built-in Function / Shortcut Method

```
# Decorator in one line explanation:
# @my_decorator
# def greet(name): ...
```

```
#
# is exactly the same as:
# def greet(name): ...
# greet = my_decorator(greet)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Before the function runs... Hello, Rahim After the function runs... --- Without the @ syntax (same thing) --- Before the function runs... Bye, Karim After the function runs... --- Decorator that returns a value --- Before the function runs... After the function runs... Result received: 15

ব্যাখ্যা

`@my_decorator` আসলে `greet = my_decorator(greet)` লেখারই শর্টকাট। `wrapper()` ফাংশনটি মূল function-কে ঘিরে (wrap করে) আগে-পরে অতিরিক্ত কাজ করেছে, কিন্তু `greet()`-এর নিজের কোড পাল্টায়নি।

Common Mistakes

- `wrapper` function-এ `*args, **kwargs` না দেওয়া — তখন argument-যুক্ত function কাজ করবে না।
- `return wrapper` না লেখা — তখন decorator `None` ফেরত দেয়।
- মূল function-এর return value `wrapper`-এ return না করা।

অনুশীলনের জন্য

1. একটি decorator বানান যা function কতবার কল হলো তা গোনে।

2. একটি decorator বানান যা function-এর ইনপুট ও আউটপুট প্রিন্ট করে।

3. একটি decorator বানান যা শুধু positive সংখ্যায় function চালায়।

সমস্যা 249: Decorator দিয়ে Function-এর Execution Time পরিমাপ করুন।

সমস্যা

সিলেবাসের 6.5 অনুযায়ী — একটি বাস্তব-জীবনের decorator তৈরি করুন যা কোনো function চলতে কত সময় নিলো তা মাপে।

কী কী জানা থাকতে হবে?

- Decorator
- time module
- *args, **kwargs

কীভাবে চিন্তা করব?

`time.time()` বর্তমান সময় (সেকেন্ডে) দেয়। function চালানোর আগে ও পরে সময় নিয়ে বিয়োগ করলে execution time পাওয়া যায়। Decorator দিয়ে এই কাজটি যেকোনো function-এ পুনর্ব্যবহারযোগ্যভাবে যোগ করা যায় — কোড বারবার লিখতে হয় না।

> **[Best Practice]** প্রফেশনাল কোডে decorator লেখার সময় `from functools import wraps` ব্যবহার করে `@wraps(func)` যোগ করুন — এতে মূল function-এর নাম ও docstring অক্ষত থাকে।

অ্যালগরিদম

Step 1: `import time` করা।

Step 2: `timer_decorator(func)` বানানো।

Step 3: wrapper-এ শুরুর সময় ও শেষের সময় নিয়ে পার্থক্য প্রিন্ট করা।

Step 4: `@timer_decorator` দিয়ে একটি ভারী কাজের function-এ প্রয়োগ করা।

Python Code

```
# Decorator to measure execution time
import time


def timer_decorator(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        elapsed = end_time - start_time
        print(func.__name__, "took", round(elapsed, 4), "seconds")
        return result
```

```
    return wrapper


@timer_decorator
def calculate_sum(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total


@timer_decorator
def slow_function():
    time.sleep(1)          # simulate a slow task
    print("Slow function finished")


result = calculate_sum(1000000)
print("Sum:", result)

print()
slow_function()
```

Built-in Function / Shortcut Method

```
# For simple timing without a decorator:
# start = time.time()
# ... code ...
# print("Time:", time.time() - start)
```

Sample Input / Output Table

Sample Input	Sample Output
None	calculate_sum took 0.0532 seconds Sum: 500000500000 Slow function finished slow_function took 1.0011 seconds

ব্যাখ্যা

প্রতিটি ফাংশনের আগে-পরে সময় নেওয়ায় execution time বের করা গেছে। `@timer_decorator` লিখলেই যেকোনো function-এ এই সুবিধা যোগ হয়ে যায় — বারবার একই কোড লিখতে হয় না। এটি বাস্তব প্রজেক্টে performance monitoring-এ ব্যবহৃত হয়।

Common Mistakes

- `func.__name__` না ব্যবহার করে wrapper-এর নাম দেখানো (decorator ব্যবহারের ফলে সব function-এর নাম 'wrapper' দেখাবে)।
- `time.time()` ব্যবহারের সময় unit ভুল করা (এটি সেকেন্ডে, মিলিসেকেন্ডে নয়)।
- একাধিক decorator একসাথে ব্যবহার করার সময় ক্রম গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. একটি decorator বানান যা function-এর result ফাইলে লগ করে।

2. `functools.wraps` ব্যবহার করে `func.__name__` ঠিক রাখুন।

3. দুইটি ভিন্ন algorithm-এর সময় তুলনা করুন decorator দিয়ে।

সমস্যা 250: Decorator দিয়ে Input Validation করুন।

সমস্যা

একটি decorator তৈরি করুন যা function-এ পাঠানো argument-গুলো ভুল (যেমন negative number) হলে function চালানোর আগেই আটকে দেয়।

কী কী জানা থাকতে হবে?

- Decorator
- *args, **kwargs
- if-else

কীভাবে চিন্তা করব?

Validation-এর কোড প্রতিটি function-এ বারবার লেখার বদলে একটি decorator বানিয়ে দিলে একবার লিখেই বহু জায়গায় ব্যবহার করা যায়। wrapper-এ argument চেক করে, ঠিক থাকলে তবেই মূল function কল করা হয়।

অ্যালগরিদম

Step 1: `validate_positive(func)` decorator বানানো।

Step 2: wrapper-এ সব argument positive কিনা যাচাই করা।

Step 3: ভুল থাকলে ValueError তোলা, ঠিক থাকলে func কল করা।

Step 4: `@validate_positive` কয়েকটি function-এ প্রয়োগ করা।

Python Code

```
# Decorator for input validation

def validate_positive(func):
    def wrapper(*args, **kwargs):
        for value in args:
            if isinstance(value, (int, float)) and value < 0:
                raise ValueError("All numbers must be positive!")
        return func(*args, **kwargs)
    return wrapper



@validate_positive
def calculate_area(length, width):
    return length * width



@validate_positive
def calculate_square_root_area(side):
    return side * side



print("Valid input:")
print("Area:", calculate_area(10, 5))

print("\nInvalid input:")
try:
    print("Area:", calculate_area(-10, 5))
except ValueError as error:
    print("Error:", error)

print("\nAnother valid call:")
print("Square area:", calculate_square_root_area(6))

try:
    calculate_square_root_area(-3)
except ValueError as error:
    print("Error:", error)
```

Built-in Function / Shortcut Method

```
# You can also validate types
# def validate_type(func):
#     def wrapper(*args):
#         for a in args:
```

```
#             if not isinstance(a, (int, float)):
#                 raise TypeError("Only numbers allowed")
#         return func(*args)
#     return wrapper
```

Sample Input / Output Table

Sample Input	Sample Output
None	Valid input: Area: 50 Invalid input: Error: All numbers must be positive! Another valid call: Square area: 36 Error: All numbers must be positive!

ব্যাখ্যা

decorator মূল function চালানোর আগেই সব argument যাচাই করে। ভুল মান পেলে `ValueError` তুলে function-টি একদম চালায়নি — এতে ভুল ডেটা নিয়ে হিসাব হওয়া আটকানো গেছে।

Common Mistakes

- শুধু প্রথম argument যাচাই করে বাকিগুলো বাদ দেওয়া।
- keyword argument (`kwargs`) যাচাই করতে ভুলে যাওয়া।
- validation ব্যর্থ হলে exception না তুলে শুধু print করে function চালিয়ে দেওয়া।

অনুশীলনের জন্য

1. একটি decorator বানান যা string argument ফাঁকা কিনা যাচাই করে।

2. একটি decorator বানান যা কেবলমাত্র নির্দিষ্ট range-এর মান গ্রহণ করে।

3. দুইটি decorator একসাথে ব্যবহার করুন (validation + timer)।

Topic: Exception & Error Handling in Python — Unit 7

সমস্যা 251: Exception কী এবং Syntax Error-এর সাথে পার্থক্য বুঝুন।

সমস্যা

সিলেবাসের 7.1 অনুযায়ী — Exception কী, Syntax Error ও Runtime Exception-এর মধ্যে পার্থক্য কী — তা উদাহরণসহ ব্যাখ্যা করুন।

কী কী জানা থাকতে হবে?

- Basic Python syntax
- print, input

কীভাবে চিন্তা করব?

Syntax Error হয় কোড লেখার নিয়ম ভাঙলে — প্রোগ্রাম শুরুই হয় না। Exception হয় প্রোগ্রাম চলাকালীন (runtime) কোনো অস্বাভাবিক পরিস্থিতিতে — যেমন শূন্য দিয়ে ভাগ, ভুল টাইপ। Exception হ্যান্ডেল করা যায়, Syntax Error আগেই ঠিক করতে হয়।

অ্যালগরিদম

Step 1: একটি Syntax Error-এর উদাহরণ (কমেন্ট আকারে) দেখানো।

Step 2: একটি Runtime Exception (ZeroDivisionError) ঘটিয়ে দেখানো।

Step 3: `type(error)` দিয়ে exception-এর নাম বের করা।

Step 4: পার্থক্য সারণি আকারে ব্যাখ্যা করা।

Python Code

```
# Understanding Exception vs Syntax Error

# --- Syntax Error example (this would NOT even run) ---
# print("Hello"          <- missing closing bracket = SyntaxError
# if True               <- missing colon = SyntaxError
#     print("test")

# --- Runtime Exception example (code IS correct syntax-wise) ---
numbers = [10, 20, 30]

print("Program started correctly (no syntax error).")

try:
    result = numbers[5]          # valid syntax, but wrong index
except IndexError as error:
    print("Exception caught:", type(error).__name__, "-", error)

try:
```

```
    value = 10 / 0                # valid syntax, but division by zero
except ZeroDivisionError as error:
    print("Exception caught:", type(error).__name__, "-", error)

try:
    number = int("hello")         # valid syntax, but invalid conversion
except ValueError as error:
    print("Exception caught:", type(error).__name__, "-", error)

print("\nProgram continued running after handling exceptions!")
```

Built-in Function / Shortcut Method

```
# Key difference:
# SyntaxError -> found BEFORE running, program does not start at all
# Exception   -> found WHILE running, program can recover with try-except
```

Sample Input / Output Table

Sample Input	Sample Output
None	Program started correctly (no syntax error). Exception caught: IndexError - list index out of range Exception caught: ZeroDivisionError - division by zero Exception caught: ValueError - invalid literal for int() with base 10: 'hello' Program continued running after handling exceptions!

ব্যাখ্যা

তিনটি ভুল কোডে syntax ঠিক ছিল, তাই প্রোগ্রাম শুরু হয়েছে; কিন্তু চলার সময় অস্বাভাবিক পরিস্থিতি ঘটায় exception এসেছে। `try-except` দিয়ে সেগুলো ধরে প্রোগ্রামকে ক্র্যাশ হতে দেওয়া হয়নি।

Common Mistakes

- Syntax Error-কে try-except দিয়ে ধরার চেষ্টা করা — এটি সম্ভব নয়, syntax error আগে ঠিক করতে হয়।

- সব ভুলকে 'Error' বলে এক করে ফেলা — নির্দিষ্ট exception-এর নাম জানা জরুরি।
- exception ধরেও কারণ analyze না করে শুধু 'Error হয়েছে' লেখা।

অনুশীলনের জন্য

1. একটি Syntax Error ও একটি Runtime Exception-এর উদাহরণ লিখুন।

2. ৩টি ভিন্ন exception ইচ্ছাকৃতভাবে ঘটিয়ে ধরুন।

3. `type(error).__name__` ব্যবহার করে exception-এর নাম বের করুন।

সমস্যা 252: Python-এর সাধারণ Built-in Exception গুলো চিহ্নিত করুন।

সমস্যা

সিলেবাসের 7.2 অনুযায়ী — `ZeroDivisionError`, `ValueError`, `TypeError`, `IndexError`, `KeyError`, `FileNotFoundError`, `NameError`, `AttributeError` — এই common exception-গুলো ঘটিয়ে চিহ্নিত করুন।

কী কী জানা থাকতে হবে?

- try-except
- Data types
- List, Dictionary

কীভাবে চিন্তা করব?

প্রতিটি exception একটি নির্দিষ্ট সমস্যার জন্য হয়। এগুলো চেনা থাকলে error message পড়েই বোঝা যায় কোথায় ভুল। একটি সারণি তৈরি করে প্রতিটি exception কখন হয় তা মনে রাখা সহজ হয়।

অ্যালগরিদম

Step 1: প্রতিটি common exception আলাদা try-except ব্লকে ঘটানো।

Step 2: exception-এর নাম ও বার্তা প্রিন্ট করা।

Step 3: একটি সারণি আকারে সারাংশ তৈরি করা।

Python Code

```
# Common built-in exceptions in Python

# 1. ZeroDivisionError
try:
    print(10 / 0)
except ZeroDivisionError as e:
    print("ZeroDivisionError:", e)

# 2. ValueError
```

```
try:
    print(int("abc"))
except ValueError as e:
    print("ValueError:", e)

# 3. TypeError
try:
    print("5" + 5)
except TypeError as e:
    print("TypeError:", e)

# 4. IndexError
try:
    my_list = [1, 2, 3]
    print(my_list[10])
except IndexError as e:
    print("IndexError:", e)

# 5. KeyError
try:
    student = {"name": "Rahim"}
    print(student["roll"])
except KeyError as e:
    print("KeyError: Key", e, "not found")

# 6. FileNotFoundError
try:
    open("no_such_file.txt", "r")
except FileNotFoundError as e:
    print("FileNotFoundError:", e)

# 7. NameError
try:
    print(undefined_variable)
except NameError as e:
    print("NameError:", e)

# 8. AttributeError
try:
    number = 5
    number.append(10)
except AttributeError as e:
    print("AttributeError:", e)
```

Built-in Function / Shortcut Method

```
# Exception summary table
```

```
# ZeroDivisionError   -> division by zero
# ValueError          -> right type, wrong value (int("abc"))
# TypeError           -> wrong type used in an operation
# IndexError          -> list index out of range
# KeyError            -> dictionary key not found
# FileNotFoundError   -> file does not exist
# NameError           -> variable not defined
# AttributeError      -> method/attribute does not exist for that type
```

Sample Input / Output Table

Sample Input	Sample Output
None	ZeroDivisionError: division by zero ValueError: invalid literal for int() with base 10: 'abc' TypeError: can only concatenate str (not "int") to str IndexError: list index out of range KeyError: Key 'roll' not found FileNotFoundError: [Errno 2] No such file or directory: 'no_such_file.txt' NameError: name 'undefined_variable' is not defined AttributeError: 'int' object has no attribute 'append'

ব্যাখ্যা

প্রতিটি exception ভিন্ন কারণে হয় — এই আটটি সবচেয়ে বেশি দেখা যায়। error message-এর শেষ লাইন পড়েই বোঝা যায় exception-এর নাম ও কারণ।

Common Mistakes

- `ValueError` ও `TypeError` গুলিয়ে ফেলা।
- `KeyError` message-এ শুধু key-টিই দেখায়, পুরো বাক্য নয় — তাই নিজে থেকে বার্তা যোগ করা ভালো।
- সব exception-এর জন্য একটিই generic `except:` ব্যবহার করে exact কারণ না জানা।

অনুশীলনের জন্য

1. ৩টি নতুন ধরনের exception (`ModuleNotFoundError`, `OverflowError`, `RecursionError`) ঘটিয়ে দেখুন।

2. একটি প্রোগ্রামে ইচ্ছাকৃতভাবে ৫টি ভিন্ন exception তৈরি করুন।

3. exception-এর নাম দেখে কারণ অনুমান করার একটি quiz বানান।

সমস্যা 253: try-except ব্যবহার করে Exception হ্যান্ডেল করুন।

সমস্যা

সিলেবাসের 7.4 অনুযায়ী — `try` ও `except` ব্লক ব্যবহার করে একটি ক্যালকুলেটর প্রোগ্রাম লিখুন যা ভুল ইনপুট দিলেও ক্র্যাশ করবে না।

কী কী জানা থাকতে হবে?

- Function
- input()
- Built-in exceptions

কীভাবে চিন্তা করব?

`try` ব্লকের ভেতরে ঝুঁকিপূর্ণ কোড রাখা হয়। কোনো সমস্যা হলে প্রোগ্রাম সাথে সাথে `except` ব্লকে চলে যায়, ক্র্যাশ করে না। একাধিক exception-এর জন্য একাধিক `except` ব্লক রাখা যায়।

অ্যালগরিদম

Step 1: ইউজারের কাছ থেকে দুইটি সংখ্যা ও অপারেটর নেওয়া।

Step 2: `try` ব্লকে হিসাব করা।

Step 3: `ZeroDivisionError` ও `ValueError` আলাদা `except`-এ ধরা।

Step 4: ফলাফল বা error message প্রিন্ট করা।

Python Code

```
# try-except for a simple calculator

def calculator():
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        operator = input("Enter operator (+, -, *, /): ")

        if operator == "+":
            result = num1 + num2
        elif operator == "-":
```

```
            result = num1 - num2
        elif operator == "*":
            result = num1 * num2
        elif operator == "/":
            result = num1 / num2
        else:
            print("Invalid operator!")
            return

        print("Result:", result)

    except ValueError:
        print("Error: Please enter valid numbers only!")
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")



print("--- Test 1: valid input ---")
calculator()

print("\n--- Test 2: division by zero ---")
calculator()

print("\n--- Test 3: invalid number ---")
calculator()
```

Built-in Function / Shortcut Method

```
# Catching multiple exceptions in one line
# except (ValueError, ZeroDivisionError) as e:
#     print("Error:", e)
```

Sample Input / Output Table

Sample Input	Sample Output
10 5 + 10 0 / abc	--- Test 1: valid input --- Enter first number: 10 Enter second number: 5 Enter operator (+, -, *, /): + Result: 15.0 --- Test 2: division by zero --- Enter first number: 10 Enter second number: 0

```
5                                   Enter operator (+, -, *, /): /
+                                   Error: Cannot divide by zero!

                                    --- Test 3: invalid number ---
                                    Enter first number: abc
                                    Error: Please enter valid numbers
                                    only!
```

ব্যাখ্যা

তিনটি টেস্টেই প্রোগ্রাম ক্র্যাশ করেনি। ভুল ইনপুট দিলে `ValueError`, শূন্য দিয়ে ভাগ করলে `ZeroDivisionError` — প্রতিটি নির্দিষ্ট `except` ব্লকে গিয়ে সঠিক বার্তা দেখিয়েছে।

Common Mistakes

- `except:` (কোনো exception টাইপ ছাড়া) ব্যবহার করা — এটি সব ভুল আড়াল করে ফেলে, ডিবাগ করা কঠিন হয়।
- try ব্লকে অপ্রয়োজনীয় বেশি কোড রাখা — শুধু ঝুঁকিপূর্ণ অংশটুকু রাখা উচিত।
- except ব্লকে কিছু না করে `pass` লিখে চুপচাপ ভুল উপেক্ষা করা।

অনুশীলনের জন্য

1. একটি বয়স ইনপুট প্রোগ্রাম বানান যা non-numeric ইনপুট handle করে।

2. একটি list index access প্রোগ্রাম বানান যা IndexError handle করে।

3. একটি ফাইল read প্রোগ্রাম বানান যা FileNotFoundError handle করে।

সমস্যা 254: একাধিক except ব্লক ও Generic Exception ব্যবহার করুন।

সমস্যা

একাধিক নির্দিষ্ট `except` ব্লকের পর একটি generic `except Exception` ব্লক রেখে যেকোনো অজানা ভুলও ধরার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- try-except
- Built-in exceptions

কীভাবে চিন্তা করব?

নির্দিষ্ট exception-গুলো আগে ধরে ফেলা ভালো, কারণ সেগুলোর জন্য নির্দিষ্ট সমাধান দেওয়া যায়। কিন্তু অপ্রত্যাশিত কোনো ভুলের জন্য সবশেষে `except Exception as e:` রাখলে প্রোগ্রাম কখনোই ক্র্যাশ করবে না।

অ্যালগরিদম

Step 1: একটি function-এ একাধিক ধরনের ভুল হতে পারে এমন কোড লেখা।

Step 2: নির্দিষ্ট exception-গুলো আগে ধরা (ValueError, KeyError)।

Step 3: সবশেষে generic `except Exception as e:` রাখা।

Step 4: বিভিন্ন ইনপুট দিয়ে যাচাই করা।

Python Code

```
# Multiple except blocks + a generic Exception catch-all

def process_student(data, key):
    try:
        roll = int(data[key])
        print("Roll number:", roll)
        return 100 / roll        # just an example calculation

    except KeyError:
        print("Error: The key", key, "does not exist in the data.")

    except ValueError:
        print("Error: Roll number must be numeric.")

    except ZeroDivisionError:
        print("Error: Roll number cannot be zero.")

    except Exception as e:
        # Catches ANY other unexpected error
        print("Unexpected error occurred:", type(e).__name__, "-", e)



student = {"name": "Rahim", "roll": "101"}

print("--- Test 1: valid data ---")
process_student(student, "roll")

print("\n--- Test 2: wrong key ---")
process_student(student, "id")

print("\n--- Test 3: non-numeric roll ---")
process_student({"roll": "abc"}, "roll")

print("\n--- Test 4: roll is zero ---")
process_student({"roll": "0"}, "roll")

print("\n--- Test 5: unexpected error (passing wrong data type) ---")
```

```
process_student(None, "roll")
```

Built-in Function / Shortcut Method

```
# Order matters! Put specific exceptions FIRST,
# then the generic Exception LAST.
# (Python checks except blocks top to bottom)
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Test 1: valid data --- Roll number: 101 --- Test 2: wrong key --- Error: The key id does not exist in the data. --- Test 3: non-numeric roll --- Error: Roll number must be numeric. --- Test 4: roll is zero --- Error: Roll number cannot be zero. --- Test 5: unexpected error (passing wrong data type) --- Unexpected error occurred: TypeError - 'NoneType' object is not subscriptable

ব্যাখ্যা

প্রতিটি চেনা ভুলের জন্য নির্দিষ্ট বার্তা দেখানো হয়েছে, আর একদম শেষের `except Exception as e:` অপ্রত্যাশিত `TypeError`-ও ধরে ফেলেছে। এভাবে প্রোগ্রাম কখনোই হুট করে ক্র্যাশ করবে না।

Common Mistakes

- generic `except Exception` কে সবার আগে রাখা — তাহলে নির্দিষ্ট except ব্লকগুলো কখনো চলবেই না।

- `except:` (কোনো টাইপ ছাড়া) ব্যবহার করা — `except Exception as e:` ব্যবহার করাই ভালো, কারণ এতে `KeyboardInterrupt`-এর মতো সিস্টেম exception বাদ থাকে।
- generic except ব্লকে exception-এর প্রকৃত কারণ প্রিন্ট না করা।

অনুশীলনের জন্য

1. একটি ব্যাংকিং function বানান যেখানে ৩টি নির্দিষ্ট exception ও একটি generic exception থাকবে।

2. generic except ব্লকে exception-এর টাইপ ও বার্তা লগ ফাইলে সেভ করুন।

3. ভুল ক্রমে except ব্লক রেখে দেখুন কী সমস্যা হয়।

সমস্যা 255: raise Keyword দিয়ে নিজে Exception তৈরি করুন।

সমস্যা

সিলেবাসের 7.3 অনুযায়ী — `raise` keyword ব্যবহার করে নির্দিষ্ট শর্তে ইচ্ছাকৃতভাবে exception তৈরি (raise) করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- try-except
- if-else
- Function

কীভাবে চিন্তা করব?

মাঝে মাঝে Python নিজে থেকে exception না দিলেও, ব্যবসায়িক নিয়ম (business rule) অনুযায়ী আমাদেরই exception 'ছুড়তে' হয়। যেমন — বয়স নেগেটিভ হতে পারে না, যদিও এটি কোনো built-in error নয়। `raise ValueError("message")` দিয়ে নিজে থেকে exception তৈরি করা যায়।

অ্যালগরিদম

Step 1: `set_age(age)` function বানানো।

Step 2: শর্ত ভুল হলে `raise ValueError("...")` করা।

Step 3: কল করার সময় `try-except` দিয়ে ধরা।

Step 4: বিভিন্ন মান দিয়ে যাচাই করা।

Python Code

```
# Using raise to create custom exceptions manually

def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    if age > 120:
        raise ValueError("Age seems unrealistic!")
```

```
    return age


def withdraw_money(balance, amount):
    if amount > balance:
        raise Exception("Insufficient balance for this withdrawal!")
    return balance - amount


print("--- Test 1: valid age ---")
try:
    age = set_age(25)
    print("Age set to:", age)
except ValueError as e:
    print("Error:", e)

print("\n--- Test 2: negative age ---")
try:
    age = set_age(-5)
    print("Age set to:", age)
except ValueError as e:
    print("Error:", e)

print("\n--- Test 3: unrealistic age ---")
try:
    age = set_age(200)
except ValueError as e:
    print("Error:", e)

print("\n--- Test 4: withdrawal ---")
try:
    new_balance = withdraw_money(1000, 1500)
    print("New balance:", new_balance)
except Exception as e:
    print("Error:", e)
```

Built-in Function / Shortcut Method

```
# raise with no message re-raises the current exception (used inside except)
# except ValueError:
#     print("Logged the error")
#     raise            # re-raise so caller also knows
```

Sample Input / Output Table

Sample Input	Sample Output

None	--- Test 1: valid age --- Age set to: 25 --- Test 2: negative age --- Error: Age cannot be negative! --- Test 3: unrealistic age --- Error: Age seems unrealistic! --- Test 4: withdrawal --- Error: Insufficient balance for this withdrawal!

ব্যাখ্যা

`raise ValueError("...")` দিয়ে ইচ্ছাকৃতভাবে exception তৈরি করা হয়েছে যখন ব্যবসায়িক নিয়ম ভঙ্গ হয়েছে (নেগেটিভ বয়স, অপর্যাপ্ত ব্যালেন্স)। এভাবে কোডে নিজস্ব নিয়ম প্রয়োগ করা যায়।

Common Mistakes

- `raise` এর পর message string না দেওয়া — ডিবাগ করা কঠিন হয়।
- সব ধরনের ভুলের জন্য generic `Exception` ব্যবহার করা — নির্দিষ্ট exception টাইপ (ValueError, TypeError) ব্যবহার করা ভালো।
- `raise` করা exception-কে try-except দিয়ে ধরতে ভুলে যাওয়া (প্রোগ্রাম ক্র্যাশ করবে)।

অনুশীলনের জন্য

1. একটি function বানান যা শূন্যের বেশি মার্কস না হলে exception তোলে।

2. একটি function বানান যা phone number ১১ ডিজিট না হলে exception তোলে।

3. raise করা exception try-except দিয়ে ধরে বার্তা প্রিন্ট করুন।

সমস্যা 256: নিজস্ব Custom Exception Class তৈরি করুন।

সমস্যা

`Exception` class থেকে inherit করে নিজস্ব একটি Custom Exception class তৈরি করুন এবং তা raise ও handle করুন।

কী কী জানা থাকতে হবে?

- Inheritance

- raise
- try-except

কীভাবে চিন্তা করব?

বড় প্রজেক্টে built-in exception যথেষ্ট নাও হতে পারে — তাই আমরা নিজেদের exception class বানাই, `Exception` থেকে inherit করে। যেমন `InsufficientBalanceError` — নামটাই বলে দেয় সমস্যা কী।

> **[Best Practice]** Custom exception-এর নাম সবসময় 'Error' দিয়ে শেষ করুন এবং docstring-এ কারণ লিখুন — এটি Python কমিউনিটির প্রথাগত নিয়ম (convention)।

অ্যালগরিদম

Step 1: `class InsufficientBalanceError(Exception):` class বানানো।

Step 2: প্রয়োজনে `__init__` override করে অতিরিক্ত তথ্য রাখা।

Step 3: `raise InsufficientBalanceError("...")` দিয়ে exception তোলা।

Step 4: `except InsufficientBalanceError as e:` দিয়ে ধরা।

Python Code

```
# Creating a custom exception class

class InsufficientBalanceError(Exception):
    """Raised when withdrawal amount is more than the balance."""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        message = "Cannot withdraw " + str(amount) + ", balance is only " +
str(balance)
        super().__init__(message)


class InvalidAgeError(Exception):
    """Raised when age is not in a valid range."""
    pass


class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError(self.balance, amount)
```

```
        self.balance = self.balance - amount
        return self.balance



def register_voter(age):
    if age < 18:
        raise InvalidAgeError("Age must be 18 or above to register. Given age:
" + str(age))
    return "Registered successfully"



print("--- Test 1: custom exception with extra data ---")
account = BankAccount(1000)
try:
    account.withdraw(1500)
except InsufficientBalanceError as e:
    print("Error:", e)
    print("Shortfall:", e.amount - e.balance, "Taka")

print("\n--- Test 2: another custom exception ---")
try:
    print(register_voter(15))
except InvalidAgeError as e:
    print("Error:", e)

print("\n--- Test 3: valid case ---")
try:
    print(register_voter(20))
except InvalidAgeError as e:
    print("Error:", e)
```

Built-in Function / Shortcut Method

```
# Simplest custom exception (no extra data needed)
# class MyError(Exception):
#     pass
#
# raise MyError("Something specific went wrong")
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Test 1: custom exception with extra data --- Error: Cannot withdraw 1500,

```
                                    balance is only 1000
                                    Shortfall: 500 Taka

                                    --- Test 2: another custom
                                    exception ---
                                    Error: Age must be 18 or above to
                                    register. Given age: 15

                                    --- Test 3: valid case ---
                                    Registered successfully
```

ব্যাখ্যা

`InsufficientBalanceError` custom exception-এ আমরা `balance` ও `amount` দুইটি অতিরিক্ত তথ্যও রেখেছি, যা built-in exception-এ পাওয়া যেত না। এটি প্রজেক্টকে আরও অর্থবহ ও ডিবাগ-বান্ধব করে তোলে।

Common Mistakes

- custom exception class-এ `Exception` থেকে inherit না করা।
- `super().__init__(message)` কল না করা — তখন `str(exception)` ফাঁকা দেখাবে।
- custom exception-এর নাম error-এর ধরন বোঝায় না এমন রাখা (যেমন `MyError1`)।

অনুশীলনের জন্য

1. `InvalidMarksError` custom exception বানান (মার্কস ০-১০০ এর বাইরে গেলে)।

2. `OutOfStockError` custom exception বানান একটি Inventory system-এর জন্য।

3. custom exception-এ অতিরিক্ত তথ্য (attribute) যোগ করুন।

সমস্যা 257: else ক্লজ ব্যবহার করুন (try-except-else)।

সমস্যা

সিলেবাসের 7.5 অনুযায়ী — `else` ক্লজ ব্যবহার করে দেখান যে কোনো Exception না হলেই কেবল `else` ব্লকের কোড চলে।

কী কী জানা থাকতে হবে?

- try-except
- if-else ধারণা

কীভাবে চিন্তা করব?

`else` ব্লক তখনই চলে যখন `try` ব্লকে কোনো exception হয়নি। এটি ব্যবহার করলে বোঝা যায় কোন কোড শুধু 'সফল' হলে চালানো উচিত — try ব্লকের ভিতরে না রেখে else-এ রাখলে কোড পরিষ্কার থাকে।

অ্যালগরিদম

Step 1: try ব্লকে ফাইল/সংখ্যা নিয়ে ঝুঁকিপূর্ণ কাজ করা।

Step 2: except ব্লকে ভুল হ্যান্ডেল করা।

Step 3: else ব্লকে 'সফল হলে যা করতে হবে' তা লেখা।

Step 4: দুই রকম ইনপুট দিয়ে যাচাই করা।

Python Code

```
# Using try-except-else

def divide_numbers(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
    else:
        # Runs ONLY if no exception occurred in try
        print("Division successful!")
        print("Result:", result)



print("--- Test 1: valid division ---")
divide_numbers(10, 2)

print("\n--- Test 2: division by zero ---")
divide_numbers(10, 0)



# Another example: reading a file
def read_file_safely(filename):
    try:
        file = open(filename, "r")
    except FileNotFoundError:
        print("Error:", filename, "not found!")
    else:
        content = file.read()
        print("File read successfully. Length:", len(content), "characters")
        file.close()



print("\n--- Test 3: file exists ---")
```

```
with open("sample_ok.txt", "w") as f:
    f.write("Hello Python")
read_file_safely("sample_ok.txt")

print("\n--- Test 4: file does not exist ---")
read_file_safely("missing_file.txt")
```

Built-in Function / Shortcut Method

```
# try-except-else structure:
# try:
#     risky_code()
# except SomeError:
#     handle_error()
# else:
#     runs_only_if_no_error()
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Test 1: valid division --- Division successful! Result: 5.0 --- Test 2: division by zero --- Error: Cannot divide by zero! --- Test 3: file exists --- File read successfully. Length: 12 characters --- Test 4: file does not exist --- Error: missing_file.txt not found!

ব্যাখ্যা

যখন division সফল হয়েছে (Test 1), তখনই `else` ব্লক চলেছে ও Result দেখিয়েছে। Test 2-এ exception হওয়ায় `else` ব্লক একদমই চলেনি — সরাসরি except-এই থেমেছে।

Common Mistakes

- try ব্লকের ভিতরেই সব কোড লিখে else ব্যবহার না করা — কাজ করলেও কোড কম পরিষ্কার হয়।
- else ব্লকের ভিতরে exception হতে পারে এমন কোড রাখা (else-এর ভুল তখন handle হবে না)।
- else-কে finally-এর মতো ভাবা — else শুধু 'no exception' হলে চলে, finally সবসময় চলে।

অনুশীলনের জন্য

1. একটি login প্রোগ্রাম বানান যেখানে সঠিক পাসওয়ার্ড দিলে else ব্লকে স্বাগত বার্তা আসবে।

2. list index access-এ try-except-else ব্যবহার করুন।

3. else ব্লকে ফাইলে ডেটা লেখার কাজ রাখুন।

সমস্যা 258: finally ক্লজ ব্যবহার করুন (সবসময় চলবে এমন কোড)।

সমস্যা

সিলেবাসের 7.5 অনুযায়ী — `finally` ক্লজ ব্যবহার করে এমন কোড লিখুন যা exception হোক বা না হোক, সবসময় চলবে (যেমন ফাইল বন্ধ করা)।

কী কী জানা থাকতে হবে?

- try-except-else
- File handling

কীভাবে চিন্তা করব?

`finally` ব্লক সবসময় চলে — exception হোক, না হোক, এমনকি `return` করলেও। এটি সাধারণত resource পরিষ্কার করতে ব্যবহার হয় — ফাইল বন্ধ করা, database connection বন্ধ করা ইত্যাদি।

> **[Important]** আধুনিক পাইথনে ফাইলের জন্য `finally` না লিখে `with open(...) as f:` ব্যবহার করাই ভালো — এটি স্বয়ংক্রিয়ভাবে finally-এর কাজ করে দেয়।

অ্যালগরিদম

Step 1: try ব্লকে ফাইল খুলে কিছু কাজ করা।

Step 2: except ব্লকে ভুল হ্যান্ডেল করা।

Step 3: finally ব্লকে ফাইল বন্ধ করা (সবসময় চলবে)।

Step 4: exception সহ ও ছাড়া উভয় ক্ষেত্রেই finally চলা প্রমাণ করা।

Python Code

```
# Using finally - always runs, error or not
```

```
def process_file(filename):
    file = None
    try:
        file = open(filename, "r")
        content = file.read()
        numbers = [int(x) for x in content.split()]
        total = sum(numbers)
        print("Total:", total)
    except FileNotFoundError:
        print("Error: File not found!")
    except ValueError:
        print("Error: File contains non-numeric data!")
    finally:
        print("Finally block: cleaning up...")
        if file is not None:
            file.close()
            print("File closed.")



# Create a valid file
with open("numbers.txt", "w") as f:
    f.write("10 20 30 40")

print("--- Test 1: valid file ---")
process_file("numbers.txt")

print("\n--- Test 2: missing file ---")
process_file("no_file.txt")

# Create an invalid file
with open("bad_numbers.txt", "w") as f:
    f.write("10 abc 30")

print("\n--- Test 3: invalid data in file ---")
process_file("bad_numbers.txt")

print("\n--- finally even runs with return ---")
def test_return():
    try:
        return "returning from try"
    finally:
        print("finally still runs before returning!")



print(test_return())
```

Built-in Function / Shortcut Method

```
# Cleaner way using 'with' (finally not needed for files)
# with open(filename, "r") as file:
#     content = file.read()
# # file auto-closes here, even if error occurs
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Test 1: valid file --- Total: 100 Finally block: cleaning up... File closed. --- Test 2: missing file --- Error: File not found! Finally block: cleaning up... --- Test 3: invalid data in file --- Error: File contains non-numeric data! Finally block: cleaning up... File closed. --- finally even runs with return --- finally still runs before returning! returning from try

ব্যাখ্যা

তিনটি টেস্টেই — সফল হোক বা exception হোক — `finally` ব্লক অবশ্যই চলেছে। এমনকি `return` করার সময়ও finally আগে চলেছে। এটি নিশ্চিত করে যে resource (ফাইল) সবসময় সঠিকভাবে বন্ধ হয়।

Common Mistakes

- `finally`-তে ফাইল close করার আগে `file is not None` চেক না করা (ফাইল খোলাই না হলে `NameError`/`AttributeError`)।
- `with` ব্যবহার করার পরেও আলাদা করে `finally` দিয়ে ফাইল বন্ধ করার চেষ্টা করা (অপ্রয়োজনীয়)।
- finally ব্লকে নতুন exception রাখা — এটি মূল exception-কে ঢেকে দিতে পারে।

অনুশীলনের জন্য

1. একটি database connection simulate করে try-except-finally লিখুন।

2. finally ব্লকে একটি লগ ফাইলে সময় লিখে রাখুন।

3. with statement ব্যবহার করে finally-এর প্রয়োজনীয়তা কমান।

সমস্যা 259: Nested try-except ব্যবহার করুন।

সমস্যা

একটি try ব্লকের ভিতরে আরেকটি try-except ব্লক (nested) রেখে জটিল পরিস্থিতি সামলানোর প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- try-except
- Function

কীভাবে চিন্তা করব?

কখনো কখনো একটি বড় কাজের ভিতরে ছোট আলাদা ঝুঁকিপূর্ণ ধাপ থাকে, যেগুলোর জন্য আলাদা exception handling দরকার। তখন try ব্লকের ভিতরে আরেকটি try-except রাখা যায় — একে nested try-except বলে।

অ্যালগরিদম

Step 1: বাইরের try ব্লকে ফাইল থেকে ডেটা read করা।

Step 2: ভিতরের try ব্লকে সেই ডেটা দিয়ে হিসাব করা।

Step 3: প্রতিটি স্তরে আলাদা except ব্যবহার করা।

Step 4: ভিন্ন পরিস্থিতিতে যাচাই করা।

Python Code

```
# Nested try-except blocks

def process_marks(filename):
    try:
        with open(filename, "r") as file:
            content = file.read()

        # Inner try-except for the calculation part
        try:
            marks = [int(x) for x in content.split(",")]
            average = sum(marks) / len(marks)
            print("Marks:", marks)
            print("Average:", average)
```

```
        except ValueError:
            print("Inner error: Marks contain non-numeric data.")
        except ZeroDivisionError:
            print("Inner error: No marks found in the file.")

    except FileNotFoundError:
        print("Outer error: File", filename, "not found.")


# Test 1: valid file
with open("marks.txt", "w") as f:
    f.write("70,80,90,65")

print("--- Test 1: valid marks ---")
process_marks("marks.txt")

# Test 2: file missing (outer exception)
print("\n--- Test 2: missing file ---")
process_marks("no_marks.txt")

# Test 3: invalid data (inner exception)
with open("bad_marks.txt", "w") as f:
    f.write("70,eighty,90")

print("\n--- Test 3: invalid marks ---")
process_marks("bad_marks.txt")

# Test 4: empty file (inner exception - zero division)
with open("empty_marks.txt", "w") as f:
    f.write("")

print("\n--- Test 4: empty file ---")
process_marks("empty_marks.txt")
```

Built-in Function / Shortcut Method

```
# Nested try-except keeps error handling specific to each step:
# outer try  -> handles file-related problems
# inner try  -> handles calculation-related problems
```

Sample Input / Output Table

Sample Input	Sample Output

```
None
```

```
--- Test 1: valid marks ---
Marks: [70, 80, 90, 65]
Average: 76.25

--- Test 2: missing file ---
Outer error: File no_marks.txt
not found.

--- Test 3: invalid marks ---
Inner error: Marks contain non-
numeric data.

--- Test 4: empty file ---
Inner error: No marks found in
the file.
```

ব্যাখ্যা

বাইরের try-except শুধু ফাইল সংক্রান্ত ভুল ধরেছে, আর ভিতরের try-except শুধু হিসাব সংক্রান্ত ভুল ধরেছে। এভাবে প্রতিটি স্তরের সমস্যা আলাদাভাবে ও স্পষ্টভাবে হ্যান্ডেল করা গেছে।

Common Mistakes

- নেস্টেড try-except অতিরিক্ত ব্যবহার করে কোড জটিল করে ফেলা — সাধারণত ২ স্তরের বেশি দরকার হয় না।
- ভিতরের exception বাইরের except ব্লকে গিয়ে ধরা পড়বে ভাবা (আসলে নিজের স্তরেই আগে ধরা পড়ে)।
- কোন স্তরে কোন exception হ্যান্ডেল হচ্ছে তা গুলিয়ে ফেলা।

অনুশীলনের জন্য

1. নেস্টেড try-except দিয়ে JSON ফাইল read ও parse করুন।

2. বাইরের try-তে database connect, ভিতরের try-তে query চালানো simulate করুন।

3. ৩ স্তরের nested try-except বানিয়ে দেখুন (সতর্কতার সাথে)।

সমস্যা 260: Exception Handling দিয়ে সম্পূর্ণ Input Validation System বানান।

সমস্যা

সিলেবাসের ব্যবহারিক প্রয়োগ হিসেবে — try-except-else-finally, custom exception ও loop একসাথে ব্যবহার করে একটি সম্পূর্ণ Student Registration ইনপুট ভ্যালিডেশন সিস্টেম বানান।

কী কী জানা থাকতে হবে?

- try-except-else-finally
- Custom Exception
- while loop

কীভাবে চিন্তা করব?

বাস্তব প্রজেক্টে ইউজার ভুল ইনপুট দিতেই পারে। তাই while loop-এর ভিতরে try-except রেখে সঠিক ইনপুট না দেওয়া পর্যন্ত বারবার জিজ্ঞাসা করা হয় — এটিই প্রফেশনাল input validation-এর মূল কৌশল।

> **[Best Practice]** প্রফেশনাল সফটওয়্যারে ইউজারকে কখনো একবার ভুল দিলেই বের করে দেওয়া হয় না — while loop + try-except দিয়ে বারবার সুযোগ দেওয়া হয়। এটি Practical প্রজেক্টগুলোতে ব্যবহার করুন।

অ্যালগরিদম

Step 1: Custom exception `InvalidRollError` বানানো।

Step 2: while loop-এ বারবার ইনপুট নেওয়া।

Step 3: try ব্লকে validation করা, ভুল হলে except-এ বার্তা দেখিয়ে আবার loop চালানো।

Step 4: সঠিক ইনপুট পেলে loop থেকে বের হওয়া।

Python Code

```
# Complete input validation system using exception handling

class InvalidRollError(Exception):
    pass



def get_valid_roll():
    while True:
        try:
            roll = int(input("Enter roll number (1-999): "))
            if roll < 1 or roll > 999:
                raise InvalidRollError("Roll must be between 1 and 999.")
        except ValueError:
            print("Invalid input! Please enter a whole number.")
        except InvalidRollError as e:
            print("Error:", e)
        else:
            print("Roll number accepted:", roll)
            return roll
        finally:
            print("-- attempt finished --\n")
```

```
def get_valid_cgpa():
    while True:
        try:
            cgpa = float(input("Enter CGPA (0.0 - 4.0): "))
            if cgpa < 0.0 or cgpa > 4.0:
                raise ValueError("CGPA must be between 0.0 and 4.0")
        except ValueError as e:
            print("Invalid CGPA:", e)
        else:
            print("CGPA accepted:", cgpa)
            return cgpa



print("=== Student Registration ===")
roll = get_valid_roll()
cgpa = get_valid_cgpa()

print("\nRegistration successful!")
print("Roll:", roll, "| CGPA:", cgpa)
```

Built-in Function / Shortcut Method

```
# This pattern (loop + try-except) is called
# "retry until valid" - very common in real applications
```

Sample Input / Output Table

Sample Input	Sample Output
abc 1500 250 5.5 3.8	=== Student Registration === Enter roll number (1-999): abc Invalid input! Please enter a whole number. -- attempt finished -- Enter roll number (1-999): 1500 Error: Roll must be between 1 and 999. -- attempt finished -- Enter roll number (1-999): 250 Roll number accepted: 250

```
-- attempt finished --

Enter CGPA (0.0 - 4.0): 5.5
Invalid CGPA: CGPA must be
between 0.0 and 4.0
Enter CGPA (0.0 - 4.0): 3.8
CGPA accepted: 3.8

Registration successful!
Roll: 250 | CGPA: 3.8
```

ব্যাখ্যা

প্রতিবার ভুল ইনপুট দিলে `while True` loop আবার জিজ্ঞাসা করেছে, আর সঠিক ইনপুট পেলে `else` ব্লকে গিয়ে `return` দিয়ে loop থেকে বের হয়েছে। `finally` প্রতিটি প্রচেষ্টার পরে চলেছে — এটি একটি বাস্তব প্রজেক্টে ব্যবহৃত পূর্ণাঙ্গ প্যাটার্ন।

Common Mistakes

- while loop-এ break/return না দিলে অসীম লুপ হয়ে যাওয়া।
- except ব্লকে continue/return ভুলভাবে ব্যবহার করে সঠিক ইনপুট নেওয়ার পরও আবার loop চালানো।
- একাধিক custom exception-এর জন্য একটিই generic exception ব্যবহার করে নির্দিষ্টতা হারানো।

অনুশীলনের জন্য

1. একটি phone number validation system বানান (১১ ডিজিট, শুধু সংখ্যা)।

2. একটি password validation system বানান (কমপক্ষে ৮ অক্ষর)।

3. email validation যোগ করে পুরো registration system সম্পূর্ণ করুন।

Topic: Logging in Python — Unit 8

সমস্যা 261: Logging কী এবং কেন print()-এর বদলে Logging ব্যবহার করবেন।

সমস্যা

সিলেবাসের 8.1 ও 8.2 অনুযায়ী — Logging-এর সংজ্ঞা, গুরুত্ব এবং print()-এর তুলনায় logging-এর সুবিধা ব্যাখ্যা করুন।

কী কী জানা থাকতে হবে?

- print() function
- File handling ধারণা

কীভাবে চিন্তা করব?

`print()` শুধু স্ক্রিনে দেখায়, প্রোগ্রাম বন্ধ হলে হারিয়ে যায়। Logging সময়, গুরুত্ব (level) ও উৎসসহ স্থায়ীভাবে ফাইলে/স্ক্রিনে রেকর্ড রাখে। বড় প্রজেক্টে সমস্যা খুঁজতে (debugging) ও সিস্টেম মনিটরিং করতে logging অপরিহার্য।

> **[Important]** ডিফল্ট logging level হলো WARNING। তাই `logging.info()` বা `logging.debug()` দেখতে হলে `basicConfig(level=logging.DEBUG)` সেট করতেই হবে।

অ্যালগরিদম

Step 1: `import logging` করা।

Step 2: `logging.basicConfig()` দিয়ে প্রাথমিক সেটআপ করা।

Step 3: `logging.info()`, `logging.warning()` ইত্যাদি ব্যবহার করা।

Step 4: print() ও logging-এর আউটপুট তুলনা করা।

Python Code

```
# Why logging instead of print()?
import logging

# Basic configuration - shows time, level and message
logging.basicConfig(level=logging.DEBUG, format="%(asctime)s - %(levelname)s -
%(message)s")

print("--- Using print() (temporary, no timestamp, no severity level) ---")
print("System started")
print("Something might be wrong")

print("\n--- Using logging (permanent record, timestamp, severity level) ---")
logging.debug("This is a debug message (detailed info for developers)")
logging.info("System started successfully")
logging.warning("Disk space is running low")
logging.error("Failed to connect to database")
logging.critical("System is shutting down unexpectedly!")

print("\nAdvantages of logging over print():")
advantages = [
    "Automatically records date and time",
    "Shows severity level (INFO, WARNING, ERROR, etc.)",
    "Can be saved permanently to a file",
    "Can be turned on/off without deleting code",
    "Different parts of a big project can be tracked separately"
]
for i, adv in enumerate(advantages, 1):
```

```
    print(i, "-", adv)
```

Built-in Function / Shortcut Method

```
# Logging levels from lowest to highest severity:
# DEBUG < INFO < WARNING < ERROR < CRITICAL
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Using print() (temporary, no timestamp, no severity level) --- System started Something might be wrong --- Using logging (permanent record, timestamp, severity level) --- 2025-08-14 10:30:15,123 - DEBUG - This is a debug message (detailed info for developers) 2025-08-14 10:30:15,124 - INFO - System started successfully 2025-08-14 10:30:15,125 - WARNING - Disk space is running low 2025-08-14 10:30:15,126 - ERROR - Failed to connect to database 2025-08-14 10:30:15,127 - CRITICAL - System is shutting down unexpectedly! Advantages of logging over print(): 1 - Automatically records date and time 2 - Shows severity level (INFO, WARNING, ERROR, etc.) 3 - Can be saved permanently to a file 4 - Can be turned on/off without deleting code 5 - Different parts of a big

```
                                        project can be tracked separately
```

ব্যাখ্যা

`logging` প্রতিটি বার্তার সাথে সময় ও গুরুত্বের মাত্রা (level) দেখায়, যা `print()` দিতে পারে না। বড় প্রজেক্টে সমস্যার উৎস দ্রুত খুঁজে বের করতে এটি অপরিহার্য।

Common Mistakes

- সব প্রজেক্টে শুধু `print()` ব্যবহার করে যাওয়া, logging শেখা এড়িয়ে যাওয়া।
- `logging.basicConfig()` একাধিকবার কল করা (প্রথমবারের পরে effect থাকে না)।
- DEBUG level ব্যবহার করেও `level=logging.DEBUG` সেট না করা (ডিফল্ট level WARNING, তাই DEBUG/INFO দেখাবে না)।

অনুশীলনের জন্য

1. ৫টি ভিন্ন logging level ব্যবহার করে একটি প্রোগ্রাম লিখুন।

2. print() ও logging-এর মধ্যে ৩টি পার্থক্য লিখুন।

3. logging.basicConfig()-এ format পরিবর্তন করে দেখুন।

সমস্যা 262: Logging-এর ৫টি Level ব্যবহার করুন (DEBUG, INFO, WARNING, ERROR, CRITICAL)।

সমস্যা

সিলেবাসের 8.3 অনুযায়ী — Logging-এর পাঁচটি level-এর প্রতিটির ব্যবহার ক্ষেত্র বুঝে একটি বাস্তব প্রোগ্রামে প্রয়োগ করুন।

কী কী জানা থাকতে হবে?

- logging module
- if-else
- try-except

কীভাবে চিন্তা করব?

প্রতিটি level নির্দিষ্ট পরিস্থিতির জন্য: DEBUG (ডেভেলপমেন্টের তথ্য), INFO (সাধারণ কার্যক্রম), WARNING (সতর্কতা কিন্তু কাজ থামেনি), ERROR (কিছু ভুল হয়েছে), CRITICAL (মারাত্মক সমস্যা, সিস্টেম বন্ধ হতে পারে)। সঠিক level ব্যবহার করলে লগ পড়ে সহজে অগ্রাধিকার বোঝা যায়।

অ্যালগরিদম

Step 1: একটি Inventory system simulate করা।

Step 2: প্রতিটি পরিস্থিতিতে উপযুক্ত level ব্যবহার করা।

Step 3: স্টক কম হলে WARNING, স্টক শেষ হলে ERROR দেখানো।

Step 4: সব লগ প্রিন্ট করে level অনুযায়ী বিশ্লেষণ করা।

Python Code

```
# Using all 5 logging levels appropriately
import logging

logging.basicConfig(level=logging.DEBUG, format="%(asctime)s - %(levelname)s -
%(message)s")

inventory = {"Mango": 50, "Apple": 5, "Banana": 0}



def sell_item(item, quantity):
    logging.debug("sell_item called with item=%s, quantity=%s", item, quantity)

    if item not in inventory:
        logging.error("Item '%s' does not exist in inventory", item)
        return

    if inventory[item] == 0:
        logging.critical("Item '%s' is completely OUT OF STOCK!", item)
        return

    if inventory[item] < 10:
        logging.warning("Low stock warning for '%s': only %s left", item,
inventory[item])

    if quantity > inventory[item]:
        logging.error("Cannot sell %s units of '%s', only %s available",
quantity, item, inventory[item])
        return

    inventory[item] = inventory[item] - quantity
    logging.info("Sold %s units of '%s'. Remaining: %s", quantity, item,
inventory[item])



print("--- Selling Mango (plenty of stock) ---")
sell_item("Mango", 10)

print("\n--- Selling Apple (low stock) ---")
sell_item("Apple", 2)

print("\n--- Selling Banana (out of stock) ---")
```

```
sell_item("Banana", 1)

print("\n--- Selling unknown item ---")
sell_item("Grapes", 1)
```

Built-in Function / Shortcut Method

```
# Level usage guide:
# DEBUG    -> detailed info, only for developers during development
# INFO     -> confirms things are working as expected
# WARNING  -> something unexpected, but the program still works
# ERROR    -> a serious problem, some function failed
# CRITICAL -> a very serious error, program may not continue
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Selling Mango (plenty of stock) --- 2025-08-14 10:31:00,001 - DEBUG - sell_item called with item=Mango, quantity=10 2025-08-14 10:31:00,002 - INFO - Sold 10 units of 'Mango'. Remaining: 40 --- Selling Apple (low stock) --- 2025-08-14 10:31:00,003 - DEBUG - sell_item called with item=Apple, quantity=2 2025-08-14 10:31:00,004 - WARNING - Low stock warning for 'Apple': only 5 left 2025-08-14 10:31:00,005 - INFO - Sold 2 units of 'Apple'. Remaining: 3 --- Selling Banana (out of stock) --- 2025-08-14 10:31:00,006 - DEBUG - sell_item called with item=Banana, quantity=1 2025-08-14 10:31:00,007 -

```
                                    CRITICAL - Item 'Banana' is
                                    completely OUT OF STOCK!

                                    --- Selling unknown item ---
                                    2025-08-14 10:31:00,008 - DEBUG -
                                    sell_item called with
                                    item=Grapes, quantity=1
                                    2025-08-14 10:31:00,009 - ERROR -
                                    Item 'Grapes' does not exist in
                                    inventory
```

ব্যাখ্যা

প্রতিটি পরিস্থিতির গুরুত্ব অনুযায়ী সঠিক level ব্যবহার করা হয়েছে — স্বাভাবিক বিক্রয়ে INFO, কম স্টকে WARNING, স্টক শেষে CRITICAL, ভুল item-এ ERROR। `%s` প্লেসহোল্ডার ব্যবহার করে ভ্যারিয়েবল বসানো হয়েছে (f-string-এর চেয়ে এটি performance-এ ভালো)।

Common Mistakes

- সব পরিস্থিতির জন্য একই level (`logging.info()`) ব্যবহার করা — গুরুত্ব বোঝা যায় না।
- `%s` প্লেসহোল্ডারের বদলে সরাসরি string concatenation করা (কম efficient)।
- CRITICAL ও ERROR গুলিয়ে ফেলা — CRITICAL মানে সিস্টেম-স্তরের বিপর্যয়, ERROR মানে নির্দিষ্ট কাজ ব্যর্থ।

অনুশীলনের জন্য

1. একটি লগইন সিস্টেমে ৫টি level ব্যবহার করুন (ভুল পাসওয়ার্ড = WARNING, একাউন্ট লক = CRITICAL)।

2. একটি ফাইল আপলোড সিস্টেমে logging level প্রয়োগ করুন।

3. নিজের প্রজেক্টের জন্য কোন পরিস্থিতিতে কোন level ব্যবহার করবেন তার তালিকা করুন।

সমস্যা 263: Logging Output ফাইলে সংরক্ষণ করুন।

সমস্যা

সিলেবাসের 8.4 ও 8.5 অনুযায়ী — logging-এর আউটপুট স্ক্রিনে না দেখিয়ে একটি .log ফাইলে সংরক্ষণ করার কনফিগারেশন করুন।

কী কী জানা থাকতে হবে?

- logging basics
- File handling ধারণা

কীভাবে চিন্তা করব?

`basicConfig()`-এ `filename` parameter দিলে logging আউটপুট ফাইলে সেভ হয়, স্ক্রিনে দেখায় না। `filemode='a'` দিলে প্রতিবার প্রোগ্রাম চালালে আগের লগ মুছে না গিয়ে যোগ হয়। এটি বাস্তব সার্ভার/অ্যাপ্লিকেশনে standard practice।

> **[Best Practice]** প্রোডাকশন সিস্টেমে সবসময় `filemode='a'` ব্যবহার করুন, যাতে সার্ভার রিস্টার্ট হলেও আগের লগ ইতিহাস হারিয়ে না যায়।

অ্যালগরিদম

Step 1: `logging.basicConfig(filename='app.log', filemode='a', level=logging.INFO)` সেট করা।

Step 2: কয়েকটি log বার্তা তৈরি করা।

Step 3: ফাইলটি খুলে ভেতরের কনটেন্ট পড়ে দেখানো।

Python Code

```
# Saving log output to a file
import logging

logging.basicConfig(
    filename="app.log",
    filemode="a",                 # 'a' = append, keeps old logs
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Application started")
logging.warning("Low memory detected")
logging.error("Failed to load configuration file")
logging.info("Application closed normally")

print("Logs have been written to app.log (not shown on screen)")

# Verify by reading the log file back
print("\n--- Contents of app.log ---")
with open("app.log", "r") as file:
    print(file.read())
```

Built-in Function / Shortcut Method

```
# filemode options:
# 'a' -> append (keeps history across runs)   <- recommended
```

```
# 'w' -> overwrite (deletes old logs each run)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Logs have been written to app.log (not shown on screen) --- Contents of app.log --- 2025-08-14 10:32:10,111 - INFO - Application started 2025-08-14 10:32:10,112 - WARNING - Low memory detected 2025-08-14 10:32:10,113 - ERROR - Failed to load configuration file 2025-08-14 10:32:10,114 - INFO - Application closed normally

ব্যাখ্যা

`filename='app.log'` দেওয়ায় logging বার্তা স্ক্রিনে না গিয়ে সরাসরি ফাইলে গিয়েছে। প্রোগ্রাম বন্ধ হয়ে গেলেও এই লগ ফাইলে থেকে যায় — পরে বিশ্লেষণ করা যায়।

Common Mistakes

- `filemode='w'` ব্যবহার করে প্রতিবার প্রোগ্রাম চালালে আগের লগ মুছে ফেলা।
- `basicConfig()` একই প্রোগ্রামে দ্বিতীয়বার আলাদা সেটিংসে কল করা (প্রথমটাই কার্যকর থাকে)।
- log ফাইলের path ভুল দেওয়া, ফলে খুঁজে না পাওয়া।

অনুশীলনের জন্য

1. নিজের প্রজেক্টের সব error শুধু 'errors.log' নামের ফাইলে সেভ করুন।

2. `filemode='w'` ও `'a'`-এর পার্থক্য প্র্যাকটিক্যালি দেখান।

3. একটি ফাংশনের প্রতিটি কল log ফাইলে রেকর্ড করুন।

সমস্যা 264: একাধিক Logger তৈরি করুন (getLogger)।

সমস্যা

`logging.getLogger(name)` ব্যবহার করে প্রজেক্টের বিভিন্ন অংশের জন্য আলাদা আলাদা logger তৈরি করুন।

কী কী জানা থাকতে হবে?

- Logging basics
- Module ধারণা

কীভাবে চিন্তা করব?

বড় প্রজেক্টে একাধিক মডিউল থাকে (যেমন database, payment, auth)। প্রতিটির জন্য আলাদা logger বানালে লগ পড়ার সময় বোঝা যায় সমস্যাটি কোন অংশে হয়েছে। `logging.getLogger(__name__)` দিয়ে module-ভিত্তিক logger বানানো হয়।

অ্যালগরিদম

Step 1: root logger বেসিক কনফিগার করা।

Step 2: `logging.getLogger('database')` ও `logging.getLogger('payment')` বানানো।

Step 3: প্রতিটি logger দিয়ে আলাদা বার্তা লেখা।

Step 4: লগে module-এর নাম কীভাবে দেখা যায় তা লক্ষ্য করা।

Python Code

```
# Multiple named loggers for different parts of a project
import logging

logging.basicConfig(level=logging.DEBUG, format="%(asctime)s - %(name)s - %
(levelname)s - %(message)s")

# Separate loggers for separate modules
db_logger = logging.getLogger("database")
payment_logger = logging.getLogger("payment")
auth_logger = logging.getLogger("auth")

db_logger.info("Connected to database successfully")
db_logger.error("Query timeout after 30 seconds")

payment_logger.info("Payment of 500 Taka received")
payment_logger.warning("Payment gateway response was slow")

auth_logger.info("User 'rahim123' logged in")
auth_logger.error("Failed login attempt for 'unknown_user'")

print("\nNotice how each log line shows which module it came from!")
print("This makes debugging large projects much easier.")
```

Built-in Function / Shortcut Method

```
# Standard practice: use __name__ so the logger name
```

```
# automatically matches the file/module name
# logger = logging.getLogger(__name__)
```

Sample Input / Output Table

Sample Input	Sample Output
None	2025-08-14 10:33:00,001 - database - INFO - Connected to database successfully 2025-08-14 10:33:00,002 - database - ERROR - Query timeout after 30 seconds 2025-08-14 10:33:00,003 - payment - INFO - Payment of 500 Taka received 2025-08-14 10:33:00,004 - payment - WARNING - Payment gateway response was slow 2025-08-14 10:33:00,005 - auth - INFO - User 'rahim123' logged in 2025-08-14 10:33:00,006 - auth - ERROR - Failed login attempt for 'unknown_user' Notice how each log line shows which module it came from! This makes debugging large projects much easier.

ব্যাখ্যা

`%(name)s` ফরম্যাটে যোগ করায় প্রতিটি লগ লাইনে কোন logger (database/payment/auth) থেকে এসেছে তা দেখা যাচ্ছে। এটি বড় প্রজেক্টে কোন অংশে সমস্যা তা দ্রুত চিহ্নিত করতে সাহায্য করে।

Common Mistakes

- সব জায়গায় root logger (`logging.info()`) ব্যবহার করে module আলাদা না করা।
- `getLogger()`-এ ভিন্ন ভিন্ন নাম না দিয়ে একই নাম বারবার ব্যবহার করা।
- `%(name)s` ফরম্যাটে যোগ না করা, ফলে কোন logger থেকে বার্তা এলো তা বোঝা না যাওয়া।

অনুশীলনের জন্য

1. একটি ৩-মডিউলের প্রজেক্টে (UI, Backend, Database) আলাদা logger বানান।

2. `__name__` ব্যবহার করে module-ভিত্তিক logger বানান।

3. প্রতিটি logger-এর জন্য আলাদা log ফাইল সেট করুন।

Topic: Unit Testing in Python — Unit 9

সমস্যা 265: Unit Testing কী এবং কেন গুরুত্বপূর্ণ তা বুঝুন।

সমস্যা

সিলেবাসের 9.1 ও 9.2 অনুযায়ী — Unit Testing-এর সংজ্ঞা, গুরুত্ব ও ম্যানুয়াল টেস্টিংয়ের সাথে পার্থক্য ব্যাখ্যা করুন।

কী কী জানা থাকতে হবে?

- Function
- if-else
- assert ধারণা

কীভাবে চিন্তা করব?

Unit Testing মানে কোডের ক্ষুদ্রতম অংশ (একটি function/method) আলাদাভাবে পরীক্ষা করা যে সেটি প্রত্যাশিত ফলাফল দেয় কিনা। ম্যানুয়ালি বারবার চালিয়ে দেখার বদলে, একবার টেস্ট লিখে রাখলে ভবিষ্যতে কোড বদলালেও সহজে পুনরায় (automatically) যাচাই করা যায়।

অ্যালগরিদম

Step 1: একটি সাধারণ function (`add`) লেখা।

Step 2: `assert` ব্যবহার করে ম্যানুয়ালি ফলাফল যাচাই করা।

Step 3: assert ব্যর্থ হলে কী হয় তা দেখানো।

Step 4: Manual testing vs Unit testing-এর পার্থক্য তালিকা করা।

Python Code

```
# Understanding Unit Testing with assert

def add(a, b):
    return a + b



def is_even(n):
    return n % 2 == 0
```

```
# Manual testing (basic idea behind unit testing)
print("--- Manual verification using assert ---")

assert add(2, 3) == 5, "Test failed: 2 + 3 should be 5"
print("Test 1 passed: add(2, 3) == 5")

assert add(-1, 1) == 0, "Test failed: -1 + 1 should be 0"
print("Test 2 passed: add(-1, 1) == 0")

assert is_even(4) == True, "Test failed: 4 should be even"
print("Test 3 passed: is_even(4) == True")

assert is_even(7) == False, "Test failed: 7 should be odd"
print("Test 4 passed: is_even(7) == False")

print("\nAll manual tests passed!")

# What happens when a test FAILS?
print("\n--- Demonstrating a failing test ---")
try:
    assert add(2, 2) == 5, "Test failed: 2 + 2 should be 4, not 5"
except AssertionError as e:
    print("AssertionError caught:", e)

print("\n--- Manual testing vs Unit testing ---")
comparison = [
    ("Speed", "Slow, done by a person", "Fast, done by the computer"),
    ("Repeatability", "Must repeat manually every time", "Run anytime with one
command"),
    ("Reliability", "Human errors possible", "Consistent and reliable"),
    ("Cost (long-term)", "Expensive (time)", "Cheap after initial setup"),
]
print("{:<20}{:<30}{:<30}".format("Aspect", "Manual Testing", "Unit Testing"))
for row in comparison:
    print("{:<20}{:<30}{:<30}".format(*row))
```

Built-in Function / Shortcut Method

```
# assert syntax:
# assert condition, "error message if condition is False"
```

Sample Input / Output Table

Sample Input	Sample Output
None	--- Manual verification using

```
assert ---
Test 1 passed: add(2, 3) == 5
Test 2 passed: add(-1, 1) == 0
Test 3 passed: is_even(4) == True
Test 4 passed: is_even(7) ==
False

All manual tests passed!

--- Demonstrating a failing test
---
AssertionError caught: Test
failed: 2 + 2 should be 4, not 5

--- Manual testing vs Unit
testing ---
Aspect              Manual
Testing               Unit
Testing
Speed               Slow, done by
a person        Fast, done by the
computer
Repeatability       Must repeat
manually every timeRun anytime
with one command
Reliability         Human errors
possible         Consistent and
reliable
Cost (long-term)    Expensive
(time)               Cheap after
initial setup
```

ব্যাখ্যা

`assert` শর্ত মিথ্যা হলে `AssertionError` তোলে — এটিই টেস্টিংয়ের মূল ভিত্তি। Unit testing এই assert-গুলোকে সংগঠিতভাবে চালানোর কাঠামো দেয়, যা পরবর্তী প্রশ্নে `unittest` module দিয়ে দেখানো হয়েছে।

Common Mistakes

- Unit Testing-কে শুধু 'কোড চালিয়ে দেখা' ভাবা — আসলে এটি স্বয়ংক্রিয় ও পুনরাবৃত্তিযোগ্য যাচাই পদ্ধতি।
- assert-এ error message না দেওয়া — ব্যর্থ হলে কারণ বোঝা কঠিন হয়।
- একটি function-এর multiple পরিস্থিতি (edge case) টেস্ট না করা।

অনুশীলনের জন্য

1. `subtract()` function-এর জন্য ৩টি assert টেস্ট লিখুন।

2. একটি ব্যর্থ হবে এমন assert তৈরি করে AssertionError ধরুন।

3. Manual ও Unit Testing-এর ৩টি পার্থক্য নিজের ভাষায় লিখুন।

সমস্যা 266: unittest Module দিয়ে Test Case লিখুন।

সমস্যা

সিলেবাসের 9.3 ও 9.4 অনুযায়ী — পাইথনের বিল্ট-ইন `unittest` module ব্যবহার করে একটি সুসংগঠিত Test Case class লিখুন।

কী কী জানা থাকতে হবে?

- Class & Object
- Function
- assert ধারণা

কীভাবে চিন্তা করব?

`unittest` module-এ `TestCase` class থেকে inherit করে একটি test class বানাতে হয়। প্রতিটি টেস্ট method-এর নাম `test_` দিয়ে শুরু করতে হয়। `self.assertEqual()`, `self.assertTrue()` ইত্যাদি ব্যবহার করে যাচাই করা হয়। `unittest.main()` দিয়ে সব টেস্ট একসাথে চালানো যায়।

> **[Best Practice]** সাধারণ .py ফাইলে `unittest.main()` লিখলে `argv` লাগে না — টার্মিনালে `python -m unittest test_file.py` দিয়ে চালানোই standard পদ্ধতি।

অ্যালগরিদম

Step 1: টেস্ট করার জন্য একটি সাধারণ function/module লেখা (`calculator.py`)।

Step 2: `import unittest` করা।

Step 3: `class TestCalculator(unittest.TestCase):` বানানো।

Step 4: `test_` দিয়ে শুরু হওয়া কয়েকটি method লেখা।

Step 5: `unittest.main()` দিয়ে চালানো।

Python Code

```
# Writing test cases using the unittest module
import unittest
```

```
# The code we want to test
def add(a, b):
    return a + b



def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b



def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True



# The test class - MUST inherit from unittest.TestCase
class TestMathFunctions(unittest.TestCase):

    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-2, -3), -5)

    def test_divide_normal(self):
        self.assertEqual(divide(10, 2), 5)

    def test_divide_by_zero_raises_error(self):
        with self.assertRaises(ValueError):
            divide(10, 0)

    def test_is_prime_true(self):
        self.assertTrue(is_prime(7))

    def test_is_prime_false(self):
        self.assertFalse(is_prime(8))



if __name__ == "__main__":
    unittest.main(argv=[""], verbosity=2, exit=False)
```

Built-in Function / Shortcut Method

```
# Common assert methods in unittest
# self.assertEqual(a, b)      -> a == b
# self.assertTrue(x)          -> x is True
# self.assertFalse(x)         -> x is False
# self.assertRaises(Error)    -> code raises that Error
# self.assertIn(a, list)      -> a is in list
```

Sample Input / Output Table

Sample Input	Sample Output
None	test_add_negative_numbers (__main__.TestMathFunctions) ... ok test_add_positive_numbers (__main__.TestMathFunctions) ... ok test_divide_by_zero_raises_error (__main__.TestMathFunctions) ... ok test_divide_normal (__main__.TestMathFunctions) ... ok test_is_prime_false (__main__.TestMathFunctions) ... ok test_is_prime_true (__main__.TestMathFunctions) ... ok ---------- ---------- ---- Ran 6 tests in 0.001s OK

ব্যাখ্যা

প্রতিটি `test_` দিয়ে শুরু হওয়া method আলাদা একটি টেস্ট। `unittest.main()` স্বয়ংক্রিয়ভাবে সবগুলো খুঁজে চালায় এবং ফলাফল (ok/FAIL) দেখায়। শেষে 'OK' মানে সব টেস্ট পাস করেছে।

Common Mistakes

- method-এর নাম `test_` দিয়ে শুরু না করা — তাহলে `unittest` সেটিকে টেস্ট হিসেবে চিনবে না।
- class-কে `unittest.TestCase` থেকে inherit না করানো।
- `.py` ফাইলে সরাসরি রান করার সময় `argv=[""]` না দিলে Jupyter/interactive পরিবেশে error আসতে পারে (সাধারণ script-এ প্রয়োজন নেই)।

অনুশীলনের জন্য

1. `subtract()` ও `multiply()` function-এর জন্য টেস্ট কেস লিখুন।

2. একটি string function (`reverse_string`)-এর জন্য টেস্ট লিখুন।

3. `assertIn()` ও `assertNotIn()` ব্যবহার করে টেস্ট লিখুন।

সমস্যা 267: setUp() ও tearDown() Method ব্যবহার করুন।

সমস্যা

সিলেবাসের 9.5 অনুযায়ী — `setUp()` ও `tearDown()` ব্যবহার করে প্রতিটি টেস্টের আগে-পরে প্রস্তুতি ও পরিষ্কার করার কাজ করুন।

কী কী জানা থাকতে হবে?

- unittest.TestCase
- Class & Object

কীভাবে চিন্তা করব?

একই ডেটা বারবার প্রতিটি টেস্টে তৈরি করা বিরক্তিকর। `setUp()` প্রতিটি টেস্টের **আগে** স্বয়ংক্রিয়ভাবে চলে (common ডেটা তৈরি করতে), আর `tearDown()` প্রতিটি টেস্টের **পরে** চলে (পরিষ্কার করতে, যেমন ফাইল মুছে ফেলা)।

অ্যালগরিদম

Step 1: `setUp(self)` method-এ একটি list/object তৈরি করা।

Step 2: একাধিক test method-এ সেই common ডেটা ব্যবহার করা।

Step 3: `tearDown(self)` method-এ পরিষ্কার করার কাজ লেখা।

Step 4: চালিয়ে দেখানো setUp/tearDown কতবার চলে।

Python Code

```
# setUp() and tearDown() in unittest
```

```
import unittest



class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def remove_item(self, item):
        if item in self.items:
            self.items.remove(item)

    def total_items(self):
        return len(self.items)



class TestShoppingCart(unittest.TestCase):

    def setUp(self):
        # Runs BEFORE every single test method
        print("setUp: creating a fresh shopping cart")
        self.cart = ShoppingCart()
        self.cart.add_item("Mango")
        self.cart.add_item("Apple")

    def tearDown(self):
        # Runs AFTER every single test method
        print("tearDown: clearing the cart\n")
        self.cart.items.clear()

    def test_initial_items(self):
        self.assertEqual(self.cart.total_items(), 2)

    def test_add_item(self):
        self.cart.add_item("Banana")
        self.assertEqual(self.cart.total_items(), 3)

    def test_remove_item(self):
        self.cart.remove_item("Mango")
        self.assertEqual(self.cart.total_items(), 1)



if __name__ == "__main__":
```

```
    unittest.main(argv=[""], verbosity=2, exit=False)
```

Built-in Function / Shortcut Method

```
# Execution order for EACH test method:
# setUp() -> test_method() -> tearDown()
# This repeats fresh for every single test!
```

Sample Input / Output Table

Sample Input	Sample Output
None	setUp: creating a fresh shopping cart tearDown: clearing the cart setUp: creating a fresh shopping cart tearDown: clearing the cart setUp: creating a fresh shopping cart tearDown: clearing the cart test_add_item (__main__.TestShoppingCart) ... ok test_initial_items (__main__.TestShoppingCart) ... ok test_remove_item (__main__.TestShoppingCart) ... ok ---------- ---------- Ran 3 tests in 0.001s OK

ব্যাখ্যা

৩টি টেস্ট মানে `setUp()` ৩ বার ও `tearDown()` ৩ বার চলেছে — প্রতিটি টেস্ট সম্পূর্ণ আলাদা, তাজা (fresh) cart দিয়ে শুরু হয়েছে। এতে একটি টেস্টের ডেটা আরেকটি টেস্টে প্রভাব ফেলে না।

Common Mistakes

- setUp()-এ ভারী কাজ (যেমন বড় ফাইল read) রাখা — প্রতিটি টেস্টেই চলবে, ধীর হয়ে যায়।
- মনে করা setUp() শুধু একবার চলে — আসলে প্রতিটি টেস্ট method-এর আগে আলাদাভাবে চলে।
- tearDown()-এ resource পরিষ্কার না করা, ফলে টেস্টগুলো একে অপরকে প্রভাবিত করা।

অনুশীলনের জন্য

1. setUp()-এ একটি ফাইল তৈরি করে tearDown()-এ মুছে ফেলুন।

2. setUp()-এ database connection simulate করুন।

3. ৪টি টেস্ট লিখে setUp/tearDown কতবার চলে গণনা করুন।

সমস্যা 268: বিভিন্ন Assert Method ব্যবহার করে সম্পূর্ণ Test Suite লিখুন।

সমস্যা

`assertEqual`, `assertTrue`, `assertFalse`, `assertRaises`, `assertIn`, `assertAlmostEqual` ইত্যাদি একসাথে ব্যবহার করে একটি Student Grade Calculator-এর সম্পূর্ণ Test Suite লিখুন।

কী কী জানা থাকতে হবে?

- unittest module
- setUp/tearDown
- Custom function

কীভাবে চিন্তা করব?

একটি বাস্তব ফাংশনের জন্য শুধু একটি নয়, একাধিক ধরনের পরিস্থিতি (normal, edge case, error case) টেস্ট করা উচিত। বিভিন্ন assert method বিভিন্ন ধরনের যাচাইয়ের জন্য উপযোগী — এই সমস্যায় সবগুলো একসাথে ব্যবহার করে একটি পূর্ণাঙ্গ Test Suite তৈরি করবো।

> **[Important]** ভালো Unit Test লিখতে ৩ ধরনের case চিন্তা করুন: (১) Normal case — সাধারণ ইনপুট, (২) Boundary case — সীমানার মান, (৩) Error case — ভুল ইনপুট।

অ্যালগরিদম

Step 1: একটি `calculate_grade(marks)` function লেখা।

Step 2: স্বাভাবিক মান, সীমানার মান (boundary), ও ভুল মান — সব ধরনের টেস্ট লেখা।

Step 3: বিভিন্ন assert method ব্যবহার করা।

Step 4: সব টেস্ট চালিয়ে ফলাফল দেখা।

Python Code

```
# A complete test suite using various assert methods
import unittest



def calculate_grade(marks):
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100")
    if marks >= 80:
        return "A+"
    elif marks >= 70:
        return "A"
    elif marks >= 60:
        return "B"
    elif marks >= 40:
        return "C"
    else:
        return "Fail"



def average_marks(marks_list):
    return sum(marks_list) / len(marks_list)



class TestGradeCalculator(unittest.TestCase):

    def test_grade_A_plus(self):
        self.assertEqual(calculate_grade(85), "A+")

    def test_grade_boundary_80(self):
        self.assertEqual(calculate_grade(80), "A+")     # exact boundary

    def test_grade_fail(self):
        self.assertEqual(calculate_grade(30), "Fail")

    def test_grade_is_not_fail_when_passing(self):
        self.assertNotEqual(calculate_grade(90), "Fail")

    def test_invalid_marks_raises_error(self):
        with self.assertRaises(ValueError):
            calculate_grade(150)

    def test_grade_in_valid_list(self):
```

```
        valid_grades = ["A+", "A", "B", "C", "Fail"]
        self.assertIn(calculate_grade(65), valid_grades)

    def test_average_marks(self):
        result = average_marks([70, 80, 90])
        self.assertAlmostEqual(result, 80.0, places=1)

    def test_average_is_true_when_passing(self):
        result = average_marks([80, 90, 100]) >= 40
        self.assertTrue(result)



if __name__ == "__main__":
    unittest.main(argv=[""], verbosity=2, exit=False)
```

Built-in Function / Shortcut Method

```
# Assert methods quick reference:
# assertEqual(a, b)          a == b
# assertNotEqual(a, b)       a != b
# assertTrue(x)              bool(x) is True
# assertFalse(x)             bool(x) is False
# assertIn(a, container)     a in container
# assertRaises(Error)        code block raises Error
# assertAlmostEqual(a, b)    a is approximately equal to b (for floats)
```

Sample Input / Output Table

Sample Input	Sample Output
None	test_average_is_true_when_passing (__main__.TestGradeCalculator) ... ok test_average_marks (__main__.TestGradeCalculator) ... ok test_grade_A_plus (__main__.TestGradeCalculator) ... ok test_grade_boundary_80 (__main__.TestGradeCalculator) ... ok test_grade_fail (__main__.TestGradeCalculator) ..

```
. ok
test_grade_in_valid_list
(__main__.TestGradeCalculator) ..
. ok
test_grade_is_not_fail_when_passi
ng (__main__.TestGradeCalculator)
... ok
test_invalid_marks_raises_error
(__main__.TestGradeCalculator) ..
. ok

----------------------------------
----------------------------------
----
Ran 8 tests in 0.002s

OK
```

ব্যাখ্যা

আটটি ভিন্ন assert পদ্ধতি ব্যবহার করে normal case (85 marks), boundary case (ঠিক 80), error case (150) — সব দিক থেকে function-টি যাচাই করা হয়েছে। সব 'ok' দেখাচ্ছে অর্থাৎ function সঠিকভাবে কাজ করছে।

Common Mistakes

- শুধু normal (সাধারণ) case টেস্ট করা, boundary ও error case বাদ দেওয়া।
- float তুলনায় `assertEqual` ব্যবহার করা (floating point এ সামান্য পার্থক্য থাকতে পারে) — `assertAlmostEqual` ব্যবহার করা উচিত।
- `assertRaises`-এর ব্যবহার ভুলভাবে করা (with ব্লক ছাড়া কল করলে কাজ করবে না)।

অনুশীলনের জন্য

1. একটি BMI calculator function-এর জন্য পূর্ণাঙ্গ test suite লিখুন।

2. boundary value (0, 100, negative) টেস্ট করে একটি validation function যাচাই করুন।

3. `assertIsInstance()` ব্যবহার করে return type যাচাই করুন।

Topic: Python RegEx (Regular Expression) — Unit 10

সমস্যা 269: RegEx কী তা বুঝুন এবং re.search() ব্যবহার করুন।

সমস্যা

সিলেবাসের 10.1 অনুযায়ী — Regular Expression (RegEx) কী তা বুঝে `re.search()` ব্যবহার করে একটি টেক্সটে নির্দিষ্ট শব্দ খুঁজে বের করুন।

কী কী জানা থাকতে হবে?

- import re
- String
- if-else

কীভাবে চিন্তা করব?

RegEx হলো টেক্সটের মধ্যে নির্দিষ্ট প্যাটার্ন (pattern) খোঁজার একটি বিশেষ ভাষা। `re.search(pattern, text)` টেক্সটের যেকোনো জায়গায় pattern খুঁজে পেলে একটি Match object ফেরত দেয়, না পেলে `None` দেয়।

অ্যালগরিদম

Step 1: `import re` করা।

Step 2: `re.search(pattern, text)` ব্যবহার করে খোঁজা।

Step 3: ফলাফল None কিনা যাচাই করা।

Step 4: `.group()` দিয়ে মিলে যাওয়া অংশ বের করা।

Python Code

```
# Introduction to RegEx using re.search()
import re

sentence = "Python is a powerful programming language for beginners"

# Search for a specific word
result = re.search("powerful", sentence)

if result:
    print("Match found:", result.group())
    print("Found at position:", result.start(), "to", result.end())
else:
    print("No match found")

# Searching for a word that does not exist
result2 = re.search("JavaScript", sentence)
if result2:
```

```
    print("Match found:", result2.group())
else:
    print("\n'JavaScript' not found in the sentence")

# Case-sensitive by default
result3 = re.search("PYTHON", sentence)
print("\nSearching 'PYTHON' (case-sensitive):", result3)

# Case-insensitive search using a flag
result4 = re.search("PYTHON", sentence, re.IGNORECASE)
print("Searching 'PYTHON' (case-insensitive):", result4.group() if result4 else
None)
```

Built-in Function / Shortcut Method

```
# re.search() vs re.match() vs re.findall()
# re.search()  -> finds the pattern ANYWHERE in the text
# re.match()   -> checks only at the BEGINNING of the text
# re.findall() -> finds ALL matches as a list
```

Sample Input / Output Table

Sample Input	Sample Output
None	Match found: powerful Found at position: 12 to 20 'JavaScript' not found in the sentence Searching 'PYTHON' (case-sensitive): None Searching 'PYTHON' (case-insensitive): Python

ব্যাখ্যা

`re.search()` টেক্সটের যেকোনো জায়গায় pattern খুঁজে বের করে। ডিফল্টভাবে এটি case-sensitive, তাই বড় হাতের 'PYTHON' ছোট হাতের 'Python'-এর সাথে মিলেনি — `re.IGNORECASE` flag ব্যবহার করে সমাধান করা হয়েছে।

Common Mistakes

- `re.search()`-এর ফলাফল সরাসরি print করে string ভাবা (আসলে এটি একটি Match object, `.group()` লাগবে)।
- None চেক না করে সরাসরি `.group()` কল করা (`AttributeError`)।
- case-sensitivity বিবেচনা না করা।

অনুশীলনের জন্য

1. একটি বাক্যে 'error' শব্দ আছে কিনা যাচাই করুন (case-insensitive)।

2. `re.match()` দিয়ে বাক্যের শুরুতে নির্দিষ্ট শব্দ আছে কিনা দেখুন।

3. `start()` ও `end()` ব্যবহার করে matched অংশের অবস্থান বের করুন।

সমস্যা 270: Meta Characters ও Sequence ব্যবহার করুন।

সমস্যা

সিলেবাসের 10.3 অনুযায়ী — RegEx-এর meta character (`.`, `^`, `$`, `*`, `+`, `?`) ও sequence (`\d`, `\w`, `\s`) ব্যবহার করে বিভিন্ন প্যাটার্ন খুঁজুন।

কী কী জানা থাকতে হবে?

- re.search()
- String

কীভাবে চিন্তা করব?

Meta character-গুলো সাধারণ অক্ষরের চেয়ে ভিন্ন অর্থ বহন করে — যেমন `.` মানে যেকোনো একটি অক্ষর, `\d` মানে যেকোনো digit (0-9), `\w` মানে letter/digit/underscore, `\s` মানে space। এগুলো একত্রে ব্যবহার করে জটিল প্যাটার্ন লেখা যায়।

> **[Best Practice]** RegEx pattern লেখার সময় সবসময় raw string ব্যবহার করুন: `r"\d+"`, `r"\w+"`। এতে backslash-জনিত সমস্যা এড়ানো যায়।

অ্যালগরিদম

Step 1: `\d+` দিয়ে একটি বা একাধিক digit খোঁজা।

Step 2: `\w+` দিয়ে word খোঁজা।

Step 3: `^` ও `$` দিয়ে শুরু/শেষ যাচাই করা।

Step 4: `.` দিয়ে যেকোনো অক্ষর মিলানো।

Python Code

```
# Meta characters and sequences in RegEx
import re

text = "My phone number is 01712345678 and my age is 25."

# \d -> matches a digit (0-9)
digits = re.findall(r"\d", text)
print("All single digits:", digits)

# \d+ -> matches one or more digits together
numbers = re.findall(r"\d+", text)
print("All numbers (grouped):", numbers)

# \w+ -> matches words (letters, digits, underscore)
words = re.findall(r"\w+", text)
print("\nAll words:", words)

# \s -> matches whitespace
spaces = re.findall(r"\s", text)
print("\nNumber of spaces:", len(spaces))

# ^ -> matches start of string
starts_with_my = re.match(r"^My", text)
print("\nStarts with 'My'?", bool(starts_with_my))

# $ -> matches end of string
ends_with_dot = re.search(r"\.$", text)
print("Ends with a dot?", bool(ends_with_dot))

# . -> matches any single character
pattern_match = re.findall(r"m.", text)
print("\n'm' followed by any character:", pattern_match)

# + -> one or more, * -> zero or more, ? -> zero or one
print("\n'a+' example:", re.findall(r"a+", "aaa bb a caaaat"))
```

Built-in Function / Shortcut Method

```
# Common sequences and meta characters
# .   -> any character except newline
# \d  -> digit [0-9]
# \D  -> non-digit
# \w  -> word character [a-zA-Z0-9_]
# \s  -> whitespace
```

```
# ^   -> start of string
# $   -> end of string
# *   -> 0 or more     + -> 1 or more     ? -> 0 or 1
```

Sample Input / Output Table

Sample Input	Sample Output

None	`All single digits: ['0', '1', '7', '1', '2', '3', '4', '5', '6', '7', '8', '2', '5']`
	`All numbers (grouped): ['01712345678', '25']`
	
	`All words: ['My', 'phone', 'number', 'is', '01712345678', 'and', 'my', 'age', 'is', '25']`
	
	`Number of spaces: 12`
	
	`Starts with 'My'? True`
	`Ends with a dot? True`
	
	`'m' followed by any character: ['m ', 'my']`
	
	`'a+' example: ['aaa', 'a', 'caaaat']`

ব্যাখ্যা

`\d` একটি করে digit ধরে, কিন্তু `\d+` পাশাপাশি থাকা সব digit একসাথে ধরে (`'01712345678'`)। `^` ও `$` যথাক্রমে শুরু ও শেষ পরীক্ষা করে। `r"..."` (raw string) ব্যবহার করা হয়েছে যাতে backslash ঠিকভাবে কাজ করে।

Common Mistakes

- raw string (`r"..."`) ব্যবহার না করে সাধারণ string দেওয়া — `\d` ভুলভাবে interpret হতে পারে।
- `\d` ও `\d+` গুলিয়ে ফেলা — একটি single digit, আরেকটি group of digits ধরে।
- `.` কে literal dot ভাবা — RegEx-এ `.` মানে 'যেকোনো অক্ষর', literal dot বোঝাতে `\.` লিখতে হয়।

অনুশীলনের জন্য

1. একটি বাক্য থেকে সব email-এর মতো শব্দ (`\w+@\w+`) বের করুন।

2. `\D` (non-digit) ব্যবহার করে সব অক্ষর বের করুন।

3. `^` ও `$` দিয়ে একটি স্ট্রিং পুরোপুরি সংখ্যা কিনা যাচাই করুন।

সমস্যা 271: findall() ও finditer() ব্যবহার করে সব Match বের করুন।

সমস্যা

`re.findall()` ও `re.finditer()` ব্যবহার করে একটি টেক্সট থেকে সব মিলে যাওয়া অংশ (matches) বের করুন।

কী কী জানা থাকতে হবে?

- Meta characters
- for loop
- List

কীভাবে চিন্তা করব?

`re.search()` শুধু প্রথম মিল খুঁজে পায়। কিন্তু `re.findall()` টেক্সটের সব মিল একটি list আকারে দেয়, আর `re.finditer()` প্রতিটি মিলের জন্য একটি Match object দেয় (অবস্থানসহ)। বড় টেক্সট থেকে সব তথ্য বের করতে এগুলো ব্যবহৃত হয়।

অ্যালগরিদম

Step 1: একটি টেক্সট থেকে `findall()` দিয়ে সব phone number বের করা।

Step 2: `finditer()` দিয়ে প্রতিটি matched অংশের অবস্থানসহ প্রিন্ট করা।

Step 3: দুটোর ফলাফলের পার্থক্য তুলনা করা।

Python Code

```
# findall() and finditer()
import re

text = "Contact us at 01712345678 or 01898765432. Office landline: 029876543"

# findall() - returns a simple list of matches
phone_numbers = re.findall(r"01\d{9}", text)
print("Mobile numbers found (findall):", phone_numbers)

# finditer() - returns match objects with position information
print("\nUsing finditer() for detailed information:")
for match in re.finditer(r"01\d{9}", text):
    print("Found:", match.group(), "at position", match.start(), "-",
match.end())
```

```
# Practical example: extract all prices from a sentence
price_text = "Mango costs 120 taka, Apple costs 250 taka, Banana costs 40 taka"
prices = re.findall(r"\d+", price_text)
print("\nAll prices found:", prices)

total = sum(int(p) for p in prices)
print("Total price:", total, "Taka")

# Extract all words that start with a capital letter
names_text = "Rahim and Karim went to Dhaka with Sadia"
capitalized_words = re.findall(r"[A-Z]\w*", names_text)
print("\nCapitalized words:", capitalized_words)
```

Built-in Function / Shortcut Method

```
# {n} means exactly n repetitions
# \d{9}   -> exactly 9 digits
# \d{2,4} -> between 2 and 4 digits
```

Sample Input / Output Table

Sample Input	Sample Output
None	Mobile numbers found (findall): ['01712345678', '01898765432'] Using finditer() for detailed information: Found: 01712345678 at position 14 - 25 Found: 01898765432 at position 29 - 40 All prices found: ['120', '250', '40'] Total price: 410 Taka Capitalized words: ['Rahim', 'Karim', 'Dhaka', 'Sadia']

ব্যাখ্যা

`findall()` সরাসরি ফলাফলের list দেয়, যা সহজে ব্যবহার করা যায়। `finditer()` প্রতিটি মিলের সঠিক অবস্থান (position) জানায়, যা বড় টেক্সটে নির্দিষ্ট জায়গা চিহ্নিত করতে দরকার হয়।

Common Mistakes

- `{9}` ভুলে যাওয়া, ফলে দৈর্ঘ্য নিয়ন্ত্রণ না হওয়া (landline নম্বরও ধরা পড়তে পারে)।
- `findall()`-এর ফলাফলকে integer ভাবা — এগুলো string, `int()` দিয়ে রূপান্তর দরকার।
- `finditer()`-এর ফলাফল সরাসরি print করে list ভাবা (এটি একটি iterator, loop চালাতে হয়)।

অনুশীলনের জন্য

1. একটি প্যারাগ্রাফ থেকে সব email address বের করুন।

2. একটি টেক্সট থেকে সব তারিখ (DD-MM-YYYY ফরম্যাট) বের করুন।

3. `finditer()` ব্যবহার করে সব মিলের মোট সংখ্যা গণনা করুন।

সমস্যা 272: Email ও Phone Number Validation করুন (RegEx)।

সমস্যা

সিলেবাসের 10.5 অনুযায়ী — RegEx ব্যবহার করে একটি Email এবং Bangladeshi Phone Number সঠিক ফরম্যাটে আছে কিনা যাচাই করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- re.match()
- re.fullmatch()
- Meta characters

কীভাবে চিন্তা করব?

Validation-এর জন্য পুরো স্ট্রিং প্যাটার্নের সাথে পুরোপুরি মিলছে কিনা যাচাই করতে হয় — এর জন্য `re.fullmatch()` ব্যবহার করা হয়। Email প্যাটার্ন: `username@domain.extension`। বাংলাদেশি ফোন নম্বর: `01` দিয়ে শুরু, মোট ১১ ডিজিট।

> **[Important]** Practical-2/6-এর মতো প্রজেক্টে ইউজারের Phone number ইনপুট নেওয়ার সময় RegEx দিয়ে validation করা professional practice — ভুল ডেটা ডাটাবেসে/CSV-তে যাওয়া রোধ করে।

অ্যালগরিদম

Step 1: Email-এর জন্য প্যাটার্ন লেখা: `\w+@\w+\.\w+`।

Step 2: Phone-এর জন্য প্যাটার্ন লেখা: `01[3-9]\d{8}`।

Step 3: `re.fullmatch()` দিয়ে পুরো স্ট্রিং যাচাই করা।

Step 4: বিভিন্ন সঠিক/ভুল ইনপুট দিয়ে টেস্ট করা।

Python Code

```
# Email and Phone number validation using RegEx
import re



def is_valid_email(email):
    pattern = r"[\w.]+@[\w]+\.[a-zA-Z]{2,}"
    return bool(re.fullmatch(pattern, email))



def is_valid_bd_phone(phone):
    # Bangladeshi mobile: starts with 01, next digit 3-9, then 8 more digits
    pattern = r"01[3-9]\d{8}"
    return bool(re.fullmatch(pattern, phone))



emails = ["rahim@gmail.com", "karim.hossain@yahoo.com", "invalid-email",
"test@@gmail.com", "rahim@gmail"]

print("--- Email Validation ---")
for email in emails:
    status = "Valid" if is_valid_email(email) else "Invalid"
    print(email, "->", status)

phones = ["01712345678", "01898765432", "0171234567", "1712345678",
"02912345678"]

print("\n--- Phone Number Validation ---")
for phone in phones:
    status = "Valid" if is_valid_bd_phone(phone) else "Invalid"
    print(phone, "->", status)

# Interactive validation
print("\n--- Interactive check ---")
user_email = input("Enter your email: ")
if is_valid_email(user_email):
    print("Email accepted!")
else:
    print("Invalid email format!")
```

Built-in Function / Shortcut Method

```
# fullmatch() vs match() vs search()
# fullmatch() -> the ENTIRE string must match the pattern (best for validation)
# match()     -> only checks from the BEGINNING
# search()    -> checks ANYWHERE in the string
```

Sample Input / Output Table

Sample Input	Sample Output
rahim@gmail.com	--- Email Validation --- rahim@gmail.com -> Valid karim.hossain@yahoo.com -> Valid invalid-email -> Invalid test@@gmail.com -> Invalid rahim@gmail -> Invalid --- Phone Number Validation --- 01712345678 -> Valid 01898765432 -> Valid 0171234567 -> Invalid 1712345678 -> Invalid 02912345678 -> Invalid --- Interactive check --- Enter your email: rahim@gmail.com Email accepted! Invalid email format!

ব্যাখ্যা

`re.fullmatch()` ব্যবহার করায় পুরো string প্যাটার্নের সাথে হুবহু মিলতে হয়েছে — আংশিক মিললে চলবে না। এতে ভুল ফরম্যাটের email/phone সঠিকভাবে বাতিল হয়েছে।

Common Mistakes

- `re.search()` ব্যবহার করে validation করা — এটি আংশিক মিল দিয়েও True দেবে, যা ভুল।
- phone number-এ দেশের কোড (+880) বিবেচনা না করা।
- email প্যাটার্নে domain extension-এর জন্য নির্দিষ্ট দৈর্ঘ্য না রাখা (`.c` মতো ভুল ফরম্যাট গ্রহণ হয়ে যেতে পারে)।

অনুশীলনের জন্য

1. +880 সহ phone number validation করুন।

2. একটি password validation করুন (কমপক্ষে ১টি বড় হাতের অক্ষর, ১টি সংখ্যা)।

3. NID (জাতীয় পরিচয়পত্র) নম্বর (১০ বা ১৭ ডিজিট) validation করুন।

সমস্যা 273: sub() ব্যবহার করে টেক্সট প্রতিস্থাপন (Replace) করুন।

সমস্যা

`re.sub()` ব্যবহার করে টেক্সটের মধ্যে প্যাটার্ন মিলে যাওয়া অংশ নতুন কিছু দিয়ে প্রতিস্থাপন করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- re.findall()
- String

কীভাবে চিন্তা করব?

`re.sub(pattern, replacement, text)` টেক্সটের মধ্যে pattern-এর সাথে মিলে যাওয়া সব অংশ replacement দিয়ে বদলে দেয়। এটি `str.replace()`-এর চেয়ে শক্তিশালী কারণ এখানে প্যাটার্ন (নির্দিষ্ট শব্দ নয়) ব্যবহার করা যায়।

অ্যালগরিদম

Step 1: টেক্সট থেকে সব digit মুছে ফেলা (`re.sub(r'\d', '', text)`)।

Step 2: phone number-এর কিছু অংশ masked (গোপন) করা।

Step 3: একাধিক স্পেসকে একটি স্পেসে রূপান্তর করা।

Python Code

```
# Replacing text using re.sub()
import re

text = "My roll is 101 and my phone is 01712345678"

# 1. Remove all digits
no_digits = re.sub(r"\d", "", text)
print("Without digits:", no_digits)

# 2. Replace multiple spaces with a single space
messy_text = "This    text   has     extra    spaces"
clean_text = re.sub(r"\s+", " ", messy_text)
print("\nBefore:", messy_text)
print("After :", clean_text)

# 3. Masking a phone number (privacy) - keep first 3 and last 2 digits
```

```
phone = "01712345678"
masked = re.sub(r"(\d{3})\d{6}(\d{2})", r"\1******\2", phone)
print("\nOriginal phone:", phone)
print("Masked phone  :", masked)

# 4. Replace bad words with asterisks (simple content filter)
comment = "This movie is stupid and bad"
bad_words = ["stupid", "bad"]
filtered_comment = comment
for word in bad_words:
    filtered_comment = re.sub(word, "***", filtered_comment,
flags=re.IGNORECASE)
print("\nOriginal comment:", comment)
print("Filtered comment:", filtered_comment)

# 5. Replace all non-alphanumeric characters
messy_string = "Hello@#$World!!!123"
clean_string = re.sub(r"[^a-zA-Z0-9]", "", messy_string)
print("\nCleaned string:", clean_string)
```

Built-in Function / Shortcut Method

```
# re.sub(pattern, replacement, text, count=0, flags=0)
# count=0 means replace ALL matches (default)
# Use \1, \2 in replacement to refer to captured groups
```

Sample Input / Output Table

Sample Input	Sample Output
None	Without digits: My roll is and my phone is Before: This text has extra spaces After : This text has extra spaces Original phone: 01712345678 Masked phone : 017******78 Original comment: This movie is stupid and bad Filtered comment: This movie is *** and ***

```
                                        Number of spaces: 12
                                        Cleaned string: HelloWorld123
```

ব্যাখ্যা

`re.sub()` তিনটি জিনিস নেয়: প্যাটার্ন, প্রতিস্থাপনকারী টেক্সট এবং মূল টেক্সট। Grouping `()` ও `\1`, `\2` ব্যবহার করে নির্দিষ্ট অংশ রেখে বাকি অংশ বদলানো গেছে (phone masking-এর উদাহরণে)।

Common Mistakes

- group রেফারেন্স (`\1`) replacement string-এ raw string না দিয়ে ব্যবহার করা (backslash সমস্যা করতে পারে)।
- `re.sub()`-এর ফলাফল ভুলে মূল ভ্যারিয়েবলে সেভ না করা (এটি নতুন string ফেরত দেয়, মূল string বদলায় না)।
- `flags=re.IGNORECASE` ভুলে যাওয়া, ফলে case-sensitive replace হওয়া।

অনুশীলনের জন্য

1. একটি টেক্সট থেকে সব HTML ট্যাগ (`<...>`) সরান।

2. একটি ফোন নম্বরের মাঝের ৫টি ডিজিট masking করুন।

3. একটি প্যারাগ্রাফের সব সংখ্যা 'X' দিয়ে replace করুন।

সমস্যা 274: split() ব্যবহার করে RegEx দিয়ে টেক্সট ভাগ করুন।

সমস্যা

`re.split()` ব্যবহার করে একাধিক ডিলিমিটার (delimiter) দিয়ে একটি টেক্সটকে ভাগ করার প্রোগ্রাম লিখুন।

কী কী জানা থাকতে হবে?

- re.sub()
- String split()

কীভাবে চিন্তা করব?

সাধারণ `str.split(',')` শুধু একটি ডিলিমিটার দিয়ে ভাগ করতে পারে। কিন্তু বাস্তব ডেটায় কমা, সেমিকোলন, স্পেস — একাধিক ডিলিমিটার মিশ্রিত থাকতে পারে। `re.split(pattern, text)` একসাথে একাধিক ডিলিমিটার দিয়ে ভাগ করতে পারে।

অ্যালগরিদম

Step 1: একটি মিশ্র-ডিলিমিটার টেক্সট তৈরি করা (কমা, সেমিকোলন, স্পেস)।

Step 2: `re.split(r'[,;\s]+', text)` ব্যবহার করে ভাগ করা।

Step 3: সাধারণ `str.split()`-এর সাথে তুলনা করা।

Python Code

```
# Splitting text using re.split()
import re

data = "Mango, Apple; Banana Orange,  Grapes"

# Normal split() can only handle ONE delimiter
normal_split = data.split(",")
print("Using str.split(',':", normal_split)

# re.split() can handle MULTIPLE delimiters at once
regex_split = re.split(r"[,;]\s*", data)
print("\nUsing re.split() with [,;]:", regex_split)

# Splitting on any whitespace or punctuation combination
mixed_text = "Rahim,Karim;Sadia Tania,,Imran"
clean_split = re.split(r"[,;\s]+", mixed_text)
print("\nMixed delimiters:", clean_split)

# Splitting a sentence into words (ignoring punctuation)
sentence = "Hello, World! How are you? I am fine."
words = re.split(r"[ ,.!?]+", sentence)
words = [w for w in words if w != ""]     # remove empty strings
print("\nWords from sentence:", words)

# Splitting with a limit (maxsplit)
limited_split = re.split(r",\s*", "a, b, c, d, e", maxsplit=2)
print("\nSplit with maxsplit=2:", limited_split)
```

Built-in Function / Shortcut Method

```
# re.split(pattern, string, maxsplit=0)
# [,;\s]+ means: split on comma, semicolon or whitespace (one or more)
```

Sample Input / Output Table

Sample Input	Sample Output
None	Using str.split(','): ['Mango', ' Apple; Banana Orange', ' Grapes'] Using re.split() with [,;]:

```
['Mango', 'Apple', 'Banana
Orange', 'Grapes']

Mixed delimiters: ['Rahim',
'Karim', 'Sadia', 'Tania',
'Imran']

Words from sentence: ['Hello',
'World', 'How', 'are', 'you',
'I', 'am', 'fine']

Split with maxsplit=2: ['a', 'b',
'c, d, e']
```

ব্যাখ্যা

সাধারণ `split(',')` শুধু কমা দিয়ে ভাগ করেছে, ফলে সেমিকোলন ও স্পেস মিশ্রিত থেকে গেছে। কিন্তু `re.split(r'[,;\s]+', ...)` কমা, সেমিকোলন ও একাধিক স্পেস — সবকিছুকে ডিলিমিটার হিসেবে ধরে সঠিকভাবে ভাগ করেছে।

Common Mistakes

- `[,;\s]+`-এর বদলে `[,;\s]` (plus চিহ্ন ছাড়া) লেখা — তাহলে একাধিক ডিলিমিটার পাশাপাশি থাকলে ফাঁকা string তৈরি হয়।
- split করার পর ফাঁকা string (`''`) filter না করা।
- raw string ব্যবহার না করে pattern লেখা।

অনুশীলনের জন্য

1. CSV-এর মতো একটি লাইনকে কমা ও ট্যাব দিয়ে ভাগ করুন।

2. একটি প্যারাগ্রাফকে বাক্যে ভাগ করুন (`.`, `!`, `?` দিয়ে)।

3. `maxsplit` ব্যবহার করে প্রথম দুইটি অংশ আলাদা করুন, বাকিটা একসাথে রাখুন।

□ Practical Experiments (Syllabus: 28531 — Practical 1 to 6)

প্রজেক্ট 1: Inventory Management for Fruits Item (CSV ব্যবহার করে)

উদ্দেশ্য (Objective)

ব্যবহারকারীর কাছ থেকে ফলের নাম, একক মূল্য ও পরিমাণ ইনপুট নিয়ে CSV ফাইলে সংরক্ষণ করা এবং সংরক্ষিত CSV ফাইল থেকে সারাংশ (summary) রিপোর্ট প্রদর্শন করা।

প্রয়োজনীয় সেটআপ (Requirements)

- Python 3 ইনস্টল করা কম্পিউটার
- কোনো অতিরিক্ত লাইব্রেরি লাগবে না — শুধু বিল্ট-ইন `csv` module
- একটি টেক্সট এডিটর বা IDE (VS Code / PyCharm / IDLE)

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: টার্মিনাল/কমান্ড প্রম্পট খুলে প্রজেক্টের জন্য একটি ফোল্ডার তৈরি করুন।

Step 2: ব্যবহারকারীর কাছ থেকে item data নিন (ফলের নাম, একক মূল্য, পরিমাণ, মোট মূল্য)।

Step 3: `csv.writer` বা `csv.DictWriter` ব্যবহার করে ডেটা 'fruits_inventory.csv' ফাইলে সংরক্ষণ করুন।

Step 4: সংরক্ষিত CSV ফাইল থেকে ডেটা পড়ে সারাংশ (মোট আইটেম, মোট মূল্য) প্রদর্শন করুন।

Step 5: একটি মেনু-ভিত্তিক লুপ তৈরি করুন যাতে ব্যবহারকারী বারবার আইটেম যোগ করতে বা রিপোর্ট দেখতে পারে।

সম্পূর্ণ Python Code

```
# ============================================================
# PRACTICAL 1: Inventory Management for Fruits Item (CSV)
# ============================================================
import csv
import os

FILENAME = "fruits_inventory.csv"
FIELDS = ["Item Name", "Unit Price", "Quantity", "Total Price"]



def setup_file():
    """Create the CSV file with headers if it does not exist yet."""
    if not os.path.exists(FILENAME):
        with open(FILENAME, "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(FIELDS)
        print("New inventory file created:", FILENAME)



def take_item_data():
    """Take fruit item data from the user."""
    print("\n--- Enter Fruit Item Details ---")
```

```
    name = input("Fruit name: ")
    unit_price = float(input("Unit price (Taka): "))
    quantity = int(input("Quantity: "))
    total_price = unit_price * quantity
    return [name, unit_price, quantity, total_price]



def store_item_to_csv(item):
    """Store one item's data into the CSV file (append mode)."""
    with open(FILENAME, "a", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(item)
    print("Item saved successfully!")



def print_summary():
    """Read the CSV file and print a summary report."""
    if not os.path.exists(FILENAME):
        print("No inventory data found yet!")
        return

    with open(FILENAME, "r", newline="") as file:
        reader = csv.reader(file)
        header = next(reader)

        print("\n" + "=" * 55)
        print("             FRUITS INVENTORY SUMMARY")
        print("=" * 55)
        print("{:<15}{:<12}{:<10}{:<12}".format(*header))
        print("-" * 55)

        total_items = 0
        grand_total = 0.0

        for row in reader:
            name, unit_price, quantity, total_price = row
            print("{:<15}{:<12}{:<10}{:<12}".format(name, unit_price, quantity,
total_price))
            total_items += 1
            grand_total += float(total_price)

        print("-" * 55)
        print("Total items entered :", total_items)
        print("Grand total value   :", round(grand_total, 2), "Taka")
        print("=" * 55)
```

```
def main():
    setup_file()
    while True:
        print("\n====== FRUITS INVENTORY MANAGEMENT ======")
        print("1. Add new fruit item")
        print("2. Show inventory summary")
        print("3. Exit")
        choice = input("Choose an option (1-3): ")

        if choice == "1":
            item = take_item_data()
            store_item_to_csv(item)
        elif choice == "2":
            print_summary()
        elif choice == "3":
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice! Please select 1, 2 or 3.")


if __name__ == "__main__":
    main()
```

Sample Output

```
New inventory file created: fruits_inventory.csv

====== FRUITS INVENTORY MANAGEMENT ======
1. Add new fruit item
2. Show inventory summary
3. Exit
Choose an option (1-3): 1

--- Enter Fruit Item Details ---
Fruit name: Mango
Unit price (Taka): 120
Quantity: 5
Item saved successfully!

====== FRUITS INVENTORY MANAGEMENT ======
Choose an option (1-3): 2

============================================================
              FRUITS INVENTORY SUMMARY
```

```
=========================================================
Item Name      Unit Price  Quantity  Total Price
---------------------------------------------------------
Mango          120.0       5         600.0
---------------------------------------------------------
Total items entered : 1
Grand total value   : 600.0 Taka
=========================================================

====== FRUITS INVENTORY MANAGEMENT ======
Choose an option (1-3): 3
Exiting program. Goodbye!
```

ব্যাখ্যা

- `setup_file()` প্রথমবার চালানোর সময় header সহ ফাইল তৈরি করে — বারবার header লেখা এড়ায়।
- 'a' (append) mode ব্যবহার করায় প্রতিবার নতুন আইটেম যোগ করলে আগের ডেটা মুছে যায় না।
- মেনু-ভিত্তিক while loop ব্যবহারকারীকে বারবার আইটেম যোগ ও রিপোর্ট দেখার সুযোগ দেয় — এটি একটি বাস্তব CLI অ্যাপ্লিকেশনের কাঠামো।
- `csv.reader`/`csv.writer` ব্যবহার করায় কোনো তৃতীয় পক্ষের লাইব্রেরি লাগেনি — শুধু বিল্ট-ইন module দিয়েই কাজ হয়েছে।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. CSV ফাইল খোলার সময় `newline=""` কেন ব্যবহার করা হয়েছে?

2. 'w' ও 'a' mode-এর মধ্যে পার্থক্য কী? এখানে কেন 'a' ব্যবহার করা হয়েছে?

3. `csv.writer` ও `csv.DictWriter`-এর মধ্যে পার্থক্য কী?

4. এই প্রোগ্রামে যদি ইউজার ভুল (non-numeric) দাম দেয়, তাহলে কী সমস্যা হতে পারে এবং কীভাবে সমাধান করা যায়?

প্রজেক্ট 2: Generate PDF Report from CSV File (Student Information System)

উদ্দেশ্য (Objective)

শিক্ষার্থীর তথ্য (ID, Name, Roll, Semester, Shift, Department, DOB, Phone) CSV ফাইলে সংরক্ষণ করে সেখান থেকে একটি সুসংগঠিত PDF রিপোর্ট তৈরি করা।

প্রয়োজনীয় সেটআপ (Requirements)

- `pip install reportlab pandas` — দুইটি এক্সটার্নাল লাইব্রেরি ইনস্টল করতে হবে
- CSV ফাইল সংরক্ষণের জন্য বিল্ট-ইন `csv` module

- একটি PDF viewer (রেজাল্ট যাচাই করার জন্য)

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: `pip install reportlab pandas` কমান্ড দিয়ে লাইব্রেরি ইনস্টল করুন।

Step 2: শিক্ষার্থীর মৌলিক তথ্য ইনপুট নিয়ে CSV ফাইলে সংরক্ষণ করুন।

Step 3: `pandas.read_csv()` দিয়ে CSV ফাইল থেকে ডেটা পড়ুন।

Step 4: `reportlab` ব্যবহার করে প্রতিটি শিক্ষার্থীর তথ্য টেবিল আকারে PDF রিপোর্টে দেখান।

Step 5: PDF ফাইলটি সেভ করে যাচাই করুন যে সব তথ্য সঠিকভাবে এসেছে।

সম্পূর্ণ Python Code

```
# ============================================================
# PRACTICAL 2: Generate PDF Report from CSV File
# Install first:  pip install reportlab pandas
# ============================================================
import csv
import os
import pandas as pd
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from reportlab.lib.units import inch

CSV_FILE = "students_info.csv"
PDF_FILE = "students_report.pdf"
FIELDS = ["Student_Id", "Name", "Roll", "Semester", "Shift",
          "Department", "Date_Of_Birth", "Phone_number"]


def get_student_data():
    """Take student basic information from the user."""
    print("\n--- Enter Student Information ---")
    return {
        "Student_Id": input("Student ID: "),
        "Name": input("Name: "),
        "Roll": input("Roll: "),
        "Semester": input("Semester: "),
        "Shift": input("Shift: "),
        "Department": input("Department: "),
        "Date_Of_Birth": input("Date of Birth (DD-MM-YYYY): "),
        "Phone_number": input("Phone number: "),
    }


def store_data_to_csv(data):
    """Store one student's data into the CSV file."""
    file_exists = os.path.exists(CSV_FILE)
```

```
    with open(CSV_FILE, "a", newline="") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=FIELDS)
        if not file_exists:
            writer.writeheader()
        writer.writerow(data)
    print("Student data added successfully!")



def generate_pdf_report():
    """Read the CSV file with pandas and generate a PDF report."""
    if not os.path.exists(CSV_FILE):
        print("No student data found. Please add students first!")
        return

    data_frame = pd.read_csv(CSV_FILE)

    c = canvas.Canvas(PDF_FILE, pagesize=letter)
    width, height = letter

    # Title
    c.setFont("Helvetica-Bold", 16)
    c.drawString(1 * inch, height - 1 * inch, "STUDENT INFORMATION REPORT")

    # Table header
    c.setFont("Helvetica-Bold", 10)
    y_position = height - 1.5 * inch
    headers = ["ID", "Name", "Roll", "Semester", "Shift", "Department"]
    x_positions = [0.7, 1.8, 3.6, 4.2, 5.0, 5.7]

    for header, x in zip(headers, x_positions):
        c.drawString(x * inch, y_position, header)

    # Table rows
    c.setFont("Helvetica", 9)
    y_position -= 0.3 * inch

    for index, row in data_frame.iterrows():
        values = [str(row["Student_Id"]), str(row["Name"]), str(row["Roll"]),
                  str(row["Semester"]), str(row["Shift"]),
str(row["Department"])]
        for value, x in zip(values, x_positions):
            c.drawString(x * inch, y_position, value)
        y_position -= 0.25 * inch

        # Start a new page if we run out of space
        if y_position < 1 * inch:
```

```
            c.showPage()
            y_position = height - 1 * inch

    c.save()
    print("PDF report generated successfully:", PDF_FILE)


def main():
    while True:
        print("\n====== STUDENT INFO MANAGEMENT ======")
        print("1. Add new student info")
        print("2. Generate PDF report")
        print("3. Exit")
        choice = input("Choose an option: ")

        if choice == "1":
            data = get_student_data()
            store_data_to_csv(data)
        elif choice == "2":
            generate_pdf_report()
        elif choice == "3":
            print("Exiting the program.")
            break
        else:
            print("Invalid choice, please try again.")


if __name__ == "__main__":
    main()
```

মূল GitHub কোডের সংশোধন নোট (Bug Fix Notes)

```
# Original GitHub code had these issues (fixed above):
#
# 1. Missing import:
#    from reportlab.lib.units import inch   <-- was missing, caused NameError
#
# 2. Canvas created twice:
#    c = canvas.Canvas(pdf_filename, pagesize=letter)   # first time
#    ...
#    c = canvas.Canvas("students_report.pdf")           # second time (BUG!
overwrites the first)
#    Fix: create the Canvas only ONCE.
#
# 3. No header row written in the PDF table (only raw row.values dumped).
#    Fix: explicitly draw header labels before the data rows.
```

```
#
# 4. No pagination - if there are many students, text would overflow the page.
#    Fix: added c.showPage() when y_position gets too low.
```

Sample Output

```
====== STUDENT INFO MANAGEMENT ======
1. Add new student info
2. Generate PDF report
3. Exit
Choose an option: 1

--- Enter Student Information ---
Student ID: 2025-CSE-01
Name: Rahim Ahmed
Roll: 101
Semester: 3rd
Shift: 1st
Department: CSE
Date of Birth (DD-MM-YYYY): 15-03-2004
Phone number: 01712345678
Student data added successfully!

====== STUDENT INFO MANAGEMENT ======
Choose an option: 2
PDF report generated successfully: students_report.pdf

====== STUDENT INFO MANAGEMENT ======
Choose an option: 3
Exiting the program.
```

ব্যাখ্যা

- মূল GitHub কোডে কিছু ভুল ছিল (যেমন `inch` import না করা এবং একই `c` ভ্যারিয়েবল দুইবার তৈরি করা) — এই সংশোধিত সংস্করণে `from reportlab.lib.units import inch` যোগ করা হয়েছে এবং Canvas একবারই তৈরি করা হয়েছে।
- `pandas.read_csv()` CSV-কে DataFrame-এ রূপান্তর করে, যা দিয়ে `.iterrows()` ব্যবহার করে প্রতিটি সারি সহজে পড়া যায়।
- `writeheader()` শুধুমাত্র ফাইল প্রথমবার তৈরি হওয়ার সময় লেখা হয় (`file_exists` চেক করে) — এতে বারবার রান করলে header ডুপ্লিকেট হয় না।
- পৃষ্ঠা শেষ হয়ে গেলে `c.showPage()` দিয়ে নতুন পৃষ্ঠা তৈরি করা হয়েছে, যাতে অনেক শিক্ষার্থী থাকলেও রিপোর্ট সঠিকভাবে দেখা যায়।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. `reportlab` কী কাজে ব্যবহার করা হয়েছে এবং কেন `pip install` করতে হলো?

2. `pandas.read_csv()` এর পরিবর্তে `csv.DictReader` ব্যবহার করা যেত কি? দুটোর পার্থক্য কী?

3. PDF-এ কোনো নতুন পৃষ্ঠা তৈরি করতে কোন function ব্যবহার করা হয়েছে?

4. যদি CSV ফাইল খালি থাকে, তাহলে প্রোগ্রামটি কী আচরণ করবে?

প্রজেক্ট 3: QR Code & Barcode Generator and Decoding

উদ্দেশ্য (Objective)

শিক্ষার্থীর ID ও নাম দিয়ে QR Code ও Barcode তৈরি করা এবং সেগুলো থেকে আবার তথ্য Decode করে দেখানো।

প্রয়োজনীয় সেটআপ (Requirements)

- `pip install qrcode[pil] python-barcode pillow pyzbar` — লাইব্রেরি ইনস্টল
- Linux-এ `pyzbar`-এর জন্য: `sudo apt-get install libzbar0` (প্রয়োজন হতে পারে)
- একটি ইমেজ ভিউয়ার (তৈরি হওয়া QR/Barcode ছবি দেখার জন্য)

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: প্রয়োজনীয় লাইব্রেরি ইনস্টল করুন।

Step 2: ব্যবহারকারীর কাছ থেকে student ID ও name নিন।

Step 3: `qrcode` লাইব্রেরি দিয়ে QR Code তৈরি করে ছবি (.png) হিসেবে সেভ করুন।

Step 4: `barcode` লাইব্রেরি দিয়ে Barcode তৈরি করে সেভ করুন।

Step 5: `pyzbar` দিয়ে তৈরি হওয়া QR/Barcode ছবি পড়ে ভেতরের ডেটা Decode করে প্রিন্ট করুন।

সম্পূর্ণ Python Code

```
# ============================================================
# PRACTICAL 3: QR Code & Barcode Generator and Decoding
# Install first:
#   pip install qrcode[pil] python-barcode pillow pyzbar
# ============================================================
import qrcode
import barcode
from barcode.writer import ImageWriter
from pyzbar.pyzbar import decode
from PIL import Image


def get_student_data():
    student_id = input("Enter Student ID: ")
```

```
    name = input("Enter Student Name: ")
    return student_id, name



def create_qr_code(student_id, name):
    data = "ID:" + student_id + " | Name:" + name
    qr = qrcode.QRCode(version=1, box_size=10, border=4)
    qr.add_data(data)
    qr.make(fit=True)

    img = qr.make_image(fill_color="black", back_color="white")
    filename = "student_qr_" + student_id + ".png"
    img.save(filename)
    print("QR Code generated:", filename)
    return filename



def create_barcode(student_id):
    # Barcode formats usually accept only numeric/alphanumeric codes
    code128 = barcode.get_barcode_class("code128")
    barcode_obj = code128(student_id, writer=ImageWriter())
    filename = barcode_obj.save("student_barcode_" + student_id)
    print("Barcode generated:", filename)
    return filename



def decode_image(filename):
    image = Image.open(filename)
    decoded_objects = decode(image)

    if not decoded_objects:
        print("No code detected in", filename)
        return

    for obj in decoded_objects:
        print("Type   :", obj.type)
        print("Data   :", obj.data.decode("utf-8"))



def main():
    student_id, name = get_student_data()

    print("\n--- Generating QR Code ---")
    qr_file = create_qr_code(student_id, name)
```

```
    print("\n--- Generating Barcode ---")
    bar_file = create_barcode(student_id)

    print("\n--- Decoding QR Code ---")
    decode_image(qr_file)

    print("\n--- Decoding Barcode ---")
    decode_image(bar_file)


if __name__ == "__main__":
    main()
```

Sample Output

```
Enter Student ID: 101
Enter Student Name: Rahim Ahmed

--- Generating QR Code ---
QR Code generated: student_qr_101.png

--- Generating Barcode ---
Barcode generated: student_barcode_101.png

--- Decoding QR Code ---
Type   : QRCODE
Data   : ID:101 | Name:Rahim Ahmed

--- Decoding Barcode ---
Type   : CODE128
Data   : 101
```

ব্যাখ্যা

- `qrcode` লাইব্রেরি টেক্সট ডেটাকে QR Code ছবিতে রূপান্তর করে — এখানে ID ও Name একসাথে একটি স্ট্রিং হিসেবে এনকোড করা হয়েছে।
- `python-barcode` লাইব্রেরির CODE128 ফরম্যাট numeric/alphanumeric ডেটার জন্য ব্যাপকভাবে ব্যবহৃত হয় — এখানে শুধু Student ID এনকোড করা হয়েছে।
- `pyzbar.decode()` ছবির মধ্যে থাকা QR/Barcode খুঁজে বের করে তার ভেতরের raw ডেটা bytes আকারে দেয়, যা `.decode('utf-8')` দিয়ে string-এ রূপান্তর করা হয়েছে।
- Decode করার পর মূল ইনপুট আর ফলাফল মিলিয়ে দেখা যায় যে এনকোডিং-ডিকোডিং প্রক্রিয়া সঠিকভাবে কাজ করেছে।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. QR Code ও Barcode-এর মধ্যে মূল পার্থক্য কী?

2. `qrcode.make_image()`-এ `fill_color` ও `back_color` কী কাজ করে?

3. `pyzbar`-এর `decode()` ফাংশন কী ফেরত দেয় এবং কেন `.decode('utf-8')` ব্যবহার করা হয়েছে?

4. যদি ছবিতে কোনো QR/Barcode না থাকে, তাহলে প্রোগ্রামটি কীভাবে সেটি সামলাবে?

প্রজেক্ট 4: Web Scraping (Book Information Extraction)

উদ্দেশ্য (Objective)

একটি ওয়েবসাইট থেকে HTML সোর্স কোড সংগ্রহ করে বইয়ের নাম, লেখকের নাম ও দাম বের করে CSV ফাইলে সংরক্ষণ করা।

প্রয়োজনীয় সেটআপ (Requirements)

- `pip install requests beautifulsoup4` — লাইব্রেরি ইনস্টল
- ইন্টারনেট সংযোগ
- যে ওয়েবসাইট স্ক্র্যাপ করা হবে তার HTML গঠন সম্পর্কে ধারণা (Inspect Element দিয়ে যাচাই)

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: `pip install requests beautifulsoup4` কমান্ড চালান।

Step 2: `requests.get(url)` দিয়ে ওয়েবসাইটের HTML সোর্স কোড সংগ্রহ করুন।

Step 3: `BeautifulSoup` দিয়ে HTML পার্স করুন।

Step 4: নির্দিষ্ট HTML ট্যাগ/ক্লাস থেকে বইয়ের নাম, লেখক ও দাম বের করুন।

Step 5: সংগৃহীত ডেটা CSV ফাইলে সংরক্ষণ করুন।

সম্পূর্ণ Python Code

```
# ============================================================
# PRACTICAL 4: Web Scraping - Extract Book Information
# Install first: pip install requests beautifulsoup4
#
# NOTE: Replace the URL and CSS selectors with the ACTUAL
# structure of the target website (use browser "Inspect" tool).
# This example uses http://books.toscrape.com - a website made
# specifically for practicing web scraping (safe to use).
# ============================================================
import csv
import requests
from bs4 import BeautifulSoup

URL = "http://books.toscrape.com/"
CSV_FILE = "books_data.csv"
```

```
def get_html_source(url):
    """Fetch the HTML source code from the given URL."""
    response = requests.get(url)
    if response.status_code == 200:
        print("Page fetched successfully! Status code:", response.status_code)
        return response.text
    else:
        print("Failed to fetch page. Status code:", response.status_code)
        return None



def extract_book_data(html):
    """Extract book name, price from the HTML source."""
    soup = BeautifulSoup(html, "html.parser")
    books = soup.find_all("article", class_="product_pod")

    book_list = []
    for book in books:
        title = book.h3.a["title"]
        price = book.find("p", class_="price_color").text
        availability = book.find("p", class_="instock
availability").text.strip()
        book_list.append({"Title": title, "Price": price, "Availability":
availability})

    return book_list



def save_to_csv(book_list):
    """Save the extracted book data into a CSV file."""
    with open(CSV_FILE, "w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=["Title", "Price",
"Availability"])
        writer.writeheader()
        writer.writerows(book_list)
    print("Data saved to", CSV_FILE)



def main():
    html = get_html_source(URL)
    if html is None:
        return

    books = extract_book_data(html)
```

```
    print("\nTotal books found:", len(books))

    print("\n--- First 5 books ---")
    for book in books[:5]:
        print(book["Title"], "|", book["Price"], "|", book["Availability"])

    save_to_csv(books)


if __name__ == "__main__":
    main()
```

রকমারি (rokomari.com)-এর মতো সাইটে প্রয়োগ করার নির্দেশনা

```
# To scrape rokomari.com/book (or any real e-commerce site) safely:
#
# 1. Always check the site's robots.txt first:
#    https://www.rokomari.com/robots.txt
#
# 2. Real websites change their HTML structure often, so:
#    - Open the page in a browser
#    - Right-click on the book title -> "Inspect"
#    - Find the exact tag/class name (it will be different from this demo)
#    - Update soup.find_all("article", class_="...") accordingly
#
# 3. Add a small delay between requests to avoid overloading the server:
#    import time
#    time.sleep(1)
#
# 4. Some sites block simple requests; you may need to send headers:
#    headers = {"User-Agent": "Mozilla/5.0"}
#    response = requests.get(url, headers=headers)
```

Sample Output

```
Page fetched successfully! Status code: 200


Total books found: 20


--- First 5 books ---
A Light in the Attic | £51.77 | In stock
Tipping the Velvet | £53.74 | In stock
Soumission | £50.10 | In stock
Sharp Objects | £47.82 | In stock
Sapiens: A Brief History of Humankind | £54.23 | In stock
```

```
Data saved to books_data.csv
```

ব্যাখ্যা

- `requests.get(url)` ওয়েবসাইটের HTML কোড ডাউনলোড করে — `status_code == 200` মানে অনুরোধ সফল হয়েছে।
- `BeautifulSoup(html, 'html.parser')` HTML-কে একটি নেভিগেট-যোগ্য গঠনে রূপান্তর করে, যাতে ট্যাগ ও ক্লাস খুঁজে বের করা সহজ হয়।
- `find_all()` দিয়ে একই ধরনের সব বই-ব্লক (article ট্যাগ) একসাথে বের করা হয়েছে, তারপর প্রতিটি থেকে নাম/দাম আলাদা করা হয়েছে।
- রকমারির মতো ওয়েবসাইট স্ক্র্যাপ করার আগে অবশ্যই তাদের robots.txt ও Terms of Service মেনে চলতে হবে — এই উদাহরণে books.toscrape.com ব্যবহার করা হয়েছে যা বিশেষভাবে স্ক্র্যাপিং অনুশীলনের জন্য তৈরি।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. `requests` ও `BeautifulSoup`-এর কাজ আলাদা করে ব্যাখ্যা করুন।

2. `response.status_code == 200` কেন যাচাই করা হয়েছে?

3. `find()` ও `find_all()`-এর মধ্যে পার্থক্য কী?

4. কোনো ওয়েবসাইট স্ক্র্যাপ করার আগে কোন বিষয়গুলো নৈতিকভাবে (ethically) বিবেচনা করা উচিত?

প্রজেক্ট 5: Weather Report (Open Weather API ব্যবহার করে)

উদ্দেশ্য (Objective)

OpenWeatherMap API-এর সাথে সংযোগ স্থাপন করে ব্যবহারকারীর দেওয়া শহরের নামের ভিত্তিতে বর্তমান আবহাওয়ার তথ্য সংগ্রহ ও প্রদর্শন করা।

প্রয়োজনীয় সেটআপ (Requirements)

- `pip install requests` — লাইব্রেরি ইনস্টল
- OpenWeatherMap-এ ফ্রি অ্যাকাউন্ট খুলে একটি API Key সংগ্রহ (https://openweathermap.org/api)
- ইন্টারনেট সংযোগ

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: `pip install requests` চালান।

Step 2: OpenWeatherMap ওয়েবসাইটে একাউন্ট খুলে বিনামূল্যে একটি API Key সংগ্রহ করুন।

Step 3: ব্যবহারকারীর কাছ থেকে location (শহরের নাম) ইনপুট নিন।

Step 4: API-তে GET request পাঠিয়ে JSON ফরম্যাটে আবহাওয়ার ডেটা সংগ্রহ করুন।

Step 5: JSON থেকে তাপমাত্রা, আর্দ্রতা, বর্ণনা ইত্যাদি বের করে সুন্দরভাবে প্রদর্শন করুন।

সম্পূর্ণ Python Code

```
# ============================================================
# PRACTICAL 5: Weather Report using OpenWeatherMap API
# Install first: pip install requests
#
# NOTE: Sign up at https://openweathermap.org/api to get a
# FREE API key, then replace "YOUR_API_KEY_HERE" below.
# ============================================================
import requests

API_KEY = "YOUR_API_KEY_HERE"
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"



def get_weather_data(city_name):
    """Connect to the OpenWeatherMap API and fetch weather data."""
    params = {
        "q": city_name,
        "appid": API_KEY,
        "units": "metric"          # get temperature in Celsius
    }

    try:
        response = requests.get(BASE_URL, params=params, timeout=10)
        data = response.json()

        if response.status_code == 200:
            return data
        else:
            print("Error:", data.get("message", "City not found!"))
            return None

    except requests.exceptions.ConnectionError:
        print("Error: No internet connection!")
        return None
    except requests.exceptions.Timeout:
        print("Error: Request timed out!")
        return None



def print_weather_report(data):
    """Print basic weather information from the collected data."""
    city = data["name"]
```

```
    country = data["sys"]["country"]
    temperature = data["main"]["temp"]
    feels_like = data["main"]["feels_like"]
    humidity = data["main"]["humidity"]
    description = data["weather"][0]["description"]
    wind_speed = data["wind"]["speed"]

    print("\n" + "=" * 40)
    print("      WEATHER REPORT -", city.upper(), ",", country)
    print("=" * 40)
    print("Condition   :", description.title())
    print("Temperature :", temperature, "\u00b0C")
    print("Feels Like  :", feels_like, "\u00b0C")
    print("Humidity    :", humidity, "%")
    print("Wind Speed  :", wind_speed, "m/s")
    print("=" * 40)



def main():
    city_name = input("Enter location (city name): ")
    weather_data = get_weather_data(city_name)

    if weather_data:
        print_weather_report(weather_data)



if __name__ == "__main__":
    main()
```

Sample Output

```
Enter location (city name): Dhaka


========================================
      WEATHER REPORT - DHAKA , BD
========================================
Condition   : Haze
Temperature : 31.5 °C
Feels Like  : 36.2 °C
Humidity    : 70 %
Wind Speed  : 2.06 m/s
========================================
```

ব্যাখ্যা

- `requests.get(url, params=params)` API-তে GET request পাঠায় — `params` dictionary স্বয়ংক্রিয়ভাবে URL query string-এ রূপান্তরিত হয়।
- `response.json()` API থেকে পাওয়া JSON রেসপন্সকে সরাসরি Python dictionary-তে রূপান্তর করে, যা দিয়ে সহজে key দিয়ে ডেটা বের করা যায়।
- `units="metric"` parameter দেওয়ায় তাপমাত্রা সরাসরি সেলসিয়াসে পাওয়া গেছে (ডিফল্ট Kelvin-এ আসতো)।
- ইন্টারনেট সংযোগ না থাকলে বা ভুল শহরের নাম দিলে `try-except` দিয়ে প্রোগ্রাম ক্র্যাশ হওয়া থেকে রক্ষা করা হয়েছে।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. API Key কী এবং এটি কেন প্রয়োজন?

2. `response.json()` কী কাজ করে এবং এটি কী টাইপের ডেটা ফেরত দেয়?

3. `units="metric"` parameter না দিলে তাপমাত্রা কোন এককে আসতো?

4. API request ব্যর্থ হলে (যেমন ভুল শহরের নাম) প্রোগ্রামটি কীভাবে তা হ্যান্ডেল করে?

প্রজেক্ট 6: Database Design and Development (GUI ফর্ম দিয়ে Student Database)

উদ্দেশ্য (Objective)

Tkinter ব্যবহার করে একটি সাধারণ GUI ফর্ম ডিজাইন করা, যেখানে ইনপুট ফিল্ড ও কমান্ড বাটন থাকবে, এবং SQLite ডাটাবেসে ডেটা Insert ও Search করা যাবে।

প্রয়োজনীয় সেটআপ (Requirements)

- Python-এর বিল্ট-ইন `tkinter` (GUI) ও `sqlite3` (ডাটাবেস) module — অতিরিক্ত ইনস্টলেশনের প্রয়োজন নেই
- একটি ডেস্কটপ পরিবেশ (GUI দেখানোর জন্য)

কর্মপদ্ধতি (Procedure)

Step 1: Setup Environment: `tkinter` ও `sqlite3` পাইথনের সাথেই থাকে, শুধু import করলেই চলবে।

Step 2: একটি সাধারণ ফর্ম ডিজাইন করুন (Window তৈরি করা)।

Step 3: ফর্মে Name, Roll, Department ইত্যাদির জন্য input field (Entry widget) যোগ করুন।

Step 4: Save ও Search-এর জন্য command button যোগ করুন।

Step 5: Button-এর সাথে function সংযুক্ত করে ডেটা Insert, Save ও Search করার কোড লিখুন।

সম্পূর্ণ Python Code

```
# ============================================================
```

```
# PRACTICAL 6: Database Design and Development (Tkinter + SQLite)
# No installation needed - tkinter and sqlite3 come with Python
# ============================================================
import tkinter as tk
from tkinter import messagebox
import sqlite3

# ---------- Step 1: Setup Database ----------
connection = sqlite3.connect("student_database.db")
cursor = connection.cursor()
create_table_query = (
    "CREATE TABLE IF NOT EXISTS students ("
    "id INTEGER PRIMARY KEY AUTOINCREMENT, "
    "name TEXT NOT NULL, "
    "roll TEXT NOT NULL, "
    "department TEXT NOT NULL)"
)
cursor.execute(create_table_query)
connection.commit()



# ---------- Step 2: Functions for buttons ----------
def save_student():
    name = entry_name.get()
    roll = entry_roll.get()
    department = entry_department.get()

    if name == "" or roll == "" or department == "":
        messagebox.showwarning("Input Error", "Please fill all fields!")
        return

    cursor.execute("INSERT INTO students (name, roll, department) VALUES (?, ?,
?)",
                   (name, roll, department))
    connection.commit()
    messagebox.showinfo("Success", "Student data saved successfully!")
    clear_fields()



def search_student():
    roll = entry_roll.get()
    if roll == "":
        messagebox.showwarning("Input Error", "Please enter a roll number to
search!")
        return

    cursor.execute("SELECT name, roll, department FROM students WHERE roll
```

```
= ?", (roll,))
    result = cursor.fetchone()

    if result:
        entry_name.delete(0, tk.END)
        entry_name.insert(0, result[0])
        entry_department.delete(0, tk.END)
        entry_department.insert(0, result[2])
        messagebox.showinfo("Found", "Student record found!")
    else:
        messagebox.showerror("Not Found", "No student found with roll: " +
roll)



def clear_fields():
    entry_name.delete(0, tk.END)
    entry_roll.delete(0, tk.END)
    entry_department.delete(0, tk.END)



# ---------- Step 3: Design the form (GUI) ----------
window = tk.Tk()
window.title("Student Database Form")
window.geometry("350x250")

tk.Label(window, text="Student Database Form", font=("Arial", 14,
"bold")).grid(
    row=0, column=0, columnspan=2, pady=10)

tk.Label(window, text="Name:").grid(row=1, column=0, padx=10, pady=5,
sticky="e")
entry_name = tk.Entry(window, width=25)
entry_name.grid(row=1, column=1, padx=10, pady=5)

tk.Label(window, text="Roll:").grid(row=2, column=0, padx=10, pady=5,
sticky="e")
entry_roll = tk.Entry(window, width=25)
entry_roll.grid(row=2, column=1, padx=10, pady=5)

tk.Label(window, text="Department:").grid(row=3, column=0, padx=10, pady=5,
sticky="e")
entry_department = tk.Entry(window, width=25)
entry_department.grid(row=3, column=1, padx=10, pady=5)

# Command buttons
tk.Button(window, text="Save", width=10, command=save_student).grid(row=4,
```

```
column=0, pady=15)
tk.Button(window, text="Search", width=10, command=search_student).grid(row=4,
column=1, pady=15)

window.mainloop()

# Close the database connection when the window is closed
connection.close()
```

Sample Output

```
(This program opens a GUI window - not text output)


+-----------------------------------------+
|       Student Database Form             |
|                                          |
|  Name:        [ Rahim Ahmed          ]   |
|  Roll:        [ 101                  ]   |
|  Department:  [ CSE                  ]   |
|                                          |
|     [ Save ]          [ Search ]         |
+-----------------------------------------+

Message box: "Student data saved successfully!"
```

ব্যাখ্যা

- `tkinter`-এ `Entry` widget দিয়ে ইনপুট ফিল্ড তৈরি করা হয়েছে, `grid()` দিয়ে ফর্মের লেআউট (row/column) সাজানো হয়েছে।
- `sqlite3` হলো পাইথনের বিল্ট-ইন হালকা ডাটাবেস — আলাদা সার্ভার ইনস্টল করার প্রয়োজন নেই, একটি `.db` ফাইলেই সব ডেটা থাকে।
- `command=save_student` দিয়ে বাটনের সাথে function যুক্ত করা হয়েছে — বাটনে ক্লিক করলেই সেই function চলে।
- `?` প্লেসহোল্ডার ব্যবহার করে SQL query-তে মান বসানো হয়েছে (parameterized query) — এটি SQL Injection আক্রমণ থেকে সুরক্ষা দেয়, সরাসরি string concatenation করা উচিত নয়।

ভাইভা / মূল্যায়ন প্রশ্ন (Viva Questions)

1. `tkinter` ও `sqlite3`-এর কাজ কী এবং কেন এগুলো ব্যবহার করা হয়েছে?

2. `grid()` layout manager কীভাবে কাজ করে?

3. SQL query-তে `?` প্লেসহোল্ডার ব্যবহার করার সুবিধা কী?

4. `CREATE TABLE IF NOT EXISTS` কেন ব্যবহার করা হয়েছে?